C# — leerboek

Programming Advanced · van tekst tot database · 42 hoofdstukken · met voorspelvragen om jezelf te testen

Inhoudsopgave

Hoe je dit gebruikt 4
Inhoud 6
Leerwijzer per les 8
Keuzegids 9

Deel 1 — De basis 12
1. Tekst 13
2. Getallen en omzetten 28
3. Datum en tijd 37
4. Beslissen en herhalen 47

Deel 2 — Verzamelingen 54
De verzamelingen naast elkaar 54
5. Array 55
6. List 72
7. Dictionary 89
8. HashSet 105
9. Queue 116
10. Stack 125

Deel 3 — Zoeken en rekenen 134
11. LINQ 135
12. IEnumerable 148

Deel 4 — Eigen types 155
13. class 156
14. record 163
15. struct 168
16. Referentie tegenover waarde 172
17. Omgaan met niets 175
18. enum 181
19. interface 186
20. Overerving en abstracte klassen 190
21. Generics 198
22. Types herkennen 204
23. Uitzonderingen 207
24. static, const en readonly 214

Deel 5 — In je API 221
25. Attributen en routes 222
26. De helpers van een controller 227
27. Program.cs en dependency injection 230

28. Logging 235
29. Asynchroon werken 239
30. JSON en bestanden 244

Deel 6 — Data 248

31. EF Core: model en database 249
32. EF Core: opvragen en bewaren 258
33. DTO's en Mapster 269
34. Dapper 276

Deel 7 — Je API, les per les 283

35. Zo groeit je project 284
36. Les 03: repository en dependency injection 289
37. Les 04: Docker, EF Core en de eerste migratie 327
38. Les 05: de repository op de database, en async 343
39. Les 06: relaties en startgegevens 368
40. Les 07: generieke repository en Unit of Work 390
41. Les 08: Include en kringlopen 396
42. Les 09: DTO's en Mapster 401

Veelgemaakte verwarringen 406

Index 412

Hoe je dit gebruikt

Dit leerboek brengt twee dingen samen die je nodig hebt om vlot in C# te programmeren: de **bouwstenen** — wat een `List`, een `Dictionary` of een `record` ís — en de **methodes** — wat je ermee kan dóen. Ze staan in een volgorde die van eenvoudig naar ingewikkeld gaat.

Elk hoofdstuk is ook volledig op zich. Bij een verzameling of een type staat niet enkel wat het zelf kan, maar ook wat het kan **via iets anders**: via LINQ, via een `foreach`, via een omzetting of via `object`. Zo mis je niets als je één hoofdstuk apart opzoekt. Wordt iets pas in een later hoofdstuk volledig uitgelegd, dan staat er een verwijzing naar dat hoofdstuk.

Er zijn drie soorten hoofdstukken.

- Een **bouwsteen** volgt altijd dezelfde stappen: wat het is, hoe het eruitziet, wat je ermee kan, wat het ook kan via iets anders, waar je op let, wanneer wel en wanneer niet, en de veelgemaakte fouten.
- Een **methodehoofdstuk** is geordend op wat je wil doen: *ik wil een stuk uit een tekst halen*, *ik wil één item uit een lijst*. Per vraag staan de methodes die ervoor bestaan, en welke je in welk geval neemt.
- Een **leshoofdstuk**, in deel 7, volgt één les van Programming Advanced: wat je bouwt, wat er bijkomt in je project, de stappen, wat er verandert in `Program.cs`, en wat je ziet als je een stap vergeet.

Elk hoofdstuk eindigt met **Voorspel de uitkomst**. Lees de code, beslis wat er op het scherm komt, en kijk pas dan naar het antwoord, op de laatste pagina van het hoofdstuk. Uitleg lezen voelt als begrijpen, maar pas als je een uitkomst juist voorspelt, weet je of het zit. Heb je er één fout, lees dan de uitleg bij het antwoord en probeer de code zelf uit in een lege consoletoepassing.

Kom je later een naam tegen en wil je weten wat hij doet, gebruik dan de index achteraan.

Volg je de lessen van Programming Advanced, dan zegt de leerwijzer welke hoofdstukken bij elke les horen.

Bouw je je eigen API mee met de lessen, begin dan bij deel 7. Per les staat daar wat je maakt, installeert, registreert en uitvoert, en welk hoofdstuk het uitlegt.

Kleuren en labels

Label of kleur	Betekenis
METHODES	een hoofdstuk geordend op wat je wil doen
VERZAMELING	een bouwsteen die meerdere dingen bewaart
EIGEN TYPE	een bouwsteen die je zelf maakt
CONCEPT	een idee dat door alle hoofdstukken heen loopt
IN JE API	wat je nodig hebt zodra je een API bouwt
DATA	werken met een database
`methode`	de naam van een methode of eigenschap
`resultaat`	wat eruit komt
`Exception`	het programma crasht, of compileert niet
Wel / Niet	wanneer je een bouwsteen kiest, en wanneer niet

> **OPMERKING**
>
> De voorbeelden zijn losse regels over fruit, steden, leerlingen, dieren, recepten en verkeerslichten, en hier en daar een onderdeel of een ventilator. Het zijn nooit stukken uit Bouwcontrole of uit je schooloefeningen. Welke bouwsteen je in je eigen projecten gebruikt, beslis je zelf — dit leerboek geeft je de kennis om die keuze te kunnen maken.

> **OPMERKING**
>
> De hoofdletters doen ertoe. In C# is het `Find`, niet `find`, en `typeof`, niet `TypeOf`. Methodes beginnen met een hoofdletter, sleutelwoorden van de taal met een kleine letter. Kleine letters vooraan zijn meestal JavaScript.

Inhoud

Deel	Hoofdstuk	Wat je leert
Vooraan	Leerwijzer per les	welke hoofdstukken bij welke les horen
	Keuzegids	welke bouwsteen je neemt, en waar je iets vindt
1. De basis	1. Tekst	lengte, hoofdletters, knippen, zoeken, vergelijken
	2. Getallen en omzetten	tekst naar getal, afronden, delen met rest
	3. Datum en tijd	datums maken, rekenen, tonen
	4. Beslissen en herhalen	`if`, `switch`, `for`, `while`
2. Verzamelingen	5. Array	een rij met vaste lengte
	6. List	een lijst die groeit en krimpt
	7. Dictionary	opzoeken op een sleutel
	8. HashSet	een verzameling zonder dubbels
	9. Queue	wie eerst komt, eerst geholpen
	10. Stack	het laatste eerst terug
3. Zoeken en rekenen	11. LINQ	zoeken, filteren, tellen en sorteren in elke verzameling
	12. IEnumerable	wat LINQ eigenlijk teruggeeft
4. Eigen types	13. class	een bouwplan voor je eigen objecten
	14. record	een pakketje gegevens dat draait om de inhoud
	15. struct	een klein pakketje dat echt gekopieerd wordt
	16. Referentie tegenover waarde	het belangrijkste idee van dit leerboek
	17. Omgaan met niets	`null`, `??`, `?.` en getallen die mogen ontbreken
	18. enum	een vaste lijst benoemde keuzes
	19. interface	een contract: wat iets moet kunnen
	20. Overerving en abstracte klassen	verder bouwen op een andere klasse, `abstract`, `override`
	21. Generics	een type met een open plaats, zoals `List<T>`
	22. Types herkennen	`is`, `as`, `typeof`, `nameof`
	23. Uitzonderingen	`try`, `catch`, `throw`: fouten opvangen en melden
	24. static, const en readonly	wat bij de klasse hoort, en wat niet meer verandert
5. In je API	25. Attributen en routes	welke URL bij welke methode uitkomt
	26. De helpers van een controller	`Ok`, `NotFound`, `BadRequest` en de rest
	27. Program.cs en dependency injection	opstarten, diensten registreren en opvragen
	28. Logging	meldingen in het terminalvenster
	29. Asynchroon werken	`async`, `await`, `Task`

Deel	Hoofdstuk	Wat je leert
	30. JSON en bestanden	lezen, schrijven, omzetten
6. Data	31. EF Core: model en database	tabellen, relaties en migraties
	32. EF Core: opvragen en bewaren	ophalen, toevoegen, wijzigen, `Include`
	33. DTO's en Mapster	enkel doorgeven wat de client nodig heeft
	34. Dapper	zelf SQL schrijven, met een micro-ORM
7. Je API, les per les	35. Zo groeit je project	de hele weg: mappen, bestanden en Program.cs na elke les
	36. Les 03: repository en dependency injection	de lijst uit je controller, achter een interface
	37. Les 04: Docker, EF Core en de eerste migratie	een database in Docker, de context, de eerste migratie
	38. Les 05: de repository op de database, en async	de overstap naar de database, en async
	39. Les 06: relaties en startgegevens	meer modellen, relaties en HasData
	40. Les 07: generieke repository en Unit of Work	de standaardbewerkingen één keer, en één keer bewaren
	41. Les 08: Include en kringlopen	gerelateerde gegevens meeladen
	42. Les 09: DTO's en Mapster	enkel tonen en ontvangen wat nodig is
Achteraan	Veelgemaakte verwarringen	tweeënveertig paren die op elkaar lijken
	Index	alles alfabetisch, met elk hoofdstuk waar het voorkomt

Leerwijzer per les

Welke hoofdstukken horen bij welke les van Programming Advanced? Lees ze vóór of naast de les. Deel 1 heb je in elke les nodig; de tabel noemt de hoofdstukken die in die les nieuw zijn of het meest terugkomen.

Les	Onderwerp	Hoofdstukken
01	Introductie	13, 20, 25, 26, 27
02	HTTP Verbs	6, 11, 17, 24, 25, 26
03	Repositories	17, 19, 24, 27, 28, 35, 36
04	Entity Framework Core	20, 27, 31, 37
05	Asynchroon programmeren	12, 23, 29, 32, 38
06	Relaties	2, 3, 17, 31, 39
07	Generics & Unit of Work	17, 19, 20, 21, 32, 40
08	Include statements	7, 13, 32, 34, 41
09	DTO's	11, 14, 33, 42

OPMERKING

De tabel loopt tot les 09. Komt er een les bij, zoek dan de namen die je erin tegenkomt op in de index achteraan.

Het laatste nummer bij les 03 tot 09 is het leshoofdstuk uit deel 7: wat je in die les bouwt, stap voor stap. Hoofdstuk 35 toont de hele weg in één keer.

Keuzegids

Welke bouwsteen neem ik?

Ik wil…	Neem	Hoofdstuk
een bedrag of prijs, exact tot op de cent	`decimal`	2
een datum of een tijdstip	`DateTime`	3
een vast aantal dingen dat nooit verandert	`array`	5
een aantal dingen waar ik bijvoeg en afhaal	`List`	6
iets terugvinden op een naam, code of nummer	`Dictionary`	7
zeker zijn dat er geen dubbels in zitten	`HashSet`	8
dingen afhandelen in de volgorde waarin ze binnenkwamen	`Queue`	9
het laatst toegevoegde als eerste terugnemen	`Stack`	10
een verzameling doorgeven zonder te zeggen welke soort	`IEnumerable`	12
een eigen ding met gegevens én gedrag	`class`	13
een pakketje gegevens dat gelijk is zodra de inhoud gelijk is	`record`	14
een klein pakketje dat bij kopiëren echt gekopieerd wordt	`struct`	15
een vaste lijst benoemde keuzes	`enum`	18
afspreken wat iets moet kunnen, zonder te zeggen hoe	`interface`	19
verwante klassen die gegevens en uitgewerkte methodes delen	`abstract class`	20
één stuk code dat werkt voor elk type dat je invult	generics, `<T>`	21
iets dat bij de klasse hoort, niet bij één object	`static`	24
een waarde die na het aanmaken nooit meer verandert	`const` of `readonly`	24
gegevens bewaren in een database, zonder zelf SQL te schrijven	EF Core	31
aan de client enkel geven wat hij nodig heeft	een DTO	33
zelf de SQL schrijven voor een snelle leesvraag	Dapper	34

Waar vind ik hoe ik…

Ik wil…	Hoofdstuk
een stuk uit een tekst halen, of tekst opsplitsen	1
twee teksten vergelijken zonder op hoofdletters te letten	1
tekst omzetten naar een getal zonder te crashen	2
iets verdelen: hoeveel per groep, hoeveel over	2
met datums rekenen, of een datum tonen	3
kiezen tussen meerdere mogelijkheden	4
iets een aantal keer herhalen	4
iets toevoegen aan of verwijderen uit een lijst	6
alles zien wat een verzameling kan, ook via LINQ	5 tot 10
weten welke lus op welke verzameling werkt, en wat ik mag veranderen terwijl ze loopt	4
per soort tellen hoeveel er van zijn	7
één item uit een verzameling halen	11
weten of er iets in zit dat voldoet	11
filteren, sorteren, optellen, het grootste vinden	11
zelf een reeks getallen maken, met of zonder stap	11
begrijpen waarom mijn resultaat plots veranderd is	12
begrijpen waarom mijn lijst na een methode veranderd is	16
een verzameling kopiëren	16
iets doen met een waarde die `null` kan zijn	17
begrijpen wat `= default!` achter een eigenschap doet	17
weten welke methodes `null` geven in plaats van te crashen	17
weten hoe mijn eigen klasse zich gedraagt in een lijst, een `HashSet` of een `Dictionary`	13, 14
mijn eigen klasse sorteerbaar maken	19
een methode van een basisklasse anders laten werken	20
één klasse schrijven die voor elk type werkt	21
nagaan van welk type een object is	22
enkel de objecten van één type uit een lijst halen	22
een fout opvangen, zodat mijn programma niet stopt	23
zelf een fout melden vanuit een methode	23
weten welke methode welke uitzondering geeft	23
een lijst bewaren tussen twee verzoeken	24
een lijst alleen-lezen maken	24

Ik wil…	Hoofdstuk
een methode aan een URL koppelen	25
een waarde uit de URL in mijn methode krijgen	25
de juiste statuscode teruggeven in een API	26
mijn repository aan mijn controller geven	27
iets melden in het terminalvenster terwijl de API draait	28
een methode asynchroon maken	29
een JSON-bestand inlezen	30
weten hoe een verzameling of een enum eruitziet in JSON	30
een tabel maken voor mijn klasse	31
een relatie tussen twee tabellen vastleggen	31
mijn database bijwerken na een wijziging aan een klasse	31
gegevens ophalen, toevoegen, wijzigen of verwijderen in de database	32
gerelateerde gegevens meeladen	32
weten welke LINQ-methodes EF Core kent, en welke async-versies er zijn	32
een entiteit omzetten naar een DTO	33

Deel 1 — De basis

Tekst, getallen, datums en beslissingen heb je in elk ander hoofdstuk nodig. Daarom komen ze eerst.

1. Tekst

METHODES

Een tekst heet in C# een `string`. Alles in dit hoofdstuk werkt op een `string`. Eén ding vooraf, omdat het overal terugkomt: **een tekst verandert nooit**. Elke methode hieronder geeft een **nieuwe** tekst terug en laat de oude zoals ze was.

Hoe lang is het, is het leeg?

Methode	Wat het doet	Voorbeeld	Resultaat
`Length`	aantal tekens; een eigenschap, geen methode, dus zonder haakjes	`"moederbord".Length`	`10`
`string.IsNullOrEmpty`	is het null of een lege tekst?	`string.IsNullOrEmpty("")`	`true`
`string.IsNullOrWhiteSpace`	is het null, leeg, of enkel spaties, tabs of regeleindes?	`string.IsNullOrWhiteSpace(" ")`	`true`
`string.Empty`	een lege tekst, hetzelfde als `""`	`string.Empty == ""`	`true`

Voor invoer die van een gebruiker komt neem je bijna altijd `IsNullOrWhiteSpace`. Iemand die een spatie intikt heeft niets ingevuld, maar `IsNullOrEmpty` vindt dat wel een waarde.

> LET OP
>
> `Length` op `null` crasht met een `NullReferenceException`. `naam?.Length` geeft dan `null` in plaats van te crashen. Zie hoofdstuk 17.

Hoofdletters, kleine letters, spaties weg

Methode	Wat het doet	Voorbeeld	Resultaat
`ToUpper`	alles hoofdletters	`"mb-b650".ToUpper()`	`"MB-B650"`
`ToLower`	alles kleine letters	`"MB-B650".ToLower()`	`"mb-b650"`
`ToUpperInvariant`, `ToLowerInvariant`	hetzelfde, maar los van de taal van de computer	`"mb-b650".ToUpperInvariant()`	`"MB-B650"`
`Trim`	spaties vooraan en achteraan weg	`"  ATX  ".Trim()`	`"ATX"`
`TrimStart`	enkel vooraan	`"  ATX".TrimStart()`	`"ATX"`
`TrimEnd`	enkel achteraan	`"ATX  ".TrimEnd()`	`"ATX"`
`Trim`	andere tekens weg dan spaties	`"--ATX--".Trim('-')`	`"ATX"`
`TrimStart`	voorloopnullen weg	`"007".TrimStart('0')`	`"7"`
`PadLeft`	vul aan tot een lengte, vooraan	`"7".PadLeft(3, '0')`	`"007"`
`PadRight`	vul aan tot een lengte, achteraan	`"AM5".PadRight(6, '.')`	`"AM5..."`

Deze methodes veranderen je tekst niet, ze geven een nieuwe terug. `naam.Trim()` op een regel alleen doet niets; je moet het resultaat ergens opvangen.

`Trim` haalt niet enkel spaties weg, maar ook tabs en regeleindes. `PadLeft` op een tekst die al lang genoeg is, doet niets: `"appel".PadLeft(3)` blijft `"appel"`.

Eén hoofdletter vooraan bestaat niet als methode. Je bouwt het zelf: `tekst[0..1].ToUpper() + tekst[1..]` maakt van `"appel"` `"Appel"` — maar crasht op een lege tekst.

Een stuk uit een tekst halen

Methode	Wat het doet	Voorbeeld	Resultaat
`Substring`	vanaf een positie, eventueel een aantal tekens	`"moederbord".Substring(6)`	`"bord"`
`Substring`	met lengte erbij	`"moederbord".Substring(0, 6)`	`"moeder"`
`[..]`	hetzelfde, met de bereikoperator	`"moederbord"[0..6]`	`"moeder"`
`[^..]`	de laatste n tekens	`"LZ-2026-8820"[^4..]`	`"8820"`
`[i]`	één teken op positie i; geeft een `char`	`"Gent"[0]`	`'G'`
`[^1]`	het laatste teken	`"kiwi"[^1]`	`'i'`
`Split`	knip op een teken, geeft een array	`"ATX,mATX,ITX".Split(',')`	`["ATX", "mATX", "ITX"]`
`Split`	knip op meerdere tekens	`"appel;peer,kiwi".Split(';', ',')`	`["appel", "peer", "kiwi"]`
`Split`	knip op een stuk tekst	`"appel en peer en kiwi".Split(" en ")`	`["appel", "peer", "kiwi"]`
`Split`	lege stukken weglaten	`"rood;groen;;blauw".Split(';', StringSplitOptions.RemoveEmptyEntries)`	`["rood", "groen", "blauw"]`
`Split`	spaties rond elk stuk weg	`"appel , peer , kiwi".Split(',', StringSplitOptions.TrimEntries)`	`["appel", "peer", "kiwi"]`
`Split`	hoogstens n stukken	`"naam=Jan=Peeters".Split('=', 2)`	`["naam", "Jan=Peeters"]`
`string.Join`	plak stukken aan elkaar met een scheider	`string.Join(" / ", delen)`	`"ATX / mATX / ITX"`
`string.Concat`	plak stukken aan elkaar zonder scheider	`string.Concat("ATX", "-", "AM5")`	`"ATX-AM5"`
`Replace`	vervang alle voorkomens	`"MB-B650-01".Replace("-", "")`	`"MBB65001"`
`Replace`	zonder op hoofdletters te letten	`"Peer en peer".Replace("PEER", "peer", StringComparison.OrdinalIgnoreCase)`	`"peer en peer"`

Methode	Wat het doet	Voorbeeld	Resultaat
`Remove`	haal weg vanaf een positie, eventueel een aantal tekens	`"moederbord".Remove(6)`	`"moeder"`
`Insert`	voeg in op een positie	`"moederbord".Insert(6, "-")`	`"moeder-bord"`

Je kan beide opties samen meegeven: `StringSplitOptions.RemoveEmptyEntries | StringSplitOptions.TrimEntries`.

> **LET OP**
>
> `Substring`, `Remove`, `Insert` en `[..]` lopen stuk zodra je buiten de tekst grijpt. `"ATX".Substring(0, 5)` en `"ATX"[0..5]` geven geen lege tekst maar een `ArgumentOutOfRangeException`. Controleer eerst de `Length`, of gebruik `Split` wanneer je niet weet hoe lang iets is.

Zit er iets in, en waar?

Methode	Wat het doet	Voorbeeld	Resultaat
`Contains`	zit het erin, ja of nee	`"MB-B650-01".Contains("650")`	`true`
`Contains`	zonder op hoofdletters te letten	`"MB-B650".Contains("b650", StringComparison.OrdinalIgnoreCase)`	`true`
`StartsWith`	begint het ermee	`"MB-B650".StartsWith("MB")`	`true`
`EndsWith`	eindigt het erop	`"MB-B650-01".EndsWith("01")`	`true`
`IndexOf`	positie van het eerste voorkomen	`"MB-B650-01".IndexOf("-")`	`2`
`IndexOf`	niet gevonden	`"MB-B650".IndexOf("x")`	`-1`
`IndexOf`	zoek pas vanaf een positie	`"MB-B650-01".IndexOf('-', 3)`	`7`
`LastIndexOf`	positie van het laatste voorkomen	`"MB-B650-01".LastIndexOf("-")`	`7`
`IndexOfAny`, `LastIndexOfAny`	eerste of laatste van meerdere tekens	`"MB-B650-01".IndexOfAny(new[] { '5', '0' })`	`5`

`Contains`, `StartsWith`, `EndsWith` en `IndexOf` werken met een stuk tekst (`"650"`) én met één teken (`'-'`), en ze aanvaarden allemaal een `StringComparison` als laatste waarde.

`Contains` als je enkel wil weten óf het erin zit. `IndexOf` als je daarna nog iets met de plaats wil doen, bijvoorbeeld alles ervoor of erna afsplitsen.

> **LET OP**
>
> `IndexOf` geeft `-1` terug als er niets gevonden is, geen `null`. Vergeet je dat te controleren en steek je die `-1` in een `Substring`, dan loopt je code stuk op een plek die er onschuldig uitziet.

Tekst vergelijken

Methode	Wat het doet	Voorbeeld	Resultaat
`==`	letterlijk gelijk, hoofdlettergevoelig	`"AM5" == "am5"`	`false`
`!=`	verschillend	`"AM5" != "am5"`	`true`
`Equals`	idem, maar je kan de regels meegeven	`"AM5".Equals("am5", StringComparison.OrdinalIgnoreCase)`	`true`
`string.Equals`	idem, en crasht niet als een van beide `null` is	`string.Equals(naam, "am5", StringComparison.OrdinalIgnoreCase)`	`true` of `false`
`CompareTo`	welke komt eerst in de sortering	`"b".CompareTo("a")`	`1`: b komt na a
`string.Compare`	idem, met regels en zonder te crashen op `null`	`string.Compare("appel", "Banaan")`	`-1`: appel komt eerst

Wil je twee sockets vergelijken die de ene keer met hoofdletters en de andere keer zonder binnenkomen, dan is `Equals` met `OrdinalIgnoreCase` het nette antwoord. `ToUpper()` op allebei werkt ook, maar maakt twee nieuwe teksten om er één vergelijking mee te doen.

`CompareTo` en `string.Compare` geven een getal: **negatief** als de eerste vooraan komt, **0** als ze gelijk zijn, **positief** als de eerste achteraan komt. Reken niet op 1 of -1: met `StringComparison.Ordinal` kan het ook 31 zijn.

`StringComparison`	Hoofdletters tellen	Sorteert zoals
`Ordinal`	ja	de codes van de tekens: alle hoofdletters vóór alle kleine letters
`OrdinalIgnoreCase`	nee	idem, zonder hoofdletters
`CurrentCulture`	ja	een woordenboek in de taal van de computer
`CurrentCultureIgnoreCase`	nee	idem, zonder hoofdletters
`InvariantCulture`, `InvariantCultureIgnoreCase`	ja / nee	een woordenboek, los van de taal van de computer

Voor codes, sleutels en wachtwoorden neem je `Ordinal` of `OrdinalIgnoreCase`. Voor namen die je alfabetisch aan een mens toont, de `Culture`-varianten.

Tekst opbouwen met waarden erin

Vorm	Wat het doet	Voorbeeld	Resultaat
`$"..."`	zet waarden rechtstreeks in de tekst	`$"Socket {socket} past"`	`"Socket AM5 past"`
`$"{x:0.00}"`	met een opmaak voor getallen	`$"{prijs:0.00} euro"`	`"19.50 euro"` of `"19,50 euro"`, zie hoofdstuk 2
`$"{x,5}"`	uitlijnen op een breedte	`$"[{getal,5}]"`	`"[ 7]"`
`+`	plakken	`"Socket " + socket`	`"Socket AM5"`
`+=`	iets achteraan bijplakken	`tekst += "!"`	`tekst` is een nieuwe tekst met `!` erachter
`string.Format`	oudere vorm met plaatshouders	`string.Format("Socket {0}", socket)`	`"Socket AM5"`
`new string`	een teken een aantal keer herhalen	`new string('-', 5)`	`"-----"`

De `$"..."`-vorm is wat je overal zal tegenkomen. Je mag er ook berekeningen in zetten: `$"Nog {sloten - modules} vrij"`.

Speciale tekens schrijf je met een backslash: `\"` is een aanhalingsteken, `\n` een nieuwe regel, `\t` een tab, `\\` een backslash. Zet je `@` voor de tekst, dan telt de backslash niet meer als speciaal: `@"C:\data\fruit.txt"`. Drie aanhalingstekens (`"""..."""`) maken een tekst waarin je gewone aanhalingstekens mag zetten.

Tekst in stukken opbouwen: StringBuilder

Elke `+=` maakt een nieuwe tekst. In een lus die duizend keer draait, zijn dat duizend teksten. Een `StringBuilder` bouwt in één stuk op en maakt pas op het einde een tekst.

```
var sb = new StringBuilder();       // using System.Text;
foreach (int i in new[] { 1, 2, 3 })
{
    sb.Append(i);
    sb.Append(';');
}
string resultaat = sb.ToString();   // "1;2;3;"
```

Methode	Wat het doet	Voorbeeld	Resultaat
`Append`	plak achteraan bij; tekst, teken of getal	`sb.Append("appel")`	de builder bevat "appel"
`AppendLine`	plak bij en begin een nieuwe regel	`sb.AppendLine("regel 1")`	"regel 1" en een regeleinde
`AppendJoin`	plak een reeks bij, met een scheider	`sb.AppendJoin(", ", fruit)`	"appel, peer, kiwi"
`Insert`, `Remove`, `Replace`	zoals bij een tekst, maar in de builder zelf	`sb.Insert(0, "> ")`	"> " staat vooraan
`Length`, `[i]`	lengte, en een teken lezen of **vervangen**	`sb[0] = 'X'`	het eerste teken is nu X
`Clear`	maak leeg	`sb.Clear()`	`Length` is 0
`ToString`	maak er een tekst van	`sb.ToString()`	een `string`

Losse tekens: char

Een tekst bestaat uit tekens. Eén teken is een `char` en staat tussen **enkele** aanhalingstekens: `'a'`. `"kiwi"[0]` geeft een `char`, geen `string`.

Methode	Wat het doet	Voorbeeld	Resultaat
`char.IsDigit`	is het een cijfer	`char.IsDigit('7')`	`true`
`char.IsLetter`	is het een letter, ook met accent	`char.IsLetter('é')`	`true`
`char.IsLetterOrDigit`	letter of cijfer	`char.IsLetterOrDigit('-')`	`false`
`char.IsWhiteSpace`	spatie, tab of regeleinde	`char.IsWhiteSpace('\t')`	`true`
`char.IsUpper`, `char.IsLower`	hoofdletter, kleine letter	`char.IsUpper('A')`	`true`; bij een cijfer allebei `false`
`char.IsPunctuation`	leesteken	`char.IsPunctuation('!')`	`true`
`char.IsAsciiDigit`, `char.IsAsciiLetter`	enkel 0–9, of enkel a–z en A–Z	`char.IsAsciiLetter('é')`	`false`
`char.ToUpper`, `char.ToLower`	één teken als hoofdletter of kleine letter	`char.ToUpper('a')`	`'A'`
`char.GetNumericValue`	het cijfer als getal	`char.GetNumericValue('7')`	`7`, als `double`; bij een letter `-1`
`'7' - '0'`	het cijfer als `int`, de korte manier	`'7' - '0'`	`7`

> **LET OP**
>
> Met tekens rekenen geeft getallen, geen tekst. `'7' + 1` is `56`: de code van het teken 7, plus 1. En `'a' + 'b'` is `195`. Wil je tekst, zorg dan dat er een `string` bij staat: `"a" + 'b'` is `"ab"`.

Tekst en getallen

Een getal uit een tekst halen doe je met `TryParse`. Hoofdstuk 2 gaat er dieper op in; de kern:

Methode	Wat het doet	Voorbeeld	Resultaat
`int.TryParse`	probeer om te zetten, zonder te crashen	`int.TryParse("120", out int aantal)`	`true`, `aantal` is 120; bij `"abc"` `false` en 0
`int.Parse`	zet om, of crasht	`int.Parse("abc")`	`FormatException`
`ToString`	een getal als tekst	`120.ToString()`	`"120"`

`TryParse` aanvaardt spaties rond het getal (`" 42 "` lukt), maar geen letters erachter (`"12abc"` lukt niet).

Wat het ook kan, via iets anders

Een tekst is ook een **reeks tekens**: je kan hem teken per teken doorlopen. In C# is een `string` een `IEnumerable<char>` (hoofdstuk 12). Daardoor werkt alles wat met een reeks werkt ook op een tekst.

Bron	Waarom het op een tekst werkt	Volledige uitleg
LINQ	een tekst is een reeks tekens	hoofdstuk 11
`foreach`	doorloopt de tekst teken per teken	hoofdstuk 4
`switch` en `is`	vergelijken met vaste teksten en patronen	hoofdstuk 4
andere verzamelingen	een lijst of set van de tekens	hoofdstukken 5 tot 8
wat elk object kan	een tekst is ook een object	hoofdstuk 13

Wat je over LINQ op een tekst moet weten:

- LINQ werkt **teken per teken**. `"kiwi".Count()` telt 4 tekens.
- Het resultaat is **geen tekst** maar een reeks tekens. `"kiwi".Reverse()` geeft geen `"iwik"`. Maak er weer tekst van met `string.Concat(...)` of `new string(....ToArray())`.
- LINQ werkt enkel met `using System.Linq;` bovenaan. In een nieuw project staat die er automatisch bij.

Tekens zoeken en nagaan

Komt van LINQ.

Methode	Wat het doet	Voorbeeld	Resultaat	Eigen tegenhanger
`First`, `FirstOrDefault`	het eerste teken (dat voldoet)	`"abc123".First(char.IsDigit)`	`'1'`	`[0]`, `IndexOfAny`
`Last`, `LastOrDefault`	het laatste teken (dat voldoet)	`"kiwi".Last()`	`'i'`	`[^1]`
`Single`, `SingleOrDefault`	het enige teken dat voldoet	`"ab1c".Single(char.IsDigit)`	`'1'`	—
`ElementAt`	het teken op een positie	`"kiwi".ElementAt(2)`	`'w'`	`[2]`
`Any`	is er een teken dat voldoet	`"abc123".Any(char.IsDigit)`	`true`	—
`All`	voldoen alle tekens	`"1234".All(char.IsDigit)`	`true`	—
`Contains`	zit dit teken erin	`"kiwi".Contains('w')`	`true`	`Contains`: de tekst heeft een eigen versie
`SequenceEqual`	dezelfde tekens in dezelfde volgorde	`"kiwi".SequenceEqual("kiwi")`	`true`	`==`

LET OP

- `FirstOrDefault` op een tekst geeft bij niets geen `null`, maar het lege teken `'\0'`.
- `"".All(char.IsDigit)` is `true`: een lege tekst heeft geen enkel teken dat géén cijfer is. Wil je weten of iets een getal is, controleer dan ook dat het niet leeg is, of gebruik `int.TryParse`.

Tekens kiezen, tellen en rekenen

Komt van LINQ.

Methode	Wat het doet	Voorbeeld	Resultaat
`Where`	enkel de tekens die voldoen	`string.Concat("a1b2c3".Where(char.IsDigit))`	`"123"`
`Select`	maak van elk teken iets anders	`"123".Select(c => c - '0')`	`1, 2, 3` als getallen
`Distinct`	elk teken één keer	`string.Concat("banaan".Distinct())`	`"ban"`
`Take`, `Skip`, `TakeLast`, `SkipLast`	de eerste of laatste n tekens	`"kiwi".TakeLast(2)`	`'w', 'i'`
`TakeWhile`, `SkipWhile`	zolang de voorwaarde klopt	`"123abc".TakeWhile(char.IsDigit)`	`'1', '2', '3'`
`Chunk`	verdeel in stukken van n tekens	`"abcdefg".Chunk(3)`	`[a, b, c], [d, e, f], [g]`
`Count`	hoeveel tekens voldoen	`"abc123".Count(char.IsDigit)`	`3`
`Count`	hoe vaak komt een teken voor	`"banaan".Count(c => c == 'a')`	`3`
`Sum`	tel de cijfers op	`"1234".Sum(c => c - '0')`	`10`
`Min`, `Max`	het kleinste of grootste teken	`"kiwi".Max()`	`'w'`
`CountBy`	tel per teken	`"banaan".CountBy(c => c)`	`b 1, a 3, n 2`
`GroupBy`	groepeer de tekens	`"banaan".GroupBy(c => c)`	b: 1, a: 3, n: 2

> **LET OP**
>
> `"1234".Sum(c => c)` geeft niet 10 maar de som van de **codes** van de tekens. Trek er `'0'` af, zoals in de tabel.

Ordenen en combineren

Komt van LINQ.

Methode	Wat het doet	Voorbeeld	Resultaat
`Reverse`	de tekens in omgekeerde volgorde	`string.Concat("kiwi".Reverse())`	`"iwik"`
`Order`, `OrderDescending`	de tekens gesorteerd	`string.Concat("kiwi".Order())`	`"iikw"`
`Concat`, `Union`, `Intersect`, `Except`	combineer de tekens van twee teksten	`string.Concat("appel".Intersect("peer"))`	`"pe"`: de letters die in beide zitten
`Append`, `Prepend`	één teken achteraan of vooraan	`string.Concat("kiwi".Append('!'))`	`"kiwi!"`
`Zip`	koppel de tekens van twee teksten	`"abc".Zip("123")`	`(a, 1), (b, 2), (c, 3)`

Tekens sorteren gebeurt op hun **code**: alle hoofdletters komen vóór alle kleine letters. `"bAaB".Order()` geeft A, B, a, b. Teksten in een lijst sorteren gebeurt wél zoals in een woordenboek: zie *Let op* hieronder.

Tekens vastleggen en doorlopen

Komt van LINQ, de taal zelf en de andere verzamelingen.

Schrijfwijze	Wat het doet	Resultaat
`"kiwi".ToCharArray()`	een array van de tekens (eigen methode)	`'k', 'i', 'w', 'i'`
`"kiwi".ToList()`, `"kiwi".ToArray()`	een lijst of array van de tekens (LINQ)	4 tekens
`"banaan".ToHashSet()`, `new HashSet<char>("banaan")`	de verschillende tekens	`'b', 'a', 'n'`
`new string(tekens)`	van een array tekens weer een tekst maken	`new string(new[] { 'k', 'i' })` is `"ki"`
`string.Concat(reeks)`	van een reeks tekens weer een tekst maken	`string.Concat("kiwi".Reverse())` is `"iwik"`
`foreach (char c in "kiwi")`	elk teken één keer	k, i, w, i
`for (int i = 0; i < tekst.Length; i++)`	elke positie, met `tekst[i]`	0, 1, 2, 3
`switch (fruit) { case "kiwi": ... }`	kiezen op een vaste tekst	zie hoofdstuk 4

> **BELANGRIJK**
>
> `tekst[0] = 'K';` compileert niet: een tekst kan je niet veranderen, ook niet één teken. Maak een nieuwe tekst, of gebruik een `StringBuilder`, waar `sb[0] = 'K'` wel mag.

Wat elk object kan

Komt van `object`. Zie hoofdstuk 13.

Methode	Wat het doet	Voorbeeld	Resultaat
`ToString`	de tekst zelf	`"kiwi".ToString()`	`"kiwi"`
`Equals` / `==`	**dezelfde inhoud**	`"kiwi" == new string(new[] { 'k', 'i', 'w', 'i' })`	`true`
`GetType`	welk type	`"kiwi".GetType().Name`	`String`
`GetHashCode`	een getal per tekst; gelijke teksten geven hetzelfde getal	`"kiwi".GetHashCode()`	een getal

Een tekst is de grote uitzondering: `==` vergelijkt bij een `string` de **inhoud**, terwijl het bij een lijst, een array of een gewone klasse vraagt of het om **hetzelfde object** gaat. Pas op: staat een van beide als `object` getypeerd, dan vergelijkt `==` weer op referentie.

Bestaat ook, maar kom je in deze cursus niet tegen

`Normalize`, `IsNormalized`, `EnumerateRunes`, `GetPinnableReference`, `AsSpan`, `CopyTo` (tekens naar een array), `ReplaceLineEndings`, `string.Create`, `string.Intern`, `CompareOrdinal`, `char.IsSymbol`, `char.IsNumber`, `char.IsControl`, `TextInfo.ToTitleCase` (elk woord met een hoofdletter), reguliere expressies (`Regex`), en uit LINQ `AsParallel` en `Cast`.

Let op bij tekst

Op de tekst zelf	Via LINQ	Het verschil
`Length`	`Count()`	Hetzelfde getal. `Length` zonder haakjes is meteen bekend; `Count()` telt. Een voorwaarde kan enkel met LINQ: `Count(char.IsDigit)`.
`Contains("650")`	`Contains('6')`	De tekst zoekt een stuk tekst of een teken. LINQ zoekt enkel één teken.
`[0]`	`First()`	Hetzelfde teken. Op een lege tekst crashen ze allebei, met een andere uitzondering.
`[1..3]`	`Skip(1).Take(2)`	`[1..3]` geeft meteen een **tekst**, en crasht buiten de tekst. LINQ geeft een reeks tekens en crasht nooit.
`ToUpper()`	`Select(char.ToUpper)`	`ToUpper` geeft een tekst. `Select` geeft een reeks tekens.
`ToCharArray()`	`ToArray()`	Hetzelfde resultaat: een `char[]`.

LET OP

- Elke methode geeft een **nieuwe** tekst. `naam.Trim();` op een regel alleen doet niets.
- `string.Join("-", "kiwi")` geeft `"kiwi"`, niet `"k-i-w-i"`: C# ziet de tekst als één waarde. Wil je de tekens scheiden, geef dan de tekens mee: `string.Join("-", "kiwi".ToCharArray())`.
- `Console.WriteLine("kiwi".Reverse())` toont geen omgekeerde tekst maar de naam van een type. Zet er `string.Concat(...)` rond.
- `"appel" == "Appel"` is `false`. Vergelijk je zonder hoofdletters, gebruik dan `StringComparison.OrdinalIgnoreCase`, niet twee keer `ToUpper()`.
- Teksten in een lijst sorteren (`Order`, `Sort`) gaat zoals in een woordenboek: `appel, Banaan, citroen`. Met `StringComparer.Ordinal` krijg je de volgorde van de codes: `Banaan, appel, citroen`. Losse **tekens** sorteren altijd op hun code.
- `ToUpper` en `ToLower` volgen de taal van de computer. Op een Turkse computer wordt `"i".ToUpper()` een İ met een punt. Voor codes en sleutels neem je `ToUpperInvariant`.
- Sommige tekens tellen voor twee: een emoji heeft `Length` 2.

Voorspel de uitkomst

Vraag 1

```
string s = "  Gent  ";
s.Trim();
Console.WriteLine("[" + s + "]");
```

Toon het antwoord ›

Vraag 2

```
string code = "ABC-123";
Console.WriteLine(code.Substring(code.IndexOf("-") + 1));
```

Toon het antwoord ›

Vraag 3

```
Console.WriteLine("appel" == "Appel");
Console.WriteLine("appel".Equals("Appel", StringComparison.OrdinalIgnoreCase));
```

Toon het antwoord ›

Vraag 4

```
string[] delen = "rood;groen;;blauw".Split(';');
Console.WriteLine(delen.Length);
```

Toon het antwoord ›

Vraag 5

```
string s = "kiwi";
var omgekeerd = s.Reverse();
Console.WriteLine(s);
Console.WriteLine(string.Concat(omgekeerd));
```

Toon het antwoord ›

Vraag 6

```
string code = "A1B2C3";
Console.WriteLine(code.Count(char.IsDigit));
Console.WriteLine(code.Any(char.IsLower));
```

Toon het antwoord ›

Vraag 7

```
Console.WriteLine('7' + 1);
Console.WriteLine('7' - '0' + 1);
```

Toon het antwoord ›

Vraag 8

```
string[] delen = " a , , b ".Split(',', StringSplitOptions.RemoveEmptyEntries);
Console.WriteLine(delen.Length);
```

Toon het antwoord ›

Vraag 9

```
Console.WriteLine(string.Join("-", "kiwi"));
Console.WriteLine(string.Join("-", "kiwi".ToCharArray()));
```

Toon het antwoord ›

Vraag 10

```
string s = "Gent";
Console.WriteLine(s[1..3]);
Console.WriteLine(s[^1]);
Console.WriteLine(s[1..9]);
```

Toon het antwoord ›

Antwoorden

VRAAG 1

`[ Gent ]`

`Trim` verandert de tekst niet. Het geeft een nieuwe tekst terug, en die werd hier nergens opgevangen. Wil je dat het werkt, dan schrijf je `s = s.Trim();`.

‹ Terug naar de vraag

VRAAG 2

`123`

Het streepje staat op positie 3. Eén verder is positie 4, en vanaf daar neemt `Substring` alles tot het einde. Dit is het patroon om alles ná een scheidingsteken te krijgen.

‹ Terug naar de vraag

VRAAG 3

`False` en daarna `True`

`==` let op hoofdletters. Met `OrdinalIgnoreCase` zeg je uitdrukkelijk dat het niet mag.

‹ Terug naar de vraag

VRAAG 4

`4`

Tussen de twee puntkomma's staat niets, en dat niets telt ook mee als stuk: een lege tekst. De vier stukken zijn `rood`, `groen`, een lege tekst en `blauw`.

‹ Terug naar de vraag

VRAAG 5

`kiwi` en daarna `iwik`

`Reverse` is LINQ: de tekst zelf verandert niet, en het resultaat is een reeks tekens. `string.Concat` maakt daar weer een tekst van.

‹ Terug naar de vraag

VRAAG 6

`3` en daarna `False`

Drie tekens zijn een cijfer: 1, 2 en 3. Kleine letters zitten er niet in: A, B en C zijn hoofdletters.

‹ Terug naar de vraag

VRAAG 7

`56` en daarna `8`

Met een teken rekenen gebeurt met zijn code, en de code van `'7'` is 55. Trek je er de code van `'0'` (48) van af, dan hou je het cijfer 7 over.

‹ Terug naar de vraag

VRAAG 8

`3`

Het middelste stuk is een spatie, geen lege tekst, dus `RemoveEmptyEntries` laat het staan. Met `RemoveEmptyEntries | TrimEntries` wordt eerst de spatie weggehaald, en dan pas het lege stuk: dan blijven er 2.

‹ Terug naar de vraag

VRAAG 9

`kiwi` en daarna `k-i-w-i`

Een tekst alleen is voor `string.Join` één waarde: er valt niets te scheiden. Een array van tekens zijn vier waarden.

‹ Terug naar de vraag

VRAAG 10

`en`, daarna `t`, en dan **`ArgumentOutOfRangeException`** — het programma crasht.

`1..3` neemt de posities 1 en 2. `^1` is het laatste teken. `1..9` gaat voorbij het einde van de tekst, en daar crasht het bereik.

‹ Terug naar de vraag

2. Getallen en omzetten

METHODES

Tekst omzetten naar een getal

Methode	Wat het doet	Voorbeeld	Resultaat
`int.Parse`	zet om, of loopt stuk	`int.Parse("120")`	`120`
`int.Parse`	bij onzin	`int.Parse("abc")`	uitzondering
`int.TryParse`	probeert om te zetten, zegt of het lukte	`int.TryParse("120", out int n)`	`true`, `n` is `120`
`int.TryParse`	bij onzin	`int.TryParse("abc", out int n)`	`false`, `n` is `0`
`double.Parse`	hetzelfde voor kommagetallen	`double.Parse("2.5")`	hangt af van je taalinstelling — zie hieronder
`decimal.Parse`, `decimal.TryParse`	hetzelfde voor een `decimal`	`decimal.TryParse("19,99", out decimal p)`	`true` op een Belgische computer
`Parse` met cultuur	lees altijd met een punt als decimaalteken	`double.Parse("2.5", CultureInfo.InvariantCulture)`	`2.5`, op elke computer
`Convert.ToInt32`	zet om; bij `null` geeft het 0 in plaats van te crashen	`Convert.ToInt32("12")`	`12`

`Parse` en `TryParse` aanvaarden spaties rond het getal (`" 42 "` lukt), maar geen letters en, zonder extra instelling, geen scheidingsteken voor duizendtallen: `int.Parse("1.000")` geeft een `FormatException`.

> **BELANGRIJK**
>
> De regel is eenvoudig: komt de tekst van een gebruiker, gebruik `TryParse`. Komt ze uit je eigen code en weet je zeker dat het een getal is, dan mag `Parse`. Een `Parse` op gebruikersinvoer is een crash die op je zit te wachten.

> **LET OP**
>
> `double.Parse` leest een kommagetal volgens de taalinstelling van je computer. Op een Belgische computer is de komma het decimaalteken en de punt het teken voor duizendtallen, zoals in 1.000. `double.Parse("2.5")` geeft daar `25` — zonder foutmelding. Komt je getal uit een bestand of een API, waar altijd een punt staat, geef dan `CultureInfo.InvariantCulture` mee: `double.Parse("2.5", CultureInfo.InvariantCulture)`.
>
> Omgekeerd toont `Console.WriteLine(2.5)` op jouw computer `2,5`. In dit leerboek staan kommagetallen in de schrijfwijze van C# zelf, met een punt.

Een getal als tekst tonen

Methode	Wat het doet	Voorbeeld	Resultaat
`ToString`	het getal als tekst	`120.ToString()`	`"120"`
`ToString("0.00")`	altijd twee cijfers na de komma	`19.5m.ToString("0.00")`	`"19,50"` op een Belgische computer
`ToString("N2")`	met scheiding voor duizendtallen	`1234.5m.ToString("N2")`	`"1.234,50"`
`ToString("C")`	als bedrag, met munteenheid	`19.5m.ToString("C")`	`"€ 19,50"`
`ToString("P0")`	als percentage	`0.25.ToString("P0")`	`"25%"`
`ToString("D3")`	geheel getal, aangevuld met nullen	`7.ToString("D3")`	`"007"`
`ToString` met cultuur	met een punt, los van de computer	`2.5.ToString(CultureInfo.InvariantCulture)`	`"2.5"`

Dezelfde opmaak werkt in een tekst met `$`: `$"{prijs:0.00} euro"`. Zie hoofdstuk 1.

Rekenen

Methode	Wat het doet	Voorbeeld	Resultaat
`Math.Max`	de grootste van twee	`Math.Max(3, 7)`	`7`
`Math.Min`	de kleinste van twee	`Math.Min(3, 7)`	`3`
`Math.Abs`	zonder minteken	`Math.Abs(-5)`	`5`
`Math.Round`	afronden	`Math.Round(2.4)`	`2`
`Math.Ceiling`	altijd naar boven	`Math.Ceiling(2.1)`	`3`
`Math.Floor`	altijd naar beneden	`Math.Floor(2.9)`	`2`
`Math.Pow`	tot de macht	`Math.Pow(2, 3)`	`8`
`/`	delen van gehele getallen	`7 / 2`	`3`
`%`	de rest na het delen	`7 % 2`	`1`
`Math.Round`	afronden op een aantal cijfers	`Math.Round(19.987m, 2)`	`19.99`
`Math.Round`	schools afronden	`Math.Round(2.5, MidpointRounding.AwayFromZero)`	`3`
`Math.Truncate`	knip alles na de komma weg	`Math.Truncate(-2.7)`	`-2`
`Math.Sqrt`	vierkantswortel	`Math.Sqrt(16)`	`4`
`Math.Clamp`	hou binnen een minimum en maximum	`Math.Clamp(15, 0, 10)`	`10`
`Math.Sign`	negatief, nul of positief	`Math.Sign(-3)`	`-1`
`++`, `--`, `+=`	één erbij, één eraf, iets erbij	`teller += 5`	`teller` is 5 groter

`Math.Floor(-2.7)` is `-3` en `Math.Truncate(-2.7)` is `-2`: `Floor` gaat altijd naar beneden, `Truncate` knipt gewoon af. Bij positieve getallen is er geen verschil.

`Math.Pow` en `Math.Sqrt` geven altijd een `double` terug, ook bij gehele getallen.

> **LET OP**
>
> Twee dingen die bijna iedereen één keer verkeerd doet. `7 / 2` geeft `3` en niet `3.5`, want twee gehele getallen delen geeft een geheel getal. En `Math.Round(2.5)` geeft `2`, niet `3` — C# rondt een halve eenheid naar het dichtstbijzijnde even getal. Wil je het schoolse afronden, dan geef je `MidpointRounding.AwayFromZero` mee.

Het deelteken en het restteken samen zijn precies wat je nodig hebt om iets te verdelen: hoeveel per groep, en hoeveel blijven er over.

Grenzen, en omzetten tussen soorten getallen

Schrijfwijze	Wat het doet	Voorbeeld	Resultaat
`int.MaxValue`, **`int.MinValue`**	het grootste en kleinste `int`	`int.MaxValue`	`2147483647`
`long`	een geheel getal voor grotere waarden	`long aantal = 3_000_000_000;`	compileert; als `int` niet
`(int)`	een kommagetal omzetten naar een geheel getal	`(int)2.9`	`2`: alles na de komma valt weg
`Convert.ToInt32`	idem, maar **afronden**	`Convert.ToInt32(2.9)`	`3`; `Convert.ToInt32(2.5)` is `2`
`(double)`	een geheel getal als kommagetal	`10 / (double)4`	`2.5`
`checked`	crash bij overlopen in plaats van stil verder te tellen	`checked(int.MaxValue + teller)`	**`OverflowException`**

> **LET OP**
>
> - Een `int` die over zijn maximum gaat, crasht niet maar springt naar het **minimum**: `int.MaxValue + 1` met een variabele wordt `-2147483648`.
> - Delen door nul: met gehele getallen en `decimal` crasht het met een **`DivideByZeroException`**. Met een `double` niet: `1.0 / 0` is `Infinity`, en `0.0 / 0` is `NaN` ("geen getal").
> - `(double)(10 / 4)` is `2`: eerst wordt geheel gedeeld, pas daarna omgezet. Zet om **vóór** je deelt.

Kommagetallen: double tegenover decimal

C# heeft twee soorten kommagetallen die je vaak tegenkomt. Een `double` is snel, maar bewaart sommige getallen net niet exact. Een `decimal` is trager, maar rekent tot op de cent juist. Vanaf les 06 staat een prijs in de cursus als `decimal`.

Vorm	Wat het doet	Voorbeeld	Resultaat
`double`	snel kommagetal, kan een heel klein beetje afwijken	`0.1 + 0.2 == 0.3`	`false`
`decimal`	exact kommagetal, voor geld en prijzen	`0.1m + 0.2m == 0.3m`	`true`
`m` achter een getal	maakt er een `decimal` van	`decimal prijs = 19.99m;`	compileert
zonder `m`	een kommagetal zonder letter is een `double`	`decimal prijs = 19.99;`	**`compileert niet`**
`(double)`	zet een `decimal` om naar een `double`	`(double)19.99m`	`19.99`
`Math.Round`	afronden op een aantal cijfers na de komma	`Math.Round(19.987m, 2)`	`19.99`

> **LET OP**
>
> Een `decimal` en een `double` mag je niet zomaar door elkaar gebruiken. `prijs * 1.21` compileert niet als `prijs` een `decimal` is, want `1.21` is een `double`. Schrijf `prijs * 1.21m`.

Een willekeurig getal

Methode	Wat het doet	Voorbeeld	Resultaat
`Random.Shared.Next`	een geheel getal vanaf het eerste, tot vóór het tweede	`Random.Shared.Next(1, 7)`	een getal van 1 tot en met 6, zoals een dobbelsteen
`Random.Shared.NextDouble`	een kommagetal van 0 tot vóór 1	`Random.Shared.NextDouble()`	bv. `0.7314`

Het tweede getal bij `Next` doet zelf niet mee: `Next(1, 7)` geeft nooit 7.

Wat het ook kan, via iets anders

Een getal is een **waarde**, geen object met methodes zoals een lijst: `int`, `double` en `decimal` zijn `struct`s (hoofdstuk 15). Wat je ermee kan, komt van drie plaatsen: de rekentekens, de klasse `Math`, en wat elk object kan. Zodra je met **meerdere** getallen werkt, komt LINQ erbij.

Bron	Wat het geeft	Volledige uitleg
LINQ op een reeks getallen	`Sum`, `Average`, `Min`, `Max`, `Order`, `Distinct`, …	hoofdstuk 11
`Enumerable.Range`	een reeks opeenvolgende getallen	hoofdstuk 11
vergelijken	`<`, `>`, `==`, `CompareTo`	hoofdstuk 4
tekst met `$`	getallen opmaken in een tekst	hoofdstuk 1
wat elk object kan	`ToString`, `Equals`, `GetType`	hoofdstuk 13

Rekenen met een reeks getallen

Komt van LINQ. In de voorbeelden is `getallen` de lijst `5, 8, 3, 8, 1`.

Methode	Wat het doet	Voorbeeld	Resultaat
`Sum`	tel alles op	`getallen.Sum()`	`25`
`Average`	het gemiddelde, altijd als `double`	`getallen.Average()`	`5`
`Min`, **`Max`**	kleinste en grootste	`getallen.Max()`	`8`
`Count`	hoeveel voldoen	`getallen.Count(g => g % 2 == 0)`	`2`: de even getallen
`Order`, **`OrderDescending`**	gesorteerd	`getallen.Order()`	`1, 3, 5, 8, 8`
`Distinct`	elk getal één keer	`getallen.Distinct()`	`5, 8, 3, 1`
`Enumerable.Range`	een reeks maken	`Enumerable.Range(1, 10).Sum()`	`55`

Het gemiddelde van gehele getallen is een `double`: `new List<int> { 1, 2 }.Average()` geeft `1.5`, ook al zijn het gehele getallen. Bij een lijst `decimal` is het gemiddelde een `decimal`.

Vergelijken en wat elk object kan

Methode	Wat het doet	Voorbeeld	Resultaat
`CompareTo`	welke is kleiner	`3.CompareTo(7)`	`-1`: negatief, want 3 is kleiner
`Equals` / **`==`**	dezelfde waarde	`3 == 3`	`true`
`GetType`	welk type	`3.GetType().Name`	`Int32`; `2.5` is `Double`, `2.5m` is `Decimal`
`ToString`	als tekst	`3.ToString()`	`"3"`

Bij getallen vergelijkt `==` de **waarde**: twee getallen zijn gelijk als ze even groot zijn. Dat is anders dan bij een lijst of een object van een klasse.

Bestaat ook, maar kom je in deze cursus niet tegen

`float` (een minder nauwkeurige `double`), `short`, `byte`, `uint`, `ulong`, `BigInteger` (willekeurig grote getallen), `Math.Log`, `Math.Sin` en hun familie, `Math.PI`, `int.IsEvenInteger`, `int.Clamp` en de andere statische methodes die sinds .NET 7 rechtstreeks op `int`, `double` en `decimal` zitten.

Let op bij getallen

LET OP

- Twee gehele getallen delen geeft een geheel getal: `7 / 2` is `3`. Ook `-7 / 2` wordt naar nul toe afgekapt: `-3`, en `-7 % 2` is `-1`.
- `Math.Round` en `Convert.ToInt32` ronden een halve eenheid af naar het **even** getal: `Math.Round(2.5)` is `2`. `(int)` rondt helemaal niet af, maar knipt.
- `Parse` en `ToString` volgen de taal van de computer. Op een Belgische computer is de komma het decimaalteken: `double.Parse("2.5")` geeft `25`, en `2.5.ToString()` geeft `"2,5"`.
- `decimal` voor geld, `double` voor metingen. Ze door elkaar gebruiken compileert niet zonder omzetting.
- `getallen.Sum()` op een lijst `int` die samen boven `int.MaxValue` uitkomt, crasht met een `OverflowException`. Gewone optelling met `+` springt stil naar het minimum.

Voorspel de uitkomst

Vraag 1

```
Console.WriteLine(7 / 2);
Console.WriteLine(7 % 2);
```

Toon het antwoord ›

Vraag 2

```
bool gelukt = int.TryParse("12a", out int getal);
Console.WriteLine(gelukt);
Console.WriteLine(getal);
```

Toon het antwoord ›

Vraag 3

```
Console.WriteLine(Math.Round(3.5));
Console.WriteLine(Math.Round(4.5));
```

Toon het antwoord ›

Vraag 4

Op een computer met Belgische taalinstelling:

```
double d = double.Parse("2.5");
Console.WriteLine(d);
```

Toon het antwoord ›

Vraag 5

```
Console.WriteLine(0.1 + 0.2 == 0.3);
Console.WriteLine(0.1m + 0.2m == 0.3m);
```

Toon het antwoord ›

Vraag 6

```
decimal prijs = 10m;
var metBtw = prijs * 1.21;
```

Toon het antwoord ›

Vraag 7

```
Console.WriteLine((int)2.9);
Console.WriteLine(Convert.ToInt32(2.9));
```

Toon het antwoord ›

Vraag 8

```
int a = 10;
int b = 4;
Console.WriteLine((double)(a / b));
Console.WriteLine((double)a / b);
```

Toon het antwoord ›

Vraag 9

```
var getallen = new List<int> { 1, 2 };
Console.WriteLine(getallen.Average());
Console.WriteLine(getallen.Sum() / getallen.Count);
```

Toon het antwoord ›

Antwoorden

VRAAG 1

`3` en daarna `1`

Twee gehele getallen delen geeft een geheel getal: 2 past 3 keer in 7. Wat er overblijft, 1, krijg je met `%`.

‹ Terug naar de vraag

VRAAG 2

`False` en daarna `0`

`"12a"` is geen getal. `TryParse` crasht niet, geeft `false` terug, en zet `getal` op nul.

‹ Terug naar de vraag

VRAAG 3

`4` en daarna `4`

Geen tikfout. C# rondt een halve eenheid af naar het dichtstbijzijnde **even** getal: 3,5 wordt 4, en 4,5 wordt ook 4. Wil je dat 4,5 een 5 wordt, dan geef je `MidpointRounding.AwayFromZero` mee.

‹ Terug naar de vraag

VRAAG 4

`25`

Op een Belgische computer is de punt het teken voor duizendtallen, zoals in 1.000. `double.Parse` leest `"2.5"` dus als vijfentwintig, zonder foutmelding. Met `double.Parse("2.5", CultureInfo.InvariantCulture)` krijg je wel twee en een half. Zie de waarschuwing hierboven.

‹ Terug naar de vraag

VRAAG 5

`False` en daarna `True`

Een `double` bewaart 0,1 en 0,2 niet exact, en de som ligt een heel klein beetje naast 0,3. Een `decimal` rekent met tientallen, zoals jij op papier, en komt wel precies op 0,3 uit. Daarom is een prijs een `decimal`.

‹ Terug naar de vraag

VRAAG 6

`Compileert niet.`

`1.21` zonder letter is een `double`, en C# weigert een `decimal` met een `double` te vermenigvuldigen. Het zou niet weten welk van de twee soorten het resultaat moet worden. Met `1.21m` lukt het wel.

‹ Terug naar de vraag

VRAAG 7

`2` en daarna `3`

`(int)` knipt alles na de komma weg. `Convert.ToInt32` rondt af.

‹ Terug naar de vraag

VRAAG 8

`2` en daarna `2.5`

In de eerste regel wordt eerst geheel gedeeld: 10 / 4 is 2. Pas daarna wordt die 2 een `double`. In de tweede regel is `a` al een `double` vóór het delen. Op een Belgische computer toont de tweede regel `2,5`.

‹ Terug naar de vraag

VRAAG 9

`1.5` en daarna `1`

`Average` geeft altijd een `double`. Zelf delen met twee gehele getallen geeft een geheel getal.

‹ Terug naar de vraag

3. Datum en tijd

METHODES

> **OPMERKING**
> In les 06 en 09 krijgen je modellen een datum, zoals de dag waarop iets aangemaakt werd. Daarvoor gebruik je `DateTime`.

Een datum maken

Vorm	Wat het doet	Voorbeeld	Resultaat
`new DateTime(j, m, d)`	een vaste datum	`new DateTime(2026, 10, 3)`	3 oktober 2026, om middernacht
`new DateTime(j, m, d, u, min, s)`	een vaste datum met uur	`new DateTime(2026, 10, 3, 14, 30, 0)`	3 oktober 2026, 14.30 uur
`DateTime.Now`	nu, op de klok van de computer	`DateTime.Now`	het huidige moment
`DateTime.Today`	vandaag, om middernacht	`DateTime.Today`	de datum van vandaag, zonder uur
`DateTime.UtcNow`	nu, in wereldtijd	`DateTime.UtcNow`	in België één of twee uur vroeger dan `Now`
`DateOnly`	enkel een datum, zonder uur	`new DateOnly(2026, 10, 3)`	3 oktober 2026
`TimeOnly`	enkel een uur, zonder datum	`new TimeOnly(14, 30)`	14.30 uur
`DateTime.MinValue`, `DateTime.MaxValue`	de vroegste en laatste mogelijke datum	`DateTime.MinValue`	1 januari van het jaar 1
`DateTime.DaysInMonth`	hoeveel dagen heeft die maand	`DateTime.DaysInMonth(2026, 2)`	`28`
`DateTime.IsLeapYear`	is het een schrikkeljaar	`DateTime.IsLeapYear(2028)`	`true`

Maanden tellen hier vanaf 1, niet vanaf 0 zoals de posities in een array. Januari is 1, december 12.

Een stuk eruit halen

Vorm	Wat het doet	Voorbeeld	Resultaat
`Year`	het jaar	`d.Year`	`2026`
`Month`	de maand, van 1 tot 12	`d.Month`	`10`
`Day`	de dag van de maand	`d.Day`	`3`
`Hour`, `Minute`	het uur en de minuut	`d.Hour`	`14`
`DayOfWeek`	de dag van de week, als enum	`d.DayOfWeek`	`Saturday`
`Date`	dezelfde dag, met het uur op middernacht	`d.Date`	3 oktober 2026, 00.00 uur
`Second`, `Millisecond`	seconden en milliseconden	`d.Second`	`0`
`DayOfYear`	de hoeveelste dag van het jaar	`d.DayOfYear`	`276`
`TimeOfDay`	enkel het uur, als `TimeSpan`	`d.TimeOfDay`	`14:30:00`
`Kind`	lokale tijd, wereldtijd, of niet gezegd	`d.Kind`	`Unspecified`; `DateTime.Now` is `Local`, `DateTime.UtcNow` is `Utc`

`DayOfWeek` is een enum zoals in hoofdstuk 18, met Engelse namen: `Monday` tot en met `Sunday`. Dat blijft zo, ook op een Nederlandstalige computer.

Rekenen met datums

Vorm	Wat het doet	Voorbeeld	Resultaat
`AddDays`	een aantal dagen erbij, of eraf met een minteken	`d.AddDays(7)`	een week later
`AddMonths`	een aantal maanden erbij	`d.AddMonths(1)`	een maand later
`AddHours`	een aantal uren erbij	`d.AddHours(-2)`	twee uur vroeger
`-`	het verschil tussen twee datums	`eind - begin`	een `TimeSpan`
`Days`	het aantal hele dagen in een `TimeSpan`	`(eind - begin).Days`	`14`
`TotalHours`	het verschil in uren, met komma	`(eind - begin).TotalHours`	`336`
`<`, `>`, `==`	vergelijken: welke komt eerst	`begin < eind`	`true`
`AddYears`, `AddMinutes`, `AddSeconds`	jaren, minuten, seconden erbij	`d.AddMinutes(45)`	15.15 uur
`Add`	een `TimeSpan` erbij	`d.Add(TimeSpan.FromHours(2))`	16.30 uur
`Subtract`	hetzelfde als `-`	`d.Subtract(begin)`	een `TimeSpan`
`CompareTo`	negatief, 0 of positief, zoals bij tekst	`d.CompareTo(new DateTime(2026, 1, 1))`	`1`: d komt later

Net als bij tekst veranderen deze methodes de datum zelf niet. `d.AddDays(1)` op een regel alleen doet niets; je moet het resultaat opvangen.

Een tijdsduur: TimeSpan

Het verschil tussen twee datums is een `TimeSpan`: een duur, geen moment. In de voorbeelden is `duur` het verschil tussen 1 oktober 0.00 uur en 15 oktober 6.00 uur.

Vorm	Wat het doet	Voorbeeld	Resultaat
`Days`, `Hours`, `Minutes`	de **delen**: hele dagen, dan de uren die overblijven ...	`duur.Days`, `duur.Hours`	`14` en `6`
`TotalDays`, `TotalHours`, `TotalMinutes`	de **hele** duur in één eenheid, met komma	`duur.TotalHours`	`342`
`TimeSpan.FromMinutes`, `FromHours`, `FromDays`	een duur maken	`TimeSpan.FromMinutes(90)`	`01:30:00`
`new TimeSpan`	een duur uit uren, minuten, seconden	`new TimeSpan(1, 30, 0).TotalMinutes`	`90`
`TimeSpan.Zero`	een duur van niets	`TimeSpan.Zero`	`00:00:00`
`Duration`	zonder minteken	`(vroeg - laat).Duration()`	een positieve duur
`<`, `>`, `+`, `-`	vergelijken en optellen	`TimeSpan.FromHours(1) < TimeSpan.FromMinutes(90)`	`true`

> **LET OP**
>
> `Hours` is niet hetzelfde als `TotalHours`. Bij 14 dagen en 6 uur is `Hours` `6` — enkel de uren bovenop de hele dagen — en `TotalHours` `342`. En een verschil met de latere datum eerst is negatief: `(1 januari - 3 januari).Days` is `-2`.

Enkel een datum of enkel een uur: DateOnly en TimeOnly

Vorm	Wat het doet	Voorbeeld	Resultaat
`DateOnly.FromDateTime`	het datumdeel van een `DateTime`	`DateOnly.FromDateTime(d)`	3 oktober 2026
`AddDays`, `AddMonths`, `AddYears`	zoals bij `DateTime`	`new DateOnly(2026, 10, 3).AddDays(30)`	2 november 2026
`DayNumber`	dagnummer; zo reken je het verschil in dagen	`eind.DayNumber - begin.DayNumber`	het aantal dagen ertussen
`ToDateTime`	maak er weer een `DateTime` van, met een uur	`dag.ToDateTime(new TimeOnly(9, 0))`	3 oktober 2026, 9.00 uur
`TimeOnly.AddMinutes`, `AddHours`	rekenen met een uur; na middernacht begint het opnieuw	`new TimeOnly(14, 30).AddHours(10)`	`00:30`
`IsBetween`	ligt dit uur tussen twee andere	`uur.IsBetween(new TimeOnly(9, 0), new TimeOnly(17, 0))`	`true` voor 14.30

Twee `DateOnly`'s van elkaar aftrekken met `-` kan niet. Gebruik `DayNumber`.

Tonen en inlezen

Vorm	Wat het doet	Voorbeeld	Resultaat
`ToString("yyyy-MM-dd")`	toon in een vaste vorm	`d.ToString("yyyy-MM-dd")`	`"2026-10-03"`
`ToString("HH:mm")`	enkel het uur	`d.ToString("HH:mm")`	`"14:30"`
`DateTime.TryParse`	probeer een tekst om te zetten, volgens de taalinstelling	`DateTime.TryParse("03/10/2026", out DateTime d)`	op een Belgische computer: 3 oktober
`DateTime.ParseExact`	zet om volgens een vorm die jij opgeeft	`DateTime.ParseExact("2026-10-03", "yyyy-MM-dd", CultureInfo.InvariantCulture)`	3 oktober 2026
`ToString("dddd")`	de naam van de dag, in de taal van de computer	`d.ToString("dddd")`	`"zaterdag"` op een Nederlandstalige computer
`ToString("d MMMM yyyy")`	met de naam van de maand	`d.ToString("d MMMM yyyy")`	`"3 oktober 2026"`
`ToString("d")`, `ToShortDateString`	de korte datum van de taalinstelling	`d.ToString("d")`	`"3/10/2026"` op een Belgische computer
`DateTime.TryParseExact`	probeer om te zetten volgens jouw vorm, zonder te crashen	`DateTime.TryParseExact("03-10-2026", "dd-MM-yyyy", CultureInfo.InvariantCulture, DateTimeStyles.None, out DateTime d)`	`true`

LET OP

In een opmaak betekent `MM` maand en `mm` minuten. `HH` is het uur van 0 tot 23, `hh` het uur van 1 tot 12. Eén verkeerde hoofdletter geeft een datum die er geloofwaardig uitziet en toch fout is.

En net als bij `double.Parse` in hoofdstuk 2 leest `DateTime.TryParse` volgens de taalinstelling. `"03/10/2026"` is in België 3 oktober, in de Verenigde Staten 10 maart. Ken je de vorm, gebruik dan `ParseExact`.

Wat het ook kan, via iets anders

Een `DateTime` is, net als een getal, een **waarde** (`struct`, hoofdstuk 15). Twee datums met hetzelfde moment zijn gelijk. Wat er via iets anders bij komt:

Bron	Wat het geeft	Volledige uitleg
LINQ op een reeks datums	`Min`, `Max`, `Order`, `Where`, `GroupBy`	hoofdstuk 11
`DayOfWeek` is een enum	vergelijken, `switch`, omzetten naar een getal	hoofdstuk 18
vergelijken	`<`, `>`, `==`, `CompareTo`	hoofdstuk 4
tekst met `$`	`$"{d:dd/MM/yyyy}"`	hoofdstuk 1
wat elk object kan	`ToString`, `Equals`, `GetType`	hoofdstuk 13

Rekenen met een reeks datums

Komt van LINQ. In de voorbeelden is `datums` de lijst 1 maart 2026, 24 december 2025 en 3 oktober 2026.

Methode	Wat het doet	Voorbeeld	Resultaat
`Max`, `Min`	de laatste of vroegste datum	`datums.Max()`	3 oktober 2026
`Order`, `OrderDescending`	gesorteerd, van vroeg naar laat	`datums.Order()`	24 december 2025, 1 maart 2026, 3 oktober 2026
`Count`, `Where`	datums die voldoen	`datums.Count(d => d.Year == 2026)`	2
`GroupBy`	groeperen per jaar, maand of dag	`datums.GroupBy(d => d.Year)`	2026: 2 datums, 2025: 1
`Contains`	zit dit moment erin	`datums.Contains(new DateTime(2026, 3, 1))`	`true`
`MaxBy`, `OrderBy`	met objecten: op hun datum	`bestellingen.OrderBy(b => b.Datum)`	oudste bestelling eerst

> **LET OP**
>
> `Sum` en `Average` bestaan niet voor datums: `datums.Average()` compileert niet. Een reeks `TimeSpan` optellen doe je met `duren.Aggregate(TimeSpan.Zero, (a, b) => a + b)`.

Vergelijken en wat elk object kan

Methode	Voorbeeld	Resultaat
`==`	`new DateTime(2026, 10, 3) == new DateTime(2026, 10, 3, 14, 30, 0)`	`false`: middernacht is niet 14.30 uur
`Date` + `==`	`d.Date == new DateTime(2026, 10, 3)`	`true`: enkel de dag vergeleken
`GetType`	`d.GetType().Name`	`DateTime`
`ToString`	`d.ToString()`	volgens de taalinstelling, bv. `3/10/2026 14:30:00`
`(int)` op `DayOfWeek`	`(int)d.DayOfWeek`	`6`; zondag is `0`, niet 7

Bestaat ook, maar kom je in deze cursus niet tegen

`DateTimeOffset` (een moment mét tijdzone), `TimeZoneInfo` (omrekenen tussen tijdzones), `Calendar` en `ISOWeek` (weeknummers), `DateTime.SpecifyKind`, `ToLocalTime`, `ToUniversalTime`.

Let op bij datums

> **LET OP**
>
> - Een `DateTime` die je niet invult, is niet leeg maar 1 januari van het jaar 1. Ook `FirstOrDefault` op een lege lijst datums geeft die datum, geen `null`. Wil je dat een datum kan ontbreken, gebruik dan `DateTime?` (hoofdstuk 17).
> - Zondag is dag `0` van `DayOfWeek`, niet 7. Wie op maandag als eerste dag rekent, moet daar rekening mee houden.
> - `DateTime.Now` is de lokale tijd van de computer; `DateTime.UtcNow` de wereldtijd. Een PostgreSQL-database met EF Core aanvaardt in een kolom met tijdzone enkel wereldtijd (`Kind` is `Utc`): een `DateTime.Now` bewaren daar crasht. Gebruik `DateTime.UtcNow`, of zet de kolom op een datum zonder tijdzone. Zie hoofdstuk 31.
> - `==` vergelijkt tot op de tick: twee keer `DateTime.Now` na elkaar is bijna nooit gelijk. Vergelijk met `.Date` als je enkel de dag bedoelt.

Voorspel de uitkomst

Vraag 1

```
var d = new DateTime(2026, 1, 31);
Console.WriteLine(d.AddMonths(1).Day);
```

Toon het antwoord ›

Vraag 2

```
var begin = new DateTime(2026, 10, 1);
var eind = new DateTime(2026, 10, 15);
Console.WriteLine((eind - begin).Days);
```

Toon het antwoord ›

Vraag 3

```
var d = new DateTime(2026, 10, 3);
d.AddDays(1);
Console.WriteLine(d.Day);
```

Toon het antwoord ›

Vraag 4

```
var d = new DateTime(2026, 10, 3, 14, 5, 0);
Console.WriteLine(d.ToString("yyyy-mm-dd"));
```

Toon het antwoord ›

Vraag 5

```
var d = new DateTime(2026, 2, 29);
```

Toon het antwoord ›

Vraag 6

```
var begin = new DateTime(2026, 10, 1);
var eind = new DateTime(2026, 10, 15, 6, 0, 0);
var duur = eind - begin;
Console.WriteLine(duur.Hours);
Console.WriteLine(duur.TotalHours);
```

Toon het antwoord ›

Vraag 7

```
var datums = new List<DateTime>();
var eerste = datums.FirstOrDefault();
Console.WriteLine(eerste.Year);
```

Toon het antwoord ›

Vraag 8

```
var d = new DateTime(2026, 10, 4);   // een zondag
Console.WriteLine((int)d.DayOfWeek);
```

Toon het antwoord ›

Antwoorden

VRAAG 1

`28`

Februari 2026 heeft geen 31ste. `AddMonths` blijft dan op de laatste dag van die maand staan, in plaats van door te schuiven naar maart.

‹ Terug naar de vraag

VRAAG 2

`14`

Van de eerste tot de vijftiende zijn het veertien dagen. Het verschil tussen twee datums is een `TimeSpan`, en `Days` geeft daarvan het aantal hele dagen.

‹ Terug naar de vraag

VRAAG 3

`3`

`AddDays` geeft een nieuwe datum terug en laat `d` met rust, net zoals `Trim` bij tekst. Wil je dat het werkt, dan schrijf je `d = d.AddDays(1);`.

‹ Terug naar de vraag

VRAAG 4

`2026-05-03`

`mm` met kleine letters zijn de minuten, en dat is hier 5. Voor de maand had er `MM` moeten staan.

‹ Terug naar de vraag

VRAAG 5

`ArgumentOutOfRangeException` — het programma crasht.

2026 is geen schrikkeljaar, dus 29 februari bestaat niet. Het compileert wel, want de compiler rekent geen kalenders na. De fout komt pas als de regel draait.

‹ Terug naar de vraag

VRAAG 6

`6` en daarna `342`

`Hours` geeft enkel de uren bovenop de 14 hele dagen. `TotalHours` zet de hele duur om naar uren: 14 maal 24 plus 6.

‹ Terug naar de vraag

VRAAG 7

`1`

Een `DateTime` kan niet `null` zijn. `FirstOrDefault` op een lege lijst geeft de standaardwaarde: 1 januari van het jaar 1.

‹ Terug naar de vraag

VRAAG 8

`0`

In `DayOfWeek` is zondag de eerste dag, met nummer 0. Maandag is 1, zaterdag 6.

‹ Terug naar de vraag

4. Beslissen en herhalen

METHODES

> **OPMERKING**
> Ken je dit al uit een andere taal, lees dan vooral de tabellen en de waarschuwingen. Een paar dingen werken in C# net iets anders, zoals de switch-expressie.

Vergelijken en combineren

Vorm	Wat het doet	Voorbeeld	Resultaat
`==`	is gelijk aan	`5 == 5`	`true`
`!=`	is niet gelijk aan	`5 != 3`	`true`
`<`, `<=`	kleiner dan, kleiner dan of gelijk aan	`3 <= 3`	`true`
`>`, `>=`	groter dan, groter dan of gelijk aan	`2 > 3`	`false`
`&&`	en: allebei waar	`leeftijd >= 3 && leeftijd <= 17`	`true` voor 3 tot en met 17
`||`	of: minstens één waar	`dag == "za" || dag == "zo"`	`true` in het weekend
`!`	niet: draait om	`!gevonden`	`true` als er niets gevonden is

`&&` en `||` stoppen zodra het antwoord vastligt. Bij `s != null && s.Length > 3` wordt de lengte enkel opgevraagd als `s` niet `null` is. Dat is geen toeval: zo schrijf je een controle die niet crasht.

Iets doen als…

```
int leeftijd = 15;

if (leeftijd < 3)
{
    Console.WriteLine("Gratis");
}
else if (leeftijd < 18)
{
    Console.WriteLine("Kindertarief");
}
else
{
    Console.WriteLine("Volle prijs");
}
```

C# loopt de voorwaarden van boven naar onder af en neemt de **eerste** die klopt. De rest wordt niet meer bekeken. De volgorde van je `else if` bepaalt dus het antwoord.

Voor een keuze tussen twee waarden bestaat er een korte vorm:

```
string tekst = leeftijd >= 18 ? "volwassen" : "minderjarig";
```

Lees het als: *klopt de voorwaarde? Dan het eerste, anders het tweede.*

Kiezen uit veel mogelijkheden: switch

Vergelijk je één waarde met een reeks vaste mogelijkheden, dan is een `switch` overzichtelijker dan een lange rij `else if`:

```
string dag = "za";

switch (dag)
{
    case "za":
    case "zo":
        Console.WriteLine("Weekend");
        break;
    case "wo":
        Console.WriteLine("Halve dag");
        break;
    default:
        Console.WriteLine("Weekdag");
        break;
}
```

Moet er gewoon een waarde uitkomen, dan is de **switch-expressie** korter. Ze werkt ook met grenzen:

```
int graden = 25;

string advies = graden switch
{
    < 10 => "Neem een jas",
    < 20 => "Neem een trui",
    _ => "Een t-shirt volstaat"
};
```

De `_` betekent *al de rest*. Ook hier geldt: de eerste die past, wint.

Herhalen

```
for (int i = 0; i < 3; i++)
{
    Console.WriteLine(i);           // 0, 1, 2
}

int teller = 10;
while (teller > 0)
{
    teller -= 3;                    // 7, 4, 1, -2: dan stopt het
}

foreach (string naam in namen)
{
    Console.WriteLine(naam);        // elk item één keer
}
```

Wat je ermee kan

Vorm	Wat het doet	Voorbeeld	Resultaat
`if` / `else if` / `else`	doe iets als een voorwaarde klopt	`if (x > 0) { ... }`	de eerste die klopt, wint
`? :`	kies tussen twee waarden	`x > 0 ? "plus" : "min"`	`"plus"` of `"min"`
`switch`	voer een blok uit per mogelijkheid	`case "za": ... break;`	elk blok eindigt met `break`
`switch`-expressie	geef een waarde per mogelijkheid	`x switch { 1 => "een", _ => "veel" }`	`"een"` of `"veel"`
`< 10 =>`	een grens in een switch-expressie	`< 10 => "koud"`	alles onder 10
`and`, `or`	grenzen combineren in een switch-expressie	`>= 3 and <= 17 => "kind"`	3 tot en met 17
`_ =>`	al de rest	`_ => "onbekend"`	wat nergens anders past
`for`	herhaal een vast aantal keer	`for (int i = 0; i < 5; i++)`	`i` gaat van 0 tot 4
`while`	herhaal zolang iets klopt	`while (teller > 0)`	misschien nul keer
`do` … `while`	herhaal, en kijk pas achteraf	`do { ... } while (x > 0);`	minstens één keer
`foreach`	elk item van een verzameling	`foreach (var n in namen)`	elk item één keer
`break`	stop de lus nu	`if (gevonden) break;`	de lus is voorbij
`continue`	sla de rest van deze ronde over	`if (x < 0) continue;`	door naar de volgende ronde

Wat het ook kan, via iets anders

Welke lus op welke verzameling

Verzameling	`foreach`	`for` met `[i]`	Iets veranderen terwijl `foreach` loopt
tekst (hoofdstuk 1)	elk teken, als `char`	ja	kan niet: een tekst verander je nooit
array (hoofdstuk 5)	ja	ja	`getallen[0] = 9` mag; de lus ziet de nieuwe waarde
`List` (hoofdstuk 6)	ja	ja	`Add`, `Remove` en zelfs `lijst[0] = 9` geven `InvalidOperationException`
`Dictionary` (hoofdstuk 7)	elk paar, als `KeyValuePair`	nee	`Add` crasht; `Remove` en een waarde overschrijven mogen
`HashSet` (hoofdstuk 8)	ja	nee	`Add` van iets nieuws crasht; `Remove` mag
`Queue`, `Stack` (hoofdstuk 9, 10)	ja; een stapel van boven naar onder	nee	`Enqueue`, `Dequeue`, `Push` en `Pop` crashen
een resultaat van LINQ (hoofdstuk 11, 12)	ja	nee	—

De variabele van een `foreach` zelf kan je nooit een nieuwe waarde geven: `foreach (var x in getallen) { x = 0; }` compileert niet. Moet je aanpassen, gebruik dan een `for` met `[i]`. Moet je uit een lijst verwijderen terwijl je ze overloopt, loop dan achteruit: `for (int i = lijst.Count - 1; i >= 0; i--)`, of gebruik `RemoveAll` uit hoofdstuk 6.

Lussen die LINQ voor je schrijft

Komt van LINQ. Veel lussen met een `if` erin bestaan al als één methode.

In plaats van een lus met …	Kan ook met	Resultaat
een `if` en een teller	`getallen.Count(x => x > 4)`	het aantal
een `if` en `break` zodra je iets vindt	`getallen.Any(x => x > 4)` of `FirstOrDefault(x => x > 4)`	`true`/`false`, of het eerste
een nieuwe lijst die je vult	`getallen.Where(x => x > 4).ToList()`	een nieuwe lijst
een som die je bijhoudt	`getallen.Sum()`	de som
`for (int i = 1; i <= 5; i++)` om getallen te maken	`Enumerable.Range(1, 5)`	1 tot en met 5
een eigen teller voor de plaats in een `foreach`	`foreach (var (i, x) in lijst.Index())`	plaats en item samen

Zie hoofdstuk 11 voor alle methodes. Een `switch` werkt ook op een enum (hoofdstuk 18), en `is` kijkt van welk type iets is (hoofdstuk 22).

Veelgemaakte fouten

LET OP

- `=` geeft een waarde, `==` vergelijkt. `if (x = 5)` compileert in C# niet, want een toewijzing is geen ja of nee.
- Een switch-expressie zonder `_` compileert met een waarschuwing. Past de waarde nergens, dan crasht ze terwijl het programma draait.
- `for (int i = 0; i <= lijst.Count; i++)` loopt één keer te veel. De laatste positie is `Count - 1`, dus het is `<`, niet `<=`.
- Een `while` waarvan de voorwaarde nooit vals wordt, stopt nooit. Kijk na of er in de lus iets verandert aan wat je controleert.
- Teksten vergelijken met `==` let op hoofdletters. Komt een waarde van een gebruiker, denk dan aan `Equals` met `OrdinalIgnoreCase` uit hoofdstuk 1.

Voorspel de uitkomst

Vraag 1

```
int leeftijd = 70;

if (leeftijd >= 18)
{
    Console.WriteLine("volwassen");
}
else if (leeftijd >= 65)
{
    Console.WriteLine("senior");
}
else
{
    Console.WriteLine("kind");
}
```

Toon het antwoord ›

Vraag 2

```
int graden = 15;

string advies = graden switch
{
    < 10 => "jas",
    < 20 => "trui",
    _ => "t-shirt"
};

Console.WriteLine(advies);
```

Toon het antwoord ›

Vraag 3

```
for (int i = 1; i <= 5; i++)
{
    if (i == 2)
    {
        continue;
    }

    if (i == 4)
    {
        break;
    }

    Console.Write(i);
}
```

Toon het antwoord ›

Vraag 4

```
int n = 10;

do
{
    Console.WriteLine(n);
}
while (n < 5);
```

Toon het antwoord ›

Vraag 5

```
int x = 5;

string woord = x switch
{
    1 => "een",
    2 => "twee"
};

Console.WriteLine(woord);
```

Toon het antwoord ›

Antwoorden

VRAAG 1

`volwassen`

70 is groter dan 18, dus de eerste voorwaarde klopt al. De tweede wordt niet meer bekeken, ook al zou die ook kloppen. Wil je dat senioren apart behandeld worden, dan moet de controle op 65 eerst komen.

‹ Terug naar de vraag

VRAAG 2

`trui`

15 is niet kleiner dan 10, maar wel kleiner dan 20. De tweede regel past als eerste.

‹ Terug naar de vraag

VRAAG 3

`13`

Bij 1 wordt er geschreven. Bij 2 slaat `continue` de rest over. Bij 3 wordt er weer geschreven. Bij 4 stopt `break` de hele lus, dus 4 en 5 komen er nooit. `Console.Write` zet alles op één regel.

‹ Terug naar de vraag

VRAAG 4

`10`

Een `do`-lus voert het blok eerst uit en kijkt pas daarna naar de voorwaarde. 10 is niet kleiner dan 5, dus het blijft bij die ene keer. Met een gewone `while` was er niets op het scherm gekomen.

‹ Terug naar de vraag

VRAAG 5

`SwitchExpressionException` — het programma crasht.

Het compileert, met een waarschuwing dat niet elke waarde gedekt is. Als het programma draait en `x` is 5, past er geen enkele regel, en dan weet C# niet wat `woord` moet worden. Een laatste regel met `_` voorkomt dat.

‹ Terug naar de vraag

Deel 2 — Verzamelingen

Zes manieren om meerdere dingen samen te bewaren. Ze verschillen in wat ze toelaten en in hoe snel je iets terugvindt.

De verzamelingen naast elkaar

	array	List	Dictionary	HashSet	Queue	Stack
Grootte	vast	groeit	groeit	groeit	groeit	groeit
Volgorde bewaard	ja	ja	niet gegarandeerd	niet gegarandeerd	ja, eerst in eerst uit	ja, laatst in eerst uit
Dubbels	mag	mag	sleutels nee, waarden wel	nee	mag	mag
Opvragen op	positie	positie	sleutel	—	enkel vooraan	enkel bovenaan
Snel nagaan of iets erin zit	nee	nee	ja, op sleutel	ja	nee	nee
Hoeveel erin zitten	`Length`	`Count`	`Count`	`Count`	`Count`	`Count`
`foreach`	ja	ja	ja, per paar	ja	ja, van voor naar achter	ja, van boven naar onder
`for` **met** `[i]`	ja	ja	nee	nee	nee	nee
Aanpassen tijdens een `foreach`	een waarde vervangen mag	crasht	`Add` crasht; `Remove` en overschrijven mogen	`Add` crasht; `Remove` mag	crasht	crasht
LINQ	ja	ja	ja, op de paren	ja	ja	ja

Die laatste rij verdient uitleg. Een `List` met tienduizend items moet bij `Contains` in het slechtste geval alle tienduizend nakijken. Een `Dictionary` of `HashSet` weet meteen waar hij moet kijken, hoe groot hij ook is. Bij tien items merk je dat niet. Bij een miljoen wel.

Elke verzameling kan ook alles van LINQ uit hoofdstuk 11. Elk hoofdstuk hieronder toont daarom eerst wat de verzameling zelf kan, en daarna wat ze kan via iets anders, met de voorbeelden op dezelfde gegevens. Zo lees je per verzameling alles op één plaats.

5. Array

VERZAMELING

Wat het is

Een rij vakjes met een vaste lengte. Zoals een eierdoos: zes plaatsen, niet meer en niet minder. Je kan een ei vervangen, maar je kan er geen zevende plaats bij maken.

Hoe het eruitziet

```
string[] dagen = { "ma", "di", "wo", "do", "vr" };

Console.WriteLine(dagen[0]);        // ma
Console.WriteLine(dagen.Length);    // 5

dagen[4] = "vrijdag";               // vervangen mag

int[] scores = new int[3];          // drie vakjes, allemaal 0

foreach (string dag in dagen)
{
    Console.WriteLine(dag);
}
```

Je zal ook de kortere schrijfwijze `int[] getallen = [1, 2, 3];` tegenkomen. Die doet hetzelfde.

De voorbeelden in dit hoofdstuk

De tabellen hieronder gebruiken telkens deze drie arrays, zodat je elk resultaat kan narekenen.

```
string[] fruit    = { "appel", "peer", "kiwi" };
int[]    getallen = { 5, 8, 3, 8, 1 };
Fan[]    fans     =
{
    new Fan { Id = 1, Merk = "NZXT",   Prijs = 24.99m },
    new Fan { Id = 2, Merk = "Noctua", Prijs = 29.99m },
    new Fan { Id = 3, Merk = "NZXT",   Prijs = 89.99m },
    new Fan { Id = 4, Merk = "Arctic", Prijs = 19.99m }
};
```

`Fan` is een klasse met drie eigenschappen. Hoe je zelf zo'n klasse maakt, staat in hoofdstuk 13.

Wat je ermee kan

Een array heeft zelf maar een handvol eigenschappen en methodes. Het meeste werk doet de klasse `Array`: je schrijft niet `getallen.Sort()` maar `Array.Sort(getallen)`. De array is een bouwsteen van de taal zelf, ouder dan `List`, en de hulpmethodes zitten daarom apart: je geeft de array mee tussen de haakjes.

Op de array zelf

Methode	Wat het doet	Voorbeeld	Resultaat
`Length`	hoeveel vakjes; een eigenschap	`getallen.Length`	`5`
`[i]`	lees of vervang het vakje op positie i	`fruit[1]`	`peer`
`CopyTo`	kopieer alles naar een andere array, vanaf een positie	`fruit.CopyTo(doel, 2)`	`doel` met vijf vakjes wordt `null, null, appel, peer, kiwi`
`Clone`	maak een kopie	`(int[])getallen.Clone()`	een nieuwe array `5, 8, 3, 8, 1`
`Rank`	hoeveel dimensies	`getallen.Rank`	`1`
`GetLength`	hoeveel vakjes in één dimensie	`getallen.GetLength(0)`	`5`
`GetUpperBound`	de laatste positie in één dimensie	`getallen.GetUpperBound(0)`	`4`
`GetLowerBound`	de eerste positie in één dimensie	`getallen.GetLowerBound(0)`	`0`

`Clone` geeft een `object` terug. Daarom staat er `(int[])` voor: je zegt de compiler dat het een array van getallen is.

Zoeken, via de klasse Array

Methode	Wat het doet	Voorbeeld	Resultaat
`Array.IndexOf`	op welke positie staat het eerst	`Array.IndexOf(getallen, 8)`	`1`, of `-1`
`Array.LastIndexOf`	op welke positie staat het laatst	`Array.LastIndexOf(getallen, 8)`	`3`, of `-1`
`Array.Exists`	is er minstens één die voldoet	`Array.Exists(fans, f => f.Prijs > 50)`	`true`
`Array.TrueForAll`	voldoen ze allemaal	`Array.TrueForAll(fans, f => f.Prijs > 20)`	`false`: de Arctic kost minder
`Array.Find`	de eerste die voldoet	`Array.Find(fans, f => f.Id == 3)`	de fan met Id 3, of `null`
`Array.FindLast`	de laatste die voldoet	`Array.FindLast(fans, f => f.Merk == "NZXT")`	de fan met Id 3
`Array.FindAll`	allemaal die voldoen	`Array.FindAll(fans, f => f.Merk == "NZXT")`	een nieuwe **array** met fan 1 en 3
`Array.FindIndex`	positie van de eerste die voldoet	`Array.FindIndex(fans, f => f.Id == 3)`	`2`, of `-1`
`Array.FindLastIndex`	positie van de laatste die voldoet	`Array.FindLastIndex(fans, f => f.Merk == "NZXT")`	`2`, of `-1`
`Array.BinarySearch`	zoek snel in een **gesorteerde** array	`Array.BinarySearch(gesorteerd, 8)`	de positie, of een negatief getal

Ordenen, vullen en kopiëren, via de klasse Array

Methode	Wat het doet	Voorbeeld	Resultaat
`Array.Sort`	sorteer de array zelf	`Array.Sort(getallen)`	`1, 3, 5, 8, 8`
`Array.Sort`	sorteer op iets dat je zelf kiest	`Array.Sort(fans, (a, b) => a.Prijs.CompareTo(b.Prijs))`	goedkoopste eerst
`Array.Sort`	sorteer twee arrays samen, op de eerste	`Array.Sort(scores, namen)`	de namen staan in de volgorde van hun score
`Array.Reverse`	keer de array zelf om	`Array.Reverse(getallen)`	`1, 8, 3, 8, 5`
`Array.Fill`	zet overal dezelfde waarde	`Array.Fill(scores, 10)`	`10, 10, 10`
`Array.Clear`	zet alles terug op de standaardwaarde	`Array.Clear(getallen)`	`0, 0, 0, 0, 0`; de lengte blijft 5
`Array.Copy`	kopieer een aantal vakjes naar een andere array	`Array.Copy(getallen, doel, 3)`	`doel` begint met `5, 8, 3`
`Array.Resize`	maak een nieuwe array met een andere lengte	`Array.Resize(ref fruit, 5)`	`appel, peer, kiwi, null, null`
`Array.ConvertAll`	maak van elk item iets anders	`Array.ConvertAll(fans, f => f.Merk)`	een nieuwe `string[]` met de merken
`Array.ForEach`	doe iets met elk item	`Array.ForEach(fruit, f => Console.WriteLine(f))`	drie regels op het scherm
`Array.AsReadOnly`	een versie die niemand kan veranderen	`Array.AsReadOnly(fruit)`	vervangen kan niet meer
`Array.Empty`	een lege array	`Array.Empty<int>()`	een array met lengte `0`

`Array.Sort`, `Array.Reverse`, `Array.Fill` en `Array.Clear` veranderen de array die je meegeeft. Ze geven niets terug.

> **BELANGRIJK**
>
> `Array.Resize` maakt de array niet groter: dat kan niet. Hij maakt een **nieuwe** array met de nieuwe lengte, kopieert de inhoud, en laat je variabele naar die nieuwe array wijzen. Daarom staat er `ref` voor. Moet je vaak iets bijvoegen, neem dan een `List`.

> **OPMERKING**
>
> In .NET 10 mag je sommige van deze methodes ook rechtstreeks op de array schrijven, zoals `getallen.Sort()` of `getallen.IndexOf(8)`. Ze komen dan uit `MemoryExtensions`, de hulpmethodes voor `Span`, en doen hetzelfde als `Array.Sort` en `Array.IndexOf`. In de cursus en in de meeste voorbeelden zie je de schrijfwijze met `Array.`.

Meerdere dimensies

Een array kan ook een rooster zijn: rijen en kolommen.

```
int[,] rooster = new int[2, 3];     // 2 rijen, 3 kolommen, allemaal 0
rooster[1, 2] = 7;

Console.WriteLine(rooster.Rank);           // 2
Console.WriteLine(rooster.Length);         // 6 – alle vakjes samen
Console.WriteLine(rooster.GetLength(0));   // 2 – rijen
Console.WriteLine(rooster.GetLength(1));   // 3 – kolommen

int[][] rijen = new int[2][];       // een array van arrays
rijen[0] = new int[] { 1, 2 };
rijen[1] = new int[] { 3, 4, 5 };   // elke rij mag een andere lengte hebben
Console.WriteLine(rijen[1][2]);     // 5
```

`Length` telt bij een rooster alle vakjes samen. Wil je het aantal rijen of kolommen, dan heb je `GetLength` nodig.

Wat het ook kan, via iets anders

Een array kan veel meer dan de methodes hierboven, omdat ze ook iets anders ís. Ze is een **reeks**: iets wat je één voor één kan doorlopen. In C# heet dat `IEnumerable` (hoofdstuk 12). Alles wat met een reeks werkt, werkt dus ook met een array.

Bron	Waarom het op een array werkt	Volledige uitleg
LINQ	LINQ werkt op elke reeks	hoofdstuk 11
`foreach`	`foreach` doorloopt elke reeks	hoofdstuk 4
`^` en `..` tussen vierkante haken	een array heeft een lengte en posities	hieronder
`string.Join`	wil een reeks teksten of waarden	hoofdstuk 1
andere verzamelingen	die kan je vullen met een reeks	hoofdstukken 6 tot 10
wat elk object kan	een array is ook een object	hoofdstuk 13

Wat je over LINQ moet weten om de tabellen te lezen:

- Een LINQ-methode **verandert je array nooit**. Ze geeft een nieuw resultaat.
- Dat resultaat is meestal **geen array**, maar een reeks. Wil je er weer een array van, zet er dan `.ToArray()` achter. Een lijst krijg je met `.ToList()`.
- `Where`, `Select`, `OrderBy` en hun familie rekenen pas op het moment dat je het resultaat gebruikt. Verandert je array tussendoor, dan rekenen ze met de nieuwe inhoud.
- LINQ werkt enkel met `using System.Linq;` bovenaan. In een nieuw project staat die er automatisch bij.

> **OPMERKING**
>
> `Index`, `CountBy` en `AggregateBy` bestaan sinds .NET 9, `Shuffle`, `LeftJoin` en `RightJoin` sinds .NET 10. Je cursus werkt met .NET 10, dus je kan ze gebruiken. In oudere voorbeelden op internet kom je ze niet tegen.

Eén item nemen

Komt van LINQ.

Methode	Wat het doet	Voorbeeld	Resultaat	Via de klasse Array
`First`	de eerste die voldoet	`fans.First(f => f.Merk == "NZXT")`	fan 1; niets gevonden geeft een `InvalidOperationException`	—
`FirstOrDefault`	de eerste die voldoet, of niets	`fans.FirstOrDefault(f => f.Id == 9)`	`null`	`Array.Find`
`Single`	de enige die voldoet	`fans.Single(f => f.Id == 2)`	fan 2; nul of twee gevonden geeft een `InvalidOperationException`	—
`SingleOrDefault`	de enige die voldoet, of niets	`fans.SingleOrDefault(f => f.Merk == "NZXT")`	`InvalidOperationException`: er zijn er twee	—
`Last`	de laatste die voldoet	`fans.Last(f => f.Merk == "NZXT")`	fan 3	—
`LastOrDefault`	de laatste die voldoet, of niets	`fans.LastOrDefault(f => f.Prijs > 100)`	`null`	`Array.FindLast`
`ElementAt`	het item op een positie	`fans.ElementAt(1)`	fan 2	`fans[1]`
`ElementAtOrDefault`	het item op een positie, of niets	`fans.ElementAtOrDefault(9)`	`null`	—

Neem `FirstOrDefault` wanneer het normaal is dat er niets gevonden wordt, zoals een Id dat niet bestaat. Neem `SingleOrDefault` wanneer er hoogstens één mág zijn: de uitzondering bij twee is dan een alarm over je gegevens. Neem `First` alleen wanneer je zeker weet dat er iets is.

Weten of iets erin zit

Komt van LINQ.

Methode	Wat het doet	Voorbeeld	Resultaat	Via de klasse Array
`Any`	is er minstens één	`fans.Any()`	`true`	`fans.Length > 0`
`Any`	is er minstens één die voldoet	`fans.Any(f => f.Prijs > 50)`	`true`	`Array.Exists`
`All`	voldoen ze allemaal	`fans.All(f => f.Prijs > 20)`	`false`	`Array.TrueForAll`
`Contains`	zit deze waarde erin	`getallen.Contains(3)`	`true`	`Array.IndexOf(...) >= 0`
`SequenceEqual`	hebben twee reeksen dezelfde inhoud, in dezelfde volgorde	`getallen.SequenceEqual(new[] { 5, 8, 3, 8, 1 })`	`true`	—

> **LET OP**
>
> `All` op een lege array geeft `true`: er is geen enkel item dat níet voldoet. Controleer eerst of er wel iets in zit.

Een deel nemen

Komt van LINQ. Het resultaat is telkens een nieuwe reeks; je array blijft zoals ze was.

Methode	Wat het doet	Voorbeeld	Resultaat	Via de klasse Array
`Where`	enkel wat voldoet	`fans.Where(f => f.Merk == "NZXT")`	fan 1 en 3	`Array.FindAll`
`Select`	maak van elk item iets anders	`fans.Select(f => f.Merk)`	`NZXT, Noctua, NZXT, Arctic`	`Array.ConvertAll`
`SelectMany`	plat een lijst van lijsten	`klassen.SelectMany(k => k.Leerlingen)`	alle leerlingen samen	—
`Distinct`	gooi dubbels weg	`getallen.Distinct()`	`5, 8, 3, 1`	—
`DistinctBy`	één item per waarde van iets	`fans.DistinctBy(f => f.Merk)`	fan 1, 2 en 4	—
`Take`	de eerste n	`getallen.Take(2)`	`5, 8`	`getallen[..2]`
`Skip`	sla de eerste n over	`getallen.Skip(2)`	`3, 8, 1`	`getallen[2..]`
`TakeLast`	de laatste n	`getallen.TakeLast(2)`	`8, 1`	`getallen[^2..]`
`SkipLast`	alles behalve de laatste n	`getallen.SkipLast(2)`	`5, 8, 3`	`getallen[..^2]`
`TakeWhile`	neem zolang de voorwaarde klopt, stop bij de eerste die niet klopt	`getallen.TakeWhile(g => g > 2)`	`5, 8, 3, 8`	—

Methode	Wat het doet	Voorbeeld	Resultaat	Via de klasse Array
SkipWhile	sla over zolang de voorwaarde klopt	`getallen.SkipWhile(g => g > 2)`	`1`	—
Chunk	verdeel in stukken van n	`getallen.Chunk(2)`	`[5, 8], [3, 8], [1]`, elk stuk een array	—
OfType	enkel de items van een bepaald type	`onderdelen.OfType<Processor>()`	enkel de processors	—
Index	elk item samen met zijn positie	`fruit.Index()`	`(0, appel), (1, peer), (2, kiwi)`	—

`OfType` heb je nodig bij overerving: een array van een basisklasse waar verschillende soorten in zitten. Zie hoofdstuk 20.

Tellen en rekenen

Komt van LINQ.

Methode	Wat het doet	Voorbeeld	Resultaat	Via de klasse Array
Count	hoeveel er zijn	`fans.Count()`	`4`	`Length`
Count	hoeveel er voldoen	`fans.Count(f => f.Prijs > 25)`	`2`	—
Sum	tel op	`fans.Sum(f => f.Prijs)`	`164.96`	—
Min	de kleinste waarde	`fans.Min(f => f.Prijs)`	`19.99`	—
Max	de grootste waarde	`fans.Max(f => f.Prijs)`	`89.99`	—
Average	het gemiddelde	`fans.Average(f => f.Prijs)`	`41.24`	—
MinBy	het item met de kleinste waarde	`fans.MinBy(f => f.Prijs)`	fan 4	—
MaxBy	het item met de grootste waarde	`fans.MaxBy(f => f.Prijs)`	fan 3	—
CountBy	tel per waarde van iets	`fans.CountBy(f => f.Merk)`	`NZXT 2, Noctua 1, Arctic 1`	—
Aggregate	voeg alles samen met een eigen regel	`fruit.Aggregate((a, b) => a + "-" + b)`	`appel-peer-kiwi`	—
AggregateBy	voeg samen per waarde van iets	`fans.AggregateBy(f => f.Merk, 0m, (som, f) => som + f.Prijs)`	`NZXT 114.98, Noctua 29.99, Arctic 19.99`	—

`Max` geeft het getal, `MaxBy` geeft het item. `Sum`, `Average`, `Min` en `Max` op een array getallen hebben geen voorwaarde nodig: `getallen.Sum()` geeft `25`.

> **LET OP**
>
> `Average`, `Min` en `Max` op een **lege** array van getallen geven een `InvalidOperationException`. `Sum` geeft dan `0`. Bij een lege array van teksten of objecten geven `Min` en `Max` `null`.

Ordenen en groeperen

Komt van LINQ. Je array blijft in haar oude volgorde; je krijgt een nieuwe, geordende reeks.

Methode	Wat het doet	Voorbeeld	Resultaat	Via de klasse Array
`Order`	sorteer de waarden oplopend	`getallen.Order()`	`1, 3, 5, 8, 8`	`Array.Sort`
`OrderDescending`	sorteer de waarden aflopend	`getallen.OrderDescending()`	`8, 8, 5, 3, 1`	—
`OrderBy`	sorteer oplopend op iets	`fans.OrderBy(f => f.Prijs)`	fan 4, 1, 2, 3	`Array.Sort` met een eigen regel
`OrderByDescending`	sorteer aflopend op iets	`fans.OrderByDescending(f => f.Prijs)`	fan 3, 2, 1, 4	—
`ThenBy`	tweede sorteersleutel, oplopend	`fans.OrderBy(f => f.Merk).ThenBy(f => f.Prijs)`	per merk, en binnen een merk de goedkoopste eerst	—
`ThenByDescending`	tweede sorteersleutel, aflopend	`fans.OrderBy(f => f.Merk).ThenByDescending(f => f.Prijs)`	per merk, en binnen een merk de duurste eerst	—
`Reverse`	omgekeerde volgorde, in een nieuwe reeks	`getallen.Reverse()`	`1, 8, 3, 8, 5`; `getallen` zelf blijft `5, 8, 3, 8, 1`	`Array.Reverse`
`Shuffle`	willekeurige volgorde	`fruit.Shuffle()`	elke keer anders	—
`GroupBy`	maak groepen	`fans.GroupBy(f => f.Merk)`	drie groepen: NZXT met 2 fans, Noctua en Arctic met elk 1	—
`ToLookup`	maak groepen, meteen vastgelegd	`fans.ToLookup(f => f.Merk)`	dezelfde groepen; `opzoek["NZXT"]` geeft fan 1 en 3	—

> **LET OP**
>
> `getallen.Reverse()` en `Array.Reverse(getallen)` lijken op elkaar, maar doen iets anders. `getallen.Reverse()` is LINQ: je krijgt een nieuwe, omgekeerde reeks en je array blijft zoals ze was. `Array.Reverse(getallen)` draait de array zelf om en geeft niets terug. Bij een `List` is het net omgekeerd: daar draait `lijst.Reverse()` de lijst zelf om (hoofdstuk 6).

Twee reeksen combineren

Komt van LINQ. In de voorbeelden is `a` de array `1, 2, 3` en `b` de array `3, 4`.

Methode	Wat het doet	Voorbeeld	Resultaat
`Concat`	plak de tweede achter de eerste	`a.Concat(b)`	`1, 2, 3, 3, 4`
`Union`	alles uit beide, zonder dubbels	`a.Union(b)`	`1, 2, 3, 4`
`Intersect`	wat in beide zit	`a.Intersect(b)`	`3`
`Except`	wat in de eerste zit en niet in de tweede	`a.Except(b)`	`1, 2`
`UnionBy`, **`IntersectBy`**, **`ExceptBy`**	hetzelfde, maar vergeleken op één eigenschap	`fans.ExceptBy(new[] { "NZXT" }, f => f.Merk)`	fan 2 en 4
`Append`	één item achteraan, in het resultaat	`a.Append(9)`	`1, 2, 3, 9`; `a` blijft drie lang
`Prepend`	één item vooraan, in het resultaat	`a.Prepend(0)`	`0, 1, 2, 3`
`Zip`	koppel de eerste aan de eerste, de tweede aan de tweede …	`fruit.Zip(getallen)`	`(appel, 5), (peer, 8), (kiwi, 3)`
`Join`	koppel items uit twee reeksen met dezelfde sleutel	`fans.Join(merken, f => f.Merk, m => m.Naam, (f, m) => ...)`	enkel de paren die bij elkaar horen
`LeftJoin`	zoals `Join`, maar elk item uit de eerste blijft, ook zonder partner	`fans.LeftJoin(merken, ...)`	alle fans, met of zonder merk
`RightJoin`	zoals `Join`, maar elk item uit de tweede blijft	`fans.RightJoin(merken, ...)`	alle merken, met of zonder fan
`GroupJoin`	koppel aan elk item uit de eerste de groep die erbij hoort	`merken.GroupJoin(fans, ...)`	elk merk met zijn fans

Een array kan niet groeien, maar met `Append`, `Concat` en `.ToArray()` erachter maak je een **nieuwe**, langere array: `getallen = getallen.Append(9).ToArray();`. `Zip` stopt bij de kortste van de twee: de 8 en de 1 uit `getallen` vallen weg.

Het resultaat vastleggen

Komt van LINQ.

Methode	Wat het doet	Voorbeeld	Resultaat
`ToArray`	maak er een nieuwe array van	`getallen.Where(g => g > 4).ToArray()`	een `int[]` met `5, 8, 8`
`ToList`	maak er een `List` van, die wél kan groeien	`fans.ToList()`	een `List<Fan>`
`ToHashSet`	maak er een `HashSet` van: dubbels vallen weg	`getallen.ToHashSet()`	`5, 8, 3, 1`
`ToDictionary`	maak er een opzoektabel van	`fans.ToDictionary(f => f.Id)`	`tabel[3]` geeft fan 3
`ToLookup`	maak er groepen van	`fans.ToLookup(f => f.Merk)`	zie hierboven

> **LET OP**
>
> `ToDictionary` crasht met een `ArgumentException` als twee items dezelfde sleutel hebben. `fans.ToDictionary(f => f.Merk)` crasht dus: NZXT komt twee keer voor.

Doorlopen en posities

Komt van de taal C# zelf.

Schrijfwijze	Wat het doet	Voorbeeld	Resultaat
`foreach`	elk item één keer	`foreach (string f in fruit) { ... }`	appel, peer, kiwi
`for` met `Length`	elke positie	`for (int i = 0; i < fruit.Length; i++) { ... }`	positie 0, 1, 2
`[i]`	het item op positie i	`fruit[0]`	`appel`
`[i] =`	vervang het item op positie i	`fruit[0] = "mango"`	de appel is een mango
`[^1]`	tellen vanaf achteren: het laatste	`fruit[^1]`	`kiwi`
`[1..3]`	een stuk, van positie 1 tot vóór 3	`fruit[1..3]`	een nieuwe array `peer, kiwi`
`[..2]`	van het begin tot vóór 2	`fruit[..2]`	een nieuwe array `appel, peer`
`[2..]`	van positie 2 tot het einde	`getallen[2..]`	een nieuwe array `3, 8, 1`
`[.. a, .. b]`	een nieuwe array uit twee arrays	`int[] samen = [.. a, .. b];`	`1, 2, 3, 3, 4`

`foreach` en `for` staan volledig in hoofdstuk 4. In een `foreach` kan je een vakje niet vervangen; daarvoor heb je `for` en `[i] =` nodig.

Omzetten naar iets anders

Komt van de andere verzamelingen en van `string`.

Schrijfwijze	Wat het doet	Voorbeeld	Resultaat
`string.Join`	zet alles achter elkaar in één tekst	`string.Join(", ", fruit)`	`appel, peer, kiwi`
`new List<T>(array)`	een lijst, die wel kan groeien	`new List<string>(fruit)`	een lijst met dezelfde inhoud
`new HashSet<T>(array)`	een `HashSet`: dubbels vallen weg	`new HashSet<int>(getallen)`	`5, 8, 3, 1`
`new Queue<T>(array)`	een wachtrij, in dezelfde volgorde	`new Queue<string>(fruit).Dequeue()`	`appel`
`new Stack<T>(array)`	een stapel: het laatste ligt bovenop	`new Stack<string>(fruit).Pop()`	`kiwi`

De lijst, de `HashSet`, de wachtrij en de stapel staan in de hoofdstukken 6, 8, 9 en 10.

Wat elk object kan

Komt van `object`: elke array is ook een object. Zie hoofdstuk 13.

Methode	Wat het doet	Voorbeeld	Resultaat
`ToString`	het object als tekst	`fruit.ToString()`	`System.String[]`
`Equals` / `==`	is dit **dezelfde** array	`getallen == new[] { 5, 8, 3, 8, 1 }`	`false`
`GetType`	welk type is dit	`fruit.GetType().Name`	`String[]`
`GetHashCode`	een getal per object, voor opzoektabellen	`fruit.GetHashCode()`	een getal; zelden zelf nodig

`ToString` geeft de naam van het type, niet de inhoud. Wil je de inhoud zien, gebruik dan `string.Join`. En `==` vergelijkt of het om dezelfde array gaat, niet of de inhoud gelijk is. Voor de inhoud is er `SequenceEqual`.

Bestaat ook, maar kom je in deze cursus niet tegen

`Array.Copy` met een beginpositie in beide arrays, `Array.ConstrainedCopy`, `Array.CreateInstance`, `GetValue` en `SetValue` (een vakje lezen of vervangen zonder `[ ]`), `Array.MaxLength`, de methodes voor `Span` (`AsSpan`, en via `MemoryExtensions` onder meer `StartsWith` en `EndsWith` op een array), en uit LINQ: `AsParallel`, `Cast`, `DefaultIfEmpty`, `LongCount`, `ToImmutableArray`, `ToFrozenSet` en hun familie.

Let op bij een array

Een array heeft voor veel LINQ-methodes een tegenhanger in de klasse `Array`. Ze lijken op elkaar, maar zijn niet altijd hetzelfde.

Klasse Array	LINQ	Het verschil
`Length`	`Count()`	Hetzelfde getal. `Length` is een eigenschap van de array; `Count()` met haakjes is LINQ. Een voorwaarde meegeven kan enkel met LINQ: `Count(f => ...)`. Een array heeft geen `Count` zonder haakjes.
`Array.Find`	`FirstOrDefault`	Hetzelfde resultaat. `FirstOrDefault` werkt op elke verzameling, ook op een databasequery.
`Array.Exists`	`Any`	Hetzelfde resultaat.
`Array.TrueForAll`	`All`	Hetzelfde resultaat.
`Array.FindAll`	`Where`	`Array.FindAll` geeft meteen een nieuwe **array**. `Where` geeft een reeks die pas rekent als je ze gebruikt.
`Array.ConvertAll`	`Select`	Zelfde verschil: meteen een array tegenover een reeks.
`Array.Sort`	`Order`, `OrderBy`	`Array.Sort` verandert je array en geeft niets terug. `Order` en `OrderBy` laten je array zoals ze is en geven een nieuwe reeks.
`Array.Reverse`	`Reverse()`	`Array.Reverse(getallen)` draait de array zelf om. `getallen.Reverse()` is LINQ en geeft een nieuwe reeks.
`Array.IndexOf(...) >= 0`	`Contains`	Hetzelfde resultaat; `Contains` leest makkelijker.
`[1..3]`	`Skip(1).Take(2)`	`[1..3]` geeft meteen een nieuwe array, en crasht als het stuk buiten de array valt. `Skip` en `Take` crashen nooit: ze geven minder terug.

LET OP

- Het resultaat van `Where`, `Select` of `OrderBy` is geen array. `int[] groot = getallen.Where(g => g > 4);` compileert niet. Zet er `.ToArray()` achter.
- `int[] b = a;` maakt **geen kopie**. `a` en `b` zijn dan twee namen voor dezelfde array. Een kopie maak je met `.ToArray()`, `Clone()` of `[.. a]`.
- `Contains`, `Array.IndexOf` en `Distinct` vergelijken items met `Equals`. Bij een array van een `class` betekent dat: is het **hetzelfde object**, niet een object met dezelfde inhoud. Bij een `record` wordt de inhoud vergeleken. Zie hoofdstuk 13 en hoofdstuk 14.
- `FirstOrDefault`, `Array.Find` en hun familie geven bij een array van getallen geen `null` maar `0`, en bij een array van `bool` geen `null` maar `false`.
- LINQ werkt niet rechtstreeks op een rooster (`int[,]`): `rooster.Sum()` compileert niet. Schrijf `rooster.Cast<int>().Sum()`. Bij een array van arrays (`int[][]`) werkt LINQ wel, per rij.

Wanneer wel, wanneer niet

Wel — de hoeveelheid ligt vast en verandert nooit: de dagen van de week, de maanden, de vakjes van een bordspel.

Niet — zodra je iets wil bijvoegen of weghalen. Een array heeft geen `Add` en geen `Remove`.

Veelgemaakte fouten

> LET OP
>
> - De eerste positie is `0`, dus de laatste is `Length - 1`. Vraag je `dagen[5]` op in een array van vijf, dan crasht je programma.
> - Het is `Length`, niet `Count`. Bij een `List` is het net omgekeerd.
> - `new int[3]` geeft drie nullen. `new string[3]` geeft drie keer `null`, geen drie lege teksten.
> - `Console.WriteLine(array)` toont de naam van het type, niet de inhoud.

Voorspel de uitkomst

Vraag 1

```
int[] a = new int[3];
Console.WriteLine(a[1]);
```

Toon het antwoord ›

Vraag 2

```
string[] s = { "a", "b", "c" };
Console.WriteLine(s[s.Length - 1]);
```

Toon het antwoord ›

Vraag 3

```
string[] s = { "a", "b", "c" };
Console.WriteLine(s[3]);
```

Toon het antwoord ›

Vraag 4

```
int[] a = { 1, 2, 3 };
int[] b = a;
b[0] = 99;
Console.WriteLine(a[0]);
```

Toon het antwoord ›

Vraag 5

```
int[] a = { 3, 1, 2 };
var gesorteerd = a.Order();
Console.WriteLine(a[0]);
Array.Sort(a);
Console.WriteLine(a[0]);
```

Toon het antwoord ›

Vraag 6

```
string[] s = { "a", "b" };
Array.Resize(ref s, 4);
Console.WriteLine(s.Length);
Console.WriteLine(s[3] == null);
```

Toon het antwoord ›

Vraag 7

```
int[] a = { 1, 2, 3 };
int[] b = { 1, 2, 3 };
Console.WriteLine(a == b);
Console.WriteLine(a.SequenceEqual(b));
Console.WriteLine(a);
```

Toon het antwoord ›

Vraag 8

```
int[] a = { 10, 20, 30, 40 };
int[] stuk = a[1..3];
stuk[0] = 99;
Console.WriteLine(a[1]);
Console.WriteLine(a[^1]);
```

Toon het antwoord ›

Vraag 9

```
int[] a = { 5, 8, 3 };
int[] b = a.Append(9).ToArray();
Console.WriteLine(a.Length);
Console.WriteLine(b.Length);
```

Toon het antwoord ›

Vraag 10

```
int[,] rooster = new int[2, 3];
Console.WriteLine(rooster.Length);
Console.WriteLine(rooster.GetLength(1));
```

Toon het antwoord ›

Antwoorden

VRAAG 1

`0`

Een nieuwe array van getallen is gevuld met de standaardwaarde, en die is voor een `int` gelijk aan nul.

‹ Terug naar de vraag

VRAAG 2

`c`

`Length` is 3, dus `Length - 1` is 2, en positie 2 is het derde vakje.

‹ Terug naar de vraag

VRAAG 3

`IndexOutOfRangeException` — het programma crasht.

Er zijn drie vakjes: 0, 1 en 2. Vakje 3 bestaat niet.

‹ Terug naar de vraag

VRAAG 4

`99`

Dit is de belangrijkste vraag van dit hoofdstuk. `b = a` maakt geen kopie van de array. Het geeft dezelfde array een tweede naam. Verander je iets via `b`, dan zie je dat ook via `a`, want het is één en dezelfde eierdoos. Zie ook hoofdstuk 16.

‹ Terug naar de vraag

VRAAG 5

`3` en daarna `1`

`Order` is LINQ: de array blijft `3, 1, 2`, het gesorteerde zit in `gesorteerd`. `Array.Sort` sorteert de array zelf.

‹ Terug naar de vraag

VRAAG 6

`4` en daarna `True`

`Array.Resize` maakte een nieuwe array van vier vakjes en kopieerde de twee teksten. De twee nieuwe vakjes zijn leeg: `null`.

‹ Terug naar de vraag

VRAAG 7

`False`, daarna `True`, en dan `System.Int32[]`

`==` vraagt of `a` en `b` dezelfde array zijn: dat zijn ze niet. `SequenceEqual` vergelijkt de inhoud. En `Console.WriteLine` toont de naam van het type: `Int32` is de echte naam van `int`.

‹ Terug naar de vraag

VRAAG 8

`20` en daarna `40`

`a[1..3]` maakt een **nieuwe** array met 20 en 30. Die veranderen raakt `a` niet. `^1` is het laatste vakje.

‹ Terug naar de vraag

VRAAG 9

`3` en daarna `4`

Een array kan niet groeien. `Append` geeft een nieuwe reeks, `ToArray` maakt daar een nieuwe array van. `a` blijft drie vakjes lang.

‹ Terug naar de vraag

VRAAG 10

`6` en daarna `3`

`Length` telt alle vakjes van het rooster: 2 rijen maal 3 kolommen. `GetLength(1)` geeft de lengte van de tweede dimensie: de kolommen.

‹ Terug naar de vraag

6. List

VERZAMELING

Wat het is

Een array die kan groeien en krimpen. Zoals een boodschappenlijstje: je schrijft erbij, je streept door, en de volgorde blijft zoals je ze opschreef.

Tussen de punthaken staat wat erin mag. `List<string>` is een lijst van teksten, `List<int>` een lijst van getallen, `List<Leerling>` een lijst van leerlingen. Iets anders erin steken compileert niet.

Hoe het eruitziet

```
List<string> fruit = new List<string>();
fruit.Add("appel");
fruit.Add("peer");
fruit.Add("kiwi");

Console.WriteLine(fruit.Count);     // 3
Console.WriteLine(fruit[1]);        // peer

fruit.Remove("peer");
Console.WriteLine(fruit[1]);        // kiwi — alles schuift een plaats op

fruit.Insert(0, "banaan");          // vooraan bijzetten

List<string> kort = new List<string> { "appel", "peer" };   // meteen gevuld
```

Je zal ook de kortere schrijfwijze `List<string> kort = ["appel", "peer"];` tegenkomen. Die doet hetzelfde.

De voorbeelden in dit hoofdstuk

De tabellen hieronder gebruiken telkens deze drie lijsten, zodat je elk resultaat kan narekenen.

```
List<string> fruit   = new List<string> { "appel", "peer", "kiwi" };
List<int>    getallen = new List<int> { 5, 8, 3, 8, 1 };
List<Fan>    fans    = new List<Fan>
{
    new Fan { Id = 1, Merk = "NZXT",   Prijs = 24.99m },
    new Fan { Id = 2, Merk = "Noctua", Prijs = 29.99m },
    new Fan { Id = 3, Merk = "NZXT",   Prijs = 89.99m },
    new Fan { Id = 4, Merk = "Arctic", Prijs = 19.99m }
};
```

`Fan` is een klasse met drie eigenschappen. Hoe je zelf zo'n klasse maakt, staat in hoofdstuk 13.

Wat je ermee kan

Dit zijn de methodes die op een `List<T>` zelf zitten. Ze werken enkel op een lijst, niet op elke verzameling. Wat er daarnaast nog werkt, staat in het volgende deel, *Wat het ook kan, via iets anders*.

Toevoegen en verwijderen

Methode	Wat het doet	Voorbeeld	Resultaat
`Add`	zet er één achteraan bij	`fans.Add(nieuweFan)`	de lijst is één langer
`AddRange`	zet er meerdere achteraan bij	`fans.AddRange(extraFans)`	de lijst groeit
`Insert`	zet er één op een vaste plaats	`fans.Insert(0, nieuweFan)`	staat nu vooraan
`InsertRange`	zet er meerdere op een vaste plaats	`fruit.InsertRange(1, extraFruit)`	ze staan vanaf positie 1
`Remove`	haal dit ene item eruit	`fans.Remove(fan)`	`true` als het erin zat
`RemoveAt`	haal weg op positie	`fans.RemoveAt(0)`	de eerste is weg
`RemoveRange`	haal een aantal weg vanaf een positie	`getallen.RemoveRange(1, 2)`	`5, 8, 1`
`RemoveAll`	haal alles weg dat voldoet	`fans.RemoveAll(f => f.Merk == "NZXT")`	`2`: er zijn er twee weg
`Clear`	maak leeg	`fans.Clear()`	de lijst is leeg

`Remove` wil het item zelf, `RemoveAt` wil een positie, `RemoveAll` wil een voorwaarde. Drie namen die op elkaar lijken en drie verschillende dingen willen.

Erin zoeken

Methode	Wat het doet	Voorbeeld	Resultaat
`Count`	hoeveel zitten erin; een eigenschap	`fans.Count`	`4`
`Contains`	zit dit exacte item erin	`getallen.Contains(3)`	`true`
`IndexOf`	op welke positie staat het eerst	`getallen.IndexOf(8)`	`1`, of `-1`
`LastIndexOf`	op welke positie staat het laatst	`getallen.LastIndexOf(8)`	`3`, of `-1`
`Exists`	is er minstens één die voldoet	`fans.Exists(f => f.Prijs > 50)`	`true`
`TrueForAll`	voldoen ze allemaal	`fans.TrueForAll(f => f.Prijs > 20)`	`false`: de Arctic kost minder
`Find`	geef de eerste die voldoet	`fans.Find(f => f.Id == 3)`	de fan met Id 3, of `null`
`FindLast`	geef de laatste die voldoet	`fans.FindLast(f => f.Merk == "NZXT")`	de fan met Id 3
`FindAll`	geef allemaal die voldoen	`fans.FindAll(f => f.Merk == "NZXT")`	een nieuwe lijst met fan 1 en 3
`FindIndex`	op welke positie staat de eerste die voldoet	`fans.FindIndex(f => f.Id == 3)`	`2`, of `-1`
`FindLastIndex`	op welke positie staat de laatste die voldoet	`fans.FindLastIndex(f => f.Merk == "NZXT")`	`2`, of `-1`
`BinarySearch`	zoek snel in een **gesorteerde** lijst	`gesorteerd.BinarySearch(8)`	de positie, of een negatief getal

Een stuk nemen of omzetten

Methode	Wat het doet	Voorbeeld	Resultaat
`GetRange`	een kopie van een stuk: vanaf positie, zoveel stuks	`getallen.GetRange(1, 3)`	een nieuwe lijst `8, 3, 8`
`Slice`	hetzelfde als `GetRange`	`getallen.Slice(1, 3)`	een nieuwe lijst `8, 3, 8`
`ConvertAll`	maak van elk item iets anders	`fans.ConvertAll(f => f.Merk)`	een nieuwe `List<string>` met de merken
`ForEach`	doe iets met elk item	`fruit.ForEach(f => Console.WriteLine(f))`	drie regels op het scherm
`ToArray`	maak er een array van	`fans.ToArray()`	een array
`CopyTo`	kopieer naar een bestaande array	`fruit.CopyTo(doel)`	`doel` bevat de drie stuks fruit
`AsReadOnly`	een versie die niemand kan veranderen	`fans.AsReadOnly()`	geen `Add` of `Remove` meer mogelijk

Ordenen

Methode	Wat het doet	Voorbeeld	Resultaat
`Sort`	sorteer de lijst zelf	`getallen.Sort()`	`1, 3, 5, 8, 8`
`Sort`	sorteer op iets dat je zelf kiest	`fans.Sort((a, b) => a.Prijs.CompareTo(b.Prijs))`	goedkoopste eerst
`Reverse`	keer de lijst zelf om	`getallen.Reverse()`	`1, 8, 3, 8, 5`

> **BELANGRIJK**
>
> `Sort` en `Reverse` veranderen je lijst zelf en geven niets terug. De LINQ-methodes hieronder doen het omgekeerde: die laten je lijst met rust en geven een nieuw resultaat. Dat verschil verklaart waarom `lijst.Sort()` werkt en `var x = lijst.Sort()` niet compileert.

Hoeveel plaats er gereserveerd is

Een lijst houdt achter de schermen meer plaats vrij dan ze gebruikt, zodat niet elke `Add` alles moet verhuizen. Je hebt dit zelden nodig.

Methode	Wat het doet	Voorbeeld	Resultaat
`Capacity`	hoeveel er passen voor de lijst moet groeien; een eigenschap	`fruit.Capacity`	minstens `3`
`EnsureCapacity`	reserveer alvast plaats	`fruit.EnsureCapacity(100)`	er passen er minstens 100
`TrimExcess`	geef de vrije plaats terug	`fruit.TrimExcess()`	`Capacity` zakt tot `Count`, als er genoeg vrij was

Wat het ook kan, via iets anders

Een lijst kan veel meer dan de methodes hierboven, omdat ze ook iets anders ís. Ze is een **reeks**: iets wat je één voor één kan doorlopen. In C# heet dat `IEnumerable` (hoofdstuk 12). Alles wat met een reeks werkt, werkt dus ook met een lijst.

Bron	Waarom het op een lijst werkt	Volledige uitleg
LINQ	LINQ werkt op elke reeks	hoofdstuk 11
`foreach`	`foreach` doorloopt elke reeks	hoofdstuk 4
`^` en `..` tussen vierkante haken	een lijst heeft een `Count`, posities en `Slice`	hieronder
`string.Join`	wil een reeks teksten of waarden	hoofdstuk 1
andere verzamelingen	die kan je vullen met een reeks	hoofdstukken 5, 7 tot 10
wat elk object kan	een lijst is ook een object	hoofdstuk 13

Wat je over LINQ moet weten om de tabellen te lezen:

- Een LINQ-methode **verandert je lijst nooit**. Ze geeft een nieuw resultaat.
- Dat resultaat is meestal **geen** `List`, maar een reeks. Wil je er weer een lijst van, zet er dan `.ToList()` achter.
- `Where`, `Select`, `OrderBy` en hun familie rekenen pas op het moment dat je het resultaat gebruikt. Verandert je lijst tussendoor, dan rekenen ze met de nieuwe inhoud. Vraag 8 onderaan toont het.
- LINQ werkt enkel met `using System.Linq;` bovenaan. In een nieuw project staat die er automatisch bij.

> **OPMERKING**
>
> `Index`, `CountBy` en `AggregateBy` bestaan sinds .NET 9, `Shuffle`, `LeftJoin` en `RightJoin` sinds .NET 10. Je cursus werkt met .NET 10, dus je kan ze gebruiken. In oudere voorbeelden op internet kom je ze niet tegen.

Eén item nemen

Komt van LINQ.

Methode	Wat het doet	Voorbeeld	Resultaat	Eigen tegenhanger
`First`	de eerste die voldoet	`fans.First(f => f.Merk == "NZXT")`	fan 1; niets gevonden geeft een `InvalidOperationException`	—
`FirstOrDefault`	de eerste die voldoet, of niets	`fans.FirstOrDefault(f => f.Id == 9)`	`null`	`Find`
`Single`	de enige die voldoet	`fans.Single(f => f.Id == 2)`	fan 2; nul of twee gevonden geeft een `InvalidOperationException`	—
`SingleOrDefault`	de enige die voldoet, of niets	`fans.SingleOrDefault(f => f.Merk == "NZXT")`	`InvalidOperationException`: er zijn er twee	—
`Last`	de laatste die voldoet	`fans.Last(f => f.Merk == "NZXT")`	fan 3	—
`LastOrDefault`	de laatste die voldoet, of niets	`fans.LastOrDefault(f => f.Prijs > 100)`	`null`	`FindLast`
`ElementAt`	het item op een positie	`fans.ElementAt(1)`	fan 2	`fans[1]`
`ElementAtOrDefault`	het item op een positie, of niets	`fans.ElementAtOrDefault(9)`	`null`	—

Neem `FirstOrDefault` wanneer het normaal is dat er niets gevonden wordt, zoals een Id dat niet bestaat. Neem `SingleOrDefault` wanneer er hoogstens één mág zijn: de uitzondering bij twee is dan een alarm over je gegevens. Neem `First` alleen wanneer je zeker weet dat er iets is.

Weten of iets erin zit

Komt van LINQ.

Methode	Wat het doet	Voorbeeld	Resultaat	Eigen tegenhanger
`Any`	is er minstens één	`fans.Any()`	`true`	`Count > 0`
`Any`	is er minstens één die voldoet	`fans.Any(f => f.Prijs > 50)`	`true`	`Exists`
`All`	voldoen ze allemaal	`fans.All(f => f.Prijs > 20)`	`false`	`TrueForAll`
`Contains`	zit deze waarde erin	`getallen.Contains(3)`	`true`	`Contains`: dezelfde
`SequenceEqual`	hebben twee reeksen dezelfde inhoud, in dezelfde volgorde	`getallen.SequenceEqual(new List<int> { 5, 8, 3, 8, 1 })`	`true`	—

> **LET OP**
>
> `All` op een lege lijst geeft `true`: er is geen enkel item dat níet voldoet. Controleer eerst of er wel iets in zit.

Een deel nemen

Komt van LINQ. Het resultaat is telkens een nieuwe reeks; je lijst blijft zoals ze was.

Methode	Wat het doet	Voorbeeld	Resultaat	Eigen tegenhanger
`Where`	enkel wat voldoet	`fans.Where(f => f.Merk == "NZXT")`	fan 1 en 3	`FindAll`
`Select`	maak van elk item iets anders	`fans.Select(f => f.Merk)`	`NZXT, Noctua, NZXT, Arctic`	`ConvertAll`
`SelectMany`	plat een lijst van lijsten	`klassen.SelectMany(k => k.Leerlingen)`	alle leerlingen samen	—
`Distinct`	gooi dubbels weg	`getallen.Distinct()`	`5, 8, 3, 1`	—
`DistinctBy`	één item per waarde van iets	`fans.DistinctBy(f => f.Merk)`	fan 1, 2 en 4	—
`Take`	de eerste n	`getallen.Take(2)`	`5, 8`	`GetRange(0, 2)`
`Skip`	sla de eerste n over	`getallen.Skip(2)`	`3, 8, 1`	—
`TakeLast`	de laatste n	`getallen.TakeLast(2)`	`8, 1`	—
`SkipLast`	alles behalve de laatste n	`getallen.SkipLast(2)`	`5, 8, 3`	—
`TakeWhile`	neem zolang de voorwaarde klopt, stop bij de eerste die niet klopt	`getallen.TakeWhile(g => g > 2)`	`5, 8, 3, 8`	—
`SkipWhile`	sla over zolang de voorwaarde klopt	`getallen.SkipWhile(g => g > 2)`	`1`	—

Methode	Wat het doet	Voorbeeld	Resultaat	Eigen tegenhanger
`Chunk`	verdeel in stukken van n	`getallen.Chunk(2)`	`[5, 8], [3, 8], [1]`	—
`OfType`	enkel de items van een bepaald type	`onderdelen.OfType<Processor>()`	enkel de processors	—
`Index`	elk item samen met zijn positie	`fruit.Index()`	`(0, appel), (1, peer), (2, kiwi)`	—

`OfType` heb je nodig bij overerving: een lijst van een basisklasse waar verschillende soorten in zitten. Zie hoofdstuk 20.

Tellen en rekenen

Komt van LINQ.

Methode	Wat het doet	Voorbeeld	Resultaat	Eigen tegenhanger
`Count`	hoeveel er zijn	`fans.Count()`	`4`	`Count`, zonder haakjes
`Count`	hoeveel er voldoen	`fans.Count(f => f.Prijs > 25)`	`2`	—
`Sum`	tel op	`fans.Sum(f => f.Prijs)`	`164.96`	—
`Min`	de kleinste waarde	`fans.Min(f => f.Prijs)`	`19.99`	—
`Max`	de grootste waarde	`fans.Max(f => f.Prijs)`	`89.99`	—
`Average`	het gemiddelde	`fans.Average(f => f.Prijs)`	`41.24`	—
`MinBy`	het item met de kleinste waarde	`fans.MinBy(f => f.Prijs)`	fan 4	—
`MaxBy`	het item met de grootste waarde	`fans.MaxBy(f => f.Prijs)`	fan 3	—
`CountBy`	tel per waarde van iets	`fans.CountBy(f => f.Merk)`	`NZXT 2, Noctua 1, Arctic 1`	—
`Aggregate`	voeg alles samen met een eigen regel	`fruit.Aggregate((a, b) => a + "-" + b)`	`appel-peer-kiwi`	—
`AggregateBy`	voeg samen per waarde van iets	`fans.AggregateBy(f => f.Merk, 0m, (som, f) => som + f.Prijs)`	`NZXT 114.98, Noctua 29.99, Arctic 19.99`	—

`Max` geeft het getal, `MaxBy` geeft het item. `Sum`, `Average`, `Min` en `Max` op een lijst getallen hebben geen voorwaarde nodig: `getallen.Sum()` geeft `25`.

> **LET OP**
>
> `Average`, `Min` en `Max` op een **lege** lijst van getallen geven een `InvalidOperationException`. `Sum` geeft dan `0`. Bij een lege lijst van teksten of objecten geven `Min` en `Max` `null`.

Ordenen en groeperen

Komt van LINQ. Je lijst blijft in haar oude volgorde; je krijgt een nieuwe, geordende reeks.

Methode	Wat het doet	Voorbeeld	Resultaat	Eigen tegenhanger
`Order`	sorteer de waarden oplopend	`getallen.Order()`	`1, 3, 5, 8, 8`	`Sort`
`OrderDescending`	sorteer de waarden aflopend	`getallen.OrderDescending()`	`8, 8, 5, 3, 1`	—
`OrderBy`	sorteer oplopend op iets	`fans.OrderBy(f => f.Prijs)`	fan 4, 1, 2, 3	`Sort` met een eigen regel
`OrderByDescending`	sorteer aflopend op iets	`fans.OrderByDescending(f => f.Prijs)`	fan 3, 2, 1, 4	—
`ThenBy`	tweede sorteersleutel, oplopend	`fans.OrderBy(f => f.Merk).ThenBy(f => f.Prijs)`	per merk, en binnen een merk de goedkoopste eerst	—
`ThenByDescending`	tweede sorteersleutel, aflopend	`fans.OrderBy(f => f.Merk).ThenByDescending(f => f.Prijs)`	per merk, en binnen een merk de duurste eerst	—
`Reverse`	omgekeerde volgorde	`getallen.AsEnumerable().Reverse()`	`1, 8, 3, 8, 5`	`Reverse`: zie de waarschuwing
`Shuffle`	willekeurige volgorde	`fruit.Shuffle()`	elke keer anders	—
`GroupBy`	maak groepen	`fans.GroupBy(f => f.Merk)`	drie groepen: NZXT met 2 fans, Noctua en Arctic met elk 1	—
`ToLookup`	maak groepen, meteen vastgelegd	`fans.ToLookup(f => f.Merk)`	dezelfde groepen; `opzoek["NZXT"]` geeft fan 1 en 3	—

> **LET OP**
>
> `getallen.Reverse()` roept altijd de `Reverse` van de lijst zelf op: die draait je lijst om en geeft niets terug. Wil je de LINQ-versie, die een nieuwe reeks geeft en je lijst laat zoals ze is, schrijf dan `getallen.AsEnumerable().Reverse()`. `var x = getallen.Reverse();` compileert niet.

Twee reeksen combineren

Komt van LINQ. In de voorbeelden is `a` de lijst `1, 2, 3` en `b` de lijst `3, 4`.

Methode	Wat het doet	Voorbeeld	Resultaat
`Concat`	plak de tweede achter de eerste	`a.Concat(b)`	`1, 2, 3, 3, 4`
`Union`	alles uit beide, zonder dubbels	`a.Union(b)`	`1, 2, 3, 4`
`Intersect`	wat in beide zit	`a.Intersect(b)`	`3`
`Except`	wat in de eerste zit en niet in de tweede	`a.Except(b)`	`1, 2`
`UnionBy`, **`IntersectBy`**, **`ExceptBy`**	hetzelfde, maar vergeleken op één eigenschap	`fans.ExceptBy(new[] { "NZXT" }, f => f.Merk)`	fan 2 en 4
`Append`	één item achteraan, in het resultaat	`a.Append(9)`	`1, 2, 3, 9`
`Prepend`	één item vooraan, in het resultaat	`a.Prepend(0)`	`0, 1, 2, 3`
`Zip`	koppel de eerste aan de eerste, de tweede aan de tweede …	`fruit.Zip(getallen)`	`(appel, 5), (peer, 8), (kiwi, 3)`
`Join`	koppel items uit twee reeksen met dezelfde sleutel	`fans.Join(merken, f => f.Merk, m => m.Naam, (f, m) => ...)`	enkel de paren die bij elkaar horen
`LeftJoin`	zoals `Join`, maar elk item uit de eerste blijft, ook zonder partner	`fans.LeftJoin(merken, ...)`	alle fans, met of zonder merk
`RightJoin`	zoals `Join`, maar elk item uit de tweede blijft	`fans.RightJoin(merken, ...)`	alle merken, met of zonder fan
`GroupJoin`	koppel aan elk item uit de eerste de groep die erbij hoort	`merken.GroupJoin(fans, ...)`	elk merk met zijn fans

`Append` en `Prepend` lijken op `Add` en `Insert`, maar veranderen je lijst niet. `Zip` stopt bij de kortste van de twee: de 8 en de 1 uit `getallen` vallen weg.

Het resultaat vastleggen

Komt van LINQ.

Methode	Wat het doet	Voorbeeld	Resultaat
`ToList`	maak er een nieuwe lijst van	`fans.Where(f => f.Prijs > 25).ToList()`	een `List<Fan>` met fan 2 en 3
`ToArray`	maak er een array van	`getallen.ToArray()`	een array; de lijst heeft ook een eigen `ToArray`
`ToHashSet`	maak er een `HashSet` van: dubbels vallen weg	`getallen.ToHashSet()`	`5, 8, 3, 1`
`ToDictionary`	maak er een opzoektabel van	`fans.ToDictionary(f => f.Id)`	`tabel[3]` geeft fan 3
`ToLookup`	maak er groepen van	`fans.ToLookup(f => f.Merk)`	zie hierboven

> **LET OP**
>
> `ToDictionary` crasht met een `ArgumentException` als twee items dezelfde sleutel hebben. `fans.ToDictionary(f => f.Merk)` crasht dus: NZXT komt twee keer voor.

Doorlopen en posities

Komt van de taal C# zelf.

Schrijfwijze	Wat het doet	Voorbeeld	Resultaat
`foreach`	elk item één keer	`foreach (string f in fruit) { ... }`	appel, peer, kiwi
`for` met `Count`	elke positie	`for (int i = 0; i < fruit.Count; i++) { ... }`	positie 0, 1, 2
`[i]`	het item op positie i	`fruit[0]`	`appel`
`[i] =`	vervang het item op positie i	`fruit[0] = "mango"`	de appel is een mango
`[^1]`	tellen vanaf achteren: het laatste	`fruit[^1]`	`kiwi`
`[1..3]`	een stuk, van positie 1 tot vóór 3	`fruit[1..3]`	een nieuwe lijst `peer, kiwi`
`[..2]`	van het begin tot vóór 2	`fruit[..2]`	een nieuwe lijst `appel, peer`
`[.. a, .. b]`	een nieuwe lijst uit twee lijsten	`List<int> samen = [.. a, .. b];`	`1, 2, 3, 3, 4`

`foreach` en `for` staan volledig in hoofdstuk 4.

Omzetten naar iets anders

Komt van de andere verzamelingen en van `string`.

Schrijfwijze	Wat het doet	Voorbeeld	Resultaat
`string.Join`	zet alles achter elkaar in één tekst	`string.Join(", ", fruit)`	`appel, peer, kiwi`
`new List<T>(lijst)`	een kopie	`new List<string>(fruit)`	een nieuwe lijst met dezelfde inhoud
`new HashSet<T>(lijst)`	een `HashSet`: dubbels vallen weg	`new HashSet<int>(getallen)`	`5, 8, 3, 1`
`new Queue<T>(lijst)`	een wachtrij, in dezelfde volgorde	`new Queue<string>(fruit).Dequeue()`	`appel`
`new Stack<T>(lijst)`	een stapel: het laatste ligt bovenop	`new Stack<string>(fruit).Pop()`	`kiwi`

De `HashSet`, de wachtrij en de stapel staan in de hoofdstukken 8, 9 en 10.

Wat elk object kan

Komt van `object`: elke lijst is ook een object. Zie hoofdstuk 13.

Methode	Wat het doet	Voorbeeld	Resultaat
`ToString`	het object als tekst	`fruit.ToString()`	`System.Collections.Generic.List`1[System.String]`
`Equals` / **`==`**	is dit **dezelfde** lijst	`fruit == new List<string> { "appel", "peer", "kiwi" }`	`false`
`GetType`	welk type is dit	`fruit.GetType().Name`	`List`1`
`GetHashCode`	een getal per object, voor opzoektabellen	`fruit.GetHashCode()`	een getal; zelden zelf nodig

`ToString` geeft de naam van het type, niet de inhoud. Wil je de inhoud zien, gebruik dan `string.Join`. En `==` vergelijkt of het om dezelfde lijst gaat, niet of de inhoud gelijk is. Voor de inhoud is er `SequenceEqual`.

Bestaat ook, maar kom je in deze cursus niet tegen

`AsParallel` (verwerken op meerdere processorkernen), `Cast` (elk item omzetten naar een type, en crashen als dat niet lukt), `DefaultIfEmpty`, `LongCount`, `TryGetNonEnumeratedCount`, `AsQueryable`, `ToAsyncEnumerable`, de methodes die een onveranderlijke kopie maken (`ToImmutableList`, `ToFrozenSet` en hun familie), en de methodes voor XML.

Let op bij een lijst

Een lijst heeft voor veel LINQ-methodes een eigen tegenhanger. Ze lijken op elkaar, maar zijn niet altijd hetzelfde.

Eigen methode	LINQ	Het verschil
`Count`	`Count()`	Hetzelfde getal. `Count` zonder haakjes is een eigenschap van de lijst en meteen bekend; `Count()` met haakjes is LINQ. Een voorwaarde meegeven kan enkel met LINQ: `Count(f => ...)`.
`Find`	`FirstOrDefault`	Hetzelfde resultaat. `FirstOrDefault` werkt op elke verzameling, ook op een databasequery; `Find` niet. Daarom gebruikt de cursus bijna altijd `FirstOrDefault`.
`FindLast`	`LastOrDefault`	Hetzelfde resultaat.
`Exists`	`Any`	Hetzelfde resultaat.
`TrueForAll`	`All`	Hetzelfde resultaat.
`FindAll`	`Where`	`FindAll` geeft meteen een nieuwe `List`. `Where` geeft een reeks die pas rekent als je ze gebruikt, en rekent dan met wat er op dat moment in de lijst zit.
`ConvertAll`	`Select`	Zelfde verschil: `ConvertAll` geeft meteen een `List`, `Select` een reeks.
`GetRange`	`Skip` + `Take`	`GetRange` geeft meteen een kopie als `List`, en crasht als het stuk buiten de lijst valt. `Skip` en `Take` crashen nooit: ze geven minder terug.
`Sort`	`Order`, `OrderBy`	`Sort` verandert je lijst en geeft niets terug. `Order` en `OrderBy` laten je lijst zoals ze is en geven een nieuwe reeks.
`Reverse()`	`Reverse()`	Zelfde naam. Op een lijst wint altijd de eigen `Reverse`: die draait je lijst om en geeft niets terug. Voor de LINQ-versie: `AsEnumerable().Reverse()`.
`ForEach`	`foreach`	`ForEach` is een methode van de lijst; `foreach` werkt op elke reeks. Je mag in geen van beide de lijst zelf veranderen.
`Contains`	`Contains`	Hetzelfde. Bij een lijst van objecten vergelijken ze met `Equals`: zie hieronder.

LET OP

- Het resultaat van `Where`, `Select` of `OrderBy` is geen `List`. `List<Fan> duur = fans.Where(f => f.Prijs > 50);` compileert niet. Zet er `.ToList()` achter.
- `Contains`, `IndexOf`, `Remove` en `Distinct` vergelijken items met `Equals`. Bij een lijst van een `class` betekent dat: is het **hetzelfde object**, niet een object met dezelfde inhoud. Een nieuwe `Fan` met Id 1, merk NZXT en prijs 24.99 zit volgens `Contains` dus níet in de lijst. Bij een `record` wel, want een record vergelijkt op inhoud. Zie hoofdstuk 13 en hoofdstuk 14.
- `FirstOrDefault`, `LastOrDefault`, `SingleOrDefault` en `ElementAtOrDefault` geven bij een lijst van getallen geen `null` maar `0`, en bij een lijst van `bool` geen `null` maar `false`. Bij een lijst van objecten of teksten is het wel `null`.

Wanneer wel, wanneer niet

Wel — de standaardkeuze voor "een aantal dingen". Twijfel je, begin dan met een `List`.

Niet — wanneer je telkens op dezelfde eigenschap zoekt (neem een `Dictionary`), of wanneer dubbels verboden zijn (neem een `HashSet`).

Veelgemaakte fouten

LET OP

- Het is `Count`, niet `Length`.
- Na `Remove` of `RemoveAt` schuift alles erachter een plaats op. Een positie die je vóór het verwijderen onthield, wijst daarna naar iets anders.
- Je mag een lijst niet veranderen terwijl je er met `foreach` doorheen loopt. Dat crasht, ook als je code er onschuldig uitziet.
- `List<string> lijst;` zonder `new` is geen lege lijst maar niets. `lijst.Add` crasht dan.
- `Console.WriteLine(lijst)` toont de naam van het type, niet de inhoud.

Voorspel de uitkomst

Vraag 1

```
var l = new List<int> { 10, 20, 30 };
l.RemoveAt(0);
Console.WriteLine(l[0]);
```

Toon het antwoord ›

Vraag 2

```
var l = new List<string> { "a", "b", "a" };
l.Remove("a");
Console.WriteLine(l.Count);
```

Toon het antwoord ›

Vraag 3

```
var l = new List<int> { 1, 2, 3 };
foreach (int x in l)
{
    if (x == 2)
    {
        l.Remove(x);
    }
}
```

Toon het antwoord ›

Vraag 4

```
var l = new List<string> { "kiwi", "appel", "peer" };
l.Sort();
Console.WriteLine(l[0]);
```

Toon het antwoord ›

Vraag 5

```
var l = new List<int> { 1, 2, 3 };
var omgekeerd = l.OrderByDescending(x => x);
Console.WriteLine(l[0]);
l.Reverse();
Console.WriteLine(l[0]);
```

Toon het antwoord ›

Vraag 6

```
var l = new List<string> { "appel", "peer" };
Console.WriteLine(l);
Console.WriteLine(string.Join(" en ", l));
```

Toon het antwoord ›

Vraag 7

```
var a = new List<int> { 1, 2 };
var b = new List<int> { 1, 2 };
Console.WriteLine(a == b);
Console.WriteLine(a.SequenceEqual(b));
```

Toon het antwoord ›

Vraag 8

```
var l = new List<int> { 1, 2, 3 };
var metFindAll = l.FindAll(x => x > 1);
var metWhere = l.Where(x => x > 1);
l.Add(4);
Console.WriteLine(metFindAll.Count);
Console.WriteLine(metWhere.Count());
```

Toon het antwoord ›

Vraag 9

```
var l = new List<string> { "a", "b", "c", "d" };
Console.WriteLine(l[^1]);
Console.WriteLine(l[1..3].Count);
```

Toon het antwoord ›

Vraag 10

```
var getallen = new List<int>();
Console.WriteLine(getallen.Sum());
Console.WriteLine(getallen.FirstOrDefault());
Console.WriteLine(getallen.Max());
```

Toon het antwoord ›

Antwoorden

VRAAG 1

`20`

De 10 op positie 0 is weg. De 20 schuift op naar positie 0.

‹ Terug naar de vraag

VRAAG 2

`2`

`Remove` haalt enkel het **eerste** voorkomen weg. De lijst is nu `b`, `a`. Wil je ze allemaal weg, dan heb je `RemoveAll` nodig.

‹ Terug naar de vraag

VRAAG 3

`InvalidOperationException` — het programma crasht.

`foreach` merkt dat de lijst veranderd is terwijl hij erdoor liep, en weigert verder te gaan. Wil je tijdens het doorlopen verwijderen, gebruik dan `RemoveAll` met een voorwaarde, of loop door een kopie.

‹ Terug naar de vraag

VRAAG 4

`appel`

`Sort` zet de lijst zelf op alfabetische volgorde. Het geeft niets terug, het verandert de lijst.

‹ Terug naar de vraag

VRAAG 5

`1` en daarna `3`

`OrderByDescending` is LINQ: je lijst blijft `1, 2, 3`, het omgekeerde zit in `omgekeerd`. `Reverse` is een methode van de lijst zelf: die draait de lijst om.

‹ Terug naar de vraag

VRAAG 6

`System.Collections.Generic.List`1[System.String]` en daarna `appel en peer`

`Console.WriteLine` roept `ToString` op, en een lijst geeft daar de naam van haar type. `string.Join` zet de inhoud achter elkaar.

‹ Terug naar de vraag

VRAAG 7

`False` en daarna `True`

`==` vraagt of `a` en `b` dezelfde lijst zijn. Het zijn twee lijsten die toevallig hetzelfde bevatten. `SequenceEqual` vergelijkt de inhoud, item per item.

‹ Terug naar de vraag

VRAAG 8

`2` en daarna `3`

`FindAll` maakte meteen een nieuwe lijst met 2 en 3. Die verandert niet meer. `Where` rekent pas bij `Count()`, en dan zit de 4 er al in. Meer hierover in hoofdstuk 12.

‹ Terug naar de vraag

VRAAG 9

`d` en daarna `2`

`^1` telt vanaf achteren: het laatste item. `1..3` neemt de posities 1 en 2, dus `b` en `c`: de 3 zelf hoort er niet meer bij.

‹ Terug naar de vraag

VRAAG 10

`0`, daarna `0`, en dan `InvalidOperationException` — het programma crasht.

Een lege som is 0. `FirstOrDefault` vindt niets en geeft de standaardwaarde van een `int`: ook 0. Maar de grootste waarde van niets bestaat niet, dus `Max` crasht.

‹ Terug naar de vraag

7. Dictionary

VERZAMELING

Wat het is

Een woordenboek. Je zoekt niet op paginanummer maar op het woord zelf. Elk woord staat er maar één keer in, en bij elk woord hoort een uitleg.

In C# heet het woord de **sleutel** en de uitleg de **waarde**. Tussen de punthaken staan ze allebei: `Dictionary<string, int>` heeft teksten als sleutel en getallen als waarde.

Hoe het eruitziet

```
Dictionary<string, string> hoofdsteden = new Dictionary<string, string>();

hoofdsteden.Add("België", "Brussel");
hoofdsteden["Frankrijk"] = "Parijs";          // toevoegen, of overschrijven

Console.WriteLine(hoofdsteden["België"]);     // Brussel
Console.WriteLine(hoofdsteden.Count);         // 2

if (hoofdsteden.TryGetValue("Spanje", out string? stad))
{
    Console.WriteLine(stad);
}
else
{
    Console.WriteLine("Onbekend land");       // dit wordt getoond
}

foreach (KeyValuePair<string, string> paar in hoofdsteden)
{
    Console.WriteLine($"{paar.Key}: {paar.Value}");
}
```

Bij het doorlopen krijg je telkens een paar terug: `.Key` is de sleutel, `.Value` de waarde. Zo'n paar heet een `KeyValuePair`. Onthoud die naam: alles wat je hieronder via LINQ doet, doe je op die paren.

Het patroon dat je het vaakst zal gebruiken: tellen

```
string[] dieren = { "kat", "hond", "kat", "vis", "kat" };
Dictionary<string, int> telling = new Dictionary<string, int>();

foreach (string dier in dieren)
{
    if (telling.ContainsKey(dier))
    {
        telling[dier]++;
    }
    else
    {
        telling[dier] = 1;
    }
}

Console.WriteLine(telling["kat"]);    // 3
Console.WriteLine(telling["vis"]);    // 1
```

Zodra je ergens *hoeveel van elk* moet weten, is dit het antwoord. Het werkt voor woorden, letters, kleuren, stemmen — alles waar je per soort wil tellen.

Hetzelfde kan korter met `GetValueOrDefault`, uit de tabel hieronder:

```
foreach (string dier in dieren)
{
    telling[dier] = telling.GetValueOrDefault(dier) + 1;
}
```

De voorbeelden in dit hoofdstuk

De tabellen hieronder gebruiken telkens dit woordenboek met ticketprijzen, zodat je elk resultaat kan narekenen.

```
Dictionary<string, int> prijzen = new Dictionary<string, int>
{
    ["dag"]   = 65,
    ["combi"] = 120,
    ["vip"]   = 210
};
```

Wat je ermee kan

Dit zijn de methodes en eigenschappen die op een `Dictionary` zelf zitten.

Toevoegen, veranderen en verwijderen

Methode	Wat het doet	Voorbeeld	Resultaat
`Add`	voeg toe, of loopt stuk bij een dubbele sleutel	`prijzen.Add("kind", 30)`	staat erin; `prijzen.Add("dag", 70)` geeft een `ArgumentException`
`TryAdd`	voeg toe als de sleutel vrij is	`prijzen.TryAdd("dag", 70)`	`false`: dag blijft 65
`[sleutel] =`	voeg toe, of overschrijf	`prijzen["dag"] = 70`	dag is nu 70
`[sleutel] +=`	verander een bestaande waarde	`prijzen["dag"] += 5`	`70`; bij een onbestaande sleutel een `KeyNotFoundException`
`Remove`	haal die sleutel weg	`prijzen.Remove("vip")`	`true`; nog eens geeft `false`
`Remove`	haal weg en geef de oude waarde	`prijzen.Remove("vip", out int oud)`	`true`, `oud` is `210`
`Clear`	maak leeg	`prijzen.Clear()`	`Count` is `0`

Opzoeken

Methode	Wat het doet	Voorbeeld	Resultaat
`[sleutel]`	haal op, of loopt stuk	`prijzen["dag"]`	`65`; `prijzen["kind"]` geeft een `KeyNotFoundException`
`TryGetValue`	haal op en zeg of het lukte	`prijzen.TryGetValue("vip", out int p)`	`true`, `p` is `210`
`GetValueOrDefault`	haal op, of geef de standaardwaarde	`prijzen.GetValueOrDefault("kind")`	`0`
`GetValueOrDefault`	haal op, of geef wat jij kiest	`prijzen.GetValueOrDefault("kind", -1)`	`-1`
`ContainsKey`	bestaat die sleutel	`prijzen.ContainsKey("vip")`	`true`
`ContainsValue`	komt die waarde ergens voor	`prijzen.ContainsValue(120)`	`true`
`Count`	hoeveel paren; een eigenschap	`prijzen.Count`	`3`
`Keys`	alle sleutels	`prijzen.Keys`	`dag, combi, vip`
`Values`	alle waarden	`prijzen.Values`	`65, 120, 210`
`Comparer`	hoe sleutels vergeleken worden	`prijzen.Comparer`	de standaardvergelijking, of wat je bij het aanmaken meegaf

Dezelfde verhouding als bij `Parse` en `TryParse`: de vierkante haken lopen stuk op een sleutel die niet bestaat, `TryGetValue` en `GetValueOrDefault` niet. Weet je niet zeker of de sleutel er is, neem dan een van die twee.

`ContainsKey` is snel, hoe groot het woordenboek ook is. `ContainsValue` moet alle waarden aflopen: een woordenboek zoekt snel op het woord, niet op de uitleg.

Aanmaken en plaats reserveren

Schrijfwijze	Wat het doet	Resultaat
`new Dictionary<string, int>(StringComparer.OrdinalIgnoreCase)`	sleutels zonder onderscheid tussen hoofdletters en kleine letters	`"Kat"` en `"kat"` zijn dezelfde sleutel
`new Dictionary<string, int>(andereTabel)`	een kopie van een ander woordenboek	een nieuw woordenboek met dezelfde paren
`prijzen.Capacity`	hoeveel er passen voor het woordenboek moet groeien	minstens `Count`
`prijzen.EnsureCapacity(100)`	reserveer alvast plaats	er passen er minstens 100
`prijzen.TrimExcess()`	geef de vrije plaats terug	zelden nodig
`prijzen.AsReadOnly()`	een versie die niemand kan veranderen	geen `Add`, geen `Remove`

Twee manieren om toe te voegen

Schrijfwijze	Sleutel bestaat nog niet	Sleutel bestaat al
`d.Add(sleutel, waarde)`	voegt toe	crasht
`d.TryAdd(sleutel, waarde)`	voegt toe, geeft `true`	doet niets, geeft `false`
`d[sleutel] = waarde`	voegt toe	overschrijft

Kies bewust. `Add` is een bewaker: hij zegt je dat er iets dubbel binnenkomt. De vierkante haken zijn soepel: ze overschrijven zonder te klagen.

Bij het aanmaken geldt hetzelfde. `new Dictionary<string, int> { { "a", 1 }, { "a", 2 } }` gebruikt `Add` en crasht. `new Dictionary<string, int> { ["a"] = 1, ["a"] = 2 }` gebruikt de vierkante haken: `a` wordt 2.

Wat het ook kan, via iets anders

Een woordenboek is ook een **reeks**: een reeks **paren**. Elk paar is een `KeyValuePair` met een `.Key` en een `.Value`. Daardoor werkt alles wat met een reeks werkt ook op een woordenboek — maar je werkt dan telkens met paren, niet met losse waarden. Daarnaast zijn `Keys` en `Values` elk een eigen reeks.

Bron	Waarom het op een woordenboek werkt	Volledige uitleg
LINQ	LINQ werkt op elke reeks, hier een reeks paren	hoofdstuk 11
LINQ op `Keys` of `Values`	elk een reeks van losse sleutels of waarden	hoofdstuk 11
`foreach`	doorloopt de paren	hoofdstuk 4
`string.Join`	zet sleutels, waarden of paren in één tekst	hoofdstuk 1
andere verzamelingen	een lijst of set van de sleutels of waarden	hoofdstukken 6 en 8
wat elk object kan	een woordenboek is ook een object	hoofdstuk 13

Wat je over LINQ moet weten om de tabellen te lezen:

- Een LINQ-methode **verandert je woordenboek nooit**. Ze geeft een nieuw resultaat.
- Dat resultaat is **geen woordenboek** maar een reeks paren. Wil je er weer een woordenboek van, zet er dan `.ToDictionary()` achter.
- Wie op de waarden wil werken, schrijft `p => p.Value`; wie op de sleutels wil werken, `p => p.Key`. Of je werkt meteen op `prijzen.Values` of `prijzen.Keys`.
- LINQ werkt enkel met `using System.Linq;` bovenaan. In een nieuw project staat die er automatisch bij.

> **OPMERKING**
>
> `CountBy` en `AggregateBy` bestaan sinds .NET 9, `Shuffle`, `LeftJoin` en `RightJoin` sinds .NET 10. Je cursus werkt met .NET 10, dus je kan ze gebruiken.

Eén paar nemen

Komt van LINQ.

Methode	Wat het doet	Voorbeeld	Resultaat
`First`	het eerste paar dat voldoet	`prijzen.First(p => p.Value > 100)`	`[combi, 120]`; niets gevonden geeft een `InvalidOperationException`
`FirstOrDefault`	het eerste paar dat voldoet, of een **leeg paar**	`prijzen.FirstOrDefault(p => p.Value > 500)`	`[, 0]`: `Key` is `null`, `Value` is `0`
`Single`	het enige paar dat voldoet	`prijzen.Single(p => p.Value > 200)`	`[vip, 210]`
`SingleOrDefault`	het enige paar, of een leeg paar	`prijzen.SingleOrDefault(p => p.Value > 100)`	`InvalidOperationException`: er zijn er twee
`Last`	het laatste paar	`prijzen.Last()`	`[vip, 210]`
`LastOrDefault`	het laatste dat voldoet, of een leeg paar	`prijzen.LastOrDefault(p => p.Value < 0)`	`[, 0]`
`ElementAt`	het paar op een positie	`prijzen.ElementAt(1)`	`[combi, 120]`
`ElementAtOrDefault`	het paar op een positie, of een leeg paar	`prijzen.ElementAtOrDefault(9)`	`[, 0]`

> **LET OP**
>
> `FirstOrDefault` en de andere `...OrDefault`-methodes geven bij een woordenboek **nooit** `null`. Een `KeyValuePair` is een waarde, geen object: niets gevonden geeft een leeg paar, met `null` of `0` als sleutel en de standaardwaarde als waarde. `if (paar == null)` werkt dus niet. Controleer `paar.Key == null`, of gebruik `Any` vooraf. Zoek je op de sleutel, gebruik dan gewoon `TryGetValue`.

Weten of iets erin zit

Komt van LINQ.

Methode	Wat het doet	Voorbeeld	Resultaat	Eigen tegenhanger
`Any`	is er minstens één paar	`prijzen.Any()`	`true`	`Count > 0`
`Any`	is er een paar dat voldoet	`prijzen.Any(p => p.Value > 200)`	`true`	—
`All`	voldoen ze allemaal	`prijzen.All(p => p.Value >= 65)`	`true`	—
`Contains`	zit dit exacte paar erin	`prijzen.Contains(new KeyValuePair<string, int>("dag", 65))`	`true`; met 70 `false`	—
`Keys.Contains`	bestaat die sleutel	`prijzen.Keys.Contains("vip")`	`true`	`ContainsKey`
`Values.Contains`	komt die waarde voor	`prijzen.Values.Contains(120)`	`true`	`ContainsValue`

Een deel nemen

Komt van LINQ. Het resultaat is een reeks paren, geen woordenboek.

Methode	Wat het doet	Voorbeeld	Resultaat
`Where`	enkel de paren die voldoen	`prijzen.Where(p => p.Value > 100)`	`[combi, 120], [vip, 210]`
`Select`	maak van elk paar iets anders	`prijzen.Select(p => $"{p.Key}: {p.Value}")`	`dag: 65, combi: 120, vip: 210`
`SelectMany`	plat de lijsten in de waarden	`klassen.SelectMany(p => p.Value)`	alle leerlingen van alle klassen samen
`Distinct`	gooi dubbels weg, hier op de waarden	`telling.Values.Distinct()`	`3, 1`
`Take`, `Skip`	de eerste n, of alles behalve de eerste n	`prijzen.Take(2)`	`[dag, 65], [combi, 120]`
`TakeWhile`, `SkipWhile`	neem of sla over zolang de voorwaarde klopt	`prijzen.TakeWhile(p => p.Value < 200)`	`[dag, 65], [combi, 120]`
`Chunk`	verdeel in stukken van n paren	`prijzen.Chunk(2)`	twee stukken: twee paren, en één paar

`Take`, `Skip`, `TakeWhile` en `Chunk` volgen de volgorde waarin je het woordenboek doorloopt. Die volgorde is niet gegarandeerd; zie *Let op* hieronder.

Tellen en rekenen

Komt van LINQ.

Methode	Wat het doet	Voorbeeld	Resultaat
`Count`	hoeveel paren voldoen	`prijzen.Count(p => p.Value > 100)`	`2`
`Sum`	tel de waarden op	`prijzen.Sum(p => p.Value)` of `prijzen.Values.Sum()`	`395`
`Min`, `Max`	de kleinste of grootste waarde	`prijzen.Values.Max()`	`210`
`Average`	het gemiddelde	`prijzen.Values.Average()`	`131.666…`
`MinBy`, `MaxBy`	het **paar** met de kleinste of grootste waarde	`prijzen.MaxBy(p => p.Value)`	`[vip, 210]`
`MaxBy(...).Key`	de **sleutel** met de grootste waarde	`telling.MaxBy(p => p.Value).Key`	`kat`
`CountBy`	tel per waarde van iets	`dieren.CountBy(d => d)`	`[kat, 3]`, `[hond, 1]`, `[vis, 1]`: het telpatroon in één regel
`Aggregate`, `AggregateBy`	voeg samen met een eigen regel, eventueel per sleutel	`fans.AggregateBy(f => f.Merk, 0m, (som, f) => som + f.Prijs)`	de totale prijs per merk

Welk dier komt het vaakst voor? is `telling.MaxBy(p => p.Value).Key`. Met `Max` krijg je enkel het getal 3, met `MaxBy` het paar, en met `.Key` erachter de naam.

Ordenen en groeperen

Komt van LINQ. Een woordenboek heeft zelf geen volgorde om te sorteren. Je krijgt een geordende reeks paren.

Methode	Wat het doet	Voorbeeld	Resultaat
`OrderBy`	sorteer oplopend op iets	`prijzen.OrderBy(p => p.Key)`	`[combi, 120]`, `[dag, 65]`, `[vip, 210]`
`OrderByDescending`	sorteer aflopend	`prijzen.OrderByDescending(p => p.Value)`	`[vip, 210]`, `[combi, 120]`, `[dag, 65]`
`ThenBy`, `ThenByDescending`	tweede sorteersleutel	`telling.OrderByDescending(p => p.Value).ThenBy(p => p.Key)`	`[kat, 3]`, `[hond, 1]`, `[vis, 1]`
`Order`, `OrderDescending`	sorteer de waarden zelf	`prijzen.Keys.Order()`	`combi, dag, vip`
`Reverse`	omgekeerde volgorde	`prijzen.Reverse()`	`[vip, 210]`, `[combi, 120]`, `[dag, 65]`
`Shuffle`	willekeurige volgorde	`prijzen.Shuffle()`	elke keer anders
`GroupBy`	maak groepen	`telling.GroupBy(p => p.Value)`	groep 3: kat; groep 1: hond en vis
`ToLookup`	maak groepen, meteen vastgelegd	`telling.ToLookup(p => p.Value, p => p.Key)`	`opzoek[1]` geeft `hond, vis`

> **LET OP**
>
> `prijzen.Order()` zonder sleutel crasht met een `InvalidOperationException`: C# weet niet hoe het twee paren moet vergelijken. Zeg altijd waarop: `OrderBy(p => p.Key)` of `OrderBy(p => p.Value)`.

Twee reeksen combineren

Komt van LINQ. In de voorbeelden is `extra` een tweede woordenboek met `kind = 30` en `dag = 70`.

Methode	Wat het doet	Voorbeeld	Resultaat
`Concat`	plak de paren achter elkaar	`prijzen.Concat(extra)`	vijf paren, `dag` twee keer
`UnionBy`	alles uit beide, één paar per sleutel	`prijzen.UnionBy(extra, p => p.Key)`	`[dag, 65]`, `[combi, 120]`, `[vip, 210]`, `[kind, 30]`
`IntersectBy`	de paren waarvan de sleutel ook in de tweede zit	`prijzen.IntersectBy(extra.Keys, p => p.Key)`	`[dag, 65]`
`ExceptBy`	de paren waarvan de sleutel niet in de tweede zit	`prijzen.ExceptBy(extra.Keys, p => p.Key)`	`[combi, 120]`, `[vip, 210]`
`Union`, `Intersect`, `Except`	hetzelfde op hele paren	`prijzen.Keys.Except(extra.Keys)`	`combi, vip`
`Append`, `Prepend`	één paar achteraan of vooraan, in het resultaat	`prijzen.Append(new KeyValuePair<string, int>("kind", 30))`	vier paren; `prijzen` blijft drie
`Zip`	koppel twee reeksen per positie	`prijzen.Keys.Zip(prijzen.Values)`	`(dag, 65)`, `(combi, 120)`, `(vip, 210)`
`Join`, `LeftJoin`, `RightJoin`, `GroupJoin`	koppel items uit twee reeksen met dezelfde sleutel	`bestellingen.Join(prijzen, b => b.Ticket, p => p.Key, (b, p) => p.Value)`	de prijs per bestelling

Een `Join` met een woordenboek is zelden nodig: met `prijzen[b.Ticket]` zoek je de prijs meteen op.

> **LET OP**
>
> `prijzen.Concat(extra).ToDictionary()` crasht met een `ArgumentException`: `dag` komt twee keer voor. Neem eerst `UnionBy(..., p => p.Key)`: dan wint het eerste woordenboek.

Een woordenboek maken, of er iets anders van maken

Komt van LINQ.

Methode	Wat het doet	Voorbeeld	Resultaat
`ToDictionary`	maak van paren weer een woordenboek	`prijzen.Where(p => p.Value > 100).ToDictionary()`	een `Dictionary` met combi en vip
`ToDictionary`	maak een woordenboek uit een andere reeks	`fruit.ToDictionary(f => f, f => f.Length)`	`[appel, 5], [peer, 4], [kiwi, 4]`
`ToDictionary`	met een eigen vergelijking van de sleutels	`woorden.ToDictionary(w => w, w => 0, StringComparer.OrdinalIgnoreCase)`	sleutels zonder onderscheid in hoofdletters
`GroupBy` + `ToDictionary`	per sleutel een lijst	`fans.GroupBy(f => f.Merk).ToDictionary(g => g.Key, g => g.ToList())`	`tabel["NZXT"]` geeft twee fans
`ToList`	een lijst van paren	`prijzen.ToList()`	`lijst[0]` is `[dag, 65]`
`ToArray`	een array van paren	`prijzen.ToArray()`	een `KeyValuePair<string, int>[]`
`Keys.ToList`, `Values.ToList`	een lijst van de sleutels of waarden	`prijzen.Values.ToList()`	`65, 120, 210`
`Values.ToHashSet`	de verschillende waarden	`telling.Values.ToHashSet()`	`3, 1`

Doorlopen

Komt van de taal C# zelf.

Schrijfwijze	Wat het doet	Resultaat
`foreach (KeyValuePair<string, int> p in prijzen)`	elk paar	`p.Key` en `p.Value`
`foreach (var (ticket, prijs) in prijzen)`	elk paar, meteen in twee variabelen	`ticket` is de sleutel, `prijs` de waarde
`foreach (string ticket in prijzen.Keys)`	elke sleutel	dag, combi, vip
`foreach (int prijs in prijzen.Values)`	elke waarde	65, 120, 210

Een woordenboek heeft **geen positie** zoals een lijst: `prijzen[0]` zoekt naar de **sleutel** 0, niet naar het eerste paar. Bij `string`-sleutels compileert dat niet; bij `int`-sleutels geeft het een `KeyNotFoundException` als er geen sleutel 0 is. Een paar op positie vraag je met `ElementAt`, maar zie *Let op*.

> **BELANGRIJK**
>
> Tijdens een `foreach` mag je **geen paar bijvoegen**: dat crasht met een `InvalidOperationException`. Een bestaande waarde overschrijven (`prijzen[p.Key] = ...`) en een paar weghalen mag wel. Moet je toch toevoegen, loop dan door een kopie: `foreach (string t in prijzen.Keys.ToList())`.

Omzetten naar iets anders

Komt van de andere verzamelingen en van `string`.

Schrijfwijze	Wat het doet	Resultaat
`string.Join(", ", prijzen.Keys)`	de sleutels in één tekst	`dag, combi, vip`
`string.Join(", ", prijzen)`	de paren in één tekst	`[dag, 65], [combi, 120], [vip, 210]`
`new List<string>(prijzen.Keys)`	een lijst van de sleutels	dag, combi, vip
`new HashSet<int>(telling.Values)`	de verschillende waarden	`3, 1`
`new Dictionary<string, int>(prijzen)`	een kopie	een nieuw woordenboek met dezelfde paren

Wat elk object kan

Komt van `object`: elk woordenboek is ook een object. Zie hoofdstuk 13.

Methode	Wat het doet	Voorbeeld	Resultaat
`ToString`	het object als tekst	`prijzen.ToString()`	de naam van het type, `System.Collections.Generic.Dictionary`2[...]`
`Equals` / **`==`**	is dit **hetzelfde** woordenboek	twee woordenboeken met dezelfde paren	`false`
`GetType`, **`GetHashCode`**	type en hashcode	`prijzen.GetType().Name`	`Dictionary`2`

Een **paar** toont zich wel netjes: `new KeyValuePair<string, int>("dag", 65).ToString()` geeft `[dag, 65]`.

Bestaat ook, maar kom je in deze cursus niet tegen

`SortedDictionary` (een woordenboek dat altijd op sleutel gesorteerd blijft), `OrderedDictionary<TKey, TValue>` (houdt de volgorde van toevoegen bij en heeft posities, sinds .NET 9), `ConcurrentDictionary` (voor meerdere taken tegelijk), `ReadOnlyDictionary`, `GetAlternateLookup`, en uit LINQ: `AsParallel`, `Cast`, `ToFrozenDictionary`, `ToImmutableDictionary`.

Let op bij een woordenboek

Op het woordenboek zelf	Via LINQ of iets anders	Het verschil
`ContainsKey`	`Keys.Contains`, `Any(p => p.Key == ...)`	Hetzelfde resultaat, maar `ContainsKey` is snel en `Any` loopt alle paren af.
`ContainsValue`	`Values.Contains`, `Any(p => p.Value == ...)`	Hetzelfde resultaat; alle drie lopen ze alles af.
`TryGetValue`	`FirstOrDefault(p => p.Key == ...)`	`TryGetValue` zoekt snel en zegt of het lukte. `FirstOrDefault` loopt alles af en geeft bij niets een leeg paar, geen `null`.
`Count`	`Count()`	Hetzelfde getal. Een voorwaarde kan enkel met LINQ: `Count(p => p.Value > 100)`.
`Remove`	`Where(p => ...)`	`Remove` haalt echt iets weg. `Where` geeft een nieuwe reeks paren en laat het woordenboek zoals het is.

LET OP

- **De volgorde is niet gegarandeerd.** Vandaag krijg je de paren meestal terug in de volgorde waarin je ze toevoegde, maar na een `Remove` en een nieuwe `Add` kan dat veranderen. `First`, `Last`, `ElementAt`, `Take`, `Skip` en `Reverse` hangen van die volgorde af. Heb je een volgorde nodig, zet er dan eerst een `OrderBy` voor.
- LINQ geeft **paren**, geen woordenboek. `Dictionary<string, int> duur = prijzen.Where(p => p.Value > 100);` compileert niet. Zet er `.ToDictionary()` achter.
- `...OrDefault` geeft een **leeg paar**, nooit `null`.
- Een `null`-sleutel kan niet: `prijzen[null]` en `Add(null, ...)` geven een `ArgumentNullException`. Een `null`-waarde mag wel.
- Een kopie met `new Dictionary<string, int>(tabel)` of `ToDictionary()` neemt de `StringComparer.OrdinalIgnoreCase` van het origineel **niet** mee. Geef hem opnieuw mee als je hem nodig hebt.
- Gebruik je een object van een `class` als **sleutel**, dan vergelijkt het woordenboek op referentie: een nieuw object met dezelfde inhoud vindt het niet. Een `record` als sleutel vergelijkt op inhoud. Zie hoofdstuk 14.

Wanneer wel, wanneer niet

Wel — je zoekt telkens op één en dezelfde eigenschap: een naam, een code, een nummer. En je wil dat snel doen, hoe groot de verzameling ook is.

Niet — wanneer de volgorde ertoe doet, of wanneer je afwisselend op verschillende eigenschappen zoekt. Een woordenboek zoekt snel op het woord, maar niet op de uitleg.

Veelgemaakte fouten

LET OP

- `hoofdsteden["Spanje"]` lezen terwijl Spanje er niet in staat, crasht. Twijfel je, gebruik dan `TryGetValue`, `GetValueOrDefault` of eerst `ContainsKey`.
- Sleutels zijn hoofdlettergevoelig. `"Kat"` en `"kat"` zijn twee verschillende sleutels. Wil je dat niet, geef dan `StringComparer.OrdinalIgnoreCase` mee bij het aanmaken.
- Reken niet op de volgorde bij het doorlopen. Vandaag komen ze toevallig terug in de volgorde waarin je ze toevoegde, maar dat is geen belofte.
- `telling[dier]++` op een sleutel die er nog niet is, crasht: er is nog niets om bij op te tellen.

Voorspel de uitkomst

Vraag 1

```
var d = new Dictionary<string, int>();
d["a"] = 1;
d["a"] = 2;
Console.WriteLine(d["a"]);
Console.WriteLine(d.Count);
```

Toon het antwoord ›

Vraag 2

```
var d = new Dictionary<string, int>();
d.Add("a", 1);
d.Add("a", 2);
```

Toon het antwoord ›

Vraag 3

```
var d = new Dictionary<string, int> { ["Kat"] = 4 };
Console.WriteLine(d.ContainsKey("kat"));
```

Toon het antwoord ›

Vraag 4

```
var d = new Dictionary<string, int> { ["x"] = 5 };
bool gevonden = d.TryGetValue("y", out int waarde);
Console.WriteLine(gevonden);
Console.WriteLine(waarde);
```

Toon het antwoord ›

Vraag 5

```
var d = new Dictionary<string, int> { ["dag"] = 65, ["vip"] = 210 };
var paar = d.FirstOrDefault(p => p.Value > 500);
Console.WriteLine(paar.Key == null);
Console.WriteLine(paar.Value);
```

Toon het antwoord ›

Vraag 6

```
var telling = new Dictionary<string, int> { ["kat"] = 3, ["hond"] = 1 };
Console.WriteLine(telling.Values.Max());
Console.WriteLine(telling.MaxBy(p => p.Value).Key);
```

Toon het antwoord ›

Vraag 7

```
var d = new Dictionary<string, int> { ["a"] = 1, ["b"] = 2 };
foreach (var p in d)
{
    d["c"] = 3;
}
```

Toon het antwoord ›

Vraag 8

```
var d = new Dictionary<string, int>(StringComparer.OrdinalIgnoreCase) { ["Kat"] = 4 };
var kopie = new Dictionary<string, int>(d);
Console.WriteLine(d.ContainsKey("kat"));
Console.WriteLine(kopie.ContainsKey("kat"));
```

Toon het antwoord ›

Vraag 9

```
var d = new Dictionary<string, int> { ["dag"] = 65, ["vip"] = 210 };
var duur = d.Where(p => p.Value > 100).ToDictionary();
Console.WriteLine(duur.Count);
Console.WriteLine(d.Count);
```

Toon het antwoord ›

Antwoorden

VRAAG 1

`2` en daarna `1`

De tweede regel overschrijft de eerste. Er is nog altijd maar één sleutel.

‹ Terug naar de vraag

VRAAG 2

`ArgumentException` — het programma crasht.

`Add` weigert een sleutel die er al in zit. Dat is het verschil met de vierkante haken uit vraag 1.

‹ Terug naar de vraag

VRAAG 3

`False`

`"Kat"` met hoofdletter en `"kat"` zonder zijn voor een gewone `Dictionary` twee verschillende sleutels.

‹ Terug naar de vraag

VRAAG 4

`False` en daarna `0`

De sleutel `"y"` bestaat niet. `TryGetValue` crasht niet maar geeft `false` terug, en zet `waarde` op de standaardwaarde van een `int`: nul.

‹ Terug naar de vraag

VRAAG 5

`True` en daarna `0`

Geen enkel paar voldoet. `FirstOrDefault` geeft dan geen `null`, maar een leeg paar: de sleutel is `null`, de waarde `0`.

‹ Terug naar de vraag

VRAAG 6

`3` en daarna `kat`

`Max` op de waarden geeft het hoogste getal. `MaxBy` geeft het paar met dat getal, en `.Key` de sleutel ervan.

‹ Terug naar de vraag

VRAAG 7

`InvalidOperationException` — het programma crasht.

`d["c"] = 3` voegt een nieuwe sleutel toe terwijl de `foreach` loopt. Een bestaande waarde overschrijven had wel gemogen; iets bijvoegen niet.

‹ Terug naar de vraag

VRAAG 8

`True` en daarna `False`

Het origineel negeert hoofdletters. De kopie kreeg de paren mee, maar niet die manier van vergelijken: daar is `"kat"` weer een andere sleutel dan `"Kat"`.

‹ Terug naar de vraag

VRAAG 9

`1` en daarna `2`

`Where` geeft de paren boven 100, `ToDictionary` maakt er een nieuw woordenboek van met enkel `vip`. Het origineel verandert niet.

‹ Terug naar de vraag

8. HashSet

VERZAMELING

Wat het is

Een verzameling waarin alles maar één keer voorkomt. Zoals een stickeralbum: een dubbele sticker plak je niet opnieuw in, je weet gewoon dat je hem al hebt.

Hoe het eruitziet

```
HashSet<string> bezocht = new HashSet<string>();

bezocht.Add("Gent");
bezocht.Add("Brugge");
bool nieuw = bezocht.Add("Gent");            // false – stond er al in

Console.WriteLine(bezocht.Count);            // 2
Console.WriteLine(bezocht.Contains("Brugge"));   // True
```

Een `HashSet` kan ook rekenen met verzamelingen:

Methode	Wat er overblijft
`a.UnionWith(b)`	alles wat in `a` óf `b` zit
`a.IntersectWith(b)`	enkel wat in `a` én `b` zit
`a.ExceptWith(b)`	wat in `a` zit maar niet in `b`

Die drie veranderen `a` zelf.

De voorbeelden in dit hoofdstuk

De tabellen hieronder gebruiken telkens deze verzamelingen, zodat je elk resultaat kan narekenen.

```
HashSet<string> steden = new HashSet<string> { "Gent", "Brugge", "Antwerpen", "Leuven" };
HashSet<int>    a      = new HashSet<int> { 1, 2, 3, 4 };
HashSet<int>    b      = new HashSet<int> { 3, 4, 5 };
```

Wat je ermee kan

Dit zijn de methodes en eigenschappen die op een `HashSet` zelf zitten.

Toevoegen en verwijderen

Methode	Wat het doet	Voorbeeld	Resultaat
`Add`	voeg toe, als het er nog niet in zit	`steden.Add("Mechelen")`	`true`; `steden.Add("Gent")` geeft `false`
`Remove`	haal dit item weg	`steden.Remove("Brugge")`	`true`; iets wat er niet in zit geeft `false`
`RemoveWhere`	haal alles weg dat voldoet	`steden.RemoveWhere(s => s.Length > 5)`	`3`: enkel Gent blijft over
`Clear`	maak leeg	`steden.Clear()`	`Count` is `0`

Opzoeken

Methode	Wat het doet	Voorbeeld	Resultaat
`Count`	hoeveel zitten erin; een eigenschap	`steden.Count`	`4`
`Contains`	zit dit erin; snel, hoe groot de set ook is	`steden.Contains("Gent")`	`true`; `"gent"` geeft `false`
`TryGetValue`	geef het exemplaar dat al in de set zit	`namen.TryGetValue("BRAM", out string? gevonden)`	`true`, `gevonden` is `Bram` (bij een set die hoofdletters negeert)
`Comparer`	hoe items vergeleken worden	`steden.Comparer`	de standaardvergelijking, of wat je bij het aanmaken meegaf

Rekenen met verzamelingen: de set zelf veranderen

Methode	Wat het doet	Voorbeeld	`a` wordt
`UnionWith`	voeg alles uit de tweede toe	`a.UnionWith(b)`	`1, 2, 3, 4, 5`
`IntersectWith`	hou enkel wat ook in de tweede zit	`a.IntersectWith(b)`	`3, 4`
`ExceptWith`	haal weg wat in de tweede zit	`a.ExceptWith(b)`	`1, 2`
`SymmetricExceptWith`	hou wat in precies één van beide zit	`a.SymmetricExceptWith(b)`	`1, 2, 5`

Deze vier veranderen `a` en geven niets terug. `b` blijft zoals het was. De tweede mag elke reeks zijn, ook een lijst met dubbels: die dubbels komen er niet bij.

Rekenen met verzamelingen: enkel een vraag stellen

Methode	Vraag	Voorbeeld	Resultaat
`IsSubsetOf`	zit alles van mij ook in de tweede	`b.IsSubsetOf(a)`	`false`: de 5 zit niet in `a`
`IsSupersetOf`	zit alles van de tweede ook in mij	`a.IsSupersetOf(new[] { 1, 2 })`	`true`
`IsProperSubsetOf`	deelverzameling, én de tweede heeft meer	`a.IsProperSubsetOf(a)`	`false`: even groot
`IsProperSupersetOf`	ik heb alles van de tweede, én meer	`a.IsProperSupersetOf(new[] { 1, 2 })`	`true`
`Overlaps`	hebben we minstens één gemeen	`a.Overlaps(b)`	`true`
`SetEquals`	dezelfde inhoud, volgorde en dubbels tellen niet	`a.SetEquals(new[] { 4, 3, 2, 1, 1 })`	`true`

Aanmaken, kopiëren en plaats reserveren

Schrijfwijze	Wat het doet	Resultaat
`new HashSet<int>(getallen)`	een set uit een reeks: dubbels vallen weg	uit `5, 8, 3, 8, 1` wordt `5, 8, 3, 1`
`new HashSet<string>(StringComparer.OrdinalIgnoreCase)`	een set die hoofdletters negeert	`"Gent"` en `"GENT"` zijn hetzelfde
`steden.CopyTo(doel)`	kopieer naar een bestaande array	`doel` bevat de vier steden
`steden.Capacity`	hoeveel er passen voor de set moet groeien	minstens `Count`
`steden.EnsureCapacity(100)`	reserveer alvast plaats	er passen er minstens 100
`steden.TrimExcess()`	geef de vrije plaats terug	zelden nodig

Wat het ook kan, via iets anders

Een `HashSet` is ook een **reeks**: iets wat je één voor één kan doorlopen (`IEnumerable`, hoofdstuk 12). Alles wat met een reeks werkt, werkt dus ook met een set — met één belangrijke kanttekening: een set heeft **geen gegarandeerde volgorde**.

Bron	Waarom het op een set werkt	Volledige uitleg
LINQ	LINQ werkt op elke reeks	hoofdstuk 11
`foreach`	`foreach` doorloopt elke reeks	hoofdstuk 4
`string.Join`	wil een reeks	hoofdstuk 1
andere verzamelingen	die kan je vullen met een reeks	hoofdstukken 5 tot 10
wat elk object kan	een set is ook een object	hoofdstuk 13

Wat je over LINQ moet weten om de tabellen te lezen:

- Een LINQ-methode **verandert je set nooit**. Ze geeft een nieuw resultaat.
- Dat resultaat is **geen** `HashSet` maar een reeks. Wil je er weer een set van, zet er dan `.ToHashSet()` achter.
- LINQ werkt enkel met `using System.Linq;` bovenaan. In een nieuw project staat die er automatisch bij.

> **OPMERKING**
>
> `Index`, `CountBy` en `AggregateBy` bestaan sinds .NET 9, `Shuffle`, `LeftJoin` en `RightJoin` sinds .NET 10. Je cursus werkt met .NET 10, dus je kan ze gebruiken.

Eén item nemen

Komt van LINQ. Alles in deze tabel hangt af van de volgorde, en die is bij een set niet gegarandeerd. De resultaten zijn die van vandaag.

Methode	Wat het doet	Voorbeeld	Resultaat
`First`, **`FirstOrDefault`**	het eerste dat voldoet, of niets	`steden.FirstOrDefault(s => s.StartsWith("Z"))`	`null`
`Single`, **`SingleOrDefault`**	het enige dat voldoet	`steden.Single(s => s.Length == 9)`	`Antwerpen`
`Last`, **`LastOrDefault`**	het laatste dat voldoet	`steden.Last(s => s.Length == 6)`	`Leuven`
`ElementAt`, **`ElementAtOrDefault`**	het item op een positie	`steden.ElementAt(0)`	`Gent`, vandaag

`Single` is bij een set het enige item uit deze tabel dat niet van de volgorde afhangt: er is er maar één die voldoet, of het crasht.

Weten of iets erin zit

Komt van LINQ.

Methode	Wat het doet	Voorbeeld	Resultaat	Eigen tegenhanger
`Any`	is er minstens één (die voldoet)	`steden.Any(s => s.Length > 8)`	`true`	`Count > 0`
`All`	voldoen ze allemaal	`steden.All(s => s.Length > 4)`	`false`: Gent heeft er 4	—
`Contains`	zit dit erin	`steden.Contains("Gent")`	`true`	`Contains`: de set gebruikt de eigen, snelle versie
`SequenceEqual`	dezelfde inhoud **in dezelfde volgorde**	`a.SequenceEqual(new[] { 4, 3, 2, 1 })`	`false`	`SetEquals` geeft `true`

Een deel nemen

Komt van LINQ. Het resultaat is een reeks, geen set.

Methode	Wat het doet	Voorbeeld	Resultaat
`Where`	enkel wat voldoet	`steden.Where(s => s.Length == 6)`	`Brugge, Leuven`
`Select`	maak van elk item iets anders	`steden.Select(s => s.Length)`	`4, 6, 9, 6`: dubbels kunnen terug opduiken
`SelectMany`	plat een lijst van lijsten	`sets.SelectMany(s => s)`	alles samen, mét dubbels
`Distinct`, `DistinctBy`	gooi dubbels weg (op iets)	`steden.DistinctBy(s => s.Length)`	`Gent, Brugge, Antwerpen`
`Take`, `Skip`, `TakeLast`, `SkipLast`	de eerste of laatste n, volgens de huidige volgorde	`steden.Order().Take(2)`	`Antwerpen, Brugge`
`TakeWhile`, `SkipWhile`, `Chunk`, `Index`	zoals bij een lijst	`steden.Chunk(2)`	twee stukken van twee
`OfType`	enkel de items van een bepaald type	`onderdelen.OfType<Processor>()`	enkel de processors

Wil je een vaste volgorde, zet er dan eerst `Order()` of `OrderBy(...)` voor, zoals in het voorbeeld bij `Take`.

Tellen en rekenen

Komt van LINQ.

Methode	Wat het doet	Voorbeeld	Resultaat
`Count`	hoeveel er voldoen	`steden.Count(s => s.Length == 6)`	`2`
`Sum`	tel op	`a.Sum()`	`10`
`Min`, `Max`, `Average`	kleinste, grootste, gemiddelde	`steden.Average(s => s.Length)`	`6.25`
`MinBy`, `MaxBy`	het item met de kleinste of grootste waarde	`steden.MaxBy(s => s.Length)`	`Antwerpen`
`CountBy`	tel per waarde van iets	`steden.CountBy(s => s.Length)`	`[4, 1], [6, 2], [9, 1]`
`Aggregate`, `AggregateBy`	voeg samen met een eigen regel	`steden.Aggregate((x, y) => x + "-" + y)`	de vier steden aan elkaar

> **LET OP**
>
> `Average`, `Min` en `Max` op een **lege** set van getallen geven een `InvalidOperationException`. `Sum` geeft dan `0`.

Ordenen en groeperen

Komt van LINQ. Een set zelf heeft geen volgorde om te sorteren; je krijgt een geordende reeks.

Methode	Wat het doet	Voorbeeld	Resultaat
`Order`, `OrderDescending`	sorteer de waarden	`steden.Order()`	`Antwerpen, Brugge, Gent, Leuven`
`OrderBy`, `OrderByDescending`	sorteer op iets	`steden.OrderBy(s => s.Length)`	`Gent, Brugge, Leuven, Antwerpen`
`ThenBy`, `ThenByDescending`	tweede sorteersleutel	`steden.OrderBy(s => s.Length).ThenByDescending(s => s)`	`Gent, Leuven, Brugge, Antwerpen`
`Reverse`, `Shuffle`	omgekeerde of willekeurige volgorde	`steden.Reverse()`	de huidige volgorde omgekeerd
`GroupBy`, `ToLookup`	maak groepen	`steden.GroupBy(s => s.Length)`	4: Gent; 6: Brugge en Leuven; 9: Antwerpen

Twee reeksen combineren

Komt van LINQ. Dit zijn de LINQ-tegenhangers van `UnionWith` en zijn familie. Ze geven een **nieuwe reeks** en laten `a` en `b` zoals ze zijn.

Methode	Wat het doet	Voorbeeld	Resultaat	Eigen tegenhanger
`Union`	alles uit beide, zonder dubbels	`a.Union(b)`	`1, 2, 3, 4, 5`	`UnionWith`
`Intersect`	wat in beide zit	`a.Intersect(b)`	`3, 4`	`IntersectWith`
`Except`	wat in de eerste zit en niet in de tweede	`a.Except(b)`	`1, 2`	`ExceptWith`
`Concat`	plak achter elkaar, **mét** dubbels	`a.Concat(b)`	`1, 2, 3, 4, 3, 4, 5`	—
`UnionBy`, `IntersectBy`, `ExceptBy`	hetzelfde, vergeleken op één eigenschap	`steden.ExceptBy(new[] { 6 }, s => s.Length)`	`Gent, Antwerpen`	—
`Append`, `Prepend`	één item erbij, in het resultaat	`a.Append(3)`	`1, 2, 3, 4, 3`: het resultaat is geen set meer	`Add`
`Zip`	koppel per positie	`steden.Zip(a)`	`(Gent, 1), (Brugge, 2), …`	—
`Join`, `LeftJoin`, `RightJoin`, `GroupJoin`	koppel items met dezelfde sleutel	`bezoekers.Join(steden, ...)`	zoals bij een lijst (hoofdstuk 6)	—

Het resultaat vastleggen

Komt van LINQ.

Methode	Wat het doet	Voorbeeld	Resultaat
`ToHashSet`	maak er (weer) een set van	`steden.Select(s => s.Length).ToHashSet()`	`4, 6, 9`
`ToList`	maak er een lijst van, met posities	`steden.Order().ToList()`	`lijst[0]` is `Antwerpen`
`ToArray`	maak er een array van	`a.ToArray()`	`1, 2, 3, 4`
`ToDictionary`, `ToLookup`	maak er een opzoektabel of groepen van	`steden.ToDictionary(s => s, s => s.Length)`	`tabel["Gent"]` is `4`

> **LET OP**
>
> `ToHashSet()` zonder meer neemt een `StringComparer.OrdinalIgnoreCase` **niet** mee. Geef hem opnieuw mee: `namen.ToHashSet(StringComparer.OrdinalIgnoreCase)`.

Doorlopen

Komt van de taal C# zelf.

Schrijfwijze	Wat het doet	Resultaat
`foreach (string s in steden)`	elk item één keer	de vier steden, in de huidige volgorde
`HashSet<int> samen = [.. a, .. b];`	een nieuwe set uit twee reeksen	`1, 2, 3, 4, 5`
`HashSet<string> s = ["Gent", "Gent"];`	korte schrijfwijze; dubbels vallen weg	`Count` is `1`

Een set heeft **geen positie**: `steden[0]` compileert niet, en een `for` met een teller kan dus niet. Wil je toch posities, maak er dan eerst een lijst van met `ToList()`.

> **BELANGRIJK**
>
> Tijdens een `foreach` mag je **niets bijvoegen**: dat crasht met een `InvalidOperationException`. Wegnemen tijdens het doorlopen doe je beter met `RemoveWhere`.

Omzetten naar iets anders

Komt van de andere verzamelingen en van `string`.

Schrijfwijze	Wat het doet	Resultaat
`string.Join(", ", steden)`	alles in één tekst	`Gent, Brugge, Antwerpen, Leuven`
`new List<string>(steden)`	een lijst	met posities en `Sort`
`new Queue<string>(steden)`	een wachtrij, in de huidige volgorde	`Dequeue` geeft `Gent`
`new Stack<string>(steden)`	een stapel	`Pop` geeft `Leuven`

Wat elk object kan

Komt van `object`: elke set is ook een object. Zie hoofdstuk 13.

Methode	Wat het doet	Voorbeeld	Resultaat
`ToString`	het object als tekst	`steden.ToString()`	`System.Collections.Generic.HashSet`1[System.String]`
`Equals` / `==`	is dit **dezelfde** set	twee sets met dezelfde inhoud	`false`; vergelijk de inhoud met `SetEquals`
`GetType`, **`GetHashCode`**	type en hashcode	`steden.GetType().Name`	`HashSet`1`

Bestaat ook, maar kom je in deze cursus niet tegen

`SortedSet<T>` (een set die altijd gesorteerd blijft, met `Min` en `Max` als eigenschap), `HashSet.CreateSetComparer`, `GetAlternateLookup`, `ToFrozenSet`, `ToImmutableHashSet`, en uit LINQ `AsParallel` en `Cast`.

Let op bij een HashSet

Op de set zelf	Via LINQ	Het verschil
`UnionWith`	`Union`	`UnionWith` verandert de set en geeft niets terug. `Union` geeft een nieuwe reeks en laat beide sets zoals ze zijn. Hetzelfde voor `IntersectWith`/`Intersect` en `ExceptWith`/`Except`.
`SetEquals`	`SequenceEqual`	`SetEquals` kijkt enkel naar de inhoud. `SequenceEqual` kijkt ook naar de volgorde, en die is bij een set niet gegarandeerd. Bij een set wil je dus bijna altijd `SetEquals`.
`Contains`	`Any(s => s == ...)`	Hetzelfde resultaat, maar `Contains` is snel en volgt de vergelijking van de set (bv. zonder hoofdletters). `Any` loopt alles af en vergelijkt met `==`: met hoofdletters.
`Add`	`Append`	`Add` voegt toe als het nog niet bestaat. `Append` zet het er gewoon achter, in een reeks die geen set meer is: dubbels zijn dan mogelijk.
`RemoveWhere`	`Where`	`RemoveWhere` haalt echt weg. `Where` geeft een nieuwe reeks met wat overblijft.
`Count`	`Count()`	Hetzelfde getal. Een voorwaarde kan enkel met LINQ.

LET OP

- **De volgorde is niet gegarandeerd.** Vandaag krijg je de items meestal terug in de volgorde van toevoegen, maar dat is geen belofte. `First`, `ElementAt`, `Take` en `Reverse` hangen daarvan af.
- Een set van objecten van een `class` vergelijkt op **referentie**: twee verschillende objecten met dezelfde inhoud zijn voor de set twee items, en beide `Add`s geven `true`. Een `record` vergelijkt op inhoud: de tweede `Add` geeft `false`. Zie hoofdstuk 13 en hoofdstuk 14.
- `Select` op een set kan **dubbels** opleveren: vier steden geven vier lengtes, waarvan twee keer 6.

Wanneer wel, wanneer niet

Wel — dubbels mogen niet voorkomen, of je moet vaak en snel nagaan of iets er al in zit.

Niet — wanneer de volgorde ertoe doet, of wanneer je iets op een positie wil opvragen. Een `HashSet` heeft geen `[0]`.

Veelgemaakte fouten

LET OP

- Een dubbele `Add` crasht niet, hij geeft gewoon `false` terug. Controleer je dat niet, dan merk je nooit dat er iets dubbel binnenkwam.
- Er bestaat geen eerste, tweede of derde. Wil je op positie opvragen, dan is dit de verkeerde bouwsteen.
- `"Gent"` en `"gent"` zijn twee verschillende items, tenzij je bij het aanmaken `StringComparer.OrdinalIgnoreCase` meegeeft.

Voorspel de uitkomst

Vraag 1

```
var s = new HashSet<int> { 1, 2, 2, 3, 3, 3 };
Console.WriteLine(s.Count);
```

Toon het antwoord ›

Vraag 2

```
var s = new HashSet<string>();
Console.WriteLine(s.Add("a"));
Console.WriteLine(s.Add("a"));
```

Toon het antwoord ›

Vraag 3

```
var a = new HashSet<int> { 1, 2, 3 };
var b = new HashSet<int> { 2, 3, 4 };
a.IntersectWith(b);
Console.WriteLine(a.Count);
```

Toon het antwoord ›

Vraag 4

```
var a = new HashSet<int> { 1, 2, 3 };
var b = new HashSet<int> { 2, 3, 4 };
var samen = a.Union(b);
Console.WriteLine(a.Count);
Console.WriteLine(samen.Count());
```

Toon het antwoord ›

Vraag 5

```
var a = new HashSet<int> { 1, 2, 3 };
Console.WriteLine(a.SetEquals(new[] { 3, 2, 1, 1 }));
Console.WriteLine(a.SequenceEqual(new[] { 3, 2, 1 }));
```

Toon het antwoord ›

Vraag 6

```
var namen = new HashSet<string>(StringComparer.OrdinalIgnoreCase) { "Anna" };
Console.WriteLine(namen.Contains("anna"));
Console.WriteLine(namen.Any(n => n == "anna"));
```

Toon het antwoord ›

Antwoorden

VRAAG 1

`3`

De dubbels worden stilzwijgend genegeerd. Er blijven drie verschillende getallen over.

‹ Terug naar de vraag

VRAAG 2

`True` en daarna `False`

De eerste keer is `"a"` nieuw. De tweede keer niet meer.

‹ Terug naar de vraag

VRAAG 3

`2`

Enkel wat in beide zit, blijft over in `a`: 2 en 3.

‹ Terug naar de vraag

VRAAG 4

`3` en daarna `4`

`Union` is LINQ: `a` blijft zoals het was, het resultaat zit in `samen`. Met `a.UnionWith(b)` was `a` zelf vier groot geworden.

‹ Terug naar de vraag

VRAAG 5

`True` en daarna `False`

`SetEquals` kijkt enkel of dezelfde getallen erin zitten. `SequenceEqual` kijkt ook naar de volgorde, en die is anders.

‹ Terug naar de vraag

VRAAG 6

`True` en daarna `False`

`Contains` gebruikt de vergelijking van de set, die hoofdletters negeert. `Any` vergelijkt met `==`, en voor `==` zijn `"Anna"` en `"anna"` verschillend.

‹ Terug naar de vraag

9. Queue

VERZAMELING

Wat het is

Een wachtrij. Wie eerst komt, wordt eerst geholpen. Je kan niet voorsteken, en je kan niemand uit het midden halen.

Hoe het eruitziet

```
Queue<string> wachtrij = new Queue<string>();

wachtrij.Enqueue("Anna");       // achteraan aansluiten
wachtrij.Enqueue("Bram");
wachtrij.Enqueue("Cleo");

Console.WriteLine(wachtrij.Peek());     // Anna — kijken wie vooraan staat
string volgende = wachtrij.Dequeue();   // Anna — en nu is ze weg
Console.WriteLine(wachtrij.Count);      // 2
```

De voorbeelden in dit hoofdstuk

De tabellen hieronder vertrekken telkens van deze wachtrij: Anna staat vooraan, Cleo achteraan.

```
Queue<string> wachtrij = new Queue<string>();
wachtrij.Enqueue("Anna");
wachtrij.Enqueue("Bram");
wachtrij.Enqueue("Cleo");
```

Wat je ermee kan

Dit zijn alle methodes en eigenschappen die op een `Queue` zelf zitten.

Methode	Wat het doet	Voorbeeld	Resultaat
`Enqueue`	sluit achteraan aan	`wachtrij.Enqueue("Dirk")`	Dirk staat achter Cleo
`Dequeue`	haal de eerste eruit en geef hem terug	`wachtrij.Dequeue()`	`Anna`, en Anna is weg; leeg geeft een `InvalidOperationException`
`Peek`	kijk wie vooraan staat, zonder weg te halen	`wachtrij.Peek()`	`Anna`; leeg geeft een `InvalidOperationException`
`TryDequeue`	haal de eerste eruit als er iemand is	`wachtrij.TryDequeue(out string? eerste)`	`true` en `eerste` is Anna; leeg geeft `false`
`TryPeek`	kijk vooraan als er iemand is	`wachtrij.TryPeek(out string? eerste)`	`true` en `eerste` is Anna; leeg geeft `false`
`Count`	hoeveel staan er; een eigenschap	`wachtrij.Count`	`3`
`Contains`	staat deze persoon in de rij	`wachtrij.Contains("Bram")`	`true`
`Clear`	maak leeg	`wachtrij.Clear()`	`Count` is `0`
`ToArray`	een array, van voor naar achter	`wachtrij.ToArray()`	`Anna, Bram, Cleo`
`CopyTo`	kopieer naar een bestaande array	`wachtrij.CopyTo(doel, 0)`	`doel` begint met Anna, Bram, Cleo
`Capacity`	hoeveel er passen voor de rij moet groeien; een eigenschap	`wachtrij.Capacity`	minstens `3`
`EnsureCapacity`, **`TrimExcess`**	reserveer plaats, of geef ze terug	`wachtrij.EnsureCapacity(100)`	zelden nodig

Er is **geen** `Remove` voor iemand uit het midden, en **geen** `[0]` om op positie te kijken. Dat is geen gemis maar de bedoeling: een wachtrij heeft maar twee plaatsen die tellen, vooraan en achteraan.

Een rij volledig afwerken doe je zo:

```
while (wachtrij.Count > 0)
{
    string klant = wachtrij.Dequeue();
    Console.WriteLine($"{klant} is geholpen");
}
```

Of korter, zonder `Count`: `while (wachtrij.TryDequeue(out string? klant)) { ... }`.

Wat het ook kan, via iets anders

Een wachtrij is ook een **reeks** (`IEnumerable`, hoofdstuk 12): je kan ze van voor naar achter doorlopen zonder iemand weg te halen. Daardoor werkt alles wat met een reeks werkt ook op een wachtrij.

Bron	Waarom het op een wachtrij werkt	Volledige uitleg
LINQ	LINQ werkt op elke reeks	hoofdstuk 11
`foreach`	doorloopt van voor naar achter, zonder iemand weg te halen	hoofdstuk 4
`string.Join`	wil een reeks	hoofdstuk 1
andere verzamelingen	die kan je vullen met een reeks	hoofdstukken 5 tot 10
wat elk object kan	een wachtrij is ook een object	hoofdstuk 13

Wat je over LINQ moet weten om de tabellen te lezen:

- Een LINQ-methode **verandert je wachtrij nooit**: ze haalt niemand weg en zet niemand erbij. `wachtrij.First()` kijkt, zoals `Peek`.
- LINQ overloopt de rij **van voor naar achter**: Anna, Bram, Cleo.
- Het resultaat is **geen wachtrij** maar een reeks. Wil je er weer een wachtrij van, geef het dan mee aan `new Queue<string>(...)`.

> **OPMERKING**
>
> `Index`, `CountBy` en `AggregateBy` bestaan sinds .NET 9, `Shuffle`, `LeftJoin` en `RightJoin` sinds .NET 10. Je cursus werkt met .NET 10, dus je kan ze gebruiken.

Eén persoon nemen of iets nagaan

Komt van LINQ. Niemand verlaat de rij.

Methode	Wat het doet	Voorbeeld	Resultaat	Eigen tegenhanger
`First`, `FirstOrDefault`	de eerste (die voldoet)	`wachtrij.First()`	`Anna`; `Count` blijft 3	`Peek`, `TryPeek`
`Last`, `LastOrDefault`	de laatste (die voldoet)	`wachtrij.Last()`	`Cleo`	—
`Single`, `SingleOrDefault`	de enige die voldoet	`wachtrij.Single(n => n.StartsWith("B"))`	`Bram`	—
`ElementAt`, `ElementAtOrDefault`	wie staat er op plaats n	`wachtrij.ElementAt(1)`	`Bram`	—
`Any`, `All`	staat er iemand (die voldoet), voldoen ze allemaal	`wachtrij.Any(n => n.Length > 4)`	`false`	`Count > 0`
`Contains`	staat deze persoon erin	`wachtrij.Contains("Bram")`	`true`	`Contains`: dezelfde
`SequenceEqual`	dezelfde mensen in dezelfde volgorde	`wachtrij.SequenceEqual(new[] { "Anna", "Bram", "Cleo" })`	`true`	—

Een deel nemen, tellen en rekenen

Komt van LINQ. Het resultaat is een reeks, geen wachtrij.

Methode	Wat het doet	Voorbeeld	Resultaat
`Where`	enkel wie voldoet	`wachtrij.Where(n => n.Contains("a"))`	`Anna, Bram`
`Select`, `SelectMany`	maak van elk iets anders	`wachtrij.Select(n => n.Length)`	`4, 4, 4`
`Distinct`, `DistinctBy`	gooi dubbels weg	`wachtrij.Distinct()`	iemand die twee keer aansloot, telt één keer
`Take`, `Skip`, `TakeLast`, `SkipLast`	de eerste of laatste n	`wachtrij.Take(2)`	`Anna, Bram`: wie straks aan de beurt komt
`TakeWhile`, `SkipWhile`, `Chunk`, `Index`, `OfType`	zoals bij een lijst	`wachtrij.Index()`	`(0, Anna), (1, Bram), (2, Cleo)`
`Count`	hoeveel er voldoen	`wachtrij.Count(n => n.Contains("a"))`	`2`
`Sum`, `Min`, `Max`, `Average`	rekenen	`wachtrij.Max()`	`Cleo`: alfabetisch de laatste
`MinBy`, `MaxBy`, `CountBy`, `Aggregate`, `AggregateBy`	zoals bij een lijst	`wachtrij.Aggregate((a, b) => a + " > " + b)`	`Anna > Bram > Cleo`

Ordenen, groeperen en combineren

Komt van LINQ. De wachtrij zelf blijft in haar volgorde: wie eerst kwam, blijft eerst.

Methode	Wat het doet	Voorbeeld	Resultaat
`Order`, `OrderDescending`, `OrderBy`, `OrderByDescending`, `ThenBy`, `ThenByDescending`	een gesorteerde kopie	`wachtrij.OrderByDescending(n => n)`	`Cleo, Bram, Anna`
`Reverse`	van achter naar voor	`wachtrij.Reverse()`	`Cleo, Bram, Anna`; de rij blijft Anna, Bram, Cleo
`Shuffle`	willekeurige volgorde	`wachtrij.Shuffle()`	elke keer anders
`GroupBy`, `ToLookup`	maak groepen	`wachtrij.GroupBy(n => n[0])`	een groep per beginletter
`Concat`, `Union`, `Intersect`, `Except` en de `By`-varianten	combineer met een andere reeks	`wachtrij.Concat(tweedeRij)`	eerst deze rij, dan de tweede
`Append`, `Prepend`	één persoon achteraan of vooraan, in het resultaat	`wachtrij.Prepend("Zoë")`	`Zoë, Anna, Bram, Cleo`; in de rij zelf steekt niemand voor
`Zip`, `Join`, `LeftJoin`, `RightJoin`, `GroupJoin`	koppel aan een andere reeks	`wachtrij.Zip(loketten)`	elke klant met zijn loket

Het resultaat vastleggen

Komt van LINQ.

Methode	Wat het doet	Voorbeeld	Resultaat
`ToList`	een lijst, van voor naar achter	`wachtrij.ToList()`	`Anna, Bram, Cleo`, met posities
`ToArray`	een array; de wachtrij heeft ook een eigen `ToArray`	`wachtrij.ToArray()`	`Anna, Bram, Cleo`
`ToHashSet`, `ToDictionary`, `ToLookup`	een set, opzoektabel of groepen	`wachtrij.ToHashSet()`	elke naam één keer

Doorlopen en omzetten

Komt van de taal C# zelf en van de andere verzamelingen.

Schrijfwijze	Wat het doet	Resultaat
`foreach (string n in wachtrij)`	iedereen van voor naar achter, **zonder** weg te halen	Anna, Bram, Cleo; `Count` blijft 3
`while (wachtrij.Count > 0) { wachtrij.Dequeue(); }`	iedereen afhandelen, en uit de rij halen	daarna is de rij leeg
`string.Join(", ", wachtrij)`	iedereen in één tekst	`Anna, Bram, Cleo`
`new Queue<string>(lijst)`	een wachtrij uit een lijst of array	de eerste van de lijst staat vooraan
`new Stack<string>(wachtrij)`	een stapel uit de wachtrij	de laatste, Cleo, ligt bovenop
`new Queue<string>(wachtrij.Where(n => n != "Bram"))`	een **nieuwe** rij zonder Bram	`Anna, Cleo`

Die laatste regel is hoe je toch iemand uit het midden haalt: je maakt een nieuwe rij zonder die persoon.

> **BELANGRIJK**
>
> In een `foreach` over de wachtrij mag je **niemand** bijzetten of weghalen: `Enqueue` en `Dequeue` crashen dan allebei met een `InvalidOperationException`. Wie de rij wil afwerken, gebruikt `while` met `Dequeue` of `TryDequeue`.

Wat elk object kan

Komt van `object`. Zie hoofdstuk 13.

Methode	Wat het doet	Voorbeeld	Resultaat
`ToString`	het object als tekst	`wachtrij.ToString()`	`System.Collections.Generic.Queue`1[System.String]`
`Equals` / `==`	is dit **dezelfde** wachtrij	twee rijen met dezelfde mensen	`false`; vergelijk de inhoud met `SequenceEqual`
`GetType`, `GetHashCode`	type en hashcode	`wachtrij.GetType().Name`	`Queue`1`

Bestaat ook, maar kom je in deze cursus niet tegen

`PriorityQueue<TElement, TPriority>` (een wachtrij waarin wie de hoogste prioriteit heeft eerst aan de beurt komt, niet wie eerst kwam), `ConcurrentQueue` (voor meerdere taken tegelijk), en uit LINQ `AsParallel` en `Cast`.

Let op bij een wachtrij

Op de wachtrij zelf	Via LINQ	Het verschil
`Peek`	`First()`	Allebei kijken ze vooraan zonder weg te halen. Bij een lege rij crashen ze allebei.
`TryPeek`	`FirstOrDefault()`	Allebei crashen ze niet bij een lege rij. `FirstOrDefault` geeft dan `null` (of `0` bij getallen); `TryPeek` zegt met `false` dat er niemand is.
`Dequeue`	`First()`	`Dequeue` haalt de eerste **weg**. `First()` kijkt enkel: wie dat verwart, verwerkt dezelfde persoon eindeloos opnieuw.
`Count`	`Count()`	Hetzelfde getal. Een voorwaarde kan enkel met LINQ.
—	`ElementAt(1)`	Een wachtrij heeft geen `[1]`. Via LINQ kan je wel kijken wie op plaats 1 staat.

> **LET OP**
>
> - LINQ en `foreach` veranderen de rij **niet**. Na `wachtrij.Take(2)` staan Anna en Bram er nog: ze zijn niet geholpen.
> - `Reverse()` op een wachtrij geeft een omgekeerde **reeks**. De rij zelf blijft staan zoals ze stond.
> - `Contains` bij een wachtrij van objecten van een `class` vergelijkt op referentie. Zie hoofdstuk 13.

Wanneer wel, wanneer niet

Wel — dingen afhandelen in de volgorde waarin ze binnenkwamen: printopdrachten, berichten, klanten aan een loket.

Niet — zodra je iets in het midden wil bereiken of wil overslaan.

Veelgemaakte fouten

LET OP

- `Dequeue` of `Peek` op een lege wachtrij crasht. Controleer eerst `Count`, of gebruik `TryDequeue`.
- `Peek` kijkt enkel, `Dequeue` haalt weg. Wie ze verwisselt, verwerkt de eerste persoon eindeloos opnieuw.
- `Dequeue` binnen een `foreach` over dezelfde wachtrij crasht. Gebruik `while`.

Voorspel de uitkomst

Vraag 1

```
var q = new Queue<int>();
q.Enqueue(1);
q.Enqueue(2);
q.Enqueue(3);
q.Dequeue();
Console.WriteLine(q.Peek());
```

Toon het antwoord ›

Vraag 2

```
var q = new Queue<int>();
q.Dequeue();
```

Toon het antwoord ›

Vraag 3

```
var q = new Queue<string>();
q.Enqueue("Anna");
q.Enqueue("Bram");
Console.WriteLine(q.First());
Console.WriteLine(q.Count);
```

Toon het antwoord ›

Vraag 4

```
var q = new Queue<string>(new[] { "Anna", "Bram", "Cleo" });
var omgekeerd = q.Reverse();
Console.WriteLine(q.Peek());
Console.WriteLine(omgekeerd.First());
```

Toon het antwoord ›

Vraag 5

```
var q = new Queue<string>(new[] { "Anna", "Bram", "Cleo" });
foreach (string n in q)
{
    q.Dequeue();
}
```

Toon het antwoord ›

Vraag 6

```
var q = new Queue<string>(new[] { "Anna", "Bram", "Cleo" });
q = new Queue<string>(q.Where(n => n != "Bram"));
Console.WriteLine(string.Join(", ", q));
```

Toon het antwoord ›

Antwoorden

VRAAG 1

`2`

De 1 stond vooraan en is weggehaald. Nu staat de 2 vooraan.

‹ Terug naar de vraag

VRAAG 2

`InvalidOperationException` — het programma crasht.

Er staat niemand in de rij.

‹ Terug naar de vraag

VRAAG 3

`Anna` en daarna `2`

`First()` is LINQ: het kijkt vooraan, zoals `Peek`, maar haalt niemand weg.

‹ Terug naar de vraag

VRAAG 4

`Anna` en daarna `Cleo`

`Reverse` geeft een omgekeerde reeks. De wachtrij zelf verandert niet: Anna staat nog altijd vooraan.

‹ Terug naar de vraag

VRAAG 5

`InvalidOperationException` — het programma crasht.

De rij verandert terwijl `foreach` erdoor loopt. Wie iedereen wil afhandelen, schrijft `while (q.Count > 0) { q.Dequeue(); }`.

‹ Terug naar de vraag

VRAAG 6

`Anna, Cleo`

Een wachtrij kan niemand uit het midden halen, maar je kan een nieuwe rij maken zonder Bram. De volgorde van de anderen blijft.

‹ Terug naar de vraag

10. Stack

VERZAMELING

Wat het is

Een stapel borden. Het bord dat je laatst neerlegt, neem je als eerste weer op. Het onderste bord raak je pas als alles erboven weg is.

Hoe het eruitziet

```
Stack<string> stappen = new Stack<string>();

stappen.Push("typ tekst");      // bovenop leggen
stappen.Push("maak vet");
stappen.Push("wis regel");

Console.WriteLine(stappen.Pop());   // wis regel – bovenste eraf
Console.WriteLine(stappen.Peek());  // maak vet – kijken, niet wegnemen
```

De voorbeelden in dit hoofdstuk

De tabellen hieronder vertrekken telkens van deze stapel. *wis regel* werd als laatste neergelegd en ligt dus bovenop; *typ tekst* ligt onderaan.

```
Stack<string> stappen = new Stack<string>();
stappen.Push("typ tekst");
stappen.Push("maak vet");
stappen.Push("wis regel");
```

Wat je ermee kan

Dit zijn alle methodes en eigenschappen die op een `Stack` zelf zitten.

Methode	Wat het doet	Voorbeeld	Resultaat
`Push`	leg bovenop	`stappen.Push("kopieer")`	kopieer ligt nu bovenop
`Pop`	neem de bovenste eraf en geef hem terug	`stappen.Pop()`	`wis regel`, en die is weg; leeg geeft een **`InvalidOperationException`**
`Peek`	kijk wat bovenop ligt, zonder weg te nemen	`stappen.Peek()`	`wis regel`; leeg geeft een **`InvalidOperationException`**
`TryPop`	neem de bovenste als er iets ligt	`stappen.TryPop(out string? bovenste)`	`true` en `bovenste` is wis regel; leeg geeft `false`
`TryPeek`	kijk bovenop als er iets ligt	`stappen.TryPeek(out string? bovenste)`	`true`; leeg geeft `false`
`Count`	hoeveel liggen er; een eigenschap	`stappen.Count`	`3`
`Contains`	ligt dit ergens op de stapel	`stappen.Contains("maak vet")`	`true`
`Clear`	maak leeg	`stappen.Clear()`	`Count` is `0`
`ToArray`	een array, **van boven naar onder**	`stappen.ToArray()`	`wis regel, maak vet, typ tekst`
`CopyTo`	kopieer naar een bestaande array, van boven naar onder	`stappen.CopyTo(doel, 0)`	`doel` begint met wis regel
`Capacity`	hoeveel er passen voor de stapel moet groeien; een eigenschap	`stappen.Capacity`	minstens `3`
`EnsureCapacity`, **`TrimExcess`**	reserveer plaats, of geef ze terug	`stappen.EnsureCapacity(100)`	zelden nodig

Er is **geen** `[0]` en **geen** manier om een bord uit het midden te trekken. Een stapel afbouwen doe je zo:

```
while (stappen.TryPop(out string? stap))
{
    Console.WriteLine($"ongedaan: {stap}");
}
```

Wat het ook kan, via iets anders

Een stapel is ook een **reeks** (`IEnumerable`, hoofdstuk 12). Alles wat met een reeks werkt, werkt dus ook op een stapel — en altijd **van boven naar onder**: eerst wat laatst werd neergelegd.

Bron	Waarom het op een stapel werkt	Volledige uitleg
LINQ	LINQ werkt op elke reeks	hoofdstuk 11
`foreach`	doorloopt van boven naar onder, zonder iets weg te nemen	hoofdstuk 4
`string.Join`	wil een reeks	hoofdstuk 1
andere verzamelingen	die kan je vullen met een reeks	hoofdstukken 5 tot 9
wat elk object kan	een stapel is ook een object	hoofdstuk 13

Wat je over LINQ moet weten om de tabellen te lezen:

- Een LINQ-methode **verandert je stapel nooit**: ze neemt niets weg en legt niets bij. `stappen.First()` kijkt, zoals `Peek`.
- LINQ overloopt de stapel **van boven naar onder**: wis regel, maak vet, typ tekst. Dat is het omgekeerde van de volgorde waarin je ze neerlegde.
- Het resultaat is **geen stapel** maar een reeks.

> **OPMERKING**
>
> `Index`, `CountBy` en `AggregateBy` bestaan sinds .NET 9, `Shuffle`, `LeftJoin` en `RightJoin` sinds .NET 10. Je cursus werkt met .NET 10, dus je kan ze gebruiken.

Eén item nemen of iets nagaan

Komt van LINQ. Er gaat niets van de stapel af.

Methode	Wat het doet	Voorbeeld	Resultaat	Eigen tegenhanger
`First`, `FirstOrDefault`	de bovenste (die voldoet)	`stappen.First()`	`wis regel`; `Count` blijft 3	`Peek`, `TryPeek`
`Last`, `LastOrDefault`	de onderste (die voldoet)	`stappen.Last()`	`typ tekst`	—
`Single`, `SingleOrDefault`	de enige die voldoet	`stappen.Single(s => s.StartsWith("m"))`	`maak vet`	—
`ElementAt`, `ElementAtOrDefault`	het item op plaats n, **van boven geteld**	`stappen.ElementAt(0)`	`wis regel`	—
`Any`, `All`	ligt er iets (dat voldoet), voldoen ze allemaal	`stappen.Any()`	`true`	`Count > 0`

Methode	Wat het doet	Voorbeeld	Resultaat	Eigen tegenhanger
`Contains`	ligt dit op de stapel	`stappen.Contains("maak vet")`	`true`	`Contains`: dezelfde
`SequenceEqual`	dezelfde inhoud in dezelfde volgorde	`stappen.SequenceEqual(new[] { "wis regel", "maak vet", "typ tekst" })`	`true`: van boven naar onder	—

Een deel nemen, tellen en rekenen

Komt van LINQ.

Methode	Wat het doet	Voorbeeld	Resultaat
`Where`	enkel wat voldoet	`stappen.Where(s => s.Contains("e"))`	`wis regel, maak vet, typ tekst`
`Select`, `SelectMany`	maak van elk item iets anders	`stappen.Select(s => s.Length)`	`9, 8, 9`
`Take`, `Skip`, `TakeLast`, `SkipLast`	de bovenste of onderste n	`stappen.Take(2)`	`wis regel, maak vet`: wat straks eerst ongedaan wordt
`Distinct`, `DistinctBy`, `TakeWhile`, `SkipWhile`, `Chunk`, `Index`, `OfType`	zoals bij een lijst, van boven naar onder	`stappen.Index()`	`(0, wis regel), (1, maak vet), (2, typ tekst)`
`Count`	hoeveel er voldoen	`stappen.Count(s => s.Length == 9)`	`2`
`Sum`, `Min`, `Max`, `Average`	rekenen	een stapel getallen 1, 2, 3: `getallen.Sum()`	`6`
`MinBy`, `MaxBy`, `CountBy`, `Aggregate`, `AggregateBy`	zoals bij een lijst	`stappen.Aggregate((a, b) => a + " > " + b)`	`wis regel > maak vet > typ tekst`

Ordenen, groeperen en combineren

Komt van LINQ. De stapel zelf verandert niet.

Methode	Wat het doet	Voorbeeld	Resultaat
`Reverse`	van onder naar boven: de volgorde waarin je ze neerlegde	`stappen.Reverse()`	`typ tekst, maak vet, wis regel`
`Order`, `OrderBy` en hun familie	een gesorteerde kopie	`stappen.Order()`	`maak vet, typ tekst, wis regel`
`Shuffle`	willekeurige volgorde	`stappen.Shuffle()`	elke keer anders
`GroupBy`, `ToLookup`	maak groepen	`stappen.GroupBy(s => s.Length)`	9: wis regel, typ tekst; 8: maak vet
`Concat`, `Union`, `Intersect`, `Except`, `Append`, `Prepend`, `Zip`, `Join` en familie	combineer met een andere reeks	`stappen.Append("start")`	vier items in het resultaat, de stapel blijft drie

Het resultaat vastleggen

Komt van LINQ.

Methode	Wat het doet	Voorbeeld	Resultaat
`ToList`	een lijst, van boven naar onder	`stappen.ToList()`	`lijst[0]` is `wis regel`
`ToArray`	een array; de stapel heeft ook een eigen `ToArray`	`stappen.ToArray()`	`wis regel, maak vet, typ tekst`
`ToHashSet`, `ToDictionary`, `ToLookup`	een set, opzoektabel of groepen	`stappen.ToHashSet()`	elke stap één keer

Doorlopen en omzetten

Komt van de taal C# zelf en van de andere verzamelingen.

Schrijfwijze	Wat het doet	Resultaat
`foreach (string s in stappen)`	alles van boven naar onder, **zonder** weg te nemen	wis regel, maak vet, typ tekst
`while (stappen.TryPop(out string? s)) { ... }`	alles afbouwen	daarna is de stapel leeg
`string.Join(", ", stappen)`	alles in één tekst, van boven naar onder	`wis regel, maak vet, typ tekst`
`new Stack<string>(lijst)`	een stapel uit een lijst: het **laatste** van de lijst ligt bovenop	uit `a, b, c` wordt een stapel met `c` bovenop
`new Stack<string>(stappen)`	een "kopie" van een stapel	de volgorde is **omgekeerd**: typ tekst ligt nu bovenop
`new Stack<string>(stappen.Reverse())`	een echte kopie, in dezelfde volgorde	wis regel ligt weer bovenop
`new Queue<string>(stappen)`	een wachtrij uit de stapel	wis regel staat vooraan

> **BELANGRIJK**
>
> `new Stack<string>(stappen)` maakt geen kopie in dezelfde volgorde. Hij doorloopt de oude stapel van boven naar onder en legt alles één voor één op de nieuwe stapel. Wat bovenaan lag, komt zo onderaan terecht.

> **BELANGRIJK**
>
> In een `foreach` over de stapel mag je **niets** bijleggen of wegnemen: `Push` en `Pop` crashen dan allebei met een `InvalidOperationException`.

Wat elk object kan

Komt van `object`. Zie hoofdstuk 13.

Methode	Wat het doet	Voorbeeld	Resultaat
`ToString`	het object als tekst	`stappen.ToString()`	`System.Collections.Generic.Stack`1[System.String]`
`Equals` / `==`	is dit **dezelfde** stapel	twee stapels met dezelfde inhoud	`false`; vergelijk de inhoud met `SequenceEqual`
`GetType`, `GetHashCode`	type en hashcode	`stappen.GetType().Name`	`Stack`1`

Bestaat ook, maar kom je in deze cursus niet tegen

`ConcurrentStack` (voor meerdere taken tegelijk), `ImmutableStack`, en uit LINQ `AsParallel` en `Cast`.

Let op bij een stapel

Op de stapel zelf	Via LINQ	Het verschil
`Peek`	`First()`	Allebei kijken ze bovenop zonder weg te nemen.
`Pop`	`First()`	`Pop` neemt de bovenste **weg**, `First()` kijkt enkel.
—	`Last()`	Wat onderaan ligt, bereik je enkel via LINQ (of door alles erboven weg te nemen).
`ToArray()`	`Reverse()`	`ToArray` geeft van boven naar onder. Wil je de volgorde waarin je ze neerlegde, gebruik dan `Reverse()`.
`Count`	`Count()`	Hetzelfde getal. Een voorwaarde kan enkel met LINQ.

LET OP

- Alles wat een stapel overloopt — `foreach`, LINQ, `ToArray`, `ToList`, `string.Join` — begint **bovenaan**. Een lijst die je met `Push` vult en daarna met `ToList()` omzet, staat dus omgekeerd.
- `new Stack<T>(stapel)` keert de volgorde om. Een kopie in dezelfde volgorde is `new Stack<T>(stapel.Reverse())`.
- `ElementAt(0)` is de **bovenste**, niet de eerste die je neerlegde.
- Een `Stack` en een `Queue` met dezelfde inhoud worden in omgekeerde volgorde overlopen: de wachtrij van de eerste naar de laatste, de stapel van de laatste naar de eerste.

Wanneer wel, wanneer niet

Wel — ongedaan maken, de terug-knop van een browser, nagaan of haakjes correct openen en sluiten. Overal waar het laatste het eerst weer aan de beurt komt.

Niet — wanneer iedereen eerlijk op zijn beurt moet wachten. Dan is het een `Queue`.

Veelgemaakte fouten

> LET OP
>
> - `Pop` of `Peek` op een lege stapel crasht, net als bij een wachtrij.
> - Een `Stack` en een `Queue` met dezelfde inhoud geven hun items in omgekeerde volgorde terug. Dat is het hele verschil, en het is groot.
> - `Pop` binnen een `foreach` over dezelfde stapel crasht. Gebruik `while` met `TryPop`.

Voorspel de uitkomst

Vraag 1

```
var s = new Stack<int>();
s.Push(1);
s.Push(2);
s.Push(3);
Console.WriteLine(s.Pop() + s.Pop());
```

Toon het antwoord ›

Vraag 2

```
var q = new Queue<char>();
var s = new Stack<char>();

foreach (char c in "abc")
{
    q.Enqueue(c);
    s.Push(c);
}

Console.WriteLine(q.Dequeue());
Console.WriteLine(s.Pop());
```

Toon het antwoord ›

Vraag 3

```
var s = new Stack<string>();
s.Push("a");
s.Push("b");
s.Push("c");
Console.WriteLine(string.Join("", s));
Console.WriteLine(s.First());
Console.WriteLine(s.Count);
```

Toon het antwoord ›

Vraag 4

```
var s = new Stack<string>();
s.Push("onder");
s.Push("boven");
var kopie = new Stack<string>(s);
Console.WriteLine(kopie.Peek());
```

Toon het antwoord ›

Vraag 5

```
var lijst = new List<int> { 1, 2, 3 };
var s = new Stack<int>(lijst);
Console.WriteLine(s.Pop());
Console.WriteLine(s.ToList()[0]);
```

Toon het antwoord ›

Antwoorden

VRAAG 1

`5`

De eerste `Pop` geeft 3, de tweede geeft 2. Samen 5. De 1 ligt nog onderaan.

‹ Terug naar de vraag

VRAAG 2

`a` en daarna `c`

Dezelfde drie letters erin, in dezelfde volgorde. De wachtrij geeft de eerste terug, de stapel de laatste.

‹ Terug naar de vraag

VRAAG 3

`cba`, daarna `c`, en dan `3`

Alles wat een stapel overloopt, begint bovenaan. `First()` kijkt enkel; er gaat niets van de stapel af.

‹ Terug naar de vraag

VRAAG 4

`onder`

De nieuwe stapel overloopt de oude van boven naar onder en legt alles één voor één neer. "boven" gaat eerst, "onder" komt er bovenop. De volgorde is omgekeerd.

‹ Terug naar de vraag

VRAAG 5

`3` en daarna `2`

De lijst wordt van voor naar achter op de stapel gelegd: de 3 ligt bovenop. Na `Pop` ligt de 2 bovenop, en `ToList` begint bovenaan.

‹ Terug naar de vraag

Deel 3 — Zoeken en rekenen

Nu je de verzamelingen kent: hoe haal je er iets uit, en wat krijg je dan eigenlijk terug?

11. LINQ

METHODES

Dit is LINQ. Deze methodes werken op elke verzameling — een lijst, een array, een resultaat van een andere LINQ-methode, en later ook op een databasetabel. Ze veranderen nooit iets aan de bron; ze geven een nieuw resultaat.

Bijna allemaal willen ze een voorwaarde in de vorm `f => f.Prijs > 50`. Lees de pijl als "geef mij, voor elke f, het antwoord op".

LINQ werkt enkel met `using System.Linq;` bovenaan. In een nieuw project staat die er automatisch bij.

De voorbeelden in dit hoofdstuk

De tabellen hieronder gebruiken telkens deze lijsten, zodat je elk resultaat kan narekenen. Dezelfde voorbeelden staan in de hoofdstukken over de verzamelingen.

```
List<string> fruit    = new List<string> { "appel", "peer", "kiwi" };
List<int>    getallen = new List<int> { 5, 8, 3, 8, 1 };
List<Fan>    fans     = new List<Fan>
{
    new Fan { Id = 1, Merk = "NZXT",   Prijs = 24.99m },
    new Fan { Id = 2, Merk = "Noctua", Prijs = 29.99m },
    new Fan { Id = 3, Merk = "NZXT",   Prijs = 89.99m },
    new Fan { Id = 4, Merk = "Arctic", Prijs = 19.99m }
};
```

> **OPMERKING**
>
> `Index`, `CountBy` en `AggregateBy` bestaan sinds .NET 9, `Shuffle`, `LeftJoin`, `RightJoin`, `Enumerable.Sequence` en `Enumerable.InfiniteSequence` sinds .NET 10. Je cursus werkt met .NET 10, dus je kan ze gebruiken. In oudere voorbeelden op internet kom je ze niet tegen.

Ik wil één item

Methode	Wat het doet	Als er niets is	Als er meerdere zijn
`First`	de eerste die voldoet	uitzondering	neemt de eerste
`FirstOrDefault`	de eerste die voldoet	`null`	neemt de eerste
`Single`	de enige die voldoet	uitzondering	uitzondering
`SingleOrDefault`	de enige die voldoet	`null`	uitzondering
`Last`	de laatste die voldoet	uitzondering	neemt de laatste
`LastOrDefault`	de laatste die voldoet	`null`	neemt de laatste
`ElementAt`	het item op een positie	uitzondering	—
`ElementAtOrDefault`	het item op een positie	`null`	—

`fans.FirstOrDefault(f => f.Id == 3)` geeft de fan met Id 3, of `null`. `fans.ElementAt(1)` geeft fan 2, en `fans.ElementAtOrDefault(9)` geeft `null`.

Aan de `...OrDefault`-varianten kan je ook zelf zeggen wat er moet komen als er niets gevonden wordt: `getallen.FirstOrDefault(g => g > 10, -1)` geeft `-1`.

> **BELANGRIJK**
>
> Neem `FirstOrDefault` wanneer het normaal is dat er niets gevonden wordt — een Id dat niet bestaat, bijvoorbeeld. Dan controleer je op `null` en geef je een nette melding.
>
> Neem `SingleOrDefault` wanneer er hoogstens één mág zijn en twee een fout in je gegevens betekent. De uitzondering die je dan krijgt is geen hinder maar een alarm: er staan twee items met hetzelfde Id in je lijst.
>
> Neem `First` alleen wanneer je zeker weet dat er iets is. Anders crasht je API met een `500` op een geval dat eigenlijk een `404` moest zijn.

> **LET OP**
>
> `FirstOrDefault` geeft niet altijd `null`. Bij een lijst van getallen geeft het `0`, en bij een lijst van waarheidswaarden `false`. De standaardwaarde van het type dus, niet noodzakelijk niets. Bij een lijst van objecten is het wel `null`. Bij een `Dictionary` krijg je een leeg paar, en bij een tekst het lege teken `'\0'`.

Ik wil weten of iets bestaat

Methode	Wat het doet	Voorbeeld	Resultaat
`Any`	is er minstens één	`fans.Any()`	`true` als de lijst niet leeg is
`Any`	met voorwaarde	`fans.Any(f => f.Prijs > 50)`	`true` of `false`
`All`	voldoen ze allemaal	`fans.All(f => f.Prijs > 0)`	`true` of `false`
`Contains`	zit deze waarde erin	`maten.Contains(120)`	`true` of `false`
`SequenceEqual`	dezelfde inhoud in dezelfde volgorde	`getallen.SequenceEqual(new[] { 5, 8, 3, 8, 1 })`	`true`

Wil je enkel weten óf er iets is, gebruik dan `Any` en niet `Count() > 0`. `Any` stopt bij het eerste item dat voldoet, `Count` loopt de hele verzameling af.

> **LET OP**
>
> `All` op een lege verzameling geeft `true`. Wiskundig klopt dat — er is geen enkel item dat níet voldoet — maar het is zelden wat je bedoelde. Controleer eerst of er wel iets in zit.

Ik wil een deel van de verzameling

Methode	Wat het doet	Voorbeeld	Resultaat
`Where`	enkel wat voldoet	`fans.Where(f => f.Merk == "NZXT")`	de NZXT-fans
`Select`	maak van elk item iets anders	`fans.Select(f => f.Merk)`	enkel de merken
`Select`	met de positie erbij	`fruit.Select((f, i) => $"{i}: {f}")`	`0: appel, 1: peer, 2: kiwi`
`Distinct`	gooi dubbels weg	`getallen.Distinct()`	`5, 8, 3, 1`
`DistinctBy`	één item per waarde van iets	`fans.DistinctBy(f => f.Merk)`	fan 1, 2 en 4
`Take`	de eerste n	`fans.Take(3)`	drie items
`Skip`	sla de eerste n over	`fans.Skip(3)`	de rest
`TakeLast`, `SkipLast`	de laatste n, of alles behalve de laatste n	`getallen.TakeLast(2)`	`8, 1`
`TakeWhile`	neem zolang de voorwaarde klopt	`getallen.TakeWhile(g => g > 2)`	`5, 8, 3, 8`
`SkipWhile`	sla over zolang de voorwaarde klopt	`getallen.SkipWhile(g => g > 2)`	`1`
`SelectMany`	plat een lijst van lijsten	`borden.SelectMany(b => b.Sloten)`	alle sloten samen
`Chunk`	verdeel in stukken van n	`getallen.Chunk(2)`	`[5, 8], [3, 8], [1]`
`OfType`	enkel de items van een bepaald type	`onderdelen.OfType<Processor>()`	enkel de processors
`Index`	elk item samen met zijn positie	`fruit.Index()`	`(0, appel), (1, peer), (2, kiwi)`

`Where` filtert, `Select` verbouwt. Dat is het hele verschil. Wil je de dure fans, dan is dat `Where`. Wil je van elke fan enkel het merk, dan is dat `Select`. Wil je de merken van de dure fans, dan is dat allebei achter elkaar.

`TakeWhile` stopt bij het eerste item dat níet voldoet, ook al voldoen er verderop nog. `Where` kijkt naar alles. `getallen.Where(g => g > 2)` geeft 5, 8, 3, 8; `getallen.TakeWhile(g => g > 6)` geeft niets, want de eerste, 5, voldoet al niet.

`OfType` hoort bij overerving: zie hoofdstuk 20.

Ik wil tellen of rekenen

Methode	Wat het doet	Voorbeeld	Resultaat
`Count`	hoeveel er zijn	`fans.Count()`	`4`
`Count`	hoeveel er voldoen	`fans.Count(f => f.Prijs > 25)`	`2`
`Sum`	tel op	`fans.Sum(f => f.Prijs)`	`164.96`
`Min`	de kleinste waarde	`fans.Min(f => f.Prijs)`	`19.99`
`Max`	de grootste waarde	`fans.Max(f => f.Prijs)`	`89.99`
`Average`	het gemiddelde	`fans.Average(f => f.Prijs)`	`41.24`
`MinBy`	het item met de kleinste waarde	`fans.MinBy(f => f.Prijs)`	de goedkoopste fan
`MaxBy`	het item met de grootste waarde	`fans.MaxBy(f => f.Prijs)`	de duurste fan
`CountBy`	tel per waarde van iets	`fans.CountBy(f => f.Merk)`	`NZXT 2, Noctua 1, Arctic 1`
`Aggregate`	voeg alles samen met een eigen regel	`fruit.Aggregate((a, b) => a + "-" + b)`	`appel-peer-kiwi`
`AggregateBy`	voeg samen per waarde van iets	`fans.AggregateBy(f => f.Merk, 0m, (som, f) => som + f.Prijs)`	`NZXT 114.98, Noctua 29.99, Arctic 19.99`
`LongCount`	zoals `Count`, voor meer dan twee miljard items	`fans.LongCount()`	`4`

`Max` geeft het getal, `MaxBy` geeft het item. Wil je weten wat de hoogste prijs is, dan is het `Max`. Wil je wéten wélke fan dat is, dan is het `MaxBy`.

Op een lijst getallen heb je geen voorwaarde nodig: `getallen.Sum()` geeft `25` en `getallen.Max()` geeft `8`.

> **LET OP**
>
> `Average`, `Min` en `Max` op een **lege** reeks getallen geven een **`InvalidOperationException`**. `Sum` en `Count` geven dan `0`. Bij een lege reeks teksten of objecten geven `Min` en `Max` `null`.

Ik wil ordenen of groeperen

Methode	Wat het doet	Voorbeeld	Resultaat
`Order`	sorteer de waarden zelf, oplopend	`getallen.Order()`	`1, 3, 5, 8, 8`
`OrderDescending`	sorteer de waarden zelf, aflopend	`getallen.OrderDescending()`	`8, 8, 5, 3, 1`
`OrderBy`	sorteer oplopend op iets	`fans.OrderBy(f => f.Prijs)`	goedkoopste eerst
`OrderByDescending`	sorteer aflopend op iets	`fans.OrderByDescending(f => f.Prijs)`	duurste eerst
`ThenBy`	tweede sorteersleutel	`fans.OrderBy(f => f.Merk).ThenBy(f => f.Prijs)`	per merk, dan op prijs
`ThenByDescending`	tweede sorteersleutel, aflopend	`fans.OrderBy(f => f.Merk).ThenByDescending(f => f.Prijs)`	per merk, duurste eerst
`Reverse`	omgekeerde volgorde	`getallen.AsEnumerable().Reverse()`	`1, 8, 3, 8, 5`
`Shuffle`	willekeurige volgorde	`fruit.Shuffle()`	elke keer anders
`GroupBy`	maak groepen	`fans.GroupBy(f => f.Merk)`	een groep per merk
`ToLookup`	maak groepen, meteen vastgelegd	`fans.ToLookup(f => f.Merk)`	`opzoek["NZXT"]` geeft fan 1 en 3

Een groep uit `GroupBy` heeft een `.Key` (het merk) en is zelf een reeks (de fans van dat merk): `fans.GroupBy(f => f.Merk).Select(g => $"{g.Key}: {g.Count()}")` geeft `NZXT: 2, Noctua: 1, Arctic: 1`.

> **LET OP**
>
> `OrderBy(...).OrderBy(...)` sorteert twee keer, en de tweede keer wint. Voor een tweede sleutel gebruik je `ThenBy`.

Ik wil twee reeksen combineren

In de voorbeelden is `a` de lijst `1, 2, 3` en `b` de lijst `3, 4`.

Methode	Wat het doet	Voorbeeld	Resultaat
`Concat`	plak de tweede achter de eerste	`a.Concat(b)`	`1, 2, 3, 3, 4`
`Union`	alles uit beide, zonder dubbels	`a.Union(b)`	`1, 2, 3, 4`
`Intersect`	wat in beide zit	`a.Intersect(b)`	`3`
`Except`	wat in de eerste zit en niet in de tweede	`a.Except(b)`	`1, 2`
`UnionBy`, `IntersectBy`, `ExceptBy`	hetzelfde, vergeleken op één eigenschap	`fans.ExceptBy(new[] { "NZXT" }, f => f.Merk)`	fan 2 en 4
`Append`	één item achteraan, in het resultaat	`a.Append(9)`	`1, 2, 3, 9`
`Prepend`	één item vooraan, in het resultaat	`a.Prepend(0)`	`0, 1, 2, 3`
`Zip`	koppel per positie	`fruit.Zip(getallen)`	`(appel, 5), (peer, 8), (kiwi, 3)`
`Join`	koppel items met dezelfde sleutel	`fans.Join(merken, f => f.Merk, m => m.Naam, (f, m) => ...)`	enkel de paren die bij elkaar horen
`LeftJoin`	zoals `Join`, elk item uit de eerste blijft	`fans.LeftJoin(merken, ...)`	alle fans, met of zonder merk
`RightJoin`	zoals `Join`, elk item uit de tweede blijft	`fans.RightJoin(merken, ...)`	alle merken, met of zonder fan
`GroupJoin`	koppel aan elk item de groep die erbij hoort	`merken.GroupJoin(fans, ...)`	elk merk met zijn fans

`Zip` stopt bij de kortste van de twee. `Union`, `Intersect` en `Except` gooien ook de dubbels binnen één reeks weg: `new[] { 1, 1 }.Union(new[] { 2 })` geeft 1, 2.

Ik wil het resultaat vastleggen

Methode	Wat het doet	Voorbeeld	Resultaat
`ToList`	maak er een echte lijst van	`fans.Where(f => f.Prijs > 50).ToList()`	een `List`
`ToArray`	maak er een array van	`fans.ToArray()`	een array
`ToDictionary`	maak er een opzoektabel van	`fans.ToDictionary(f => f.Id)`	een `Dictionary`
`ToHashSet`	maak er een set van: dubbels vallen weg	`getallen.ToHashSet()`	`5, 8, 3, 1`
`ToLookup`	maak er groepen van	`fans.ToLookup(f => f.Merk)`	zie hierboven

BELANGRIJK

`Where`, `Select` en `OrderBy` voeren op het moment dat je ze schrijft nog niets uit. Wat dat betekent, en waarom het je kan verrassen, is het onderwerp van het volgende hoofdstuk. De vuistregel voor nu: wat je aan de client van een API teruggeeft, geef je pas terug na een `ToList()`.

LET OP

`ToDictionary` crasht met een `ArgumentException` als twee items dezelfde sleutel hebben. `fans.ToDictionary(f => f.Merk)` crasht: NZXT komt twee keer voor.

Ik wil zelf een reeks maken

Methode	Wat het doet	Voorbeeld	Resultaat
`Enumerable.Range`	getallen van een begin, zoveel stuks	`Enumerable.Range(1, 5)`	`1, 2, 3, 4, 5`
`Enumerable.Sequence`	getallen van een begin tot en met een einde, met een stap	`Enumerable.Sequence(1, 10, 3)`	`1, 4, 7, 10`
`Enumerable.InfiniteSequence`	getallen zonder einde, met een stap; altijd samen met `Take`	`Enumerable.InfiniteSequence(0, 5).Take(4)`	`0, 5, 10, 15`
`Enumerable.Repeat`	dezelfde waarde een aantal keer	`Enumerable.Repeat("x", 3)`	`x, x, x`
`Enumerable.Empty`	een lege reeks	`Enumerable.Empty<int>()`	niets
`DefaultIfEmpty`	een lege reeks vervangen door één standaardwaarde	`new List<int>().DefaultIfEmpty()`	`0`
`Cast`	elk item omzetten naar een type, of crashen	`rooster.Cast<int>()`	een reeks `int` uit een rooster `int[,]`

`Enumerable.Range(1, 5)` lees je als *begin bij 1, geef er 5*, niet als *van 1 tot 5*. `Enumerable.Range(3, 2)` geeft dus 3 en 4.

`Enumerable.Sequence` werkt wel met een einde, en dat einde telt mee. Gaat de stap de verkeerde kant op, zoals `Enumerable.Sequence(1, 10, -1)`, dan krijg je een `ArgumentOutOfRangeException`.

Waar LINQ op werkt

LINQ werkt op alles wat je één voor één kan doorlopen (`IEnumerable`, hoofdstuk 12). Dat is meer dan een lijst. Per soort is er iets anders om op te letten.

Waarop	Wat elk item is	Waar je op let	Hoofdstuk
`List`	een item	De lijst heeft eigen tegenhangers (`Find`, `Exists`, `Sort`, `Reverse`). `lijst.Reverse()` draait de lijst zelf om.	6
array	een item	`Length` in plaats van `Count`; `array.Reverse()` is wél LINQ en geeft een nieuwe reeks.	5
`Dictionary`	een paar (`Key`, `Value`)	`...OrDefault` geeft een leeg paar, nooit `null`. Werk op `p.Value`, of op `Keys` of `Values`.	7
`HashSet`	een item	De volgorde is niet gegarandeerd. `Union` en zijn familie veranderen de set niet; `UnionWith` wel.	8
`Queue`	een item, van voor naar achter	LINQ haalt niemand uit de rij: `First()` is zoals `Peek`.	9
`Stack`	een item, **van boven naar onder**	Alles begint bij wat laatst werd neergelegd.	10
`string`	een teken (`char`)	Het resultaat is geen tekst. Maak er weer een van met `string.Concat`.	1
een LINQ-resultaat	een item	Rekent pas als je het gebruikt, en telkens opnieuw.	12
een `DbSet` (EF Core)	een rij uit de database	Wordt een SQL-query. Gebruik de async-varianten (`ToListAsync`, `FirstOrDefaultAsync`, ...), en niet elke C#-methode kan naar SQL.	32

Let op bij LINQ

Eigen methode van de verzameling	LINQ	Het verschil
`lijst.Count`, `array.Length`	`Count()`	Hetzelfde getal. Zonder haakjes is meteen bekend; `Count()` met een voorwaarde kan enkel met LINQ.
`Find`, `Array.Find`	`FirstOrDefault`	Hetzelfde resultaat. `FirstOrDefault` werkt op elke reeks, ook op een databasequery.
`Exists`, `TrueForAll`	`Any`, `All`	Hetzelfde resultaat.
`FindAll`	`Where`	`FindAll` geeft meteen een nieuwe lijst. `Where` rekent pas als je het resultaat gebruikt.
`Sort`	`Order`, `OrderBy`	`Sort` verandert de lijst zelf en geeft niets terug. `OrderBy` laat de lijst zoals ze is.
`lijst.Reverse()`	`Reverse()`	Op een `List` wint altijd de eigen `Reverse`: die draait de lijst om. Op een array en de andere verzamelingen is `Reverse()` LINQ.
`UnionWith`	`Union`	`UnionWith` verandert de set, `Union` geeft een nieuwe reeks.

LET OP

- Het resultaat van `Where`, `Select` of `OrderBy` is geen lijst. `List<Fan> duur = fans.Where(f => f.Prijs > 50);` compileert niet. Zet er `.ToList()` achter.
- `Contains`, `Distinct`, `Union`, `Intersect` en `Except` vergelijken met `Equals`. Bij objecten van een `class` betekent dat: **hetzelfde object**, niet een object met dezelfde inhoud. Een `record` vergelijkt op inhoud. Zie hoofdstuk 13 en hoofdstuk 14. Wil je op één eigenschap vergelijken, gebruik dan de `By`-varianten: `DistinctBy`, `ExceptBy`.
- `Order()` zonder sleutel op objecten die C# niet kan vergelijken (een eigen klasse, of een paar uit een `Dictionary`) crasht met een `InvalidOperationException`. Zeg waarop: `OrderBy(f => f.Prijs)`.
- LINQ op een lege reeks: `First`, `Single`, `Min`, `Max` en `Average` crashen, `FirstOrDefault`, `Sum`, `Count`, `Any` en `All` niet.

Bestaat ook, maar kom je in deze cursus niet tegen

`AsParallel` (verwerken op meerdere processorkernen), `AsQueryable`, `TryGetNonEnumeratedCount`, `ToImmutableList`, `ToFrozenSet` en hun familie.

Er bestaat ook een tweede schrijfwijze voor LINQ, die op SQL lijkt. Je zal ze tegenkomen in oudere voorbeelden:

```
var duur = from f in fans
           where f.Prijs > 25
           orderby f.Prijs
           select f.Merk;
```

Dat is precies hetzelfde als `fans.Where(f => f.Prijs > 25).OrderBy(f => f.Prijs).Select(f => f.Merk)`. De cursus gebruikt de schrijfwijze met de methodes.

Voorspel de uitkomst

Vraag 1

```
var getallen = new List<int> { 5, 8, 3, 8 };
Console.WriteLine(getallen.First(g => g > 4));
Console.WriteLine(getallen.Last(g => g > 4));
```

Toon het antwoord ›

Vraag 2

```
var getallen = new List<int> { 5, 8, 3 };
Console.WriteLine(getallen.FirstOrDefault(g => g > 10));
```

Toon het antwoord ›

Vraag 3

```
var getallen = new List<int> { 5, 8, 3, 8 };
var x = getallen.SingleOrDefault(g => g == 8);
```

Toon het antwoord ›

Vraag 4

```
var leeg = new List<int>();
Console.WriteLine(leeg.All(g => g > 100));
Console.WriteLine(leeg.Any());
```

Toon het antwoord ›

Vraag 5

```
var namen = new List<string> { "Bram", "Anna", "Cleo" };
var gesorteerd = namen.OrderBy(n => n);
Console.WriteLine(namen[0]);
```

Toon het antwoord ›

Vraag 6

```
var getallen = new List<int> { 1, 2, 3, 4 };
Console.WriteLine(getallen.Where(g => g > 2).Count());
Console.WriteLine(getallen.Select(g => g > 2).Count());
```

Toon het antwoord ›

Vraag 7

```
var woorden = new List<string> { "kat", "olifant", "muis" };
Console.WriteLine(woorden.Max(w => w.Length));
Console.WriteLine(woorden.MaxBy(w => w.Length));
```

Toon het antwoord ›

Vraag 8

```
var namen = new List<string> { "Bram", "Anna", "Bert", "Axel" };
Console.WriteLine(string.Join(", ", namen.OrderBy(n => n[0])));
Console.WriteLine(string.Join(", ", namen.OrderBy(n => n[0]).ThenBy(n => n)));
```

Toon het antwoord ›

Vraag 9

```
var getallen = new List<int> { 5, 8, 3, 8, 1 };
Console.WriteLine(getallen.Where(g => g > 4).Count());
Console.WriteLine(getallen.TakeWhile(g => g > 4).Count());
```

Toon het antwoord ›

Vraag 10

```
var r = Enumerable.Range(3, 4);
Console.WriteLine(string.Join(", ", r));
```

Toon het antwoord ›

Vraag 11

```
var woorden = new List<string> { "kat", "hond", "kat", "vis", "kat" };
var top = woorden.GroupBy(w => w).MaxBy(g => g.Count());
Console.WriteLine(top.Key);
Console.WriteLine(top.Count());
```

Toon het antwoord ›

Antwoorden

VRAAG 1

`5` en daarna `8`

Groter dan 4 zijn 5, 8 en 8. `First` neemt de eerste daarvan, `Last` de laatste.

‹ Terug naar de vraag

VRAAG 2

`0`

Er is geen getal groter dan 10. `FirstOrDefault` geeft dan de standaardwaarde van het type terug, en bij een `int` is dat nul, niet `null`. Pas op: een 0 kan hier dus zowel "niets gevonden" als "ik vond een nul" betekenen.

‹ Terug naar de vraag

VRAAG 3

`InvalidOperationException` — het programma crasht.

Er zijn twee achten. `SingleOrDefault` mag er nul of één vinden, twee is voor hem een fout. Dat is precies waarvoor je hem gebruikt: als alarm wanneer iets dat uniek moet zijn het niet is.

‹ Terug naar de vraag

VRAAG 4

`True` en daarna `False`

In een lege lijst is er geen enkel getal dat níet groter is dan 100, dus `All` zegt `true`. Tegelijk zit er niets in, dus `Any` zegt `false`. Twee antwoorden die elkaar lijken tegen te spreken, en allebei correct.

‹ Terug naar de vraag

VRAAG 5

`Bram`

`OrderBy` laat de oorspronkelijke lijst met rust. Het gesorteerde resultaat zit in `gesorteerd`, niet in `namen`. Vergelijk met `Sort` in hoofdstuk 6, die de lijst zelf verandert.

‹ Terug naar de vraag

VRAAG 6

`2` en daarna `4`

`Where` houdt enkel 3 en 4 over. `Select` houdt alles over, maar maakt van elk getal een ja of nee: `false`, `false`, `true`, `true`. Vier antwoorden, dus 4.

‹ Terug naar de vraag

VRAAG 7

`7` en daarna `olifant`

`Max` geeft de grootste lengte. `MaxBy` geeft het woord dat die lengte heeft.

‹ Terug naar de vraag

VRAAG 8

`Anna, Axel, Bram, Bert` en daarna `Anna, Axel, Bert, Bram`

`OrderBy(n => n[0])` sorteert enkel op de eerste letter. Namen met dezelfde eerste letter blijven in hun oorspronkelijke volgorde: Bram stond vóór Bert. `ThenBy` sorteert binnen elke letter verder op de hele naam.

‹ Terug naar de vraag

VRAAG 9

`3` en daarna `2`

`Where` kijkt naar alle getallen: 5, 8 en 8 zijn groter dan 4. `TakeWhile` stopt bij het eerste getal dat niet voldoet, de 3, en houdt dus enkel 5 en 8.

‹ Terug naar de vraag

VRAAG 10

`3, 4, 5, 6`

`Range(3, 4)` betekent: begin bij 3 en geef er 4. Niet: van 3 tot 4.

‹ Terug naar de vraag

VRAAG 11

`kat` en daarna `3`

`GroupBy` maakt een groep per woord. `MaxBy` kiest de groep met de meeste items. `.Key` is het woord van die groep, `Count()` het aantal.

‹ Terug naar de vraag

12. IEnumerable

VERZAMELING

Wat het is

Geen verzameling, maar een belofte: "je kan mij één voor één doorlopen". Zoals een playlist: je weet dat je nummer na nummer kan afspelen, maar niet noodzakelijk hoeveel er zijn of wat het vijfde is.

Elke verzameling uit deel 2 is óók een `IEnumerable`. Een array, een `List`, een `HashSet` — ze kunnen allemaal doorlopen worden. En alles wat LINQ teruggeeft, zoals het resultaat van `Where`, is een `IEnumerable`.

Hoe het eruitziet

```
List<int> getallen = new List<int> { 1, 2, 3, 4 };

IEnumerable<int> even = getallen.Where(g => g % 2 == 0);

getallen.Add(6);

Console.WriteLine(even.Count());    // 3 — de 6 telt mee!
```

Dat laatste verrast iedereen de eerste keer. `even` is geen lijst met 2 en 4. Het is een vraag — *geef mij de even getallen uit die lijst* — die pas gesteld wordt op het moment dat je het antwoord nodig hebt. En op dat moment zit de 6 er al in.

Wat je ermee kan

Een kale `IEnumerable<T>` belooft maar één ding: dat je hem van voor naar achter kan doorlopen. Daarvoor heeft hij één methode, `GetEnumerator`, en die gebruik je nooit zelf: `foreach` doet het voor jou.

Wat je wil	Op een `IEnumerable`	Wat je in de plaats doet
alles doorlopen	`foreach (int g in reeks)`	dit is waarvoor hij bestaat
weten hoeveel	**Niet**: `reeks.Count` compileert niet	`reeks.Count()`, met haakjes: LINQ telt ze
het item op positie 0	**Niet**: `reeks[0]` compileert niet	`reeks.First()` of `reeks.ElementAt(0)`
iets bijvoegen	**Niet**: er is geen `Add`	`reeks.Append(9)` geeft een nieuwe reeks; of eerst `ToList()` en dan `Add`
iets weghalen	**Niet**: er is geen `Remove`	`reeks.Where(g => g != 3)` geeft een nieuwe reeks zonder 3
sorteren	**Niet**: er is geen `Sort`	`reeks.Order()` of `reeks.OrderBy(...)`

Alles in de rechterkolom komt van LINQ. Zonder LINQ kan je met een kale reeks enkel `foreach` doen.

Welke types een IEnumerable zijn

Alles wat je kan doorlopen. Veel types beloven ook méér. Die extra beloftes hebben een eigen naam, en je komt ze tegen als parameter of als returntype.

Interface	Belooft bovenop het doorlopen	Wie het belooft
`IEnumerable<T>`	niets: enkel doorlopen	alles hieronder, plus `string` (een reeks `char`) en elk LINQ-resultaat
`IReadOnlyCollection<T>`	`Count`	`List`, array, `Dictionary`, `HashSet`, `Queue`, `Stack`
`IReadOnlyList<T>`	`Count` en `[i]` om te lezen	`List`, array
`ICollection<T>`	`Count`, `Add`, `Remove`, `Contains`, `Clear`	`List`, array, `Dictionary` (van paren), `HashSet`
`IList<T>`	alles van `ICollection<T>`, plus `[i]`, `Insert`, `RemoveAt`, `IndexOf`	`List`, array
`IQueryable<T>`	wordt een databasequery in plaats van in het geheugen te lopen	een `DbSet` in EF Core: zie hoofdstuk 32

> **LET OP**
>
> Een array belooft `ICollection<T>` en `IList<T>`, maar kan niet groeien. Roep je via die belofte `Add` of `Insert` op, dan crasht het met een `NotSupportedException`. Vervangen met `[i] =` mag wel.

Een `Queue` en een `Stack` beloven geen `ICollection<T>`: je kan er niets in steken via `Add`, enkel via hun eigen `Enqueue` en `Push`.

Wat het ook kan, via iets anders

Bron	Wat het geeft	Volledige uitleg
LINQ	alle methodes om te zoeken, kiezen, tellen, ordenen en combineren	hoofdstuk 11
`foreach`	alles één keer doorlopen	hoofdstuk 4
`string.Join`	alles in één tekst	hoofdstuk 1
de constructors van de verzamelingen	een echte verzameling uit de reeks	hoofdstukken 5 tot 10
`[.. reeks]`	een nieuwe lijst of array uit de reeks	hoofdstuk 6

De voorbeelden hieronder vertrekken van `IEnumerable<int> groot = getallen.Where(g => g > 4);` met `getallen` gelijk aan `5, 8, 3, 8, 1`. `groot` is dus 5, 8, 8.

LINQ: alles uit hoofdstuk 11

Groep	Methodes	Voorbeeld	Resultaat
één item	`First`, `FirstOrDefault`, `Single`, `SingleOrDefault`, `Last`, `LastOrDefault`, `ElementAt`, `ElementAtOrDefault`	`groot.First()`	`5`
nagaan	`Any`, `All`, `Contains`, `SequenceEqual`	`groot.Any(g => g > 7)`	`true`
een deel	`Where`, `Select`, `SelectMany`, `Distinct`, `DistinctBy`, `Take`, `Skip`, `TakeLast`, `SkipLast`, `TakeWhile`, `SkipWhile`, `Chunk`, `OfType`, `Index`	`groot.Distinct()`	`5, 8`
tellen en rekenen	`Count`, `Sum`, `Min`, `Max`, `Average`, `MinBy`, `MaxBy`, `CountBy`, `Aggregate`, `AggregateBy`	`groot.Max()`	`8`
ordenen en groeperen	`Order`, `OrderDescending`, `OrderBy`, `OrderByDescending`, `ThenBy`, `ThenByDescending`, `Reverse`, `Shuffle`, `GroupBy`, `ToLookup`	`getallen.GroupBy(g => g % 2 == 0)`	oneven: 5, 3, 1; even: 8, 8
combineren	`Concat`, `Union`, `Intersect`, `Except` en hun `By`-varianten, `Append`, `Prepend`, `Zip`, `Join`, `LeftJoin`, `RightJoin`, `GroupJoin`	`groot.Append(0)`	`5, 8, 8, 0`
vastleggen	`ToList`, `ToArray`, `ToHashSet`, `ToDictionary`, `ToLookup`	`groot.ToList()`	een `List<int>` met 5, 8, 8

Een echte verzameling maken

Schrijfwijze	Resultaat
`groot.ToList()` of `new List<int>(groot)` of `List<int> l = [.. groot];`	een lijst met 5, 8, 8
`groot.ToArray()`	een array met 5, 8, 8
`groot.ToHashSet()` of `new HashSet<int>(groot)`	een set met 5, 8
`new Queue<int>(groot)`, `new Stack<int>(groot)`	een wachtrij of stapel
`string.Join(", ", groot)`	de tekst `"5, 8, 8"`

Wanneer rekent een reeks?

Sommige LINQ-methodes stellen enkel de vraag; andere vragen meteen het antwoord. Dat verschil verklaart het verrassende voorbeeld bovenaan.

Stelt de vraag, rekent nog niet	Vraagt het antwoord, rekent meteen
`Where`, `Select`, `SelectMany`, `OrderBy`, `ThenBy`, `Reverse`, `Take`, `Skip`, `Distinct`, `Concat`, `Union`, `Append`, `Zip`, `Chunk`, `GroupBy`, `Join`	`ToList`, `ToArray`, `ToHashSet`, `ToDictionary`, `ToLookup`, `Count`, `Sum`, `Min`, `Max`, `Average`, `First`, `Single`, `Last`, `ElementAt`, `Any`, `All`, `Contains`, `Aggregate`, en `foreach`

Een reeks die nog niet gerekend heeft, rekent **elke keer opnieuw** als je ze gebruikt: twee keer `Count()` is twee keer het werk, en tussendoor kan de bron veranderd zijn. Wil je één vast antwoord, zet er dan `ToList()` achter.

Wat elk object kan

Komt van `object`. Zie hoofdstuk 13.

Methode	Voorbeeld	Resultaat
`ToString`	`groot.ToString()`	de naam van een intern type, zoals `System.Linq.Enumerable+WhereListIterator`1[System.Int32]`, niet de inhoud
`Equals` / `==`	twee keer dezelfde `Where`	`false`: twee verschillende vragen
`GetType`	`groot.GetType().Name`	`WhereListIterator`1`: geen `List`

Een LINQ-resultaat is geen lijst, ook al lijkt het er soms op. `(List<int>)groot` crasht met een `InvalidCastException`. `ToList()` maakt er wel een.

Zelf een reeks maken: yield return

Een methode kan zelf een reeks teruggeven, item per item, met `yield return`.

```
IEnumerable<int> Even(int tot)
{
    for (int i = 0; i <= tot; i += 2)
    {
        yield return i;
    }
}

Console.WriteLine(string.Join(", ", Even(6)));   // 0, 2, 4, 6
```

De methode loopt pas als iemand de reeks doorloopt, en elke keer opnieuw — net als een `Where`. `yield break;` stopt de reeks vroegtijdig. In de cursus kom je dit zelden tegen; je herkent het aan het woord `yield`.

Let op bij een IEnumerable

Je schrijft	Wat er gebeurt
`reeks.Count`	compileert niet; het is `reeks.Count()`
`reeks[0]`	compileert niet; het is `reeks.First()` of `reeks.ElementAt(0)`
`reeks.Count()` twee keer	rekent twee keer
`List<int> l = getallen.Where(...)`	compileert niet; zet er `.ToList()` achter
de bron veranderen tijdens een `foreach` over de reeks	crasht met een `InvalidOperationException`
`Select` met een `Console.WriteLine` erin, zonder het resultaat te gebruiken	er wordt niets getoond: de vraag werd nooit gesteld

Wanneer wel, wanneer niet

Wel — als parameter van een methode die enkel moet doorlopen. Wie die methode aanroept, mag dan een array, een `List` of een `HashSet` meegeven.

Niet — zodra je `Count`, een positie of `Add` nodig hebt. Maak er dan eerst met `ToList()` een echte lijst van.

Veelgemaakte fouten

> LET OP
>
> - Elke keer dat je een `IEnumerable` doorloopt, wordt de vraag opnieuw gesteld. Twee keer `Count()` op hetzelfde resultaat is twee keer het werk.
> - Verandert de bron tussendoor, dan verandert het antwoord mee.
> - Er is geen `[0]` en geen `.Count` zonder haakjes. Dat zijn dingen van een `List`, niet van een belofte.

Voorspel de uitkomst

Vraag 1

```
var getallen = new List<int> { 1, 2, 3 };
var groot = getallen.Where(g => g > 1);
getallen.Add(10);
Console.WriteLine(groot.Count());
```

Toon het antwoord ›

Vraag 2

```
var getallen = new List<int> { 1, 2, 3 };
var groot = getallen.Where(g => g > 1).ToList();
getallen.Add(10);
Console.WriteLine(groot.Count);
```

Toon het antwoord ›

Vraag 3

```
IEnumerable<int> reeks = new List<int> { 3, 1, 2 };
Console.WriteLine(reeks.Count());
Console.WriteLine(reeks.First());
```

Toon het antwoord ›

Vraag 4

```
var fruit = new List<string> { "appel", "peer" };
var luid = fruit.Select(f =>
{
    Console.WriteLine("bezig met " + f);
    return f.ToUpper();
});
Console.WriteLine("klaar");
```

Toon het antwoord ›

Vraag 5

```
int[] getallen = { 1, 2, 3 };
ICollection<int> verzameling = getallen;
verzameling.Add(4);
```

Toon het antwoord ›

Antwoorden

VRAAG 1

`3`

2, 3 en 10. De vraag werd pas gesteld bij `Count()`, en toen zat de 10 er al in.

‹ Terug naar de vraag

VRAAG 2

`2`

Enkel 2 en 3. `ToList()` stelde de vraag meteen en legde het antwoord vast. Wat er nadien bijkomt, telt niet meer mee. Vergelijk met vraag 1: één woord verschil, een ander antwoord.

‹ Terug naar de vraag

VRAAG 3

`3` en daarna `3`

`Count()` en `First()` zijn LINQ en werken op elke reeks. `reeks.Count` zonder haakjes en `reeks[0]` hadden niet gecompileerd: de variabele is een `IEnumerable`, ook al zit er een lijst achter.

‹ Terug naar de vraag

VRAAG 4

`klaar`

Enkel dat. `Select` stelt de vraag, maar niemand gebruikt het resultaat. Er wordt dus niets gerekend, en de `Console.WriteLine` binnen `Select` loopt nooit. Zet er `.ToList()` achter, en je ziet eerst de twee "bezig met"-regels.

‹ Terug naar de vraag

VRAAG 5

`NotSupportedException` — het programma crasht.

Het compileert, want een array belooft `ICollection<int>`. Maar een array kan niet groeien, dus `Add` weigert bij het uitvoeren.

‹ Terug naar de vraag

Deel 4 — Eigen types

Tot hier ging het over wat C# je geeft. Nu over de dingen die je zelf maakt — en over het idee dat bepaalt hoe ze zich gedragen.

13. class

EIGEN TYPE

Wat het is

Een bouwplan. De klasse is het plan, het object is het huis dat ernaar gebouwd is. Met één plan bouw je zoveel huizen als je wil, en elk huis heeft zijn eigen bewoners, zijn eigen kleur deur.

Hoe het eruitziet

```
public class Leerling
{
    public string Naam { get; set; } = "";
    public int Leeftijd { get; set; }

    public bool IsVolwassen()
    {
        return Leeftijd >= 18;
    }
}
```

En zo gebruik je het:

```
Leerling anna = new Leerling { Naam = "Anna", Leeftijd = 17 };

Console.WriteLine(anna.Naam);            // Anna
Console.WriteLine(anna.IsVolwassen());   // False

anna.Leeftijd = 18;                      // aanpassen mag
Console.WriteLine(anna.IsVolwassen());   // True
```

Wil je afdwingen dat een leerling altijd met een naam gemaakt wordt, dan geef je de klasse een **constructor** — een methode met dezelfde naam als de klasse, die draait op het moment van `new`:

```
public class Leerling
{
    public string Naam { get; set; }

    public Leerling(string naam)
    {
        Naam = naam;
    }
}

Leerling bram = new Leerling("Bram");
```

Eigenschappen die zichzelf uitrekenen: =>

Een eigenschap hoeft geen waarde te bewaren. Met `=>` rekent ze haar waarde uit, telkens opnieuw wanneer je erom vraagt:

```
public class Persoon
{
    public string Voornaam { get; set; } = "";
    public string Achternaam { get; set; } = "";

    public string VolledigeNaam => Voornaam + " " + Achternaam;
}
```

`VolledigeNaam` heeft geen `set`: je kan ze lezen, maar niet invullen. Verander je de voornaam, dan verandert de volledige naam vanzelf mee. Dezelfde pijl kan ook een korte methode schrijven: `public bool IsVolwassen() => Leeftijd >= 18;` doet hetzelfde als de versie met `return` hierboven.

In les 07 kom je deze schrijfwijze tegen bij de eigenschappen van de Unit of Work.

Waar een klasse woont: namespace en using

Elke klasse staat in een **namespace**, een soort map voor code. Zo kunnen twee klassen dezelfde naam hebben, zolang ze in een andere namespace staan.

```
namespace School.Models;

public class Leerling
{
    // ...
}
```

Wil je die klasse in een ander bestand gebruiken, dan zeg je bovenaan met `using` in welke map C# moet kijken:

```
using School.Models;

var anna = new Leerling();
```

Je zal twee schrijfwijzen tegenkomen. `namespace School.Models;` met een puntkomma geldt voor het hele bestand. De oudere vorm zet de klasse tussen accolades: `namespace School.Models { ... }`. Ze doen hetzelfde.

Een paar namespaces staan in een nieuw project al klaar zonder dat je ze typt, zoals `System`, `System.Linq` en `System.Collections.Generic`. Daarom kan je `List` en `Where` gebruiken zonder één `using`.

Een object zonder klasse: anonieme objecten

Soms wil je even een paar waarden samen doorgeven zonder er een klasse voor te schrijven:

```
var p = new { Naam = "Anna", Leeftijd = 17 };

Console.WriteLine(p.Naam);       // Anna
// p.Naam = "Bram";              compileert niet: enkel lezen
```

Het type heeft geen naam, daarom moet de variabele `var` zijn. In les 02 zie je dit bij `CreatedAtAction(..., new { id = laptop.Id }, ...)`: dat kleine object zegt welke waarde er in de route moet komen.

Wat je ermee kan

Vorm	Wat het doet	Voorbeeld	Resultaat
`{ get; set; }`	een eigenschap die je kan lezen en invullen	`public int Leeftijd { get; set; }`	`anna.Leeftijd = 18;` mag
`{ get; }`	enkel lezen; invullen kan alleen in de constructor	`public string Naam { get; }`	`anna.Naam = "x";` compileert niet
`=>` (eigenschap)	een eigenschap die zichzelf uitrekent	`public string VolledigeNaam => ...`	rekent telkens opnieuw
`=>` (methode)	een methode van één regel	`public bool IsVolwassen() => Leeftijd >= 18;`	zoals met `return`
`new Klasse { ... }`	maak een object en vul meteen eigenschappen in	`new Leerling { Naam = "Anna" }`	een leerling met een naam
constructor	draait bij `new` en kan iets afdwingen	`public Leerling(string naam) { ... }`	`new Leerling()` compileert niet meer
`namespace`	de map waarin een klasse woont	`namespace School.Models;`	volledige naam `School.Models.Leerling`
`using`	zeg waar C# moet zoeken	`using School.Models;`	`Leerling` is bruikbaar
`new { ... }`	een anoniem object	`new { id = 5 }`	enkel te lezen

> **LET OP**
>
> Krijg je de fout dat een type of namespace niet gevonden wordt, dan ontbreekt meestal een `using`. Visual Studio stelt die zelf voor: zet je cursor op de rode naam en druk op `Ctrl+.`.

Wat het ook kan, via iets anders

Elke klasse die je schrijft, krijgt er een paar dingen gratis bij, ook al staan ze niet in je code. Ze komen van `object`, de klasse waar elke klasse in C# van erft (hoofdstuk 20). Daarnaast bepaalt je klasse hoe ze zich gedraagt in een verzameling en in LINQ.

In de voorbeelden staat `Leerling` in de namespace `School.Models`, met de eigenschappen `Naam` en `Leeftijd`, en zijn `anna` en `kopie` twee aparte objecten met dezelfde inhoud: Anna, 18 jaar.

Wat elk object kan

Komt van `object`.

Methode	Wat het doet	Voorbeeld	Resultaat
`ToString`	het object als tekst	`anna.ToString()`	`School.Models.Leerling`: de naam van de klasse, niet de inhoud
`Equals` / **`==`**	is dit **hetzelfde object**	`anna == kopie`	`false`, ook al is de inhoud gelijk
`GetHashCode`	een getal per object, voor opzoektabellen	`anna.GetHashCode() == kopie.GetHashCode()`	`false`
`GetType`	welk type	`anna.GetType().Name`	`Leerling`
`ReferenceEquals`	zijn het echt dezelfde	`ReferenceEquals(anna, anna)`	`true`

`ToString` kan je zelf invullen. Dan toont `Console.WriteLine(anna)` wat jij wil:

```
public override string ToString() => $"{Naam} ({Leeftijd})";
```

Met `Equals` en `GetHashCode` kan dat ook, maar dat is zelden nodig: wil je vergelijken op inhoud, neem dan een `record` (hoofdstuk 14).

In een verzameling en in LINQ

Een lijst, een `HashSet` en een `Dictionary` vergelijken objecten met `Equals`. Bij een klasse is dat: **hetzelfde object**. `anna` staat in `lijst`, `kopie` niet.

Methode	Voorbeeld	Resultaat	Wat je dan doet
`Contains`, **`IndexOf`**, **`Remove`**	`lijst.Contains(kopie)`	`false`	zoek op een eigenschap: `lijst.Any(l => l.Naam == "Anna")`
`Distinct`	`new[] { anna, kopie }.Distinct().Count()`	`2`	`DistinctBy(l => l.Naam)` geeft `1`
`HashSet`	`new HashSet<Leerling> { anna, kopie }.Count`	`2`	een set van de namen, of een `record`
`Dictionary`-sleutel	`tabel.ContainsKey(kopie)` als `anna` de sleutel is	`false`	gebruik een eigenschap als sleutel: `Dictionary<string, Leerling>`

Methode	Voorbeeld	Resultaat	Wat je dan doet
`Sort`, `Order`	`leerlingen.Sort()`	`InvalidOperationException`: C# weet niet waarop	`OrderBy(l => l.Naam)` of `Sort((a, b) => a.Naam.CompareTo(b.Naam))`
`Max`, `Min` zonder sleutel	`leerlingen.Max()`	`ArgumentException`	`Max(l => l.Leeftijd)` of `MaxBy(l => l.Leeftijd)`

Alle andere LINQ-methodes werken gewoon op een lijst van objecten, telkens met een eigenschap in de voorwaarde: `leerlingen.Where(l => l.Leeftijd >= 18)`, `leerlingen.Select(l => l.Naam)`. Zie hoofdstuk 11.

Nog meer via iets anders

Bron	Wat het geeft	Volledige uitleg
een `interface`	een belofte van methodes, zodat je klasse ergens past	hoofdstuk 19
overerving	eigenschappen en methodes van een basisklasse	hoofdstuk 20
`is`, `as`, `switch` op type	nagaan van welke klasse een object is	hoofdstuk 22
`null`	een variabele van een klasse kan leeg zijn	hoofdstuk 17
JSON	een object als tekst bewaren en terug inlezen; enkel publieke eigenschappen tellen	hoofdstuk 30

LET OP

- `Console.WriteLine(anna)` toont de naam van de klasse, tot je zelf `ToString` invult.
- Twee objecten met dezelfde inhoud zijn voor `==`, `Contains`, `Distinct`, `HashSet` en `Dictionary` **niet** gelijk. Dat is de bedoeling bij een klasse: twee leerlingen die toevallig even oud zijn en dezelfde naam hebben, blijven twee leerlingen.

Wanneer wel, wanneer niet

Wel — de standaardkeuze. Iets met gegevens die in de loop van de tijd veranderen, en met gedrag: een leerling die ouder wordt, een bankrekening waarop je stort.

Niet — wanneer het object enkel gegevens draagt en twee objecten met dezelfde inhoud als gelijk moeten gelden. Dan is een `record` handiger.

Veelgemaakte fouten

LET OP

- `Leerling b = a;` maakt geen kopie. Het geeft hetzelfde object een tweede naam, net als bij een array.
- `a == b` zegt of het **hetzelfde** object is, niet of ze **dezelfde inhoud** hebben. Twee leerlingen met dezelfde naam zijn voor een klasse niet gelijk.
- Een klassevariabele zonder `new` is `null`. Ernaar vragen crasht.

Voorspel de uitkomst

Deze vragen gebruiken de eerste versie van `Leerling`, zonder constructor.

Vraag 1

```
var a = new Leerling { Naam = "Anna" };
var b = a;
b.Naam = "Bram";
Console.WriteLine(a.Naam);
```

Toon het antwoord ›

Vraag 2

```
var a = new Leerling { Naam = "Anna" };
var b = new Leerling { Naam = "Anna" };
Console.WriteLine(a == b);
```

Toon het antwoord ›

Vraag 3

Deze vraag gebruikt de klasse `Persoon` van hierboven.

```
var p = new Persoon { Voornaam = "Anna", Achternaam = "Peeters" };
p.Voornaam = "Bram";
Console.WriteLine(p.VolledigeNaam);
```

Toon het antwoord ›

Vraag 4

```
var p = new { Naam = "Anna" };
p.Naam = "Bram";
```

Toon het antwoord ›

Vraag 5

```
var a = new Leerling { Naam = "Anna" };
var b = new Leerling { Naam = "Anna" };
var lijst = new List<Leerling> { a };
Console.WriteLine(lijst.Contains(b));
Console.WriteLine(lijst.Any(l => l.Naam == b.Naam));
```

Toon het antwoord ›

Antwoorden

VRAAG 1

`Bram`

`a` en `b` zijn twee namen voor één en dezelfde leerling. Hernoem je haar via `b`, dan heet ze ook via `a` anders.

‹ Terug naar de vraag

VRAAG 2

`False`

Twee keer `new` is twee objecten. Dezelfde naam maakt ze niet tot dezelfde leerling — net zoals twee echte mensen die Anna heten niet dezelfde persoon zijn.

‹ Terug naar de vraag

VRAAG 3

`Bram Peeters`

`VolledigeNaam` bewaart niets. Ze wordt pas samengesteld op het moment dat je erom vraagt, en dan is de voornaam al Bram.

‹ Terug naar de vraag

VRAAG 4

`Compileert niet.`

De eigenschappen van een anoniem object kan je enkel lezen. Moet een waarde kunnen veranderen, schrijf dan een echte klasse.

‹ Terug naar de vraag

VRAAG 5

`False` en daarna `True`

`Contains` vraagt of `b` zelf in de lijst staat, en dat is niet zo: `b` is een ander object. `Any` vergelijkt de namen, en die zijn gelijk.

‹ Terug naar de vraag

14. record

EIGEN TYPE

Wat het is

Een klasse die draait om haar inhoud. Twee records met exact dezelfde gegevens zijn gelijk. Zoals twee briefjes van tien euro: het maakt niet uit welk exemplaar je hebt, enkel wat erop staat.

Hoe het eruitziet

```
public record Punt(int X, int Y);
```

Die ene regel maakt een type met twee eigenschappen, een constructor, en een nette weergave. En zo gebruik je het:

```
var p1 = new Punt(1, 2);
var p2 = new Punt(1, 2);

Console.WriteLine(p1 == p2);     // True — zelfde inhoud, dus gelijk
Console.WriteLine(p1);           // Punt { X = 1, Y = 2 }

var p3 = p1 with { Y = 5 };      // een kopie met één waarde anders
Console.WriteLine(p3);           // Punt { X = 1, Y = 5 }
Console.WriteLine(p1);           // Punt { X = 1, Y = 2 } — ongewijzigd
```

Wat je ermee kan

Uit die ene regel `public record Punt(int X, int Y);` maakt C# zelf:

Wat C# maakt	Wat het doet	Voorbeeld	Resultaat
eigenschappen	`X` en `Y`, enkel te lezen	`p1.X`	`1`
constructor	maakt een record met alle waarden	`new Punt(1, 2)`	een punt
`ToString`	toont de naam én de inhoud	`p1.ToString()`	`Punt { X = 1, Y = 2 }`
`Equals`, `==`, `!=`	vergelijkt de **inhoud**	`p1 == p2`	`true`
`GetHashCode`	gelijke inhoud geeft hetzelfde getal	`p1.GetHashCode() == p2.GetHashCode()`	`true`
`with`	een kopie met een paar waarden anders	`p1 with { Y = 5 }`	`Punt { X = 1, Y = 5 }`
`Deconstruct`	de waarden in losse variabelen	`var (x, y) = p1;`	`x` is 1, `y` is 2

Een record kan ook gewone eigenschappen en methodes hebben, net als een klasse: `public record Punt(int X, int Y) { public int Som => X + Y; }`.

`record struct` maakt een record dat zich als een struct gedraagt (hoofdstuk 15): hij wordt gekopieerd in plaats van gedeeld, en heeft toch `==` en een nette `ToString`.

Wat het ook kan, via iets anders

Een record is een klasse (hoofdstuk 13): alles van daar geldt ook hier. Het grote verschil zit in hoe hij zich gedraagt in een verzameling.

In de voorbeelden is `Leerling` nu een record: `public record Leerling(string Naam, int Leeftijd);`. `a` en `b` zijn twee aparte records `new Leerling("Anna", 18)`.

Methode	Voorbeeld	Resultaat	Bij een klasse
`Contains`, **`IndexOf`**, **`Remove`**	`new List<Leerling> { a }.Contains(b)`	`true`	`false`
`Distinct`	`new[] { a, b }.Distinct().Count()`	`1`	`2`
`HashSet`	`new HashSet<Leerling> { a, b }.Count`	`1`	`2`
`Dictionary`-sleutel	`tabel.ContainsKey(b)` als `a` de sleutel is	`true`	`false`
`Union`, `Intersect`, `Except`	vergelijken op inhoud	zoals hierboven	op referentie
`Sort`, **`Order`**	`leerlingen.Sort()`	**`InvalidOperationException`**	ook

Sorteren gaat ook bij een record niet vanzelf: zeg waarop, met `OrderBy(l => l.Naam)`.

Voor JSON, LINQ en `is`/`switch` gedraagt een record zich als een klasse. Les 09 toont records ook als DTO (hoofdstuk 33).

Let op bij een record

LET OP

- Een record vergelijkt de inhoud **eigenschap per eigenschap**, met de `Equals` van elke eigenschap. Zit er een `List` in je record, dan wordt die lijst op referentie vergeleken: twee records met elk een eigen lijst met dezelfde namen zijn **niet** gelijk.
- `ToString` toont zo'n lijst ook niet: je ziet `Leerlingen = System.Collections.Generic.List`1[System.String]`.
- `with` maakt een **ondiepe** kopie. De nieuwe record wijst naar dezelfde lijst: voeg je iets toe via de kopie, dan zie je het ook in het origineel.

Wanneer wel, wanneer niet

Wel — een pakketje gegevens dat je doorgeeft en niet meer verandert. In les 09 van Programming Advanced kom je records tegen als een van de twee manieren om DTO's te schrijven; zie hoofdstuk 33.

Niet — wanneer het object in de loop van de tijd verandert en zijn eigen identiteit heeft. Een leerling die ouder wordt blijft dezelfde leerling; dat is een `class`.

Veelgemaakte fouten

> LET OP
>
> - De eigenschappen van `record Punt(int X, int Y)` kan je na het aanmaken niet meer aanpassen. `p1.X = 5;` compileert niet.
> - `with` verandert niets aan het origineel. Het maakt een nieuw record. Wie het resultaat niet opvangt, heeft niets gedaan.

Voorspel de uitkomst

Vraag 1

```
var p1 = new Punt(1, 2);
var p2 = p1 with { X = 9 };
Console.WriteLine(p1.X);
```

Toon het antwoord ›

Vraag 2

```
var p1 = new Punt(1, 2);
p1.X = 5;
```

Toon het antwoord ›

Vraag 3

```
Console.WriteLine(new Punt(3, 4));
```

Toon het antwoord ›

Vraag 4

```
public record Leerling(string Naam, int Leeftijd);

var a = new Leerling("Anna", 18);
var b = new Leerling("Anna", 18);
var set = new HashSet<Leerling> { a, b };
Console.WriteLine(set.Count);
```

Toon het antwoord ›

Vraag 5

```
public record Klas(string Naam, List<string> Leerlingen);

var k1 = new Klas("1A", new List<string> { "Anna" });
var k2 = new Klas("1A", new List<string> { "Anna" });
Console.WriteLine(k1 == k2);
```

Toon het antwoord ›

Antwoorden

VRAAG 1

`1`

`with` maakte een kopie waarin `X` 9 is. Het origineel bleef zoals het was.

‹ Terug naar de vraag

VRAAG 2

`Compileert niet.`

Bij deze korte schrijfwijze kan je de eigenschappen enkel invullen bij het aanmaken. Wil je een ander punt, maak dan een nieuw — met `with` of met `new`.

‹ Terug naar de vraag

VRAAG 3

```
Punt { X = 3, Y = 4 }
```

Een record weet uit zichzelf hoe het zich moet tonen. Een gewone klasse toont enkel haar naam — dat is een van de redenen waarom records zo handig zijn om mee te testen.

‹ Terug naar de vraag

VRAAG 4

`1`

Een record vergelijkt op inhoud. Voor de `HashSet` is `b` dus een dubbel van `a`. Met een gewone klasse was het 2 geweest.

‹ Terug naar de vraag

VRAAG 5

`False`

De namen zijn gelijk, maar de lijsten zijn twee verschillende objecten. Een record vergelijkt elke eigenschap met haar eigen `Equals`, en een lijst vergelijkt op referentie.

‹ Terug naar de vraag

15. struct

EIGEN TYPE

Wat het is

Een klein pakketje waarden dat bij kopiëren écht gekopieerd wordt. Zoals een fotokopie: wat je op de kopie schrijft, staat niet op het origineel.

Je gebruikt ze al zonder het te weten. Een `int`, een `double` en een `bool` zijn allemaal structs.

Hoe het eruitziet

```
public struct Kleur
{
    public int R;
    public int G;
    public int B;
}

Kleur a = new Kleur { R = 255, G = 0, B = 0 };
Kleur b = a;         // een echte kopie
b.R = 0;

Console.WriteLine(a.R);   // 255 — a is niet veranderd
Console.WriteLine(b.R);   // 0
```

Vergelijk dit met vraag 1 in hoofdstuk 13. Dezelfde drie regels, een tegengesteld resultaat.

Wat je ermee kan

Een struct heeft wat je er zelf in zet: velden, eigenschappen, methodes en constructors, net als een klasse. Wat anders is:

Wat	Bij een struct	Bij een klasse
`Kleur b = a;`	een **kopie**	een tweede naam voor hetzelfde object
`new Kleur()` zonder waarden	alle velden op hun standaardwaarde: `R` is `0`	een object met de standaardwaarden
een variabele zonder waarde	kan niet `null` zijn	kan `null` zijn
erven	kan niet	kan wel

Wat het ook kan, via iets anders

Komt van `object`, via `ValueType`. In de voorbeelden zijn `a` en `b` twee `Kleur`-structs met `R = 255`.

Methode	Voorbeeld	Resultaat	Bij een klasse
`Equals`	`a.Equals(b)`	`true`: veld per veld vergeleken	`false`
`==`	`a == b`	**`compileert niet`**	vergelijkt de referentie
`ToString`	`a.ToString()`	de naam van het type, bv. `School.Models.Kleur`	ook
`GetType`	`a.GetType().Name`	`Kleur`	ook
`Contains`, `Distinct`, `HashSet`	`new[] { a, b }.Distinct().Count()`	`1`: vergelijken via `Equals`	`2`
LINQ	`kleuren.Where(k => k.R > 100)`	werkt zoals bij elk object	ook

`int`, `double`, `decimal`, `bool`, `char`, `DateTime` en `TimeSpan` zijn allemaal structs. Daarom gedragen ze zich als één waarde: zie hoofdstuk 16.

Let op bij een struct

LET OP

- `==` bestaat niet vanzelf voor je eigen struct. Gebruik `Equals`, of maak er een `record struct` van (hoofdstuk 14): die krijgt `==` en een nette `ToString`.
- `lijst[0].R = 0;` op een lijst van structs **compileert niet**. `lijst[0]` geeft een kopie, en een kopie aanpassen heeft geen zin. Haal hem eruit, pas hem aan, en zet hem terug: `var k = lijst[0]; k.R = 0; lijst[0] = k;`.
- In een `foreach` over structs kan je de variabele niet aanpassen.

Wanneer wel, wanneer niet

Wel — zelden zelf nodig. Een klein ding van een paar getallen dat zich gedraagt als één waarde: een kleur, een coördinaat.

Niet — bijna altijd. Twijfel je, neem een `class` of een `record`.

Veelgemaakte fouten

> **LET OP**
>
> - Een struct in een lijst aanpassen past de kopie aan, niet wat in de lijst staat. Dat is de klassieke valkuil, en de reden waarom je ze best klein en onveranderlijk houdt.

Voorspel de uitkomst

Vraag 1

```
int x = 5;
int y = x;
y = 10;
Console.WriteLine(x);
```

Toon het antwoord ›

Vraag 2

```
var kleuren = new List<Kleur> { new Kleur { R = 255 } };
var eerste = kleuren[0];
eerste.R = 0;
Console.WriteLine(kleuren[0].R);
```

Toon het antwoord ›

Antwoorden

VRAAG 1

5

Een `int` is een struct. `y = x` kopieerde de waarde 5. Wat je daarna met `y` doet, raakt `x` niet. Dit gedrag ken je al sinds je eerste les — het heeft nu gewoon een naam.

‹ Terug naar de vraag

VRAAG 2

255

`kleuren[0]` gaf een kopie. `eerste` aanpassen raakt de struct in de lijst niet. Wil je hem echt aanpassen, zet `eerste` dan terug met `kleuren[0] = eerste;`.

‹ Terug naar de vraag

16. Referentie tegenover waarde

CONCEPT

Dit is het belangrijkste idee uit dit hele leerboek. Als je één ding onthoudt, laat het dit zijn.

Er zijn twee soorten types in C#, en ze gedragen zich tegengesteld wanneer je ze aan een tweede variabele geeft.

	Referentietype	Waardetype
Voorbeelden	`class`, `record`, array, `List`, `Dictionary`, alle verzamelingen	`int`, `double`, `bool`, `struct`, `enum`
`b = a` doet	geeft hetzelfde ding een tweede naam	maakt een echte kopie
Via `b` iets aanpassen	zie je ook via `a`	raakt `a` niet
Vergelijking	een huissleutel: twee sleutels, één huis	een fotokopie: twee vellen, elk apart
Kan `null` zijn	ja	nee, tenzij je `?` achter het type zet

> OPMERKING
>
> `string` is technisch een referentietype, maar gedraagt zich als een waarde: je kan een tekst nooit aanpassen, enkel vervangen door een nieuwe. Daarom verrast hij je nooit op de manier waarop een lijst dat doet.

Test jezelf: kijk terug naar vraag 4 in hoofdstuk 5, vraag 1 in hoofdstuk 13 en de vraag in hoofdstuk 15. Drie keer dezelfde drie regels, en het antwoord hangt enkel af van de soort type. De vraag onderaan dit hoofdstuk brengt het samen.

class, record en struct naast elkaar

	`class`	`record`	`struct`
Soort	referentie	referentie	waarde
`a == b` vergelijkt	is het hetzelfde object	is de inhoud gelijk	compileert niet; `Equals` vergelijkt de inhoud
Aanpassen na het maken	ja	met de korte schrijfwijze: nee	ja, maar liever niet
Zichzelf tonen	enkel de naam van het type	alle eigenschappen	enkel de naam van het type
Typisch gebruik	iets met gedrag en een identiteit	een pakketje gegevens	een klein getalachtig ding

Een verzameling kopiëren

Een verzameling is een referentietype. `var b = a;` geeft dus geen kopie. Wil je er wel een, dan zijn er een paar wegen, maar ze kopiëren enkel de verzameling, niet de objecten erin.

Je schrijft	Een nieuwe verzameling?	Dezelfde objecten erin?
`var b = a;`	nee: een tweede naam voor dezelfde	ja
`new List<Leerling>(a)`, `a.ToList()`, `a.ToArray()`	ja: `b.Add(...)` raakt `a` niet	ja: `b[0].Naam = "X"` zie je ook in `a`
`(int[])getallen.Clone()`, `getallen.ToArray()`	ja	bij getallen, tekst en structs: elke waarde is een kopie

Hetzelfde zie je bij `with` van een record (hoofdstuk 14): een lijst in de record wordt niet mee gekopieerd. En bij structs in een lijst (hoofdstuk 15): `lijst[0]` geeft een kopie.

Voorspel de uitkomst

Vraag 1

Dit is de vraag die alles samenbrengt. Twee methodes: de ene krijgt een lijst mee, de andere een getal.

```
void VoegToe(List<int> lijst)
{
    lijst.Add(99);
}

void Verhoog(int getal)
{
    getal = getal + 1;
}

var l = new List<int> { 1 };
int n = 1;

VoegToe(l);
Verhoog(n);

Console.WriteLine(l.Count);
Console.WriteLine(n);
```

Toon het antwoord ›

Antwoorden

VRAAG 1

`2` en daarna `1`

Een lijst is een referentietype. `VoegToe` krijgt geen kopie van de lijst, maar een tweede sleutel tot dezelfde lijst. Wat de methode erin stopt, zit er daarna nog in.

Een `int` is een waardetype. `Verhoog` krijgt een fotokopie van het getal. Het verhoogt die kopie, en de kopie verdwijnt zodra de methode klaar is. `n` blijft 1.

Dit is de reden waarom een lijst die je aan een methode meegeeft na afloop soms "vanzelf" veranderd is.

‹ Terug naar de vraag

17. Omgaan met niets

CONCEPT

In het vorige hoofdstuk zag je dat een referentietype `null` kan zijn en een waardetype niet — tenzij je er een vraagteken achter zet. Dit hoofdstuk gaat over hoe je daar veilig mee omgaat.

Vorm	Wat het doet	Voorbeeld	Resultaat
`??`	neem dit, tenzij het null is	`naam ?? "onbekend"`	`"onbekend"` als `naam` null is
`??=`	zet enkel als het nog null is	`_repo ??= new Repository()`	vult één keer in
`?.`	roep enkel aan als het niet null is	`fan?.Merk`	`null` in plaats van een crash
`is null`	is het niets	`if (fan is null)`	`true` of `false`
`is not null`	is het iets	`if (fan is not null)`	`true` of `false`
`int?`	een getal dat ook niets mag zijn	`int? hoogte = null`	mag leeg blijven
`HasValue`	zit er een waarde in	`hoogte.HasValue`	`false`
`GetValueOrDefault`	de waarde, of een terugval	`hoogte.GetValueOrDefault(0)`	`0`
`!` achter een waarde	zeg tegen de compiler: dit is niet null	`naam!.Length`	geen waarschuwing; is het tóch null, dan crasht het
`= default!`	begin met niets, zonder waarschuwing	`public Stad Stad { get; set; } = default!;`	de eigenschap is null tot iemand ze invult

Het vraagteken achter een type betekent: deze waarde mag ontbreken. Dat is de manier waarop je in code het verschil vastlegt tussen "nul" en "niet ingevuld" — precies het onderscheid waar een controle op stuk loopt als je het niet maakt.

Het uitroepteken: vertrouw me

In een nieuw project waarschuwt de compiler zodra een referentietype zonder vraagteken `null` zou kunnen zijn, zoals een `string`-eigenschap zonder beginwaarde. Het uitroepteken zet die waarschuwing uit. Aan wat er gebeurt terwijl het programma draait, verandert het niets: het is een belofte aan de compiler, geen controle.

In de cursus zie je het op twee plaatsen:

- `= default!` achter een eigenschap, vanaf les 06. De eigenschap begint als null, maar wordt later ingevuld, bijvoorbeeld door EF Core bij het ophalen. Zie hoofdstuk 32. `= null!` betekent hetzelfde.
- `src.Product!.Naam` in een regel voor Mapster, in les 09. Daar zet het enkel de waarschuwing uit. Mapster kijkt zelf na of het product er is, en laat de naam leeg als het ontbreekt. Zie hoofdstuk 33.

In gewone code geldt: klopt de belofte niet, dan krijg je dezelfde `NullReferenceException` als zonder uitroepteken, alleen zonder waarschuwing vooraf.

Waar null vandaan komt, via iets anders

Methodes die niets teruggeven in plaats van te crashen

Methode	Bij een klasse of tekst	Bij een getal	Hoofdstuk
`FirstOrDefault`, `LastOrDefault`, `SingleOrDefault`, `ElementAtOrDefault`	`null`	`0`	11
`FirstOrDefault(-1)`	—	je eigen terugval: `-1`	11
`List.Find`, `Array.Find`	`null`	`0`	5, 6
`GetValueOrDefault`	`null`	`0`, of je eigen terugval	7
`TryGetValue`, `TryParse`	de `out`-variabele wordt `null`	de `out`-variabele wordt `0`	2, 7
`Max`, `Min`, `MaxBy`, `MinBy` op een lege verzameling	`null`	`InvalidOperationException`	11
`as`	`null` als het type niet klopt	—	22
`FindAsync`, `FirstOrDefaultAsync`	`null`	—	32
`QueryFirstOrDefaultAsync`	`null`	—	34

LINQ op getallen met een vraagteken

In een `List<int?>` slaan `Sum`, `Average`, `Max` en `Min` de lege plaatsen over. Met `{ 1, null, 3 }`:

Methode	Resultaat	Waarom
`Sum()`	`4`	1 + 3
`Average()`	`2`	(1 + 3) / 2: de lege telt niet mee
`Max()`, `Min()`	`3`, `1`	
`Count()`	`3`	tellen telt wél alle plaatsen
`Max()` op een lege `List<int?>`	`null`	geen crash, anders dan bij `List<int>`
`Average()` als alles leeg is	`null`	`Sum()` geeft dan `0`

Tekst en null

Je schrijft	Resultaat
`"a" + naam + "b"` met `naam` gelijk aan null	`"ab"`: null telt als lege tekst
`string.Join(",", ...)` op `"a"`, null, `"b"`	`"a,,b"`
`namen.OfType<string>()`	enkel de teksten die niet null zijn
`namen.Contains(null)`	`true` als er een null in zit
`string.IsNullOrEmpty(naam)`	null of leeg in één keer; zie hoofdstuk 1

LET OP

- `int? x = null;` en dan `x.Value`: **`InvalidOperationException`**. Gebruik `GetValueOrDefault` of `??`.
- `lijst?.Count` geeft een `int?`: null als de lijst null is.
- `int.Parse(null)` crasht met **`ArgumentNullException`**, maar `Convert.ToInt32(null)` geeft gewoon `0`.

Voorspel de uitkomst

Vraag 1

```
string? naam = null;
Console.WriteLine(naam ?? "onbekend");
```

Toon het antwoord ›

Vraag 2

```
string? naam = null;
Console.WriteLine(naam?.Length);
```

Toon het antwoord ›

Vraag 3

```
string? naam = null;
Console.WriteLine(naam.Length);
```

Toon het antwoord ›

Vraag 4

```
int? score = null;
Console.WriteLine(score.HasValue);
Console.WriteLine(score.GetValueOrDefault());
```

Toon het antwoord ›

Vraag 5

```
string? a = null;
a ??= "eerste";
a ??= "tweede";
Console.WriteLine(a);
```

Toon het antwoord ›

Vraag 6

```
string? naam = null;
Console.WriteLine(naam!.Length);
```

Toon het antwoord ›

Vraag 7

```
var hoogtes = new List<int?> { 4, null, 2 };
Console.WriteLine(hoogtes.Average());
Console.WriteLine(hoogtes.Count());
```

Toon het antwoord ›

Vraag 8

```
var namen = new List<string>();
var getallen = new List<int>();
Console.WriteLine(namen.Max() is null);
Console.WriteLine(getallen.Max());
```

Toon het antwoord ›

Antwoorden

VRAAG 1

`onbekend`

`naam` is null, dus `??` neemt wat rechts staat.

‹ Terug naar de vraag

VRAAG 2

Een lege regel.

`naam?.Length` vraagt de lengte enkel op als `naam` niet null is. Anders is het resultaat zelf null, en `Console.WriteLine` toont null als niets. Geen crash.

‹ Terug naar de vraag

VRAAG 3

`NullReferenceException` — het programma crasht.

Het enige verschil met vraag 2 is het vraagteken. Zonder `?.` vraag je de lengte op van iets dat er niet is. Visual Studio waarschuwt je hier wel met een groene kronkellijn, maar compileren gaat.

‹ Terug naar de vraag

VRAAG 4

`False` en daarna `0`

Er zit geen waarde in. `GetValueOrDefault` geeft dan de standaardwaarde van een `int`: nul. Merk op dat je nu wél kan zien dat de score ontbrak, via `HasValue`. Bij een gewone `int` kon dat niet.

‹ Terug naar de vraag

VRAAG 5

`eerste`

`??=` vult enkel in als er nog niets staat. De eerste keer was `a` null, de tweede keer niet meer.

‹ Terug naar de vraag

VRAAG 6

`NullReferenceException` — het programma crasht.

Hetzelfde als bij vraag 3, maar nu zonder groene kronkellijn. Het uitroepteken zegt de compiler dat `naam` niet null is. De compiler gelooft het, en zwijgt. Het programma zelf kijkt niet na, en loopt vast op de lengte van iets dat er niet is.

‹ Terug naar de vraag

VRAAG 7

`3` en daarna `3`

`Average` slaat de lege plaats over: (4 + 2) / 2. `Count` telt alle plaatsen, ook de lege.

‹ Terug naar de vraag

VRAAG 8

`True`, en daarna `InvalidOperationException` — het programma crasht.

Bij een lege verzameling teksten geeft `Max` gewoon null. Een `int` kan niet null zijn, dus daar crasht het.

‹ Terug naar de vraag

18. enum

EIGEN TYPE

Wat het is

Een vaste lijst benoemde keuzes. Een verkeerslicht staat op rood, oranje of groen. Niet op paars, niet op "roood" met drie o's.

Hoe het eruitziet

```
public enum Verkeerslicht
{
    Rood,
    Oranje,
    Groen
}
```

En zo gebruik je het:

```
Verkeerslicht licht = Verkeerslicht.Rood;

if (licht == Verkeerslicht.Rood)
{
    Console.WriteLine("Stop");
}

switch (licht)
{
    case Verkeerslicht.Rood:
        Console.WriteLine("Wachten");
        break;
    case Verkeerslicht.Groen:
        Console.WriteLine("Rijden");
        break;
}

Console.WriteLine(licht);           // Rood
Console.WriteLine((int)licht);      // 0
```

Tekst omzetten naar een enum gaat met `Enum.TryParse`, net als bij getallen:

```
if (Enum.TryParse("Groen", out Verkeerslicht gekozen))
{
    Console.WriteLine(gekozen);     // Groen
}
```

Wat je ermee kan

Een enum heeft zelf geen methodes die je schrijft. Wat je ermee doet, komt van de klasse `Enum` en van de taal. In de voorbeelden is het `Verkeerslicht` van hierboven: `Rood` (0), `Oranje` (1), `Groen` (2).

Methode	Wat het doet	Voorbeeld	Resultaat
`ToString`	de naam als tekst	`Verkeerslicht.Oranje.ToString()`	`"Oranje"`
`(int)`	het getal erachter	`(int)Verkeerslicht.Groen`	`2`
`(Verkeerslicht)`	van een getal naar een keuze	`(Verkeerslicht)1`	`Oranje`
`Enum.GetValues`	alle keuzes	`Enum.GetValues<Verkeerslicht>()`	`Rood, Oranje, Groen`
`Enum.GetNames`	alle namen, als tekst	`Enum.GetNames<Verkeerslicht>()`	`"Rood", "Oranje", "Groen"`
`Enum.TryParse`	tekst omzetten, zonder te crashen	`Enum.TryParse("groen", true, out Verkeerslicht v)`	`true`: met `true` negeert het hoofdletters
`Enum.Parse`	tekst omzetten, of crashen	`Enum.Parse<Verkeerslicht>("Paars")`	**`ArgumentException`**
`Enum.IsDefined`	bestaat deze keuze echt	`Enum.IsDefined((Verkeerslicht)7)`	`false`
`==`, `!=`, `<`, `>`	vergelijken, op het getal erachter	`Verkeerslicht.Rood < Verkeerslicht.Groen`	`true`
`CompareTo`, **`Equals`**, **`GetType`**	wat elk object kan	`Verkeerslicht.Rood.GetType().Name`	`Verkeerslicht`
`switch`	per keuze iets anders doen	`licht switch { Verkeerslicht.Rood => "stop", _ => "rijden" }`	`"stop"` bij rood

Wat het ook kan, via iets anders

Bron	Wat het geeft	Voorbeeld	Resultaat
LINQ op `Enum.GetValues`	werken met alle keuzes	`Enum.GetValues<Verkeerslicht>().Where(v => v != Verkeerslicht.Oranje)`	`Rood, Groen`
LINQ op een lijst keuzes	tellen, sorteren, groeperen	`lichten.Count(v => v == Verkeerslicht.Groen)`	het aantal groene
`Order`	sorteert op het getal erachter, niet op de naam	`lichten.Order()`	eerst alle `Rood`, dan `Oranje`, dan `Groen`
een tekst	alles van hoofdstuk 1 op de naam	`licht.ToString().ToLower()`	`"groen"`
`default`	de keuze met getal 0	`default(Verkeerslicht)`	`Rood`

Een enum is een waarde, zoals een getal: `==` vergelijkt de keuze, en `Contains`, `Distinct` en `HashSet` werken zoals je verwacht.

Bestaat ook, maar kom je in deze cursus niet tegen

`[Flags]`: een enum waarvan je meerdere keuzes tegelijk kan aanzetten, zoals `Rechten.Lezen | Rechten.Schrijven`, met `HasFlag` om er één na te gaan. En `Enum.GetUnderlyingType`, `Enum.Format`.

Let op bij een enum

> **LET OP**
>
> - `Enum.TryParse` aanvaardt ook getallen: `Enum.TryParse("7", out Verkeerslicht v)` geeft `true`, met `v` gelijk aan 7 — een keuze die niet bestaat. Controleer achteraf met `Enum.IsDefined`.
> - Een eigenschap van een enum-type die je niet invult, staat op de keuze met getal 0. Bij `Verkeerslicht` is dat `Rood`, zonder dat iemand het koos.
> - `Order` sorteert op het getal, niet alfabetisch op de naam.

Wanneer wel, wanneer niet

Wel — een eigenschap heeft een klein, vast aantal mogelijke waarden die niet veranderen terwijl je programma draait: dagen van de week, een status, een richting. Het grote voordeel tegenover een `string`: een tikfout wordt een compileerfout in plaats van een fout die pas opduikt als het te laat is.

Niet — wanneer de lijst van keuzes groeit terwijl het programma draait, of door gebruikers aangevuld wordt. Een enum wijzig je enkel door je code te herschrijven.

Veelgemaakte fouten

LET OP

- Onder de motorkap is elke keuze een getal: `Rood` is 0, `Oranje` 1, `Groen` 2. Voeg je vooraan een keuze toe, dan schuiven alle getallen op. Staan die getallen ergens opgeslagen, dan betekenen ze plots iets anders.
- `(Verkeerslicht)7` compileert, ook al bestaat er geen zevende kleur.
- `Enum.TryParse` is standaard hoofdlettergevoelig.

Voorspel de uitkomst

Vraag 1

```
Console.WriteLine((int)Verkeerslicht.Groen);
```

Toon het antwoord ›

Vraag 2

```
Verkeerslicht l = (Verkeerslicht)7;
Console.WriteLine(l);
```

Toon het antwoord ›

Vraag 3

```
bool ok = Enum.TryParse("groen", out Verkeerslicht l);
Console.WriteLine(ok);
```

Toon het antwoord ›

Vraag 4

```
bool gelukt = Enum.TryParse("1", out Verkeerslicht v);
Console.WriteLine(gelukt);
Console.WriteLine(v);
```

Toon het antwoord ›

Vraag 5

```
var licht = (Verkeerslicht)7;
Console.WriteLine(licht);
Console.WriteLine(Enum.IsDefined(licht));
```

Toon het antwoord ›

Antwoorden

VRAAG 1

`2`

Tellen begint bij nul: Rood is 0, Oranje 1, Groen 2.

‹ Terug naar de vraag

VRAAG 2

`7`

C# laat het toe, en omdat er geen naam bij 7 hoort, toont het gewoon het getal. Een enum beschermt je dus tegen tikfouten in je code, maar niet tegen elk getal dat van buiten binnenkomt.

‹ Terug naar de vraag

VRAAG 3

`False`

`"groen"` met kleine letter is niet `Groen`. Wil je dat het wel lukt, dan geef je een extra `true` mee om hoofdletters te negeren.

‹ Terug naar de vraag

VRAAG 4

`True` en daarna `Oranje`

`TryParse` aanvaardt ook het getal achter een keuze. 1 is `Oranje`.

‹ Terug naar de vraag

VRAAG 5

`7` en daarna `False`

Omzetten van een getal lukt altijd, ook als er geen keuze bij hoort. `ToString` toont dan gewoon het getal. `IsDefined` zegt dat de keuze niet bestaat.

‹ Terug naar de vraag

19. interface

EIGEN TYPE

Wat het is

Een contract. Het zegt wát iets moet kunnen, niet hóe. Zoals een stopcontact: elk toestel met de juiste stekker past erin, of het nu een lamp is of een waterkoker. Het stopcontact hoeft niet te weten wat je aansluit.

Hoe het eruitziet

```
public interface IGeluid
{
    string MaakGeluid();
}
```

Het contract zegt enkel: *wie mij belooft, kan een geluid maken*. Geen uitwerking. Die komt in de klassen die de belofte doen:

```
public class Hond : IGeluid
{
    public string MaakGeluid()
    {
        return "Woef";
    }
}

public class Kat : IGeluid
{
    public string MaakGeluid()
    {
        return "Miauw";
    }
}
```

En nu het nut ervan:

```
List<IGeluid> dieren = new List<IGeluid> { new Hond(), new Kat() };

foreach (IGeluid dier in dieren)
{
    Console.WriteLine(dier.MaakGeluid());
}
```

De lus weet niet of hij een hond of een kat voor zich heeft. Dat hoeft ook niet: hij weet dat ze allebei een geluid kunnen maken, en meer heeft hij niet nodig.

Waarom dit in je cursus zo belangrijk is

In les 03 van Programming Advanced maak je een repository met een interface ervoor. Dat is precies dit principe. De controller praat met het contract, niet met de klasse. Daardoor kan de klasse achter het contract later vervangen worden — een lijst in het geheugen wordt een database — zonder dat de controller één letter verandert.

Wat het ook kan, via iets anders

.NET kent zelf een paar interfaces. Belooft je klasse er een, dan werken er methodes uit andere hoofdstukken die anders niet werken.

Interface	Je belooft	Wat je er gratis bij krijgt	Hoofdstuk
`IComparable<T>`	`int CompareTo(T ander)`	`Sort()`, `Order()`, `Max()` en `Min()` zonder sleutel	6, 11, 13
`IEnumerable<T>`	een manier om je items één voor één te geven	`foreach` en heel LINQ	12

```
public class Leerling : IComparable<Leerling>
{
    public string Naam { get; set; } = "";

    public int CompareTo(Leerling? ander) => Naam.CompareTo(ander?.Naam);
}
```

Zonder die belofte crasht `leerlingen.Sort()` (hoofdstuk 13). Met de belofte sorteert het op naam, en geeft `leerlingen.Max()` de leerling met de naam die alfabetisch het laatst komt.

Een `List<IGeluid>` is een gewone lijst: alles van hoofdstuk 6 en LINQ werkt erop, zoals `dieren.Select(d => d.MaakGeluid())`. Wil je enkel de katten, gebruik dan `OfType<Kat>()` uit hoofdstuk 22.

Bestaat ook, maar kom je in deze cursus niet tegen

`IEquatable<T>` (zelf bepalen wanneer twee objecten gelijk zijn), `IDisposable` (opruimen met `using`), `IComparer<T>` (een aparte vergelijker die je aan `Sort` geeft).

Wanneer wel, wanneer niet

Wel — wanneer verschillende klassen hetzelfde moeten kunnen, of wanneer je wil dat een deel van je programma later vervangbaar is zonder de rest aan te passen.

Niet — voor een klasse waarvan er maar één soort ooit zal bestaan en die je nooit zal vervangen. Een contract met maar één ondertekenaar voegt weinig toe.

Moeten verwante klassen niet alleen hetzelfde kunnen, maar ook gegevens of uitgewerkte methodes delen, dan is een abstracte klasse het betere antwoord. Zie hoofdstuk 20.

Veelgemaakte fouten

LET OP

- Van een interface kan je geen object maken. `new IGeluid()` compileert niet — je kan geen stopcontact aansluiten op een ander stopcontact.
- Belooft een klasse het contract, dan moet ze **alles** uitwerken wat erin staat. Eén methode vergeten en het compileert niet.
- Volgens de afspraak begint de naam met een hoofdletter `I`.

Voorspel de uitkomst

Vraag 1

Wat verschijnt er op het scherm bij de lus hierboven?

Toon het antwoord ›

Vraag 2

```
IGeluid g = new IGeluid();
```

Toon het antwoord ›

Vraag 3

```
IGeluid g = new Kat();
Console.WriteLine(g is Kat);
```

Toon het antwoord ›

Vraag 4

`Leerling` belooft `IComparable<Leerling>` zoals hierboven.

```
var leerlingen = new List<Leerling>
{
    new Leerling { Naam = "Cleo" },
    new Leerling { Naam = "Anna" },
    new Leerling { Naam = "Bert" }
};
leerlingen.Sort();
Console.WriteLine(leerlingen[0].Naam);
Console.WriteLine(leerlingen.Max()!.Naam);
```

Toon het antwoord ›

Antwoorden

VRAAG 1

`Woef` en daarna `Miauw`

Elke klasse gebruikt haar eigen uitwerking, ook al weet de lus enkel dat ze `IGeluid` beloven.

‹ Terug naar de vraag

VRAAG 2

`Compileert niet.`

Een interface is een belofte zonder uitwerking. Er valt niets te bouwen.

‹ Terug naar de vraag

VRAAG 3

`True`

De variabele zegt enkel "iets dat een geluid maakt", maar het object dat erin zit is nog altijd een kat. Met `is` kan je dat nagaan.

‹ Terug naar de vraag

VRAAG 4

`Anna` en daarna `Cleo`

`Sort` en `Max` gebruiken jouw `CompareTo`, en die vergelijkt de namen. Zonder de interface was `Sort` gecrasht met een `InvalidOperationException`.

‹ Terug naar de vraag

20. Overerving en abstracte klassen

EIGEN TYPE

Wat het is

Een klasse die verder bouwt op een andere. Zoals een basisrecept voor deeg: pizza en brood beginnen allebei met hetzelfde deeg, en voegen er elk hun eigen stappen aan toe. Wie het basisrecept aanpast, past het voor allebei aan.

De klasse waarop je verder bouwt, heet de **basisklasse**. De klasse die erop verder bouwt, heet de **afgeleide klasse**. Die krijgt alles mee wat de basisklasse heeft, en voegt er haar eigen dingen aan toe.

Hoe het eruitziet

```
public class Voertuig
{
    public int Wielen { get; set; }

    public string Toeter()
    {
        return "Tuut";
    }
}

public class Fiets : Voertuig
{
    public bool HeeftBel { get; set; }
}
```

Het dubbelpunt na `Fiets` betekent: *een fiets is een voertuig*. Een fiets heeft dus ook `Wielen` en kan ook `Toeter()`, zonder dat die er opnieuw in staan:

```
Fiets f = new Fiets { Wielen = 2, HeeftBel = true };

Console.WriteLine(f.Wielen);      // 2 — komt van Voertuig
Console.WriteLine(f.Toeter());    // Tuut — komt van Voertuig
Console.WriteLine(f.HeeftBel);    // True — van de fiets zelf

Voertuig v = f;                   // mag: een fiets ís een voertuig
Console.WriteLine(v.Wielen);      // 2
// v.HeeftBel compileert niet: via v zie je enkel wat elk voertuig heeft
```

Een basisklasse die zelf nooit bestaat: abstract

Een "voertuig" op zich rijdt nergens rond. Wat op de weg staat, is altijd een fiets, een bus of iets anders. Met `abstract` leg je dat vast: van de basisklasse zelf kan je geen object meer maken, enkel van wat ervan erft.

```
public abstract class Voertuig
{
    public int Wielen { get; }
    protected int Kilometers { get; set; }

    protected Voertuig(int wielen)
    {
        Wielen = wielen;
    }

    public void Rij(int km)
    {
        Kilometers += km;
    }

    public virtual string Toeter()
    {
        return "Tuut";
    }

    public abstract int MaxPassagiers();
}

public class Fiets : Voertuig
{
    public Fiets() : base(2) { }

    public override string Toeter()
    {
        return "Tring";
    }

    public override int MaxPassagiers()
    {
        return 1;
    }
}

public class Bus : Voertuig
{
    public Bus() : base(6) { }

    public override int MaxPassagiers()
    {
        return 50;
    }
}
```

Wat daar gebeurt:

- `abstract` vóór `class`: een object maken van `Voertuig` zelf compileert niet meer.
- `protected Voertuig(int wielen)` is de constructor van de basisklasse. Met `: base(2)` roept de fiets hem aan met twee wielen.
- `protected int Kilometers` is zichtbaar in `Voertuig` en in elke klasse die ervan erft, maar niet daarbuiten.
- `virtual` geeft `Toeter` een standaardversie die een afgeleide klasse mag vervangen. De fiets doet dat met `override`, de bus niet.
- `abstract` vóór een methode betekent: geen uitwerking. Elke afgeleide klasse moet ze zelf uitwerken, met `override`.

En het nut ervan:

```
List<Voertuig> vloot = new List<Voertuig> { new Fiets(), new Bus() };

foreach (Voertuig v in vloot)
{
    Console.WriteLine($"{v.Wielen} wielen, {v.Toeter()}, max {v.MaxPassagiers()}");
}
// 2 wielen, Tring, max 1
// 6 wielen, Tuut, max 50
```

De lus weet niet of ze een fiets of een bus voor zich heeft. Dat hoeft ook niet: elk voertuig heeft wielen, kan toeteren en weet hoeveel passagiers erin passen. Het is hetzelfde idee als bij de interface in hoofdstuk 19, met één verschil: de basisklasse geeft ook gegevens en uitgewerkte methodes mee.

Wat je ermee kan

Vorm	Wat het doet	Voorbeeld	Resultaat
`: Basisklasse`	erf van een klasse	`class Fiets : Voertuig`	een fiets heeft alles van een voertuig
`: Basisklasse, IContract`	erf van één klasse en beloof daarnaast contracten	`class Fiets : Voertuig, IGeluid`	eerst de klasse, dan de interfaces
`base(...)`	roep de constructor van de basisklasse aan	`public Fiets() : base(2) { }`	`Wielen` is `2`
`protected`	zichtbaar in de klasse en in wie ervan erft, niet daarbuiten	`protected int Kilometers`	`fiets.Kilometers` compileert niet
`virtual`	deze methode mag vervangen worden	`public virtual string Toeter()`	de bus gebruikt de standaardversie
`override`	vervang een methode die `virtual` of `abstract` is	`public override string Toeter()`	`"Tring"`
`base.Methode()`	gebruik de versie van de basisklasse, binnen je eigen versie	`return base.Toeter() + "!";`	`"Tuut!"`

Vorm	Wat het doet	Voorbeeld	Resultaat
`abstract class`	een klasse waarvan je geen object kan maken	`new Voertuig(4)`	compileert niet
`abstract` (methode)	een methode zonder uitwerking; elke afgeleide klasse moet ze uitwerken	`public abstract int MaxPassagiers();`	vergeet je ze, dan compileert het niet
`sealed`	van deze klasse mag niemand meer erven	`public sealed class Bus : Voertuig`	`class Dubbeldekker : Bus` compileert niet
`is`	is dit object van dat type, of ervan afgeleid	`new Fiets() is Voertuig`	`true`
`GetType`	het echte type van het object, niet dat van de variabele	`Voertuig v = new Fiets();` en dan `v.GetType().Name`	`"Fiets"`

Wat het ook kan, via iets anders

Bron	Wat het geeft	Voorbeeld	Resultaat
een lijst van de basisklasse	fietsen en bussen samen in één lijst	`var voertuigen = new List<Voertuig> { new Fiets(), new Bus() };`	—
LINQ op die lijst	werken met wat alle voertuigen hebben	`voertuigen.Sum(v => v.Wielen)`	`8`
`OfType`	enkel één soort	`voertuigen.OfType<Fiets>()`	enkel de fietsen, als `Fiets`
`is` in LINQ	tellen per soort	`voertuigen.Count(v => v is Bus)`	`1`
`object`	elke klasse erft er uiteindelijk van: `ToString`, `Equals`, `GetType`	`override string ToString()`	`Console.WriteLine(v)` en `$"{v}"` tonen jouw tekst

Zie hoofdstuk 13 voor wat je van `object` krijgt, en hoofdstuk 22 voor `is`, `as` en `OfType`.

Abstracte klasse tegenover interface

	Abstracte klasse	Interface
Zegt	wat iets **is**	wat iets **kan**
Bewaart gegevens	ja	nee — ze kan enkel zeggen dát er een eigenschap moet zijn
Uitgewerkte methodes	ja, naast de abstracte	nee, op een zeldzame uitzondering na die je in deze cursus niet tegenkomt
Constructor	ja	nee
Hoeveel per klasse	één	zoveel je wil
Een object van maken	nee	nee
Typisch gebruik	verwante klassen die gegevens en werk delen: een fiets en een bus	klassen die hetzelfde moeten kunnen, ook als ze niets met elkaar te maken hebben: een hond en een deurbel maken allebei geluid

Waar je dit in de cursus tegenkomt

Les	Wat je ziet	Wat er gebeurt
01	een controller die erft van `ControllerBase`	je controller erft `Ok`, `NotFound` en de rest uit hoofdstuk 26
04	`: base(options)` in je context	de opties gaan door naar de constructor van `DbContext`; zie hoofdstuk 31
04	`protected override void OnModelCreating(...)`	je vervangt een methode die `DbContext` daarvoor openzet
07	een specifieke repository die erft van de generieke	ze krijgt alle gewone methodes mee; `protected` maakt de context ook voor haar zichtbaar

Wanneer wel, wanneer niet

Wel — verwante klassen die echt hetzelfde zijn en gegevens of uitgewerkte methodes delen. De test: klinkt *een X is een Y* juist? Een fiets is een voertuig: ja. Maak de basisklasse `abstract` zodra ze op zichzelf niets betekent.

Niet — enkel om een paar methodes te hergebruiken tussen klassen die niets met elkaar te maken hebben. Een leerling is geen klaslokaal, ook al hebben ze allebei een naam. En gaat het enkel om wat iets kan, dan is een interface lichter.

Veelgemaakte fouten

LET OP

- Wat `private` is in de basisklasse, kan de afgeleide klasse niet zien. Heeft ze het nodig, maak het dan `protected` — niet `public`, want dan kan iedereen erbij.
- Heeft de basisklasse enkel een constructor met parameters, dan moet elke afgeleide klasse `: base(...)` aanroepen. Vergeet je dat, dan compileert het niet.
- `override` kan enkel op een methode die `virtual` of `abstract` is. Schrijf je in de afgeleide klasse gewoon een methode met dezelfde naam, dan vervang je niets: je verstopt de oude alleen. Visual Studio waarschuwt je, maar het compileert. Zie vraag 4.
- Een klasse erft van hoogstens één klasse. Interfaces mag je er zoveel bij zetten als je wil.
- Via een variabele van de basisklasse zie je enkel wat de basisklasse heeft, ook als het object meer kan. Met `is` uit hoofdstuk 22 ga je na wat het echt is.

Voorspel de uitkomst

Vragen 1, 2, 3 en 6 gebruiken de tweede versie van `Voertuig`, met `abstract`, en de klassen `Fiets` en `Bus` die erbij horen.

Vraag 1

```
Voertuig v = new Fiets();
Console.WriteLine(v.Toeter());
```

Toon het antwoord ›

Vraag 2

```
Voertuig v = new Bus();
Console.WriteLine(v.Toeter());
```

Toon het antwoord ›

Vraag 3

```
Voertuig v = new Voertuig(4);
```

Toon het antwoord ›

Vraag 4

```
public class Basis
{
    public string Wie()
    {
        return "basis";
    }
}

public class Afgeleid : Basis
{
    public string Wie()
    {
        return "afgeleid";
    }
}

Basis b = new Afgeleid();
Afgeleid a = new Afgeleid();
Console.WriteLine(b.Wie());
Console.WriteLine(a.Wie());
```

Toon het antwoord ›

Vraag 5

```
public class A
{
    public A()
    {
        Console.WriteLine("A");
    }
}

public class B : A
{
    public B()
    {
        Console.WriteLine("B");
    }
}

new B();
```

Toon het antwoord ›

Vraag 6

```
Voertuig v = new Fiets();
Console.WriteLine(v is Voertuig);
Console.WriteLine(v.GetType() == typeof(Voertuig));
Console.WriteLine(v.GetType().Name);
```

Toon het antwoord ›

Antwoorden

VRAAG 1

`Tring`

De variabele zegt enkel "een voertuig", maar het object is een fiets. Omdat `Toeter` `virtual` is en de fiets ze met `override` vervangt, telt de versie van het object, niet die van de variabele.

‹ Terug naar de vraag

VRAAG 2

`Tuut`

De bus vervangt `Toeter` niet. Ze gebruikt dus de standaardversie uit `Voertuig`. Een `virtual` methode mag je vervangen, je moet niet.

‹ Terug naar de vraag

VRAAG 3

`Compileert niet.`

`Voertuig` is `abstract`. Van de basisklasse zelf kan je geen object maken, enkel van een klasse die ervan erft: een fiets of een bus.

‹ Terug naar de vraag

VRAAG 4

`basis` en daarna `afgeleid`

Twee keer hetzelfde object, twee verschillende antwoorden. Zonder `virtual` en `override` vervangt `Afgeleid` de methode niet, maar verstopt ze haar enkel. Welke versie je krijgt, hangt dan af van de variabele, niet van het object. Zet `virtual` bij `Basis` en `override` bij `Afgeleid`, en je krijgt twee keer `afgeleid`.

‹ Terug naar de vraag

VRAAG 5

`A` en daarna `B`

Eerst de fundering, dan de muren. Vóór de constructor van `B` draait die van de basisklasse. Staat er geen `: base(...)`, dan roept C# zelf de constructor zonder parameters van `A` aan.

‹ Terug naar de vraag

VRAAG 6

`True`, `False` en daarna `Fiets`

`is` vraagt: *is dit een voertuig, of iets dat ervan erft?* Een fiets is dat. `GetType()` geeft het echte type van het object, en dat is precies `Fiets`, niet `Voertuig`. Wil je weten of iets tot een familie behoort, gebruik dan `is`.

‹ Terug naar de vraag

21. Generics

EIGEN TYPE

> **OPMERKING**
> Zelf generics schrijven komt aan bod in les 07 van Programming Advanced. Het staat hier al, zodat je het klaar hebt wanneer je het nodig hebt.

Wat het is

Een type met een open plaats. Zoals een doos met een etiket *inhoud: ___*. De doos werkt hetzelfde voor schoenen, boeken of speelgoed; pas wanneer je hem gebruikt, vul je in wat erin gaat.

Je gebruikt het al sinds hoofdstuk 6. `List<T>` is één klasse die werkt voor `List<string>`, `List<int>` en `List<Leerling>`. De `T` is de open plaats. In dit hoofdstuk maak je zelf zo'n type.

Hoe het eruitziet

```
public class Doos<T>
{
    private T _inhoud;

    public Doos(T inhoud)
    {
        _inhoud = inhoud;
    }

    public T Open()
    {
        return _inhoud;
    }
}
```

Overal waar `T` staat, komt straks het type dat je invult:

```
Doos<string> brief = new Doos<string>("Proficiat!");
Doos<int> getal = new Doos<int>(42);

string tekst = brief.Open();      // Proficiat!
int n = getal.Open();             // 42 — een echt getal, je kan ermee rekenen

// new Doos<int>("42") compileert niet: dat is een tekst, geen getal
```

Ook een losse methode kan een open plaats hebben:

```
T Eerste<T>(List<T> lijst)
{
    return lijst[0];
}

string s = Eerste(new List<string> { "kiwi", "peer" });   // kiwi
int g = Eerste(new List<int> { 7, 3 });                   // 7
```

C# leidt zelf af wat `T` is uit wat je meegeeft. Je hoeft `Eerste<string>(...)` niet uit te schrijven, al mag het.

De open plaats beperken: where

Zonder meer weet C# niets over `T`. Het kan een getal zijn, een tekst of een leerling, dus je mag er niets mee doen wat niet voor álle types werkt. Met `where` beperk je wat er ingevuld mag worden — en daar krijg je iets voor terug:

```
public class Garage<T> where T : Voertuig
{
    private List<T> _voertuigen = new List<T>();

    public void Zet(T voertuig)
    {
        _voertuigen.Add(voertuig);
    }

    public int TotaalWielen()
    {
        return _voertuigen.Sum(v => v.Wielen);   // mag: elke T is een voertuig
    }
}

Garage<Fiets> stalling = new Garage<Fiets>();    // mag: een fiets is een voertuig
```

`Voertuig` en `Fiets` komen uit hoofdstuk 20. Zonder `where T : Voertuig` zou `v.Wielen` niet compileren, want dan kan `T` ook een tekst zijn.

Wat je ermee kan

Vorm	Wat het doet	Voorbeeld	Resultaat
`class Naam<T>`	een klasse met een open plaats	`class Doos<T>`	`Doos<int>`, `Doos<string>`, …
`Methode<T>(...)`	een methode met een open plaats	`T Eerste<T>(List<T> lijst)`	`Eerste(namen)` geeft een tekst
`interface INaam<T>`	een contract met een open plaats	`interface IHouder<T>`	elk type belooft het voor zijn eigen `T`
`class X<T> : IY<T>`	geef de open plaats door aan het contract	`class Doos<T> : IHouder<T>`	de doos belooft het contract, voor elke `T`
`class X : IY<int>`	vul de open plaats meteen in	`class Spaarpot : IHouder<int>`	enkel voor getallen
`<T1, T2>`	meerdere open plaatsen	`class Paar<TLinks, TRechts>`	zoals `Dictionary<TKey, TValue>`
`where T : class`	`T` moet een referentietype zijn	`class Doos<T> where T : class`	`Doos<int>` compileert niet
`where T : struct`	`T` moet een waardetype zijn	`class Doos<T> where T : struct`	`Doos<string>` compileert niet
`where T : Basisklasse`	`T` moet die klasse zijn, of ervan erven	`where T : Voertuig`	binnenin mag je `Wielen` gebruiken
`where T : IContract`	`T` moet dat contract beloven	`where T : IGeluid`	binnenin mag je `MaakGeluid()` aanroepen
`where T : new()`	`T` moet een constructor zonder parameters hebben	`where T : new()`	binnenin mag je `new T()` schrijven
`default(T)`	de standaardwaarde van `T`	`default(int)`	`0`; bij een klasse `null`, bij een bool `false`
`typeof(T)`	het type dat ingevuld werd	`typeof(T).Name` in een `Doos<int>`	`"Int32"`

Meerdere beperkingen zet je achter elkaar, gescheiden door een komma: `where T : class, new()`.

> **OPMERKING**
>
> `string` is een klasse, en dus een referentietype. Bij `where T : class` mag je `string` dus wél invullen. Enkel waardetypes zoals `int`, `double` en `bool` vallen erbuiten.

Generics die je al kent

Bijna elke verzameling en veel methodes uit dit leerboek hebben een open plaats.

Waar	Voorbeeld	Hoofdstuk
verzamelingen	`List<T>`, `Dictionary<TKey, TValue>`, `HashSet<T>`, `Queue<T>`, `Stack<T>`	6 tot 10
reeksen	`IEnumerable<T>`	12
LINQ	elke methode, zoals `Select<TSource, TResult>`: C# vult de types zelf in	11
omzetten	`Enum.Parse<T>`, `JsonSerializer.Deserialize<T>`	18, 30
asynchroon	`Task<T>`	29
in je API	`ActionResult<T>`, `DbSet<T>`, `Set<T>()`, `Adapt<T>()`, `QueryAsync<T>`	26, 31, 32, 33, 34

Met `where T : IComparable<T>` mag je binnenin `CompareTo` gebruiken: zie hoofdstuk 19.

Waarom dit in je cursus zo belangrijk is

In les 07 van Programming Advanced schrijf je één repository die werkt voor al je entiteiten, in plaats van één per entiteit. De open plaats is de entiteit; de cursus noemt ze `TEntity` in plaats van `T`. De beperking `where TEntity : class` staat erbij omdat EF Core enkel met klassen werkt. Specifieke repositories erven daarna van die generieke, zoals in hoofdstuk 20. Hoe zo'n repository aan de juiste tabel komt, staat in hoofdstuk 32.

Wanneer wel, wanneer niet

Wel — dezelfde code werkt voor meerdere types, en enkel het type verschilt. Zonder generics zou je dezelfde klasse per type kopiëren.

Niet — wanneer de code per type iets anders moet doen. Schrijf je in een generieke methode `if (typeof(T) == typeof(int))`, dan heb je geen open plaats nodig maar aparte methodes, of overerving.

Veelgemaakte fouten

LET OP

- Zonder `where` kan je binnen de klasse niets specifieks met `T` doen. `voertuig.Wielen` compileert pas met `where T : Voertuig`, ook al stop je er in de praktijk enkel voertuigen in.
- `return null;` compileert niet als `T` ook een getal kan zijn. Gebruik `default`, of beperk `T` met `where T : class`.
- `Doos<Fiets>` is geen `Doos<Voertuig>`, ook al is een fiets een voertuig. Hetzelfde geldt voor `List<Fiets>` en `List<Voertuig>`. Zie vraag 4.
- Volgens de afspraak begint de naam van een open plaats met een `T`: `T`, `TSleutel`, `TEntity`.

Voorspel de uitkomst

De vragen gebruiken `Doos<T>` en `Garage<T>` van hierboven, en `Voertuig` en `Fiets` uit hoofdstuk 20.

Vraag 1

```
var a = new Doos<int>(5);
var b = new Doos<string>("5");
Console.WriteLine(a.Open() + 1);
Console.WriteLine(b.Open() + 1);
```

Toon het antwoord ›

Vraag 2

```
Console.WriteLine(default(int));
Console.WriteLine(default(bool));
Console.WriteLine(default(string) == null);
```

Toon het antwoord ›

Vraag 3

```
var g = new Garage<string>();
```

Toon het antwoord ›

Vraag 4

```
Doos<Voertuig> d = new Doos<Fiets>(new Fiets());
```

Toon het antwoord ›

Vraag 5

```
string Soort<T>(T waarde)
{
    return typeof(T).Name;
}

Console.WriteLine(Soort(42));
Console.WriteLine(Soort("hallo"));
```

Toon het antwoord ›

Antwoorden

VRAAG 1

`6` en daarna `51`

Dezelfde klasse, twee verschillende types. De eerste doos geeft een echt getal terug, en 5 + 1 is 6. De tweede geeft een tekst terug, en een tekst + 1 plakt de 1 erachter.

‹ Terug naar de vraag

VRAAG 2

`0`, `False` en daarna `True`

Elk type heeft een standaardwaarde: nul voor getallen, `false` voor een bool en `null` voor referentietypes. Het is dezelfde waarde die `FirstOrDefault` uit hoofdstuk 11 teruggeeft wanneer het niets vindt.

‹ Terug naar de vraag

VRAAG 3

Compileert niet.

`Garage<T>` eist met `where T : Voertuig` dat `T` een voertuig is. Een tekst is dat niet. De fout valt bij het bouwen, niet pas wanneer het programma draait.

‹ Terug naar de vraag

VRAAG 4

Compileert niet.

Voor C# zijn `Doos<Fiets>` en `Doos<Voertuig>` twee aparte types, zonder verband. Dat een fiets een voertuig is, maakt een doos met een fiets nog geen doos met een voertuig. Wat wel mag: een `Doos<Voertuig>` maken en daar een fiets in stoppen, met `new Doos<Voertuig>(new Fiets())`.

‹ Terug naar de vraag

VRAAG 5

`Int32` en daarna `String`

C# leidt `T` af uit wat je meegeeft: eerst een `int`, daarna een `string`. Dat de namen er anders uitzien, komt doordat `int` en `string` korte namen zijn voor `Int32` en `String`. Het zijn dezelfde types.

‹ Terug naar de vraag

22. Types herkennen

METHODES

De voorbeelden gebruiken `IGeluid`, `Hond` en `Kat` uit hoofdstuk 19.

Vorm	Wat het doet	Voorbeeld	Resultaat
`typeof`	het type zelf, als je de naam kent	`typeof(Kat)`	het type
`GetType`	het type van een bestaand object	`dier.GetType()`	het type
`is`	is dit object van dat type	`if (dier is Kat)`	`true` of `false`
`is` met naam	idem, en geef het meteen die naam	`if (dier is Kat k)`	`k` is bruikbaar
`as`	behandel als dat type, of `null`	`dier as Kat`	het object of `null`
`nameof`	de naam van iets als tekst	`nameof(GetFanById)`	`"GetFanById"`

`nameof` lijkt onnozel tot je het nodig hebt. Het voordeel tegenover `"GetFanById"` tussen aanhalingstekens is dat je code niet meer compileert wanneer je die methode hernoemt. Een typfout in een naam vind je zo bij het bouwen in plaats van bij het testen.

Types herkennen in een verzameling, via iets anders

Komt van LINQ. In de voorbeelden is `dieren` een `List<IGeluid>` met een hond en twee katten.

Methode	Voorbeeld	Resultaat
`OfType`	`dieren.OfType<Kat>()`	de twee katten, als `Kat`; ook null wordt overgeslagen
`Cast`	`dieren.Cast<Kat>().ToList()`	`InvalidCastException`: de hond is geen kat
`Where` met `is`	`dieren.Where(d => d is Kat)`	de twee katten, maar nog als `IGeluid`
`Count` met `is`	`dieren.Count(d => d is Kat)`	`2`
`GroupBy` op het type	`dieren.GroupBy(d => d.GetType().Name)`	`Hond: 1, Kat: 2`

> LET OP
>
> `Cast` controleert pas als je de items echt gebruikt. `dieren.Cast<Kat>().Count()` geeft `3`, zonder fout: tellen kan zonder om te zetten. `ToList()` of een `foreach` crasht wel. Wil je enkel de katten, neem dan `OfType`.

Voorspel de uitkomst

Vraag 1

```
IGeluid dier = new Hond();
Console.WriteLine(dier is Kat);
```

Toon het antwoord ›

Vraag 2

```
IGeluid dier = new Hond();
Kat? k = dier as Kat;
Console.WriteLine(k is null);
```

Toon het antwoord ›

Vraag 3

```
Console.WriteLine(nameof(Kat));
Console.WriteLine(typeof(Kat).Name);
```

Toon het antwoord ›

Vraag 4

```
var dieren = new List<IGeluid> { new Hond(), new Kat(), new Kat() };
Console.WriteLine(dieren.OfType<Kat>().Count());
var katten = dieren.Cast<Kat>().ToList();
```

Toon het antwoord ›

Antwoorden

VRAAG 1

`False`

De variabele zegt enkel "iets dat een geluid maakt". Wat erin zit, is een hond.

‹ Terug naar de vraag

VRAAG 2

`True`

`as` probeert het dier als kat te behandelen. Lukt dat niet, dan krijg je null in plaats van een crash. Daarom controleer je na een `as` altijd op null.

‹ Terug naar de vraag

VRAAG 3

`Kat` en daarna `Kat`

Twee wegen naar dezelfde tekst. `nameof` wordt al bij het compileren ingevuld, `typeof(...).Name` pas als het programma draait. Voor een naam in een melding of in `CreatedAtAction` is `nameof` de gewoonte.

‹ Terug naar de vraag

VRAAG 4

`2`, en daarna `InvalidCastException` — het programma crasht.

`OfType` slaat de hond over. `Cast` probeert elk dier om te zetten naar een kat, ook de hond, en dat lukt niet.

‹ Terug naar de vraag

23. Uitzonderingen

CONCEPT

Wat het is

Een noodrem. Wanneer een methode iets gevraagd wordt dat ze niet kan — een tekst omzetten die geen getal is, een sleutel opzoeken die er niet is — dan stopt ze meteen en gooit ze een **uitzondering**. Die gaat terug naar wie de methode aanriep, en naar wie díe aanriep, tot iemand ze opvangt. Vangt niemand ze op, dan stopt het hele programma.

Je bent ze in dit leerboek al vaak tegengekomen, in het rood. Dit hoofdstuk gaat over wat je ermee doet.

Hoe het eruitziet

```
try
{
    int leeftijd = int.Parse("twaalf");
    Console.WriteLine($"Leeftijd: {leeftijd}");
}
catch (FormatException)
{
    Console.WriteLine("Dat is geen getal.");
}
finally
{
    Console.WriteLine("Klaar.");
}
```

`int.Parse` gooit een `FormatException`. De rest van het `try`-blok wordt overgeslagen, het `catch`-blok vangt de uitzondering op, en `finally` draait altijd — fout of geen fout. Op het scherm komt `Dat is geen getal.` en daarna `Klaar.`

Je kan ook zelf een uitzondering gooien, wanneer je methode iets gevraagd wordt dat ze niet kan of mag doen:

```
void ZetThermostaat(int graden)
{
    if (graden < 5 || graden > 30)
    {
        throw new ArgumentException("Kies tussen 5 en 30 graden.");
    }

    Console.WriteLine($"Thermostaat op {graden} graden.");
}

try
{
    ZetThermostaat(40);
}
catch (ArgumentException ex)
{
    Console.WriteLine(ex.Message);     // Kies tussen 5 en 30 graden.
}
```

`throw` werkt zoals `return`: de methode stopt daar. Het verschil is dat er geen waarde teruggaat maar een uitzondering, en dat wie de methode aanriep ze moet opvangen — of zelf ook stopt.

Wat je ermee kan

Vorm	Wat het doet	Voorbeeld	Resultaat
`try`	probeer dit blok	`try { ... }`	loopt tot de eerste uitzondering
`catch (Soort)`	vang deze soort op	`catch (FormatException)`	het programma loopt verder na het blok
`catch (Soort ex)`	vang op en geef ze een naam	`catch (FormatException ex)`	`ex` is bruikbaar in het blok
`catch (Exception)`	vang elke soort op	`catch (Exception ex)`	vangt alles, ook wat je niet verwachtte
`finally`	draait altijd, fout of geen fout	`finally { ... }`	ook na een `return` in het `try`-blok
`throw new`	gooi zelf een uitzondering	`throw new ArgumentException("Te warm.")`	de methode stopt hier
`throw;`	gooi dezelfde uitzondering verder, binnen een `catch`	`catch (Exception) { ...; throw; }`	wie hoger staat, krijgt ze alsnog
`Message`	de uitleg bij de uitzondering	`ex.Message`	`"Te warm."`
`GetType`	welke soort het is	`ex.GetType().Name`	`"FormatException"`

De uitzonderingen die je al kent

Uitzondering	Wanneer	Hoofdstuk
`FormatException`	`int.Parse`, `Convert.ToInt32` of `DateTime.Parse` op een tekst die niet past	2, 3
`ArgumentNullException`	`int.Parse(null)`, of null als sleutel in een `Dictionary`	2, 7
`DivideByZeroException`	een `int` delen door 0; bij een `double` krijg je ∞ of NaN, zonder fout	2
`OverflowException`	een berekening in `checked`, of `Sum` in LINQ die boven `int.MaxValue` gaat	2, 11
`IndexOutOfRangeException`	een positie buiten een array, of buiten een tekst: `"abc"[5]`	1, 5
`ArgumentOutOfRangeException`	een positie buiten een `List`, `ElementAt` buiten de reeks, of `Substring` buiten de tekst	1, 6, 11
`ArgumentException`	`Add` of `ToDictionary` met een sleutel die al bestaat, `Enum.Parse` met een onbekende naam, `Max()` op objecten die niet vergelijkbaar zijn	7, 11, 13, 18
`KeyNotFoundException`	`[sleutel]` die niet bestaat	7
`InvalidOperationException`	een verzameling veranderen in een `foreach`, `Dequeue`, `Pop` of `Peek` op iets leegs, `First` zonder resultaat, `Single` met twee, `Max` of `Average` op een lege lijst getallen, `Sort()` op objecten die niet vergelijkbaar zijn, `.Value` op een lege `int?`, een vraag die EF Core niet kan vertalen	4, 6, 9, 10, 11, 13, 17, 32
`InvalidCastException`	`(int)` op iets dat geen getal is, of `Cast<T>` met een item van een ander type	22

Uitzondering	Wanneer	Hoofdstuk
`NullReferenceException`	iets opvragen van `null`	17
`FileNotFoundException`	`File.ReadAllText` op een bestand dat niet bestaat	30
`JsonException`	`JsonSerializer.Deserialize` op tekst die geen geldige JSON is, of met een waarde van het verkeerde type; een kring in je objecten	30, 32
`NotImplementedException`	een methode die nog niet uitgewerkt is	les 05 van de cursus

Ontbreekt niet alleen het bestand maar ook de map waarin het moet staan, dan krijg je een `DirectoryNotFoundException` in plaats van een `FileNotFoundException`.

Voorkomen tegenover opvangen

Veel uitzonderingen kan je voorkomen door eerst te kijken. Dat is bijna altijd beter: het zegt duidelijker wat je bedoelt, en een `catch` die te veel opvangt, verbergt fouten die je had moeten zien.

In plaats van opvangen	Kijk vooraf met	Hoofdstuk
`FormatException` van `Parse`	`TryParse`	2
`KeyNotFoundException`	`TryGetValue` of `ContainsKey`	7
`InvalidOperationException` van `First`	`FirstOrDefault` en een controle op `null`	11
`NullReferenceException`	`?.` of `is null`	17
`ArgumentException` van `Add` in een `Dictionary`	`TryAdd` of `ContainsKey`	7
`InvalidOperationException` van `Max` op een lege lijst	`Any()` vooraf, of `MaxBy` op objecten	11
`InvalidCastException`	`is`, `as` of `OfType`	22
`DivideByZeroException`	kijk of de deler 0 is	2

Opvangen blijft nodig bij alles wat van buiten je programma komt: een bestand, een netwerk, een database. Of een bestand er nog staat op het moment dat je het leest, en of de inhoud geldige JSON is, kan je vooraf niet zeker weten.

De volgorde van je catch-blokken

Alle uitzonderingen erven van `Exception` — overerving uit hoofdstuk 20. Daarom vangt `catch (Exception)` elke soort op. Zet je meerdere `catch`-blokken onder elkaar, dan loopt C# ze van boven naar onder af en neemt het het eerste dat past. De specifieke soorten komen dus eerst, de algemene als laatste:

```
try
{
    int aantal = int.Parse(invoer);
    Console.WriteLine(100 / aantal);
}
catch (FormatException)
{
    Console.WriteLine("Geef een getal.");
}
catch (DivideByZeroException)
{
    Console.WriteLine("Nul mag niet.");
}
catch (Exception ex)
{
    Console.WriteLine($"Onverwacht: {ex.Message}");
}
```

In je API

Een uitzondering die in een controller niemand opvangt, legt niet de hele server plat. ASP.NET Core vangt ze op het laatste moment op en stuurt `500 Internal Server Error` terug — dat zag je in vraag 3 van hoofdstuk 26. Voor de client betekent 500: *de server heeft een fout gemaakt*. Lag de fout eigenlijk bij de client — een Id dat niet bestaat, een ongeldige waarde — dan hoort daar een 404 of een 400 bij.

Wanneer wel, wanneer niet

Wel — `try` en `catch` bij alles wat van buiten je programma komt en wat je vooraf niet volledig kan nakijken: bestanden, netwerk, een database, JSON van elders. En `throw` wanneer je methode iets gevraagd wordt dat niet kan, en ze zelf niet weet wat ze dan moet doen.

Niet — als vervanger van een controle die je vooraf kan doen. En nooit om een fout stil te laten verdwijnen.

Veelgemaakte fouten

LET OP

- Een lege `catch { }` slikt de fout in. Het programma loopt verder alsof er niets gebeurd is, met verkeerde of ontbrekende gegevens, en niemand weet waarom.
- `catch (Exception)` vangt ook de fouten die je niet verwachtte, zoals een `NullReferenceException` uit je eigen code. Vang op wat je verwacht en waar je iets zinnigs mee kan doen.
- Een variabele die je in het `try`-blok maakt, bestaat niet meer na het blok. Heb je hem nadien nodig, maak hem dan vóór de `try`.
- Code die in hetzelfde blok na een `throw` staat, wordt nooit uitgevoerd.

Voorspel de uitkomst

Vraag 1

```
try
{
    Console.WriteLine("A");
    int.Parse("x");
    Console.WriteLine("B");
}
catch (FormatException)
{
    Console.WriteLine("C");
}
finally
{
    Console.WriteLine("D");
}
Console.WriteLine("E");
```

Toon het antwoord ›

Vraag 2

```
try
{
    var d = new Dictionary<string, int>();
    Console.WriteLine(d["x"]);
}
catch (FormatException)
{
    Console.WriteLine("opgevangen");
}
```

Toon het antwoord ›

Vraag 3

```
try
{
    int.Parse("x");
}
catch (Exception)
{
    Console.WriteLine("algemeen");
}
catch (FormatException)
{
    Console.WriteLine("formaat");
}
```

Toon het antwoord ›

Vraag 4

```
try
{
    throw new InvalidOperationException("Rij is leeg");
}
catch (Exception ex)
{
    Console.WriteLine(ex.GetType().Name);
    Console.WriteLine(ex.Message);
}
```

Toon het antwoord ›

Vraag 5

```
string Test()
{
    try
    {
        return "uit try";
    }
    finally
    {
        Console.WriteLine("finally");
    }
}

Console.WriteLine(Test());
```

Toon het antwoord ›

Antwoorden

VRAAG 1

`A`, `C`, `D` en daarna `E`

`A` komt nog op het scherm. Daarna gooit `int.Parse` een uitzondering, en de rest van het `try`-blok — de `B` — wordt overgeslagen. Het `catch`-blok toont `C`, `finally` toont `D`, en omdat de uitzondering opgevangen is, loopt het programma gewoon verder met `E`.

‹ Terug naar de vraag

VRAAG 2

`KeyNotFoundException` — het programma crasht.

De sleutel `"x"` bestaat niet, en dat geeft een `KeyNotFoundException`. Het `catch`-blok vangt enkel een `FormatException` op. Een andere soort gaat er gewoon langs.

‹ Terug naar de vraag

VRAAG 3

`Compileert niet.`

`FormatException` erft van `Exception`. Het eerste blok vangt dus al alles op, en het tweede kan nooit aan de beurt komen. C# weigert dat. Draai de volgorde om: de specifieke soort eerst, de algemene als laatste.

‹ Terug naar de vraag

VRAAG 4

`InvalidOperationException` en daarna `Rij is leeg`

`catch (Exception ex)` vangt elke soort op, maar `ex` blijft wat het echt is. Net als bij de fiets in hoofdstuk 20 geeft `GetType()` het echte type, niet dat van de variabele.

‹ Terug naar de vraag

VRAAG 5

`finally` en daarna `uit try`

De `return` legt het antwoord klaar, maar vóór de methode echt stopt, draait `finally` nog. Pas daarna komt het antwoord bij `Console.WriteLine` aan. Dat is wat *altijd* betekent.

‹ Terug naar de vraag

24. static, const en readonly

CONCEPT

Wat het is

Wat bij de klasse hoort, en wat bij één object. Denk aan een school. Elk klaslokaal heeft zijn eigen bord: wat erop staat, verschilt per lokaal. Maar er is maar één schoolbel, en als die gaat, hoort iedereen hem.

Een gewoon veld is het bord: elk object heeft er zijn eigen exemplaar van. Een `static` veld is de bel: er is er één, en die hoort bij de klasse zelf.

Daarnaast zijn er twee manieren om te zeggen dat iets niet meer mag veranderen: `const` en `readonly`.

Hoe het eruitziet

```
public class Teller
{
    public static int Totaal = 0;    // één voor alle tellers samen
    public int Eigen = 0;            // één per teller

    public void Tik()
    {
        Totaal++;
        Eigen++;
    }
}

var a = new Teller();
var b = new Teller();

a.Tik();
a.Tik();
b.Tik();

Console.WriteLine(Teller.Totaal);   // 3 — via de klasse, niet via a of b
Console.WriteLine(a.Eigen);         // 2
Console.WriteLine(b.Eigen);         // 1
```

Je kent `static` al zonder het te weten. `Math.Max`, `int.Parse` en `Console.WriteLine` roep je op via de naam van de klasse, zonder eerst een object te maken. Dat zijn `static` methodes.

En zo zien `const` en `readonly` eruit:

```
public class Bestelling
{
    public const int MaxAantal = 10;                // ligt vast bij het compileren

    private readonly DateTime _aangemaakt;           // ligt vast na de constructor

    public Bestelling()
    {
        _aangemaakt = DateTime.Now;                 // hier mag het nog
    }

    public void Wijzig()
    {
        // _aangemaakt = DateTime.Now;               compileert niet
    }
}
```

Waarom dit in je cursus zo belangrijk is

Les 02. De lijst in de controller is daar `private static List<Laptop>`. ASP.NET Core maakt voor elk verzoek een nieuw controller-object. Zonder `static` hoort de lijst bij dat ene object en begint elk verzoek met een verse lijst: wat je met een POST toevoegde, is bij de volgende GET verdwenen. Met `static` hoort de lijst bij de klasse, en blijft ze bestaan zolang de applicatie draait.

Vanaf les 03. Een repository of logger die je via de constructor krijgt, bewaar je in een `private readonly` veld. Zo kan niemand dat veld later in de klasse nog vervangen. Hoe dat precies werkt, staat in hoofdstuk 27.

Wat je ermee kan

Vorm	Wat het doet	Voorbeeld	Resultaat
`static` veld	één waarde voor de hele klasse	`public static int Totaal;`	alle objecten delen dezelfde
`static` methode	oproepen zonder object	`Teller.Reset()`	zoals `Math.Max`
`static class`	een klasse met enkel `static` leden	`public static class Omrekenen`	`new Omrekenen()` compileert niet
Klasse.Lid	een `static` lid gebruik je via de klassenaam	`Teller.Totaal`	3
`const`	een vaste waarde, al bekend bij het compileren	`const int MaxAantal = 10;`	kan nooit veranderen
`readonly`	invullen bij de declaratie of in de constructor, daarna nooit meer	`private readonly ILogger _logger;`	toewijzen in een andere methode compileert niet
`static readonly`	één vaste waarde voor de hele klasse, die geen `const` kan zijn	`static readonly DateTime Start = DateTime.Now;`	wordt één keer ingevuld

Wat het ook kan, via iets anders

static, const en readonly die je al kent

Klasse	Voorbeelden	Soort	Hoofdstuk
`string`	`string.Join`, `string.IsNullOrEmpty`, `string.Empty`	`static` methodes; `Empty` is `static readonly`	1
`int`, `double`	`int.Parse`, `int.TryParse`, `int.MaxValue`	`static` methodes; `MaxValue` is een `const`	2
`Math`, `Convert`	`Math.Round`, `Math.PI`, `Convert.ToInt32`	`static class`; `PI` is een `const`	2
`DateTime`	`DateTime.Now`, `DateTime.Parse`	`static`	3
`Array`	`Array.Sort`, `Array.IndexOf`	`static` methodes	5
`Enumerable`	`Enumerable.Range`, en elke LINQ-methode	`static`: `getallen.Where(...)` is een kortere schrijfwijze voor `Enumerable.Where(getallen, ...)`	11
`Enum`	`Enum.Parse`, `Enum.GetValues`	`static` methodes	18
`Task`	`Task.Delay`	`static` methode	29
`File`, `Path`, `JsonSerializer`	`File.ReadAllText`, `Path.Combine`, `JsonSerializer.Serialize`	`static class`	30

Een lijst echt alleen-lezen maken

`readonly` beschermt het veld, niet wat erin zit. Wil je dat niemand nog iets toevoegt, geef dan een alleen-lezen kijk op de lijst:

Je schrijft	Wat het doet
`lijst.AsReadOnly()`	een `ReadOnlyCollection` zonder `Add` en `Remove` (hoofdstuk 6)
een eigenschap van het type `IReadOnlyList<T>`	wie de eigenschap gebruikt, ziet enkel lezende methodes (hoofdstuk 12)

Het blijft een kijk op dezelfde lijst: voeg je iets toe aan de originele lijst, dan zie je het ook in de alleen-lezen versie.

Wanneer wel, wanneer niet

Wel — `static` voor wat echt bij de klasse hoort: een teller over alle objecten, een hulpmethode die geen gegevens van een object nodig heeft. `const` voor een waarde die nooit verandert, zoals een maximum. `readonly` voor een veld dat je één keer invult en daarna met rust laat, zoals alles wat je via de constructor krijgt.

Niet — `static` voor gegevens die bij één object horen. De `static` lijst uit les 02 is een tijdelijke oplossing: zodra er een database is, verdwijnt ze.

Veelgemaakte fouten

LET OP

- Een `static` methode kan niet aan de gewone velden van de klasse. Ze hoort bij de klasse, niet bij één object, dus ze weet niet wélk object je bedoelt.
- `readonly` beschermt het veld, niet wat erin zit. Een `readonly` lijst kan je niet vervangen door een andere lijst, maar je kan er wel nog items aan toevoegen.
- `const` werkt enkel met waarden die al vastliggen bij het compileren: getallen, tekst, `true` en `false`. `const List<int>` of `const DateTime` compileert niet. Gebruik dan `static readonly`.
- Een `static` lid roep je op via de klassenaam. `a.Totaal` compileert niet, `Teller.Totaal` wel.

Voorspel de uitkomst

Vraag 1

Deze vraag gebruikt de klasse `Teller` van hierboven.

```
var a = new Teller();
var b = new Teller();
var c = new Teller();

a.Tik();
b.Tik();
b.Tik();

Console.WriteLine(Teller.Totaal);
Console.WriteLine(c.Eigen);
```

Toon het antwoord ›

Vraag 2

```
public class Mand
{
    private List<string> _items = new List<string>();
    private static List<string> _gedeeld = new List<string>();

    public void Voeg(string item)
    {
        _items.Add(item);
        _gedeeld.Add(item);
    }

    public string Tel()
    {
        return _items.Count + " en " + _gedeeld.Count;
    }
}

new Mand().Voeg("appel");
new Mand().Voeg("peer");
Console.WriteLine(new Mand().Tel());
```

Toon het antwoord ›

Vraag 3

```
public class Klas
{
    private readonly List<string> _namen = new List<string>();

    public void Voeg(string naam)
    {
        _namen.Add(naam);
    }

    public int Aantal()
    {
        return _namen.Count;
    }
}

var k = new Klas();
k.Voeg("Anna");
k.Voeg("Bram");
Console.WriteLine(k.Aantal());
```

Toon het antwoord ›

Vraag 4

```
public class Rekenhulp
{
    private int _laatste = 0;

    public static int Dubbel(int x)
    {
        _laatste = x * 2;
        return _laatste;
    }
}
```

Toon het antwoord ›

Vraag 5

```
var basis = new List<int> { 1, 2, 3 };
var alleenLezen = basis.AsReadOnly();
basis.Add(4);
Console.WriteLine(alleenLezen.Count);
```

Toon het antwoord ›

Antwoorden

VRAAG 1

`3` en daarna `0`

Er werd drie keer getikt, en het totaal is van iedereen samen. `c` heeft zelf nooit getikt, dus zijn eigen teller staat nog op nul.

‹ Terug naar de vraag

VRAAG 2

`0 en 2`

Elke `new Mand()` is een nieuwe mand met een lege eigen lijst. De gedeelde lijst hoort bij de klasse en onthoudt alles. Dit is precies wat er in een controller gebeurt: elk verzoek krijgt een nieuw object, en enkel wat `static` is, blijft.

‹ Terug naar de vraag

VRAAG 3

`2`

`readonly` zegt enkel dat `_namen` altijd naar dezelfde lijst wijst. Wat er in die lijst zit, mag gewoon veranderen. `_namen = new List<string>();` in `Voeg` had niet gecompileerd.

‹ Terug naar de vraag

VRAAG 4

`Compileert niet.`

`Dubbel` is `static` en hoort bij de klasse. `_laatste` is een gewoon veld en hoort bij één object. Als je `Rekenhulp.Dubbel(5)` oproept zonder object, is er geen `_laatste` om in te schrijven.

‹ Terug naar de vraag

VRAAG 5

`4`

`AsReadOnly` maakt geen kopie, maar een kijk op dezelfde lijst. Via `alleenLezen` kan je niets toevoegen, via `basis` wel, en dat zie je overal.

‹ Terug naar de vraag

Deel 5 — In je API

Wat je nodig hebt zodra je van een consoletoepassing naar een API overstapt.

25. Attributen en routes

IN JE API

Wat het is

Een etiket op je code. Een attribuut staat tussen vierkante haken boven een klasse, een methode of een parameter. Het verandert niets aan wat de code doet, maar het vertelt het framework iets erover. `[HttpGet]` boven een methode zegt: *deze methode antwoordt op een GET-verzoek*.

Voor een API is het belangrijkste wat attributen regelen de **route**: welke URL bij welke methode uitkomt.

Hoe het eruitziet

```
[ApiController]
[Route("api/[controller]")]
public class ReceptController : ControllerBase
{
    [HttpGet]
    public ActionResult<List<Recept>> GetAlle() { /* ... */ }

    [HttpGet("{id}")]
    public ActionResult<Recept> GetById(int id) { /* ... */ }

    [HttpGet("keuken/{keuken}")]
    public ActionResult<List<Recept>> GetPerKeuken(string keuken) { /* ... */ }

    [HttpPost]
    public ActionResult<Recept> VoegToe(Recept recept) { /* ... */ }

    [HttpPut("{id}")]
    public IActionResult Wijzig(int id, Recept recept) { /* ... */ }

    [HttpDelete("{id}")]
    public IActionResult Verwijder(int id) { /* ... */ }
}
```

De route van de klasse en die van de methode worden aan elkaar geplakt:

Verzoek	Komt uit bij	Met
`GET api/recept`	`GetAlle`	—
`GET api/recept/5`	`GetById`	`id` is 5
`GET api/recept/keuken/italiaans`	`GetPerKeuken`	`keuken` is `"italiaans"`
`POST api/recept`	`VoegToe`	`recept` komt uit de JSON in de body
`PUT api/recept/5`	`Wijzig`	`id` uit de route, `recept` uit de body
`DELETE api/recept/5`	`Verwijder`	`id` is 5

`[controller]` wordt vervangen door de naam van de klasse zonder het woord *Controller*: van `ReceptController` blijft `Recept` over. Hoofdletters maken in een route niet uit: `api/Recept` en `api/recept` komen bij dezelfde methode uit. De waarde zelf behoudt wel haar hoofdletters: bij `keuken/Italiaans` is `keuken` gelijk aan `"Italiaans"`.

Waar een parameter vandaan komt

Met `[ApiController]` op de klasse beslist ASP.NET Core zelf waar elke parameter vandaan komt:

1. Staat de naam tussen accolades in de route, dan komt hij uit de route.
2. Is het een object, zoals een `Recept`, dan komt het uit de body.
3. Al de rest komt uit de querystring: het stuk na het vraagteken, zoals `?zoek=pasta`.

Je kan het ook zelf zeggen met `[FromRoute]`, `[FromBody]` of `[FromQuery]` vóór de parameter. Met `[ApiController]` heb je dat zelden nodig.

Wat je ermee kan

Vorm	Wat het doet	Voorbeeld	Resultaat
`[ApiController]`	zet het API-gedrag aan: parameters herkennen, automatisch 400 bij ongeldige invoer	boven de klasse	—
`[Route("api/[controller]")]`	de basisroute van de klasse	op `ReceptController`	`api/recept`
`[Route("api/keuken")]`	een vaste route, los van de klassenaam	op `ReceptController`	`api/keuken`
`[HttpGet]`	antwoordt op GET, op de basisroute	`GetAlle()`	`GET api/recept`
`[HttpGet("{id}")]`	met een routeparameter; de naam tussen accolades moet die van de parameter zijn	`GetById(int id)`	`GET api/recept/5`
`[HttpGet("{id:int}")]`	routeparameter met een beperking: enkel een geheel getal	`GetById(int id)`	`GET api/recept/abc` geeft 404
`[HttpGet("vast/{x}")]`	een vast stuk en een parameter	`keuken/{keuken}`	`GET api/recept/keuken/thais`
`[HttpGet("{a}/{b}")]`	twee parameters in één route	`Bereken(int a, int b)`	`GET api/recept/2/3`
`[HttpPost]`	antwoordt op POST	`VoegToe(Recept r)`	`POST api/recept`
`[HttpPut("{id}")]`	antwoordt op PUT	`Wijzig(int id, Recept r)`	`PUT api/recept/5`
`[HttpDelete("{id}")]`	antwoordt op DELETE	`Verwijder(int id)`	`DELETE api/recept/5`
`[FromBody]`	haal deze parameter uit de body	`VoegToe([FromBody] Recept r)`	de JSON wordt een `Recept`
`[FromQuery]`	haal deze parameter uit de querystring	`Zoek([FromQuery] string zoek)`	`api/recept?zoek=pasta`
`[FromRoute]`	haal deze parameter uit de route	`GetById([FromRoute] int id)`	zoals zonder attribuut

Veelgemaakte fouten

LET OP

- Twee methodes met hetzelfde verb en dezelfde route compileren, maar zodra een verzoek binnenkomt, weet ASP.NET Core niet welke het moet nemen. Dat geeft `500` met een `AmbiguousMatchException`. `{id}` en `{naam}` op dezelfde plaats tellen als dezelfde route.
- De naam tussen accolades moet exact die van de parameter zijn. `[HttpGet("{id}")]` met `GetById(int nummer)` vult `nummer` niet uit de route, en dan blijft het op 0.
- Een route in een methode-attribuut die begint met `/` negeert de route van de klasse. `[HttpGet("/zoek")]` is gewoon `zoek`, zonder `api/recept` ervoor.
- Een waarde uit de URL behoudt haar hoofdletters. Vergelijk je ze met een vaste tekst, denk dan aan `Equals` met `OrdinalIgnoreCase` uit hoofdstuk 1.

Voorspel de uitkomst

Bij een API voorspel je wat de client terugkrijgt. Deze vragen gebruiken de `ReceptController` van hierboven.

Vraag 1

```
GET api/recept/abc
```

Toon het antwoord ›

Vraag 2

Dezelfde controller, maar nu met `[HttpGet("{id:int}")]` boven `GetById`.

```
GET api/recept/abc
```

Toon het antwoord ›

Vraag 3

Iemand voegt deze methode toe aan de `ReceptController`:

```
[HttpGet("{naam}")]
public ActionResult<Recept> GetByNaam(string naam) { /* ... */ }

GET api/recept/5
```

Toon het antwoord ›

Antwoorden

VRAAG 1

`400 Bad Request`

De route `{id}` past: er staat iets na `api/recept/`. Maar `abc` kan geen `int` worden. `[ApiController]` merkt dat en stuurt zelf een 400 terug, nog voor `GetById` draait.

‹ Terug naar de vraag

VRAAG 2

`404 Not Found`

Door de beperking `:int` past de route niet meer bij `abc`. Er is dus geen enkele methode voor dit adres, en dat is een 404. Vergelijk met vraag 1: een kleine beperking, een andere statuscode.

‹ Terug naar de vraag

VRAAG 3

`500 Internal Server Error`

`api/recept/{id}` en `api/recept/{naam}` zijn voor de route hetzelfde: een stuk tekst na `api/recept/`. Er passen nu twee methodes, en ASP.NET Core weigert te kiezen. Het compileerde nochtans zonder fout.

‹ Terug naar de vraag

26. De helpers van een controller

IN JE API

Deze erf je van `ControllerBase`. Ze maken een antwoord met de juiste statuscode.

Methode	Statuscode	Voorbeeld
`Ok`	`200 OK`	`Ok(fans)`
`CreatedAtAction`	`201 Created`	`CreatedAtAction(nameof(GetFanById), new { id = fan.Id }, fan)`
`NoContent`	`204 No Content`	`NoContent()`
`BadRequest`	`400 Bad Request`	`BadRequest("De prijs moet groter zijn dan 0.")`
`NotFound`	`404 Not Found`	`NotFound($"Geen fan met Id {id}.")`
`StatusCode`	wat je zelf opgeeft	`StatusCode(500)`

Ze nemen allemaal een tekst of een object mee, en dat is wat de client in de body ziet. Een `NotFound()` zonder tekst is geldig maar zegt niets; een `NotFound()` met een zin erbij scheelt de volgende persoon tien minuten zoeken.

> **OPMERKING**
>
> Geef je een verzameling mee, zoals `Ok(fans)`, dan wordt die JSON zoals in hoofdstuk 30: een `List`, array of `HashSet` wordt een rij tussen `[ ]`, een `Dictionary` een object tussen `{ }`. ASP.NET Core schrijft de namen van eigenschappen met een kleine beginletter.

`ActionResult<T>` tegenover `IActionResult`

Een methode die een van deze helpers teruggeeft, heeft een terugkeertype nodig dat zowel een statuscode als gegevens kan dragen. In de cursus kom je er twee tegen: `ActionResult<T>` vanaf les 02, en `IActionResult` vanaf les 06, bij PUT en DELETE.

	`ActionResult<T>`	`IActionResult`
Zegt wat er terugkomt	ja: een `T`	nee
`return recept;`	mag, wordt `200 OK`	`compileert niet` — schrijf `return Ok(recept);`
`return NotFound();`	mag	mag
Scalar toont het type van het antwoord	ja	nee
Typisch gebruik	GET en POST, die iets teruggeven	PUT en DELETE, die enkel een statuscode teruggeven

> **LET OP**
>
> `ActionResult<T>` kan een waarde enkel rechtstreeks teruggeven als `T` een klasse is, geen interface. Bij `ActionResult<IEnumerable<Recept>>` compileert `return resultaat;` niet als `resultaat` zelf als `IEnumerable` getypt is. Maak er eerst met `ToList()` een lijst van, of kies meteen `ActionResult<List<Recept>>`.

Voorspel de uitkomst

Bij een API voorspel je geen tekst op het scherm, maar wat de client terugkrijgt: een statuscode en een body.

Vraag 1

Een methode eindigt met:

```
return NotFound("Onbekend");
```

Toon het antwoord ›

Vraag 2

Na een geslaagde DELETE eindigt de methode met:

```
return NoContent();
```

Toon het antwoord ›

Vraag 3

Een methode zoekt met `First` en het gevraagde Id bestaat niet:

```
var dier = dieren.First(d => d.Id == id);
return Ok(dier);
```

Toon het antwoord ›

Antwoorden

VRAAG 1

`404 Not Found`, met in de body de tekst `Onbekend`.

‹ Terug naar de vraag

VRAAG 2

`204 No Content`, met een lege body.

Dat is geen vergetelheid. Er valt na een verwijdering niets meer te tonen, en de statuscode zegt al dat het gelukt is.

‹ Terug naar de vraag

VRAAG 3

`500 Internal Server Error`

`First` crasht zodra er niets gevonden wordt, en die crash wordt nergens opgevangen. De server meldt dan een fout aan zijn kant — terwijl de fout eigenlijk bij de client lag, die een Id vroeg dat niet bestaat. Met `FirstOrDefault`, een controle op null en een `NotFound` was dit een nette 404 geworden. Kijk nog eens naar hoofdstuk 11.

‹ Terug naar de vraag

27. Program.cs en dependency injection

IN JE API

Wat het is

Het startpunt en de schakelkast van je API. `Program.cs` draait één keer, bij het opstarten. Het doet vier dingen, altijd in deze volgorde:

1. een **bouwer** maken;
2. bij die bouwer **registreren** wat je API nodig heeft: de diensten;
3. de app **bouwen**;
4. vastleggen welke weg elk verzoek aflegt, en de app **starten**.

Hoe het eruitziet

```
var builder = WebApplication.CreateBuilder(args);

// 2. Diensten registreren
builder.Services.AddControllers();
builder.Services.AddOpenApi();
builder.Services.AddScoped<IReceptRepository, ReceptRepository>();

// 3. Bouwen
var app = builder.Build();

// 4. De weg van een verzoek, en starten
if (app.Environment.IsDevelopment())
{
    app.MapOpenApi();
    app.MapScalarApiReference();
}

app.UseHttpsRedirection();
app.UseAuthorization();
app.MapControllers();

app.Run();
```

Alles wat met `builder.Services` begint, staat vóór `builder.Build()`. Alles wat met `app.` begint, staat erna.

Dependency injection: vragen in plaats van zelf maken

Een controller heeft vaak iets anders nodig, zoals een repository. Hij maakt dat niet zelf met `new`. Hij **vraagt** erom in zijn constructor:

```
public class ReceptController : ControllerBase
{
    private readonly IReceptRepository _repository;

    public ReceptController(IReceptRepository repository)
    {
        _repository = repository;
    }
}
```

Komt er een verzoek binnen, dan ziet ASP.NET Core dat de controller een `IReceptRepository` nodig heeft. Het zoekt in de registraties welke klasse daarbij hoort, maakt er een object van en geeft dat mee. De controller kent enkel het contract uit hoofdstuk 19. Zo kan je later een andere klasse registreren, bijvoorbeeld een die een database gebruikt, zonder één letter in de controller te veranderen.

Het `private readonly` veld uit hoofdstuk 24 bewaart wat je kreeg, zodat geen andere methode het nog kan vervangen.

Hoe lang een dienst leeft

Registratie	Hoe vaak er een nieuw object komt	Typisch voor
`AddTransient<I, K>()`	elke keer dat iemand erom vraagt	kleine diensten zonder eigen gegevens
`AddScoped<I, K>()`	één per verzoek	repositories en de databasecontext; de cursus gebruikt dit
`AddSingleton<I, K>()`	één voor de hele levensduur van de app	iets dat iedereen moet delen

> **BELANGRIJK**
>
> Bij `AddScoped` krijgt elk verzoek een nieuwe repository. Bewaart die repository haar gegevens in een gewone lijst, dan begint elk verzoek met een verse lijst: wat je met een POST toevoegde, is bij de volgende GET weg. Dat is hetzelfde probleem als bij de controller in hoofdstuk 24. Met een database speelt het niet, want die onthoudt de gegevens zelf.

Wat je ermee kan

Vorm	Wat het doet	Voorbeeld	Result
`WebApplication.CreateBuilder`	maak de bouwer	`var builder = WebApplication.CreateBuilder(args);`	de bou standa
`AddControllers`	zet controllers aan	`builder.Services.AddControllers();`	zonde contro
`AddOpenApi`	beschrijf je API voor Scalar	`builder.Services.AddOpenApi();`	—
`AddScoped`	registreer: één object per verzoek	`AddScoped<IReceptRepository, ReceptRepository>()`	wie he vraagt
`AddTransient`	registreer: elke keer een nieuw object	`AddTransient<IKlok, Klok>()`	—
`AddSingleton`	registreer: één object voor de hele app	`AddSingleton<ITeller, Teller>()`	—
`GetConnectionString`	lees een verbinding uit `appsettings.json`	`builder.Configuration.GetConnectionString("Standaard")`	de tek Conne
`Build`	bouw de app; daarna liggen de registraties vast	`var app = builder.Build();`	de app
`IsDevelopment`	draait de app op je eigen computer	`if (app.Environment.IsDevelopment())`	`true` ontwik
`MapOpenApi`, `MapScalarApiReference`	maak de beschrijving en de Scalar-pagina bereikbaar	`app.MapScalarApiReference();`	de Sca bereik
`UseHttpsRedirection`	stuur `http` door naar `https`	`app.UseHttpsRedirection();`	—
`UseAuthorization`	controleer wie wat mag	`app.UseAuthorization();`	belang beveili
`MapControllers`	koppel de routes van je controllers	`app.MapControllers();`	zonde route
`Run`	start de app en blijf luisteren	`app.Run();`	wat er pas als

> **OPMERKING**
>
> In nieuwere code zie je soms een kortere schrijfwijze, een *primary constructor*: `public class ReceptController(IReceptRepository repository) : ControllerBase`. De parameter staat dan meteen achter de klassenaam, en het veld en de constructor vallen weg. Het werkt op dezelfde manier.

Veelgemaakte fouten

LET OP

- Vraagt een controller iets dat niet geregistreerd is, dan compileert alles. Pas wanneer er een verzoek binnenkomt, krijg je een `InvalidOperationException`: *Unable to resolve service for type…* De client ziet `500`.
- Een registratie na `builder.Build()` komt te laat. Zet ze er altijd vóór.
- Vergeet je `app.MapControllers()`, dan start de app gewoon, maar geeft elke route 404.
- `new ReceptRepository()` in een controller werkt, maar breekt het hele idee. De controller hangt dan weer vast aan één klasse.

Voorspel de uitkomst

Vraag 1

Een repository bewaart recepten in een gewone lijst, geen `static`. Ze is geregistreerd met `AddScoped`. Je doet een POST die één recept toevoegt, en daarna een GET die alle recepten opvraagt. De lijst begon leeg.

Hoeveel recepten geeft de GET terug? En als de repository met `AddSingleton` geregistreerd was?

Toon het antwoord ›

Vraag 2

In `Program.cs` ontbreekt de regel met `AddScoped<IReceptRepository, ReceptRepository>()`. De `ReceptController` vraagt in zijn constructor een `IReceptRepository`. De app start, en je doet `GET api/recept`.

Toon het antwoord ›

Vraag 3

```
app.MapControllers();

app.Run();

Console.WriteLine("De API draait.");
```

Toon het antwoord ›

Antwoorden

VRAAG 1

Met `AddScoped`: `0`. Met `AddSingleton`: `1`.

`AddScoped` geeft elk verzoek een nieuwe repository, met een nieuwe, lege lijst. Het recept van de POST zat in de repository van dat ene verzoek. Bij `AddSingleton` is er één repository voor de hele app, en die onthoudt het recept.

‹ Terug naar de vraag

VRAAG 2

`500 Internal Server Error`

De app start gewoon, want niemand heeft de controller al nodig. Bij het eerste verzoek wil ASP.NET Core er een maken, vindt geen klasse voor `IReceptRepository`, en geeft een `InvalidOperationException`.

‹ Terug naar de vraag

VRAAG 3

Zolang de API draait, verschijnt er niets.

`app.Run()` keert pas terug wanneer de app stopt. Wil je iets melden bij het opstarten, dan moet het vóór `app.Run()` staan.

‹ Terug naar de vraag

28. Logging

IN JE API

Wat het is

Een logboek dat je API zelf bijhoudt. Terwijl de API draait, schrijft hij in het terminalvenster wat er gebeurt: welk verzoek er binnenkwam, wat er niet gevonden werd, wat er misliep. Zo zie je achteraf wat er gebeurd is, ook als niemand op dat moment keek.

Elke melding krijgt een **niveau**, van een detail dat enkel tijdens het ontwikkelen nuttig is, tot een fout die meteen aandacht vraagt.

Hoe het eruitziet

```
public class ReceptController : ControllerBase
{
    private readonly ILogger<ReceptController> _logger;

    public ReceptController(ILogger<ReceptController> logger)
    {
        _logger = logger;
    }

    [HttpGet("{id}")]
    public ActionResult<Recept> GetById(int id)
    {
        _logger.LogInformation("Recept {Id} opgevraagd", id);

        // ...

        _logger.LogWarning("Recept {Id} niet gevonden", id);
        return NotFound();
    }
}
```

De logger vraag je in de constructor, net als een repository in hoofdstuk 27. Registreren hoeft niet: ASP.NET Core heeft hem al klaarstaan. Tussen de punthaken staat de klasse die logt. Die naam komt mee in het logboek, zodat je ziet waar een melding vandaan komt.

De niveaus

Methode	Wanneer	Standaard zichtbaar
`LogTrace`	de kleinste details	nee
`LogDebug`	details die enkel tijdens het ontwikkelen helpen	nee
`LogInformation`	de normale werking: een verzoek, een geslaagde actie	ja
`LogWarning`	iets onverwachts, maar de app werkt verder: een Id dat niet bestaat	ja
`LogError`	iets is misgelopen: een actie lukte niet	ja
`LogCritical`	de app zelf zit in de problemen	ja

Welke niveaus je ziet, staat in `appsettings.json`, onder `Logging` en `LogLevel`. Een nieuw project zet `Default` op `Information`: alles vanaf dat niveau verschijnt, `Debug` en `Trace` niet.

Twee schrijfwijzen

In les 03 schrijft de cursus de melding met een `$` ervoor, zoals in hoofdstuk 1: de waarden staan meteen in de tekst. Dat werkt.

Microsoft raadt een andere vorm aan, met een **sjabloon**: de tekst bevat plaatshouders tussen accolades, en de waarden komen erachter, gescheiden door komma's. Bijvoorbeeld `_logger.LogInformation("Recept {Id} opgevraagd", id);`. In het terminalvenster zie je hetzelfde. Het verschil is dat de logger de waarden apart bewaart, zodat een loggingprogramma later kan zoeken op *alle meldingen over recept 5*. Visual Studio stelt daarom soms voor om de eerste vorm om te zetten.

Wat je ermee kan

Vorm	Wat het doet	Voorbeeld	Resultaat
`ILogger<T>`	een logger voor klasse `T`, via de constructor	`ILogger<ReceptController> logger`	de logger staat klaar
`LogInformation`	meld de normale werking	`_logger.LogInformation("Alle recepten opgevraagd");`	`info: ... Alle recepten opgevraagd`
`LogWarning`	meld iets onverwachts	`_logger.LogWarning("Recept {Id} niet gevonden", id);`	`warn: ... Recept 5 niet gevonden`
`LogError`	meld een fout	`_logger.LogError("Verwijderen van {Id} mislukt", id);`	`fail: ...`
`LogError` met uitzondering	meld een fout en geef de uitzondering mee	`_logger.LogError(ex, "Bestand niet gelezen");`	de melding, met de details van `ex` eronder
`{Naam}`	plaatshouder in een sjabloon	`"{Aantal} recepten", aantal`	`12 recepten`

Veelgemaakte fouten

LET OP

- In een sjabloon telt de **volgorde** van de waarden, niet de naam tussen de accolades. De eerste waarde komt in de eerste plaatshouder, wat er ook tussen de accolades staat.
- Een `LogDebug` die je nooit ziet, is meestal geen fout in je code. Het niveau staat in `appsettings.json` hoger ingesteld.
- Zet nooit wachtwoorden of andere geheime gegevens in een logboek. Wie het logboek kan lezen, kan ze dan ook lezen.

Voorspel de uitkomst

Vraag 1

```
_logger.LogInformation("{Peren} peren en {Appels} appels", 3, 8);
```

Toon het antwoord ›

Vraag 2

In `appsettings.json` staat `"Default": "Information"`. Een methode bevat:

```
_logger.LogDebug("Begin van de methode");
_logger.LogWarning("Geen recepten gevonden");
```

Wat verschijnt er in het terminalvenster?

Toon het antwoord ›

Antwoorden

VRAAG 1

`3 peren en 8 appels`

De eerste waarde gaat naar de eerste plaatshouder, de tweede naar de tweede. Dat de namen `Peren` en `Appels` heten, speelt geen rol bij welke waarde waar komt.

‹ Terug naar de vraag

VRAAG 2

Enkel `Geen recepten gevonden`, als waarschuwing.

`Debug` ligt lager dan `Information`, dus die melding valt weg. `Warning` ligt hoger en komt erdoor.

‹ Terug naar de vraag

29. Asynchroon werken

IN JE API

Vorm	Wat het doet
`async`	markeert een methode die mag wachten
`await`	wacht op het resultaat zonder de server te blokkeren
`Task`	een belofte dat er ooit iets klaar is
`Task<T>`	een belofte dat er ooit een `T` klaar is

De asynchrone tegenhangers die je in de latere lessen tegenkomt dragen dezelfde naam met `Async` erachter: `ToListAsync`, `FirstOrDefaultAsync`, `FindAsync`, `SaveChangesAsync`, `AnyAsync`, `CountAsync`. Ze doen hetzelfde als hun gewone versie; het verschil zit in wat de server intussen mag doen.

> **OPMERKING**
>
> Deze komen pas aan bod in les 05 van Programming Advanced. Ze staan hier zodat je ze herkent wanneer je ze in een voorbeeld ziet staan, niet om ze nu al te gebruiken. Hoe je ze met een database gebruikt, staat in hoofdstuk 32.

Hoe het eruitziet

```
async Task<int> TelLangzaamAsync()
{
    await Task.Delay(1000);    // wacht een seconde, zonder de thread vast te houden
    return 42;
}

int n = await TelLangzaamAsync();
Console.WriteLine(n);          // 42, na een seconde
```

In een controller ziet dat er zo uit:

```
[HttpGet]
public async Task<ActionResult<List<Recept>>> GetAlle()
{
    List<Recept> recepten = await _repository.GetAllAsync();
    return recepten;
}
```

Drie dingen veranderen tegenover de gewone versie: `async` voor het terugkeertype, het terugkeertype zelf in een `Task<...>`, en `await` voor elke aanroep die tijd kost.

Async is besmettelijk

Les 05 noemt het het domino-effect. Gebruik je ergens `await`, dan moet die methode `async` worden en een `Task` teruggeven. Wie die methode oproept, moet dan zelf ook `await` gebruiken, en dus ook `async` worden. Zo schuift het naar boven: van de repository, via de interface, tot in de controller.

Wat je ermee kan

Vorm	Wat het doet	Voorbeeld	Resultaat
`async Task`	een asynchrone methode die niets teruggeeft	`async Task BewaarAsync()`	—
`async Task<T>`	een asynchrone methode die een `T` teruggeeft	`async Task<int> TelAsync()`	`await TelAsync()` geeft een `int`
`await`	wacht op een `Task` en pak het resultaat uit	`int n = await TelAsync();`	`n` is een getal
`Task.Delay`	wacht een aantal milliseconden, zonder te blokkeren	`await Task.Delay(500);`	een halve seconde later gaat het verder
`Async` achter de naam	de afspraak voor asynchrone methodes	`GetAllAsync`, `SaveChangesAsync`	je ziet meteen dat er een `await` bij hoort
`.Result`, **`.Wait()`**	wacht door de thread vast te houden	`TelAsync().Result`	werkt, maar blokkeert; niet doen
`async void`	een asynchrone methode zonder `Task`	`async void Bewaar()`	fouten erin kan niemand opvangen; niet doen

Wat het ook kan, via iets anders

Asynchrone methodes die je tegenkomt

Methode	Geeft na `await`	Hoofdstuk
`ToListAsync`, `FirstOrDefaultAsync`, `CountAsync`, …	een `List<T>`, een object of null, een getal	32
`SaveChangesAsync`	het aantal geschreven rijen	32
`QueryAsync<T>`, `ExecuteAsync`	een `IEnumerable<T>`, het aantal geraakte rijen	34
`File.ReadAllTextAsync`, `File.WriteAllTextAsync`	de tekst, of niets	30
`Task.Delay`	niets	hierboven

Pas na `await` heb je wat erin zit. Zonder `await` heb je een `Task<List<Recept>>`, en die heeft geen `Count`, geen `[0]` en geen LINQ. Met `await` heb je een gewone `List<Recept>` met alles van hoofdstuk 6.

Bestaat ook, maar kom je in deze cursus niet tegen

`Task.WhenAll` (wacht op meerdere taken tegelijk), `Task.WhenAny`, `await foreach` over een `IAsyncEnumerable<T>`, `CancellationToken` (een taak afbreken).

Veelgemaakte fouten

> LET OP
>
> - Een vergeten `await` compileert vaak gewoon, met een waarschuwing. Je code gaat dan verder voor het werk klaar is.
> - `.Result` en `.Wait()` maken van asynchrone code weer gewone, wachtende code. Alle winst is weg, en in sommige toepassingen loopt het programma vast, omdat twee stukken op elkaar wachten (een *deadlock*). Les 05 zegt het kort: hou het bij `async` en `await`.
> - `async void` geeft geen `Task` terug. Wie de methode oproept, kan er dus niet op wachten en geen fout opvangen. Gebruik altijd `async Task` of `async Task<T>`.

Voorspel de uitkomst

Stel dat er een methode bestaat met deze kop:

```
Task<int> TelAsync()
```

Vraag 1

```
var x = await TelAsync();
```

Van welk type is `x`?

Toon het antwoord ›

Vraag 2

```
var x = TelAsync();
```

Van welk type is `x` nu?

Toon het antwoord ›

Vraag 3

```
async Task SchrijfAsync()
{
    Console.WriteLine("B");
    await Task.Delay(100);
    Console.WriteLine("D");
}

Console.WriteLine("A");
Task t = SchrijfAsync();
Console.WriteLine("C");
await t;
Console.WriteLine("E");
```

Toon het antwoord ›

Vraag 4

```
async Task<int> GeefAsync()
{
    await Task.Delay(10);
    return 5;
}

int x = GeefAsync() + 1;
```

Toon het antwoord ›

Vraag 5

`GetAllAsync` geeft een `Task<List<Recept>>`.

```
var recepten = _repository.GetAllAsync();
Console.WriteLine(recepten.Count);
```

Toon het antwoord ›

Antwoorden

VRAAG 1

`int`

`await` wacht tot de belofte ingelost is en geeft je wat erin zit: het getal.

‹ Terug naar de vraag

VRAAG 2

`Task<int>`

Zonder `await` krijg je de belofte zelf, niet het getal. Probeer je er daarna mee te rekenen, dan compileert het niet. Een vergeten `await` is een van de meest voorkomende fouten zodra je asynchroon begint te werken.

‹ Terug naar de vraag

VRAAG 3

`A`, `B`, `C`, `D` en daarna `E`

`SchrijfAsync` begint meteen en schrijft `B`. Bij de eerste `await` die echt moet wachten, geeft de methode de beurt terug. Daardoor komt `C` vóór `D`. Pas bij `await t` wacht het hoofdprogramma tot de methode helemaal klaar is, en dus komt `E` als laatste.

‹ Terug naar de vraag

VRAAG 4

Compileert niet.

Zonder `await` is `GeefAsync()` een `Task<int>`, een belofte, en een belofte plus 1 bestaat niet. Met `await GeefAsync() + 1` lukt het wel: eerst het getal uitpakken, dan optellen.

‹ Terug naar de vraag

VRAAG 5

Compileert niet

Zonder `await` is `recepten` een `Task<List<Recept>>`: een belofte, nog geen lijst. Een `Task` heeft geen `Count`. Met `var recepten = await _repository.GetAllAsync();` lukt het wel.

‹ Terug naar de vraag

30. JSON en bestanden

IN JE API

Methode	Wat het doet
`JsonSerializer.Serialize`	maak van een object een JSON-tekst
`JsonSerializer.Deserialize`	maak van een JSON-tekst een object
`File.ReadAllText`	lees een heel bestand als tekst
`File.WriteAllText`	schrijf een tekst naar een bestand
`File.Exists`	bestaat dat bestand
`Path.Combine`	plak padstukken correct aan elkaar

Ontbreekt het bestand, of staat er geen geldige JSON in, dan krijg je een uitzondering. Hoe je die opvangt, staat in hoofdstuk 23.

> **LET OP**
>
> `JsonSerializer.Deserialize` is standaard hoofdlettergevoelig. Staat er `"krakenId"` in je bestand en heet je eigenschap `KrakenId`, dan blijft die leeg zonder één foutmelding. Dat gedrag zet je om met `PropertyNameCaseInsensitive`. Dat ASP.NET Core zelf wél soepel is met hoofdletters geldt enkel voor verzoeken die het framework zelf uitleest, niet voor een bestand dat je zelf inleest.

Wat het ook kan, via iets anders

Wat je terugkrijgt

Methode	Geeft	Daarmee werk je verder in
`File.ReadAllText`	één `string`	hoofdstuk 1
`File.ReadAllLines`	een `string[]`, één regel per vakje	hoofdstuk 5
`File.ReadLines`	een `IEnumerable<string>`: leest de regels pas terwijl je ze overloopt	hoofdstuk 12
`File.WriteAllLines`	schrijft een verzameling teksten, één per regel	—
`File.AppendAllText`	voegt tekst toe aan het einde, in plaats van te vervangen	—
`JsonSerializer.Deserialize<List<T>>`	een `List<T>`, of null als er `null` staat	hoofdstuk 6
`JsonSerializer.Deserialize<Dictionary<string, int>>`	een `Dictionary`	hoofdstuk 7

Hoe verzamelingen en types in JSON staan

In C#	In JSON
`List`, array, `HashSet`, `Queue`	een rij: `[1,2]`
`Stack`	een rij, van boven naar onder: `[3,2,1]`
`Dictionary<string, int>`	een object: `{"a":1,"b":2}`
een enum	het getal erachter: `"Licht":2`
een publiek veld (zonder `{ get; set; }`)	niets: enkel eigenschappen komen mee
een anoniem object, een klasse, een record	een object met de eigenschappen

LET OP

- Een `Stack` die je bewaart en terug inleest, staat omgekeerd: wat bovenaan lag, ligt nu onderaan.
- Een enum wordt een getal. Verander je later de volgorde van de keuzes, dan betekenen oude bestanden iets anders.

Bestaat ook, maar kom je in deze cursus niet tegen

`File.ReadAllTextAsync` en `File.WriteAllTextAsync` (asynchroon, zie hoofdstuk 29), `JsonStringEnumConverter` (een enum als naam in plaats van getal), `JsonSerializerDefaults.Web` (kleine beginletters, zoals ASP.NET Core), `Directory` (mappen).

Voorspel de uitkomst

Vraag 1

```
var p = new { Naam = "Anna", Leeftijd = 17 };
Console.WriteLine(JsonSerializer.Serialize(p));
```

Toon het antwoord ›

Vraag 2

```
public class Persoon
{
    public string Naam { get; set; } = "";
}

var p = JsonSerializer.Deserialize<Persoon>("{\"naam\":\"Anna\"}");
Console.WriteLine("[" + p!.Naam + "]");
```

Toon het antwoord ›

Vraag 3

```
var stapel = new Stack<int>();
stapel.Push(1);
stapel.Push(2);
stapel.Push(3);
string json = JsonSerializer.Serialize(stapel);
var terug = JsonSerializer.Deserialize<Stack<int>>(json);
Console.WriteLine(json);
Console.WriteLine(terug!.Peek());
```

Toon het antwoord ›

Vraag 4

```
public enum Licht { Rood, Oranje, Groen }

public class Lamp
{
    public int Nummer = 5;
    public string Naam { get; set; } = "";
    public Licht Licht { get; set; }
}

Console.WriteLine(JsonSerializer.Serialize(new Lamp { Naam = "Gang", Licht = Licht.Groen
}));
```

Toon het antwoord ›

Antwoorden

VRAAG 1

`{"Naam":"Anna","Leeftijd":17}`

Alles op één regel, en de namen precies zoals in je code, met hoofdletter. Dat een API zijn JSON met kleine letters teruggeeft, komt doordat ASP.NET Core dat voor jou instelt; `JsonSerializer` zelf doet het niet.

‹ Terug naar de vraag

VRAAG 2

`[]`

In de JSON staat `naam` met kleine letter, in de klasse `Naam` met hoofdletter. Standaard is dat voor `JsonSerializer` een andere naam, dus `Naam` blijft leeg. Er komt geen enkele foutmelding. Met `PropertyNameCaseInsensitive = true` in de opties lost je dit op.

‹ Terug naar de vraag

VRAAG 3

`[3,2,1]` en daarna `1`

Een stapel wordt van boven naar onder geschreven. Bij het inlezen gaat elk getal er opnieuw met `Push` op, dus de 1 eindigt bovenaan.

‹ Terug naar de vraag

VRAAG 4

`{"Naam":"Gang","Licht":2}`

`Nummer` is een veld, geen eigenschap, en komt niet mee. De enum wordt het getal erachter.

‹ Terug naar de vraag

Deel 6 — Data

Van gegevens in het geheugen naar gegevens in een database: bewaren, opvragen, en enkel doorgeven wat nodig is.

31. EF Core: model en database

DATA

Wat het is

Een tolk tussen je klassen en je tabellen. Entity Framework Core, kort EF Core, zet je C#-klassen om naar tabellen in een database, en de rijen uit die tabellen weer naar objecten. Je schrijft zelf geen SQL. Je beschrijft je klassen, en EF Core maakt de tabellen erbij. De cursus noemt die werkwijze *code first*: eerst de code, dan de database.

Drie stukken werken samen:

1. je **entiteiten**: gewone klassen, één per tabel;
2. de **context**: één klasse die erft van `DbContext`, met per tabel een `DbSet`;
3. de **migraties**: scripts die je database gelijkzetten met je klassen.

Dit hoofdstuk gaat over het model: hoe je tabellen en relaties vastlegt. Hoe je gegevens opvraagt en bewaart, staat in hoofdstuk 32.

Hoe het eruitziet

Twee entiteiten: een keuken heeft veel recepten, een recept hoort bij één keuken.

```
public class Keuken
{
    public int Id { get; set; }
    public string Naam { get; set; } = default!;

    public List<Recept> Recepten { get; set; } = new();
}

public class Recept
{
    public int Id { get; set; }
    public string Titel { get; set; } = default!;
    public decimal Kostprijs { get; set; }

    public int KeukenId { get; set; }                  // de vreemde sleutel
    public Keuken Keuken { get; set; } = default!;     // de navigatie-eigenschap
}
```

De context, met de verfijning in `OnModelCreating`:

```
public class ReceptContext : DbContext
{
    public ReceptContext(DbContextOptions<ReceptContext> options) : base(options) { }

    public DbSet<Keuken> Keukens { get; set; }
    public DbSet<Recept> Recepten { get; set; }

    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        base.OnModelCreating(modelBuilder);

        modelBuilder.Entity<Recept>(entity =>
        {
            entity.Property(r => r.Titel).IsRequired().HasMaxLength(100);
            entity.Property(r => r.Kostprijs).HasPrecision(8, 2);

            entity.HasOne(r => r.Keuken)
                  .WithMany(k => k.Recepten)
                  .HasForeignKey(r => r.KeukenId)
                  .OnDelete(DeleteBehavior.Restrict);
        });
    }
}
```

De registratie in `Program.cs`, met de verbinding uit `appsettings.json`:

```
var connectionString = builder.Configuration.GetConnectionString("Standaard");
builder.Services.AddDbContext<ReceptContext>(options =>
options.UseNpgsql(connectionString));

"ConnectionStrings": {
  "Standaard":
"Host=localhost;Port=5432;Database=ReceptenDb;Username=postgres;Password=..."
}
```

`AddDbContext` registreert de context zoals `AddScoped` uit hoofdstuk 27: één context per verzoek. Een repository vraagt hem gewoon in haar constructor.

Wat EF Core zelf afleidt

Veel hoef je niet te zeggen. EF Core volgt **conventies**: vaste afspraken over namen en types.

Je schrijft	EF Core maakt ervan
`DbSet<Recept> Recepten` in de context	een tabel `Recepten`
een eigenschap `Id` of `ReceptId`	de primaire sleutel; bij een `int` telt de database zelf op
`string`	een verplichte kolom van onbeperkte lengte
`string?`, `int?`	een kolom die leeg mag blijven
`decimal`	een kolom voor kommagetallen, exact, zonder afrondingsfouten
`int KeukenId` naast `Keuken Keuken`	een vreemde sleutel naar de tabel `Keukens`
`int KeukenId`	een **verplichte** relatie: elk recept heeft een keuken
`int? KeukenId`	een **optionele** relatie: een recept mag zonder keuken

Alles wat afwijkt van die afspraken, of wat je strenger wil, zet je in `OnModelCreating`. Die schrijfwijze, met methodes achter elkaar, heet de **Fluent API**. `override` en `base` ken je uit hoofdstuk 20.

Relaties

Een **navigatie-eigenschap** is een eigenschap die naar een ander object wijst, zoals `Recept.Keuken`, of naar een lijst objecten, zoals `Keuken.Recepten`. De **vreemde sleutel** is het getal in de tabel dat die koppeling bewaart, zoals `KeukenId`.

Soort	Voorbeeld	Waar de vreemde sleutel staat	Navigatie-eigenschappen
één op één	een leerling en zijn kluisje	aan één kant, die je zelf kiest	aan elke kant één object
één op veel	een keuken en haar recepten	aan de *veel*-kant: `Recept.KeukenId`	`Recept.Keuken` en de lijst `Keuken.Recepten`
veel op veel	recepten en ingrediënten	in een aparte koppeltabel, met twee vreemde sleutels	elke kant een lijst koppelobjecten

Bij veel op veel maakt de cursus de koppeltabel als een eigen klasse, met twee één-op-veel-relaties. Zo kan de koppeling zelf ook gegevens dragen, zoals de hoeveelheid van een ingrediënt in een recept.

Een relatie in de Fluent API lees je vanuit de entiteit die je configureert:

Regel	Lees als
`HasOne(r => r.Keuken)`	een recept heeft één keuken
`.WithMany(k => k.Recepten)`	die keuken heeft veel recepten
`.HasForeignKey(r => r.KeukenId)`	de koppeling zit in `KeukenId`
`.OnDelete(DeleteBehavior.Restrict)`	een keuken met recepten mag niet weg

Verwijderen: Cascade of Restrict

	`Cascade`	`Restrict`
Standaard bij	een verplichte relatie, zoals `int KeukenId`	nooit, je kiest het zelf
Een keuken met recepten verwijderen	de recepten verdwijnen mee	`SaveChanges` weigert, met een uitzondering
Het gevaar	gegevens verdwijnen zonder dat iemand het merkt	geen; je ruimt eerst zelf op

Les 06 vertelt het verhaal van klant Gerrit, die verwijderd wil worden. Met `Cascade` gaan al zijn bestellingen stilletjes mee. Met `Restrict` houdt de database tegen.

Migraties

Een migratie is een script dat beschrijft hoe de database moet veranderen om weer bij je klassen te passen. Elk script heeft een `Up`, die de wijziging uitvoert, en een `Down`, die ze terugdraait.

Package Manager Console (Visual Studio)	Terminal	Wat het doet
`Add-Migration Naam`	`dotnet ef migrations add Naam`	maak een nieuw script in de map `Migrations`
`Update-Database`	`dotnet ef database update`	voer de scripts uit die de database nog niet kent
`Remove-Migration`	`dotnet ef migrations remove`	verwijder het laatste script, zolang de database het nog niet uitvoerde

De vaste cyclus, telkens je een entiteit toevoegt of aanpast:

1. pas je klasse aan, en voeg indien nodig een `DbSet` toe;
2. `Add-Migration` met een naam die zegt wat er verandert;
3. kijk het script na: heeft EF Core begrepen wat je bedoelde?
4. `Update-Database`, met Docker aan.

Je hebt drie packages nodig: `Npgsql.EntityFrameworkCore.PostgreSQL`, `Microsoft.EntityFrameworkCore.Design` en, voor de Package Manager Console, `Microsoft.EntityFrameworkCore.Tools`. EF Core zelf komt mee met het eerste. Les 04 installeert het ook apart, en dat kan geen kwaad. In de terminal installeer je eenmalig ook het programma zelf, met `dotnet tool install --global dotnet-ef`.

> **LET OP**
>
> In les 04 staat op een paar plaatsen `dotnet Entity Framework migrations add` en `dotnet Entity Framework database update`. De commando's heten `dotnet ef migrations add` en `dotnet ef database update`. De lange vorm herkent de terminal niet.

Startgegevens met HasData

Met `HasData` geef je rijen mee die de migratie meteen in de tabel zet. Zo heeft je database na `Update-Database` al gegevens om mee te testen. Twee regels: je vult het `Id` zelf in, ook al telt de database het anders zelf op, en een relatie leg je via de vreemde sleutel, niet via de navigatie-eigenschap.

```
modelBuilder.Entity<Keuken>().HasData(
    new Keuken { Id = 1, Naam = "Italiaans" },
    new Keuken { Id = 2, Naam = "Thais" });
```

> **LET OP**
>
> PostgreSQL telt de Ids met een teller die niet weet dat jouw startgegevens 1 en 2 al gebruiken. De eerste keuken die je daarna via de API toevoegt, krijgt ook `Id` 1, en dat geeft een fout over een dubbele sleutel: een `500`. De documentatie van Npgsql noemt twee uitwegen: de teller laten beginnen boven je startgegevens, met `HasIdentityOptions`, of je startgegevens negatieve Ids geven.

Wat je ermee kan

Vorm	Wat het doet	Voorbeeld	Resultaat
`DbContext`	de basisklasse van je context	`class ReceptContext : DbContext`	de brug naar de database
`DbSet<T>`	één tabel	`public DbSet<Recept> Recepten { get; set; }`	de tabel `Recepten`
`: base(options)`	geef de opties door aan `DbContext`	in de constructor van de context	de context weet welke database
`AddDbContext`	registreer de context, één per verzoek	`builder.Services.AddDbContext<ReceptContext>(...)`	repositories kunnen hem vragen
`UseNpgsql`	gebruik PostgreSQL, met deze verbinding	`options.UseNpgsql(connectionString)`	—
`OnModelCreating`	de plaats voor de Fluent API	`protected override void OnModelCreating(ModelBuilder modelBuilder)`	—

Vorm	Wat het doet	Voorbeeld	Resultaat
`Entity<T>`	configureer één entiteit	`modelBuilder.Entity<Recept>(entity => { ... })`	—
`ToTable`	kies zelf de naam van de tabel	`entity.ToTable("Recepten")`	—
`HasKey`	kies zelf de primaire sleutel	`entity.HasKey(r => r.Code)`	nodig als de sleutel geen `Id` heet
`Property`	configureer één eigenschap	`entity.Property(r => r.Titel)`	—
`IsRequired`	de kolom mag niet leeg zijn	`.IsRequired()`	—
`HasMaxLength`	een maximale lengte voor tekst	`.HasMaxLength(100)`	hoogstens 100 tekens
`HasPrecision`	het aantal cijfers van een kommagetal	`.HasPrecision(8, 2)`	8 cijfers, waarvan 2 na de komma
`HasOne` + `WithMany`	één op veel, gelezen vanaf de *veel*-kant	`entity.HasOne(r => r.Keuken).WithMany(k => k.Recepten)`	—
`HasOne` + `WithOne`	één op één	`.HasOne(l => l.Kluisje).WithOne(k => k.Leerling)`	—
`HasForeignKey`	welke eigenschap de vreemde sleutel is	`.HasForeignKey(r => r.KeukenId)`	bij één op één met het type erbij: `HasForeignKey<Kluisje>(k => k.LeerlingId)`
`OnDelete`	wat er gebeurt als de andere kant verdwijnt	`.OnDelete(DeleteBehavior.Restrict)`	—
`HasData`	startgegevens via een migratie	`modelBuilder.Entity<Keuken>().HasData(...)`	rijen in de tabel na `Update-Database`

Wat het ook kan, via iets anders

Wat	Waar het vandaan komt	Hoofdstuk
een navigatie-eigenschap `List<Recept> Recepten`	een gewone `List`: `Count`, `Any` en LINQ werken, zodra ze geladen is	6, 32
`DbSet<T>`	een `IQueryable<T>`: LINQ wordt SQL	32
een enum als eigenschap	wordt standaard het getal erachter in de database	18
`DateTime` als eigenschap	PostgreSQL wil UTC; zie de waarschuwing in hoofdstuk 3	3

Veelgemaakte fouten

LET OP

- Een klasse aangepast, maar geen migratie gemaakt of `Update-Database` vergeten? Alles compileert, maar bij het eerste verzoek meldt PostgreSQL dat een kolom of tabel niet bestaat. De client ziet `500`.
- Heet de sleutel geen `Id`, en ook niet de naam van de klasse met `Id` erachter, en staat er geen `HasKey`, dan stopt `Add-Migration` met *The entity type '…' requires a primary key to be defined.*
- Pas de database nooit met de hand aan. De volgende migratie gaat ervan uit dat de database er nog uitziet zoals de vorige migratie hem achterliet.
- Draait Docker niet, dan kan `Update-Database` de database niet bereiken.
- Zet `base.OnModelCreating(modelBuilder)` altijd bovenaan. Bij een gewone `DbContext` merk je het verschil niet, maar erft je context later van een klasse die zelf tabellen configureert, zoals bij de beveiliging, dan krijg je zonder die regel fouten over ontbrekende sleutels.

Voorspel de uitkomst

Vraag 1

De context bevat `public DbSet<Ingredient> Ingredienten { get; set; }`, en verder niets over ingrediënten.

```
public class Ingredient
{
    public int Nummer { get; set; }
    public string Naam { get; set; } = default!;
}

Add-Migration Ingredienten
```

Toon het antwoord ›

Vraag 2

`Recept` heeft `public int KeukenId`, en in `OnModelCreating` staat niets over `OnDelete`. De keuken met Id 1 heeft drie recepten. Je verwijdert die keuken en roept `SaveChangesAsync` op.

Hoeveel recepten van die keuken staan er daarna nog in de database?

Toon het antwoord ›

Vraag 3

Hetzelfde, maar nu met `.OnDelete(DeleteBehavior.Restrict)` op de relatie. Het verwijderen gebeurt in een DELETE-methode van een controller, zonder `try` en `catch`.

Toon het antwoord ›

Vraag 4

Je voegt `public int Bereidingstijd { get; set; }` toe aan `Recept`. Je maakt een migratie met `Add-Migration`, maar vergeet `Update-Database`. Dan start je de API en doe je `GET api/recept`.

Toon het antwoord ›

Antwoorden

VRAAG 1

`Er komt geen migratie.` EF Core meldt dat `Ingredient` een primaire sleutel nodig heeft.

EF Core zoekt een eigenschap die `Id` of `IngredientId` heet. `Nummer` past bij geen van beide. Hernoem de eigenschap, of wijs ze aan met `HasKey`.

‹ Terug naar de vraag

VRAAG 2

`0`

`int KeukenId` kan niet leeg zijn, dus de relatie is verplicht. Bij een verplichte relatie kiest EF Core standaard `Cascade`: de recepten gaan mee met hun keuken. Geen foutmelding, geen waarschuwing.

‹ Terug naar de vraag

VRAAG 3

`500 Internal Server Error`, en de keuken staat nog in de database, met haar drie recepten.

`SaveChangesAsync` gooit een uitzondering, omdat er nog recepten aan de keuken hangen. Er wordt dus niets verwijderd. Wil je de client een duidelijker antwoord geven, kijk dan vóór het verwijderen of er nog recepten zijn, en geef een **`400 Bad Request`** met uitleg.

‹ Terug naar de vraag

VRAAG 4

`500 Internal Server Error`

De migratie staat klaar in je project, maar de database kent ze nog niet. EF Core vraagt de kolom `Bereidingstijd` op, en PostgreSQL meldt dat die niet bestaat. Het script maken en het script uitvoeren zijn twee aparte stappen.

‹ Terug naar de vraag

32. EF Core: opvragen en bewaren

DATA

Wat het is

Een winkelmandje. Wat je erin legt, eruit haalt of omruilt, is nog niet betaald. Pas aan de kassa gaat alles in één keer door. Bij EF Core is de context het mandje: hij noteert wat je toevoegt, wijzigt en verwijdert. Dat noteren heet de **change tracker**. De kassa is `SaveChangesAsync`: pas dan maakt EF Core er SQL van en stuurt hij alles naar de database.

Opvragen werkt anders. Een vraag gaat meteen naar de database, op het moment dat je het antwoord opeist.

De voorbeelden gebruiken de `ReceptContext` uit hoofdstuk 31, bewaard in een veld `_context`.

Hoe het eruitziet

```
// opvragen
List<Recept> alle = await _context.Recepten.ToListAsync();
Recept? vijf = await _context.Recepten.FindAsync(5);
List<Recept> goedkoop = await _context.Recepten
    .Where(r => r.Kostprijs < 5)
    .OrderBy(r => r.Titel)
    .ToListAsync();

// bewaren
_context.Recepten.Add(nieuwRecept);
await _context.SaveChangesAsync();
```

`Where`, `OrderBy` en `Select` ken je uit hoofdstuk 11. Ze werken hier net zo, met één verschil: EF Core vertaalt ze naar SQL, en de database doet het filteren. De asynchrone methodes zoals `ToListAsync` staan in de namespace `Microsoft.EntityFrameworkCore`. Zonder die `using` bovenaan kent je code ze niet.

Opvragen: de vraag vertrekt pas als je het antwoord opeist

`_context.Recepten.Where(...)` geeft een `IQueryable<Recept>`. Net als de `IEnumerable` uit hoofdstuk 12 is dat een vraag die nog niet gesteld is. Het verschil: een `IQueryable` wordt vertaald naar SQL, zodat de database het werk doet in plaats van je programma.

Bouwt de vraag verder	Stuurt de vraag naar de database
`Where`, `OrderBy`, `Select`, `Include`, `Take`, `Skip`	`ToListAsync`, `FirstOrDefaultAsync`, `FindAsync`, `CountAsync`, `AnyAsync`, een `foreach`

Elke keer dat je dezelfde `IQueryable` uitvoert, vertrekt er een nieuwe vraag naar de database. Les 05 noemt dit *deferred execution*: uitgestelde uitvoering.

> **OPMERKING**
>
> Geeft een repository een lijst terug, en filter je daarna in de controller met `Where`, dan gebeurt dat filteren in het geheugen, op alles wat de database stuurde. Zet een filter dus in de vraag zelf, vóór `ToListAsync`.

Bewaren: wat de context noteert

Je doet	De context noteert	Bij `SaveChangesAsync`
`Add(recept)`	nieuw	een `INSERT`
een opgevraagd object aanpassen	gewijzigd, vanzelf	een `UPDATE` van de gewijzigde kolommen
`Update(recept)`, of `Attach` met `State = EntityState.Modified`	gewijzigd	een `UPDATE` van alle kolommen
`Remove(recept)`	te verwijderen	een `DELETE`
niets	ongewijzigd	niets

`SaveChangesAsync` geeft het aantal rijen terug dat het schreef. Een nieuw object heeft na `Add` nog geen `Id`: de database kiest dat getal pas bij het bewaren. Daarom staat een `CreatedAtAction` met het nieuwe `Id` altijd ná `SaveChangesAsync`.

`FindAsync` zoekt op de primaire sleutel, en kijkt eerst of de context dat object al kent. Is dat zo, dan vraagt hij niets aan de database.

Gerelateerde gegevens: Include en ThenInclude

Vraag je een recept op, dan komt alleen het recept mee. `recept.Keuken` blijft `null`. EF Core laadt gerelateerde gegevens niet vanzelf: je zegt het met `Include`.

Stel dat een recept ook een lijst `ReceptIngredienten` heeft, de koppelklasse uit hoofdstuk 31, en dat elk koppelobject zijn `Ingredient` kent:

```
Recept? recept = await _context.Recepten
    .Include(r => r.Keuken)
    .Include(r => r.ReceptIngredienten)
        .ThenInclude(ri => ri.Ingredient)
    .FirstOrDefaultAsync(r => r.Id == id);
```

`Include` vertrekt altijd van het recept. `ThenInclude` gaat één stap verder vanaf de vorige `Include`: van het koppelobject naar zijn ingrediënt. Alles samen wordt één vraag aan de database.

Zonder `Include` zou je per recept een aparte vraag moeten stellen om de keuken op te halen: één vraag voor de lijst, en dan nog eens N vragen, één per recept. Les 08 noemt dat het **N+1-probleem**. Het gebeurt ongemerkt bij *lazy loading*, waarbij gegevens pas geladen worden zodra je de eigenschap gebruikt. In EF Core staat lazy loading standaard uit.

> **OPMERKING**
>
> Bovenaan les 08 staat dat EF Core standaard werkt "via het Lazy Loading principe". Daar bedoelt de les: gelinkte gegevens komen niet vanzelf mee. Verderop in de les, en in de documentatie van Microsoft, betekent *lazy loading* iets anders: vanzelf laden zodra je de eigenschap aanraakt. Dat doet EF Core standaard niet. Zonder `Include` blijft de eigenschap gewoon `null`.

Kringlopen in JSON

Een recept wijst naar zijn keuken, en die keuken heeft een lijst met haar recepten, waaronder hetzelfde recept. Geef je zo'n object rechtstreeks terug uit een controller, dan probeert de JSON-omzetter die kring eindeloos te volgen. Hij stopt met een `JsonException`: *A possible object cycle was detected.*

Les 08 lost dat snel op met één regel in `Program.cs`: `.AddJsonOptions(o => o.JsonSerializerOptions.ReferenceHandler = ReferenceHandler.IgnoreCycles)` achter `AddControllers()`, met `using System.Text.Json.Serialization;` bovenaan. Les 08 schrijft de volledige naam uit, dan heb je die `using` niet nodig. Waar de kring zich sluit, komt dan `null`.

> **BELANGRIJK**
>
> Les 08 zegt het zelf: `IgnoreCycles` is een snelle oplossing, maar op het examen verwacht men DTO's. Die staan in hoofdstuk 33.

In een generieke repository en een Unit of Work

Les 07 maakt één repository voor alle entiteiten, met de open plaats uit hoofdstuk 21. Omdat de repository niet weet welke tabel ze krijgt, schrijft ze niet `_context.Recepten` maar `_context.Set<TEntity>()`: de `DbSet` van het type dat ingevuld werd.

De repositories roepen zelf geen `SaveChangesAsync` meer op. Dat doet de **Unit of Work**: één klasse die alle repositories aanbiedt en ze allemaal dezelfde context meegeeft. Omdat ze één context delen, noteert die ene context alle wijzigingen. Eén `SaveChangesAsync` op de Unit of Work bewaart ze dan samen, of geen enkele.

Wat je ermee kan

Vorm	Wat het doet	Voorbeeld	Resultaat
`ToListAsync`	voer de vraag uit, alles in een lijst	`await _context.Recepten.ToListAsync()`	een `List<Recept>`
`FindAsync`	zoek op de primaire sleutel	`await _context.Recepten.FindAsync(5)`	het recept, of `null`
`FirstOrDefaultAsync`	de eerste die voldoet	`.FirstOrDefaultAsync(r => r.Titel == "Soep")`	het recept, of `null`
`AnyAsync`	is er minstens één	`.AnyAsync(r => r.KeukenId == 3)`	`true` of `false`
`CountAsync`	hoeveel zijn er	`.CountAsync(r => r.Kostprijs > 10)`	een getal
`Add`	noteer als nieuw	`_context.Recepten.Add(recept)`	het `Id` is nog `0`
`Update`	noteer als gewijzigd	`_context.Recepten.Update(recept)`	—
`Attach`	laat de context een object volgen	`_context.Set<TEntity>().Attach(entity)`	gevolgd als ongewijzigd
`Entry(...).State`	zet zelf de toestand van een object	`_context.Entry(entity).State = EntityState.Modified;`	gevolgd als gewijzigd
`Remove`	noteer om te verwijderen	`_context.Recepten.Remove(recept)`	—
`SaveChangesAsync`	stuur alle genoteerde wijzigingen naar de database	`int n = await _context.SaveChangesAsync();`	het aantal geschreven rijen
`Include`	laad een navigatie-eigenschap mee	`.Include(r => r.Keuken)`	`recept.Keuken` is ingevuld
`ThenInclude`	ga één stap dieper vanaf de vorige `Include`	`.ThenInclude(ri => ri.Ingredient)`	—
`Set<T>`	de `DbSet` van een type dat je pas later kent	`_context.Set<TEntity>()`	in een generieke repository
`IQueryable<T>`	een vraag die naar SQL vertaald wordt	`var vraag = _context.Recepten.Where(...);`	nog niets uitgevoerd

Wat het ook kan, via iets anders

Een `DbSet` is meer dan de tabel hierboven laat zien. Hij heeft eigen methodes, hij is een `IQueryable` waarop bijna heel LINQ werkt, en EF Core voegt nog een reeks asynchrone methodes toe. Wat je na `ToListAsync` overhoudt, is een gewone lijst.

Wat een DbSet zelf nog kan

Methode	Wat het doet	Voorbeeld	Resultaat
`AddRange`	noteer meerdere als nieuw	`_context.Recepten.AddRange(r1, r2)`	twee `INSERT`s bij het bewaren
`UpdateRange`, `RemoveRange`, `AttachRange`	hetzelfde als `Update`, `Remove`, `Attach`, voor meerdere	`_context.Recepten.RemoveRange(oude)`	—
`AddAsync`, `AddRangeAsync`	zoals `Add`; enkel nodig bij speciale sleutels	`await _context.Recepten.AddAsync(r)`	—
`Find`	`FindAsync` zonder `await`	`_context.Recepten.Find(5)`	het recept, of `null`
`Entry`	de notities van de context over één object	`_context.Entry(r).State`	bv. `Unchanged`, `Modified`
`Local`	wat de context nu al kent, zonder de database te vragen	`_context.Recepten.Local.Count`	het aantal gevolgde recepten

Een vraag opbouwen

Komt van LINQ. Deze methodes geven op een `IQueryable` opnieuw een `IQueryable`: er vertrekt nog niets. EF Core vertaalt ze naar SQL.

Methode	Voorbeeld	Wordt in SQL
`Where`	`.Where(r => r.Kostprijs < 5)`	`WHERE`
`OrderBy`, `OrderByDescending`, `ThenBy`, `ThenByDescending`	`.OrderBy(r => r.Titel)`	`ORDER BY`
`Order`, `OrderDescending`	`.Order()`: sorteert op de primaire sleutel (EF Core 9 en later)	`ORDER BY "Id"`
`Select`	`.Select(r => r.Titel)`	enkel die kolommen
`Skip`, `Take`	`.Skip(10).Take(10)`: de tweede pagina van tien	`OFFSET` en `LIMIT`
`Distinct`	`.Select(r => r.KeukenId).Distinct()`	`SELECT DISTINCT`
`GroupBy`	`.GroupBy(r => r.KeukenId).Select(g => new { g.Key, Aantal = g.Count() })`	`GROUP BY`
`Include`, `ThenInclude`	zie hierboven	een `JOIN`
`AsNoTracking`	`.AsNoTracking()`: de context volgt de resultaten niet	dezelfde SQL, maar sneller en enkel om te lezen

Binnen een voorwaarde mag je ook tekst- en datummethodes gebruiken die EF Core kent. Met PostgreSQL wordt `r.Titel.Contains("soep")` een `LIKE`, `StartsWith` en `EndsWith` ook, en `ToLower()` wordt `lower()`.

Een vraag uitvoeren

Komt van EF Core, in `Microsoft.EntityFrameworkCore`. Elke methode van LINQ die één antwoord of een lijst geeft, heeft hier een versie met `Async`. Die stuurt de vraag naar de database.

Methode	Wat het geeft	Als er niets is
`ToListAsync`, `ToArrayAsync`	een `List<T>` of een `T[]`	een lege lijst
`ToDictionaryAsync`	een `Dictionary`: `.ToDictionaryAsync(r => r.Id)`	een lege tabel
`ToHashSetAsync`	een `HashSet<T>`	een lege set
`FirstAsync`, `FirstOrDefaultAsync`	het eerste dat voldoet	`InvalidOperationException` of `null`
`SingleAsync`, `SingleOrDefaultAsync`	het enige dat voldoet; meer dan één geeft `InvalidOperationException`	`InvalidOperationException` of `null`
`LastAsync`, `LastOrDefaultAsync`	het laatste; enkel na een `OrderBy`	`InvalidOperationException` of `null`
`ElementAtAsync`, `ElementAtOrDefaultAsync`	het element op een plaats, na een `OrderBy`	een uitzondering of `null`
`AnyAsync`, `AllAsync`	`true` of `false`	`false` en `true`
`ContainsAsync`	zit deze waarde erin: `.Select(r => r.Titel).ContainsAsync("Soep")`	`false`
`CountAsync`, `LongCountAsync`	hoeveel	`0`
`SumAsync`	de som: `.SumAsync(r => r.Kostprijs)`	`0`
`MinAsync`, `MaxAsync`, `AverageAsync`	kleinste, grootste, gemiddelde	`InvalidOperationException`
`ForEachAsync`	voer iets uit per resultaat	er gebeurt niets
`LoadAsync`	laad de resultaten in de context, zonder ze terug te geven	—

De versies zonder `Async`, zoals `ToList()`, `First()` en `Count()`, werken ook op een `IQueryable`. Ze laten je programma wachten tot de database klaar is. In een API gebruik je de `Async`-versie, met `await`.

Na ToListAsync: een gewone lijst

Wat `ToListAsync` teruggeeft, is een `List<T>` uit hoofdstuk 6, en alles van daar en van hoofdstuk 11 werkt erop. Maar het gebeurt dan in je programma, niet meer in de database:

Je schrijft	Waar het gebeurt
`(await _context.Recepten.Where(r => r.Kostprijs > 10).ToListAsync()).Count`	filteren in de database, tellen in je programma
`(await _context.Recepten.ToListAsync()).Where(r => r.Kostprijs > 10)`	**alle** recepten komen over, en je programma filtert
`await _context.Recepten.CountAsync(r => r.Kostprijs > 10)`	alles in de database; enkel het getal komt over

Bestaat ook, maar kom je in deze cursus niet tegen

Methode	Wat het doet
`ExecuteUpdateAsync`, `ExecuteDeleteAsync`	wijzig of verwijder alle rijen die voldoen, **meteen**, zonder ze op te halen en zonder `SaveChangesAsync`: `.Where(r => r.Kostprijs > 100).ExecuteDeleteAsync()`. Geeft het aantal rijen terug. De context merkt er niets van.
`ToQueryString`	de SQL die EF Core zou sturen, als tekst: handig om te kijken
`FromSql`, `SqlQuery`	zelf SQL schrijven en toch EF Core gebruiken
`AsSplitQuery`	een vraag met veel `Includes` in meerdere stukken sturen
`AsAsyncEnumerable`	resultaten één per één verwerken met `await foreach`

Let op bij EF Core

Wat	Waarom
een nieuw object na `Add`	zit nog niet in het antwoord op een vraag: de database kent het pas na `SaveChangesAsync`
`LastAsync` of `LastOrDefaultAsync` zonder `OrderBy`	`InvalidOperationException`: *must have a deterministic sort order*
een eigen methode in `Where` of `OrderBy`	`InvalidOperationException`: *could not be translated*
een eigen `static` methode in de laatste `Select`	werkt: EF Core haalt de kolommen op en roept je methode op in je programma; zonder `static` weigert EF Core ze
een object aanpassen na `AsNoTracking`	`SaveChangesAsync` bewaart niets, want de context volgt het niet
`Contains`, `StartsWith`, `==` op tekst in PostgreSQL	maken onderscheid tussen hoofdletters en kleine letters: `"soep"` vindt *Soep* niet

LET OP

- Een filter ná `ToListAsync` laat eerst de hele tabel overkomen. Zet `Where` altijd vóór `ToListAsync`.
- `ExecuteDeleteAsync` en `ExecuteUpdateAsync` wachten niet op `SaveChangesAsync`. Wat ze doen, staat meteen in de database, ook als je daarna niet bewaart.

Veelgemaakte fouten

LET OP

- `Add`, `Update` of `Remove` zonder `SaveChangesAsync` erachter: er gebeurt niets in de database, en je krijgt geen enkele foutmelding.
- Het `Id` van een nieuw object gebruiken vóór `SaveChangesAsync`. Het staat dan nog op 0.
- `recept.Keuken.Naam` zonder `Include`: de keuken is niet geladen, en je krijgt een `NullReferenceException`.
- `Include` werkt niet samen met `FindAsync`. Wil je gerelateerde gegevens mee, gebruik dan `FirstOrDefaultAsync` met een voorwaarde op het `Id`.
- Een eigen C#-methode in een `Where` kan EF Core niet naar SQL vertalen. Het compileert, maar bij het uitvoeren krijg je *The LINQ expression … could not be translated.*
- Een `await` vergeten bij een EF-methode, en meteen een tweede oproepen? Dan werkt de context aan twee dingen tegelijk, en dat weigert hij met een uitzondering. Zet `await` voor elke asynchrone EF-methode.

Voorspel de uitkomst

Vraag 1

De tabel `Recepten` is net aangemaakt en nog leeg.

```
var r = new Recept { Titel = "Soep", KeukenId = 1 };
_context.Recepten.Add(r);
Console.WriteLine(r.Id);
await _context.SaveChangesAsync();
Console.WriteLine(r.Id);
```

Toon het antwoord ›

Vraag 2

De tabel `Recepten` bevat vier recepten.

```
_context.Recepten.Add(new Recept { Titel = "Curry", KeukenId = 2 });
int aantal = await _context.Recepten.CountAsync();
Console.WriteLine(aantal);
```

Toon het antwoord ›

Vraag 3

```
var r = await _context.Recepten.FindAsync(2);
r!.Titel = "Tomatensoep";
await _context.SaveChangesAsync();
```

Er staat nergens `Update`. Heet recept 2 nu in de database *Tomatensoep*?

Toon het antwoord ›

Vraag 4

In een nieuwe context, waarin nog niets geladen werd:

```
var r = await _context.Recepten.FirstOrDefaultAsync(x => x.Id == 2);
Console.WriteLine(r!.Keuken.Naam);
```

Toon het antwoord ›

Vraag 5

```
var duur = _context.Recepten.Where(x => x.Kostprijs > 10);
int aantal = await duur.CountAsync();
List<Recept> lijst = await duur.ToListAsync();
```

Hoeveel keer vraagt deze code iets aan de database?

Toon het antwoord ›

Vraag 6

Een GET-methode geeft een recept terug, opgehaald met `.Include(x => x.Keuken)`. De klasse `Keuken` heeft een lijst `Recepten`. In `Program.cs` staat niets over `ReferenceHandler`.

Toon het antwoord ›

Vraag 7

```
var r = await _context.Recepten.AsNoTracking().FirstAsync(x => x.Id == 2);
r.Titel = "Tomatensoep";
int n = await _context.SaveChangesAsync();
Console.WriteLine(n);
```

Toon het antwoord ›

Vraag 8

```
var laatste = await _context.Recepten.LastOrDefaultAsync();
```

Toon het antwoord ›

Antwoorden

VRAAG 1

`0` en daarna `1`

Na `Add` is het recept enkel genoteerd. Het `Id` kiest de database, en die heeft het recept nog niet gezien. Na `SaveChangesAsync` vult EF Core het `Id` in met wat de database koos.

‹ Terug naar de vraag

VRAAG 2

4

`CountAsync` stelt de vraag aan de database, en daar staat de curry nog niet in. Het recept ligt nog in het mandje, want `SaveChangesAsync` is niet opgeroepen.

‹ Terug naar de vraag

VRAAG 3

Ja.

De context heeft het recept zelf opgehaald, en volgt het sindsdien. Hij merkt dat de titel veranderd is, en `SaveChangesAsync` stuurt een `UPDATE`. `Update` heb je enkel nodig voor een object dat de context niet zelf ophaalde, zoals een object dat uit de body van een verzoek komt.

‹ Terug naar de vraag

VRAAG 4

`NullReferenceException` — het programma crasht.

Het recept is gevonden, maar zijn keuken niet meegeladen. `r.Keuken` is `null`, ook al staat er `= default!` achter de eigenschap. Met `.Include(x => x.Keuken)` vóór `FirstOrDefaultAsync` lukt het wel.

‹ Terug naar de vraag

VRAAG 5

2

`duur` is een vraag, geen antwoord. `CountAsync` stelt ze één keer, `ToListAsync` stelt ze opnieuw. Komt er tussen die twee een recept bij, dan kan `aantal` zelfs verschillen van `lijst.Count`. Heb je beide nodig, vraag dan de lijst op en tel die.

‹ Terug naar de vraag

VRAAG 6

`500 Internal Server Error`

Het recept wijst naar de keuken, en de keuken heeft het recept in haar lijst `Recepten`: EF Core vult die lijst aan met wat hij al geladen heeft. De JSON-omzetter loopt in een kring en stopt met een `JsonException`. Met `IgnoreCycles` krijg je wel een antwoord, met `null` waar de kring zich sluit. Met een DTO heb je het probleem niet.

‹ Terug naar de vraag

VRAAG 7

`0`

Met `AsNoTracking` volgt de context het recept niet. Hij merkt de nieuwe titel dus niet op, en `SaveChangesAsync` heeft niets te bewaren. Vergelijk met vraag 3.

‹ Terug naar de vraag

VRAAG 8

`InvalidOperationException` — het programma crasht.

Een tabel heeft geen vaste volgorde, dus "het laatste" bestaat niet. EF Core weigert de vraag zonder `OrderBy`. Met `.OrderBy(x => x.Id).LastOrDefaultAsync()` lukt het wel.

‹ Terug naar de vraag

33. DTO's en Mapster

DATA

Wat het is

De etalage, niet het magazijn. Je entiteit is het magazijn: alles wat je bewaart, met alle relaties en velden die niemand buiten je API hoeft te zien. Een **DTO**, een *Data Transfer Object*, is de etalage: een klasse of record met enkel wat de client te zien krijgt, in de vorm die hem past.

Een DTO lost vier dingen tegelijk op:

- geen kringlopen in je JSON, want de DTO wijst niet terug naar zijn ouder (zie hoofdstuk 32);
- je stuurt niet meer dan nodig;
- velden die binnen moeten blijven, komen er gewoon niet in;
- je database mag veranderen zonder dat de client het merkt.

Van een entiteit een DTO maken is veel overtypen: `Titel = recept.Titel`, en zo verder voor elke eigenschap. **Mapster** doet dat voor je.

Hoe het eruitziet

```
public class ReceptDto
{
    public int Id { get; set; }
    public string Titel { get; set; } = default!;
    public string KeukenNaam { get; set; } = default!;
}
```

Les 09 toont ook de kortere vorm als `record`, uit hoofdstuk 14:

```
public record ReceptDto(int Id, string Titel, string KeukenNaam);
```

Omzetten, na `dotnet add package Mapster` en `using Mapster;`:

```
ReceptDto dto = recept.Adapt<ReceptDto>();
List<ReceptDto> dtos = recepten.Adapt<List<ReceptDto>>();
```

`Adapt` is een *extension method*: een methode die Mapster toevoegt aan elk object, zodat je ze kan oproepen alsof ze er altijd al stond.

Hoe Mapster een eigenschap vult

Mapster overloopt elke eigenschap van de DTO en zoekt in de bron:

1. een eigenschap met **exact dezelfde naam**, hoofdletters inbegrepen: `Titel` uit `Titel`;
2. een naam die aan elkaar geplakt is uit een eigenschap en de eigenschap daarvan: `KeukenNaam` uit `Keuken.Naam`. Dat heet **platslaan** (*flattening*);
3. vindt hij niets, dan laat hij de eigenschap op haar standaardwaarde staan, zonder foutmelding.

Wat in de bron staat en niet in de DTO, laat hij gewoon liggen.

> **OPMERKING**
>
> Platslaan werkt enkel als de keuken ook geladen is. Zonder `Include` is de keuken `null`. Mapster vangt dat op, zonder crash, en laat `KeukenNaam` gewoon leeg.

Eigen regels: een MapperProfile

Past een naam niet, of moet een waarde ergens anders vandaan komen, dan schrijf je een regel. Les 09 zet die regels samen in één klasse die `IRegister` belooft. Stel een `ReceptDetailDto` met een tekst `Herkomst` en een lijst `Ingredienten`:

```
public class MapperProfile : IRegister
{
    public void Register(TypeAdapterConfig config)
    {
        config.NewConfig<Recept, ReceptDetailDto>()
            .Map(dest => dest.Herkomst, src => src.Keuken.Naam)
            .Map(dest => dest.Ingredienten, src => src.ReceptIngredienten);
    }
}
```

Lees `.Map(dest => dest.Herkomst, src => src.Keuken.Naam)` als: *vul in het doel* `Herkomst` *met de naam van de keuken uit de bron*. Wat wel overeenkomt, blijft Mapster zelf doen. Voor de lijst `Ingredienten` zoekt Mapster op zijn beurt een regel om één koppelobject om te zetten.

Die klasse doet pas iets als `Program.cs` haar laat zoeken, met één regel:

```
TypeAdapterConfig.GlobalSettings.Scan(Assembly.GetExecutingAssembly());
```

`Scan` doorzoekt je project naar klassen die `IRegister` beloven, en leest hun regels. Bovenaan heb je `using Mapster;` en `using System.Reflection;` nodig.

> **OPMERKING**
>
> Les 09 raadt aan om in een profiel geen berekeningen te doen, zoals een totaalprijs. Code in een profiel is moeilijk te testen en te debuggen. Hou het profiel bij overzetten; het rekenwerk hoort in een controller, een service of een hulpklasse.

Wanneer wel, wanneer niet

Wel — alles wat je API teruggeeft aan de client. Voor wat binnenkomt maak je vaak een aparte DTO zonder `Id`, want dat kiest de database.

Niet — binnen je eigen code, tussen repository en database. Daar werk je met de entiteiten zelf.

Wat je ermee kan

Vorm	Wat het doet	Voorbeeld	Resultaat
`Adapt<T>`	maak een nieuw object van type `T`	`recept.Adapt<ReceptDto>()`	een `ReceptDto`
`Adapt` op een lijst	zet elk item om	`recepten.Adapt<List<ReceptDto>>()`	een `List<ReceptDto>`
`bron.Adapt(doel)`	kopieer in een object dat al bestaat	`dto.Adapt(recept)`	`recept` krijgt de waarden van `dto`
`IRegister`	de interface voor een klasse met eigen regels	`class MapperProfile : IRegister`	—
`Register`	de methode waarin je de regels zet	`public void Register(TypeAdapterConfig config)`	—
`NewConfig`	begin een regel voor dit paar	`config.NewConfig<Recept, ReceptDto>()`	vervangt een eerdere regel voor hetzelfde paar
`ForType`	maak of vul een regel aan	`config.ForType<Recept, ReceptDto>()`	een eerdere regel blijft gelden
`Map`	vul deze doel-eigenschap met dit	`.Map(dest => dest.Herkomst, src => src.Keuken.Naam)`	—
`Ignore`	laat deze doel-eigenschap met rust	`.Ignore(dest => dest.Id)`	`Id` wordt niet overschreven
`Scan`	zoek alle `IRegister`-klassen en lees hun regels	`GlobalSettings.Scan(...)`	één regel in `Program.cs`, zie hierboven

Wat het ook kan, via iets anders

Wat	Voorbeeld	Resultaat
`Adapt` naar elke soort verzameling	`recepten.Adapt<ReceptDto[]>()`, `Adapt<List<ReceptDto>>()`	een array of een lijst, ook vanuit een `IEnumerable` van Dapper
een enum naar tekst	`Status` is een enum in de entiteit en een `string` in de DTO	de naam van de keuze, zoals `"Groen"`
tekst naar een enum	omgekeerd	de keuze met die naam
een getal naar een ander getal	`int` in de entiteit, `decimal` in de DTO	omgezet
een lijst in een lijst	`Ingredienten` met elk een eigen DTO	elk item omgezet, zie hierboven

Wat `Adapt` teruggeeft, is een gewone lijst of array: alles van hoofdstuk 5, 6 en 11 werkt erop.

Veelgemaakte fouten

LET OP

- Een naam die net anders is, zoals `Naam` in de bron en `Titel` in de DTO, geeft geen fout. De eigenschap blijft gewoon leeg. Kijk bij een lege eigenschap eerst naar de namen.
- De `Scan`-regel vergeten in `Program.cs`: alles compileert en draait, maar je eigen regels worden nooit gelezen. Enkel wat vanzelf overeenkomt, wordt ingevuld.
- Twee keer `NewConfig` voor hetzelfde paar: de tweede vervangt de eerste helemaal. Wil je aanvullen, gebruik dan `ForType`, of zet alles in één regel.
- DTO's gebruiken én `IgnoreCycles` laten staan. Les 09 haalt die regel weer weg: een DTO zonder terugwijzende eigenschappen heeft hem niet nodig.

Voorspel de uitkomst

Deze vragen gebruiken gewone klassen met eigenschappen, zonder database.

Vraag 1

```
public class Dier
{
    public int Id { get; set; }
    public string Naam { get; set; } = "";
    public string Soort { get; set; } = "";
}

public class DierDto
{
    public string Naam { get; set; } = "";
    public string Ras { get; set; } = "";
}

var dto = new Dier { Id = 3, Naam = "Bo", Soort = "hond" }.Adapt<DierDto>();
Console.WriteLine("[" + dto.Naam + "] [" + dto.Ras + "]");
```

Toon het antwoord ›

Vraag 2

```
public class Stad
{
    public string Naam { get; set; } = "";
}

public class Leerling
{
    public string Naam { get; set; } = "";
    public Stad Stad { get; set; } = new();
}

public class LeerlingDto
{
    public string Naam { get; set; } = "";
    public string StadNaam { get; set; } = "";
}

var anna = new Leerling { Naam = "Anna", Stad = new Stad { Naam = "Gent" } };
var dto = anna.Adapt<LeerlingDto>();
Console.WriteLine(dto.StadNaam);
```

Toon het antwoord ›

Vraag 3

`LeerlingDto` krijgt er twee teksten bij: `Woonplaats` en `Klas`. `Leerling` krijgt een eigenschap `Klasgroep`, met daarin een `Code`. Het profiel bevat:

```
config.NewConfig<Leerling, LeerlingDto>()
    .Map(dest => dest.Woonplaats, src => src.Stad.Naam);

config.NewConfig<Leerling, LeerlingDto>()
    .Map(dest => dest.Klas, src => src.Klasgroep.Code);
```

Welke van de twee wordt ingevuld?

Toon het antwoord ›

Vraag 4

Het profiel van vraag 3 is verbeterd en zet beide regels achter één `NewConfig`. Maar in `Program.cs` ontbreekt de regel met `Scan`.

Wat staat er in `Naam`, `Woonplaats` en `Klas` van de DTO?

Toon het antwoord ›

Antwoorden

VRAAG 1

`[Bo] []`

`Naam` staat aan beide kanten, dus die wordt gekopieerd. `Ras` vindt geen bron en houdt zijn beginwaarde, een lege tekst. `Id` en `Soort` hebben geen plaats in de DTO en vallen weg. Nergens een foutmelding.

‹ Terug naar de vraag

VRAAG 2

`Gent`

`StadNaam` bestaat niet in `Leerling`, maar `Stad` gevolgd door `Naam` wel. Mapster slaat dat vanzelf plat. Een eigen regel heb je pas nodig als de naam dat patroon niet volgt, zoals bij `Woonplaats`.

‹ Terug naar de vraag

VRAAG 3

Enkel `Klas`.

De tweede `NewConfig` voor hetzelfde paar gooit de eerste regel weg. `Woonplaats` blijft leeg. Zet beide `Map`-regels achter één `NewConfig`, of begin de tweede met `ForType`.

‹ Terug naar de vraag

VRAAG 4

`Naam` is ingevuld, `Woonplaats` en `Klas` blijven leeg.

Zonder `Scan` leest niemand het profiel. Mapster doet enkel wat het zelf kan: eigenschappen met dezelfde naam kopiëren, en platslaan. Er is geen enkele foutmelding die je op het spoor zet.

‹ Terug naar de vraag

34. Dapper

DATA

> **OPMERKING**
> Les 08 zegt het duidelijk: op het examen ligt de focus op EF Core. Dapper hoef je niet zelf te kunnen schrijven, maar je moet kunnen uitleggen wat het is, en waarom en wanneer iemand ervoor kiest. Dit hoofdstuk helpt je de code te lezen en het verschil te begrijpen.

Wat het is

Zelf de taal van de database spreken. EF Core is een tolk: jij schrijft C#, hij maakt er SQL van. Met Dapper schrijf je de SQL zelf, en Dapper doet enkel de laatste stap: de rijen die terugkomen omzetten in objecten. Zo'n kleine tussenpersoon heet een **micro-ORM**. EF Core is een volledige ORM.

	EF Core	Dapper
SQL	maakt het zelf, uit LINQ	schrijf je zelf
Wijzigingen bijhouden	ja, de change tracker	nee
Tabellen en migraties	ja	nee; de database moet al bestaan
Gerelateerde gegevens	`Include`	een `JOIN` in je SQL, en zelf de objecten koppelen
Sterk in	schrijven, en de structuur van je database	snel lezen, ingewikkelde rapporten

Grotere toepassingen combineren ze vaak. EF Core doet het schrijven, met change tracking en een Unit of Work. Dapper doet het lezen waar elke milliseconde telt. Les 08 noemt dat het hybride patroon, naar het principe *Command Query Separation*.

Hoe het eruitziet

```
using Dapper;

string sql = """
    SELECT * FROM "Recepten"
    WHERE "Id" = @Id
    """;

Recept? recept = await _dbConnection.QueryFirstOrDefaultAsync<Recept>(sql, new { Id = id
});
```

`_dbConnection` is een `IDbConnection`: een verbinding met de database, die de repository in haar constructor krijgt. Dapper voegt zijn methodes toe aan die verbinding.

De drie aanhalingstekens openen een *raw string*: een tekst, op één regel of over meerdere regels, waarin gewone aanhalingstekens gewoon mogen staan. Dat komt van pas, want PostgreSQL wil de namen van tabellen en kolommen die EF Core maakte tussen aanhalingstekens. Zie de waarschuwing verderop.

Parameters: altijd met @

`@Id` in de SQL is een plaatshouder. De waarde komt uit het **anonieme object** `new { Id = id }` uit hoofdstuk 13: de naam na `@` moet de naam van een eigenschap in dat object zijn. Dapper geeft de waarde apart door aan de database, nooit als stuk tekst in de SQL.

Plak je een waarde zelf in de SQL, met een `$` en accolades zoals in hoofdstuk 1, dan kan wie die waarde aanlevert, eigen SQL meesturen. Dat heet **SQL-injectie**. Vraag 1 toont hoe weinig daarvoor nodig is.

Een rij in twee objecten: multi-mapping

Met een `JOIN` komen een recept en zijn keuken samen op één rij. Dapper moet weten waar het recept ophoudt en de keuken begint:

```
string sql = """
    SELECT r.*, k.*
    FROM "Recepten" r
    INNER JOIN "Keukens" k ON r."KeukenId" = k."Id"
    WHERE r."Id" = @Id
    """;

var resultaat = await _dbConnection.QueryAsync<Recept, Keuken, Recept>(
    sql,
    (recept, keuken) =>
    {
        recept.Keuken = keuken;
        return recept;
    },
    param: new { Id = id },
    splitOn: "Id");
```

- Tussen de punthaken staan de types per stuk van de rij, en als laatste wat je terugkrijgt.
- `splitOn: "Id"` zegt: zodra je opnieuw een kolom `Id` tegenkomt, begint het volgende object. `Id` is ook wat Dapper aanneemt als je niets opgeeft.
- De lambda koppelt de stukken aan elkaar, zoals `Include` dat bij EF Core voor je doet.

Bij één op veel staat de ouder op elke rij opnieuw, één keer per kind. Les 08 houdt daarom een `Dictionary` bij uit hoofdstuk 7, met het `Id` als sleutel, zodat elke ouder maar één keer in het resultaat komt.

Wat je ermee kan

In de voorbeelden is `db` de verbinding, `sql` de tekst van de query en `param` een anoniem object met de parameters, zoals `new { Id = 5 }`. Voor elke oproep staat nog een `await`.

Vorm	Wat het doet	Voorbeeld	Resultaat
`QueryAsync<T>`	alle rijen, elk als object	`db.QueryAsync<Recept>(sql)`	een `IEnumerable<Recept>`
`QueryFirstOrDefaultAsync<T>`	de eerste rij, of niets	`db.QueryFirstOrDefaultAsync<Recept>(sql, param)`	het recept, of `null`
`QueryFirstAsync<T>`	de eerste rij, die er moet zijn	`db.QueryFirstAsync<Recept>(sql, param)`	geen rij: `InvalidOperationException`
`QuerySingleOrDefaultAsync<T>`	precies één rij, of niets	`db.QuerySingleOrDefaultAsync<Recept>(sql, param)`	meer dan één rij: `InvalidOperationException`
`ExecuteAsync`	voer een `INSERT`, `UPDATE` of `DELETE` uit	`db.ExecuteAsync(sql, param)`	het aantal geraakte rijen
`ExecuteScalarAsync<T>`	voer uit en geef één waarde terug	`db.ExecuteScalarAsync<int>(sql)`	één getal of tekst
`@Naam` + `new { Naam = ... }`	een parameter	`WHERE "Titel" = @Titel` met `new { Titel = titel }`	veilig voor SQL-injectie
`QueryAsync<A, B, A>`	één rij verdelen over twee objecten	met een lambda en `splitOn`	het object `A`, met `B` eraan gekoppeld
`splitOn`	de kolom waar het volgende object begint	`splitOn: "Id"`	—

Dapper koppelt elke kolom aan de eigenschap met dezelfde naam. Anders dan bij Mapster maken hoofdletters daarbij niet uit: een kolom `titel` vult `Titel`. Vindt hij geen eigenschap, dan slaat hij de kolom over, zonder foutmelding.

Wat het ook kan, via iets anders

`QueryAsync<T>` geeft een `IEnumerable<T>` uit hoofdstuk 12. Dapper leest standaard alle rijen in één keer in, en bewaart ze in een lijst. Daarna is de database niet meer betrokken: alles wat je verder doet, gebeurt in je programma.

Op het resultaat van QueryAsync

Komt van LINQ. Alles van hoofdstuk 11 werkt erop. In de voorbeelden is `recepten` het resultaat van `await db.QueryAsync<Recept>(sql)`.

Methode	Voorbeeld	Resultaat
`Count`, **`Any`**, **`First`**, **`FirstOrDefault`**	`recepten.Count()`	het aantal rijen; de SQL loopt niet opnieuw
`Where`, **`OrderBy`**, **`Select`**	`recepten.Where(r => r.Kostprijs > 10)`	gefilterd in je programma, niet in de database
`ToList`	`recepten.ToList()`	een `List<Recept>`, met alles van hoofdstuk 6
`AsList`	`recepten.AsList()`	ook een `List<Recept>`; Dapper's eigen versie, die geen kopie maakt als het al een lijst is
`ToDictionary`	`recepten.ToDictionary(r => r.Id)`	een `Dictionary<int, Recept>`, zoals bij multi-mapping
`foreach`	`foreach (var r in recepten)`	overloopt de rijen

Wat **niet** werkt: `recepten.Add(...)`, `recepten[0]` en `recepten.Count` zonder haakjes. Dat zijn dingen van een lijst, en het type is `IEnumerable<Recept>`. Maak er eerst een lijst van.

Bestaat ook, maar kom je in deze cursus niet tegen

Methode	Wat het doet
`QuerySingleAsync`	precies één rij, anders een **`InvalidOperationException`**
`QueryMultipleAsync`	meerdere `SELECT`s in één keer, en de resultaten één voor één lezen
`ExecuteReaderAsync`	de rijen zelf uitlezen, zonder objecten
`buffered: false`	rijen één voor één inlezen in plaats van allemaal tegelijk, voor heel grote resultaten
een lijst als parameter	`new { Ids = new[] { 1, 2, 3 } }`: Dapper vult de waarden zelf in, veilig
`ExecuteAsync` met een lijst objecten	voert dezelfde SQL uit voor elk object, en telt alle geraakte rijen samen

Let op bij Dapper

Wat	Waarom
`Where` op het resultaat van `QueryAsync`	alle rijen kwamen al over; filter liever in de SQL, met `WHERE`
een kolom die anders heet dan de eigenschap	Dapper slaat ze over, zonder fout; gebruik `AS` in je SQL
geen change tracker	een object aanpassen doet niets in de database; schrijf zelf een `UPDATE`

Veelgemaakte fouten

LET OP

- **Een waarde in de SQL plakken** met `$` en accolades. Gebruik altijd `@Naam` en een anoniem object.
- **Namen zonder aanhalingstekens.** Tabellen die EF Core in PostgreSQL maakte, heten exact `Recepten`, met hoofdletter. PostgreSQL maakt van elke naam zonder aanhalingstekens kleine letters, en zoekt dan `recepten`. Die bestaat niet. Schrijf `"Recepten"` en `"KeukenId"`.
- **`SCOPE_IDENTITY()` in PostgreSQL.** Het voorbeeld in les 08 haalt het nieuwe `Id` op met `SELECT CAST(SCOPE_IDENTITY() as int)`. Dat is SQL Server. In PostgreSQL zet je `RETURNING "Id"` achter de `INSERT`, en lees je dat getal met `ExecuteScalarAsync<int>`.
- **Een `IDbConnection` vragen die niet geregistreerd is.** Les 08 toont niet hoe die in `Program.cs` komt. Je registreert ze met `AddScoped`, met een lambda die een `NpgsqlConnection` maakt met je connection string. Zonder registratie krijg je dezelfde fout als in hoofdstuk 27.

Voorspel de uitkomst

`db` is een verbinding met een database waarin EF Core de tabel `"Recepten"` maakte. Die bevat twaalf recepten, en geen enkel recept heeft een vreemde titel.

Vraag 1

```
string titel = "' OR '1'='1";
string sql = $"SELECT * FROM \"Recepten\" WHERE \"Titel\" = '{titel}'";
var recepten = await db.QueryAsync<Recept>(sql);
Console.WriteLine(recepten.Count());
```

Toon het antwoord ›

Vraag 2

```
int n = await db.ExecuteAsync(
    """DELETE FROM "Recepten" WHERE "Id" = @Id""",
    new { Id = 999 });
Console.WriteLine(n);
```

Toon het antwoord ›

Vraag 3

```
var r = await db.QueryFirstOrDefaultAsync<Recept>(
    """SELECT "Id", "Titel" AS naam FROM "Recepten" WHERE "Id" = 1""");
Console.WriteLine("[" + r!.Titel + "]");
```

Toon het antwoord ›

Vraag 4

```
var alle = await db.QueryAsync<Recept>("SELECT * FROM Recepten");
```

Toon het antwoord ›

Vraag 5

```
var r = await db.QueryFirstOrDefaultAsync<Recept>(
    """SELECT * FROM "Recepten" WHERE "Id" = @Id""",
    new { Id = 999 });
Console.WriteLine(r is null);
```

Toon het antwoord ›

Vraag 6

```
var recepten = await db.QueryAsync<Recept>("""SELECT * FROM "Recepten" """);
Console.WriteLine(recepten.Count());
Console.WriteLine(recepten.Count(r => r.Kostprijs > 10));
```

Hoeveel keer vraagt deze code iets aan de database?

Toon het antwoord ›

Antwoorden

VRAAG 1

`12`

Na het invullen staat er `WHERE "Titel" = '' OR '1'='1'`. Het tweede deel is altijd waar, dus elke rij voldoet. Met `WHERE "Titel" = @Titel` en `new { Titel = titel }` zoekt de database letterlijk naar die vreemde titel, en vindt ze er `0`.

‹ Terug naar de vraag

VRAAG 2

`0`

Er is geen recept 999, dus er wordt niets verwijderd. Dat is geen fout: de database meldt gewoon dat nul rijen geraakt zijn. In een controller is dat het moment voor een `NotFound`.

‹ Terug naar de vraag

VRAAG 3

`[]`

De kolom heet in het antwoord `naam`, en `Recept` heeft geen eigenschap met die naam. Dapper slaat de kolom over. `Titel` krijgt dus niets, en blijft `null`. Het `Id` is wel ingevuld.

‹ Terug naar de vraag

VRAAG 4

`Een uitzondering van PostgreSQL`: *relation "recepten" does not exist*. In een API wordt dat een **`500`**.

Zonder aanhalingstekens maakt PostgreSQL er `recepten` van, met kleine letters. EF Core maakte de tabel als `"Recepten"`. Voor PostgreSQL zijn dat twee verschillende namen.

‹ Terug naar de vraag

VRAAG 5

`True`

Geen rij gevonden, dus `null`, net als `FirstOrDefault` in hoofdstuk 11. Met `QueryFirstAsync` was het een `InvalidOperationException` geweest.

‹ Terug naar de vraag

VRAAG 6

`1`

Dapper leest alle rijen in bij `QueryAsync`. De twee `Count`s tellen in je programma. Vergelijk met vraag 5 van hoofdstuk 32: daar was het een `IQueryable`, en vertrok de vraag twee keer.

‹ Terug naar de vraag

Deel 7 — Je API, les per les

Hoofdstuk 25 tot 34 leggen elke bouwsteen apart uit. Dit deel zet ze in de volgorde van de lessen. Per les staat er wat je maakt en in welke map, wat je installeert, wat je registreert, welke commando's je uitvoert, en wat je ziet als je een stap vergeet.

Het voorbeeld is dat van de rest van dit boek: een API met recepten en keukens, `ReceptenApi`. De lessen zelf werken met laptops en bestellingen in `WebAPIDemo`. Lees eerst de les, en hou dan het hoofdstuk van die les open terwijl je bouwt. Krijg je een foutmelding, kijk dan in de tabel *Als een stap ontbreekt*.

35. Zo groeit je project

IN JE API

Wat het is

De kaart van de hele weg. Welke mappen en bestanden er na elke les bijkomen, en hoe `Program.cs` meegroeit. De details staan per les in hoofdstuk 36 tot hoofdstuk 42.

Je vertrekpunt na les 02

Na les 01 en 02 heb je één project, gemaakt met `dotnet new webapi --use-controllers`, met Scalar erbij. Er is een map `Models` met één model, en een controller met een `private static` lijst.

```
ReceptenApi/
├── Controllers/
│   └── ReceptController.cs
├── Models/
│   └── Recept.cs
├── Properties/
│   └── launchSettings.json
├── appsettings.json
├── Program.cs
└── ReceptenApi.csproj
```

Wat er per les bijkomt

Les	Nieuwe mappen	Nieuwe bestanden	In Program.cs	Installeren en uitvoeren
03	`Repositories`	de interface en de klasse van je repository	`AddScoped` voor de repository	niets
04	`Data`, `Migrations`	`docker-compose.yml`, de context, de eerste migratie	de verbinding ophalen en `AddDbContext`	Docker Desktop, vier EF Core-packages, `docker compose up -d`, een migratie en `Update-Database`
05	geen	een repository die met de database werkt	de registratie wijst naar die nieuwe repository	niets
06	geen	nieuwe modellen, een repository en een controller per nieuw model	`AddScoped` voor elke nieuwe repository	een migratie en `Update-Database` per stap
07	geen	de generieke repository, de Unit of Work	enkel nog `AddScoped` voor de Unit of Work	niets
08	geen	specifieke repositories met `Include`	`AddJsonOptions` met `IgnoreCycles`, tijdelijk	niets
09	`DTOs` met een submap per model, `Configuration`	de DTO's, de `MapperProfile`	`IgnoreCycles` weg, Mapster laten zoeken	het package `Mapster`

Het project na les 09

```
ReceptenApi/
├── Configuration/
│   └── MapperProfile.cs
├── Controllers/
│   ├── KeukenController.cs
│   └── ReceptController.cs
├── Data/
│   ├── IUnitOfWork.cs
│   ├── ReceptContext.cs
│   └── UnitOfWork.cs
├── DTOs/
│   ├── Keukens/
│   │   ├── KeukenDto.cs
│   │   └── KeukenWriteDto.cs
│   └── Recepten/
│       ├── ReceptDetailDto.cs
│       ├── ReceptDto.cs
│       └── ReceptWriteDto.cs
├── Migrations/
├── Models/
│   ├── Ingredient.cs
│   ├── Keuken.cs
│   ├── Recept.cs
│   └── ReceptIngredient.cs
├── Repositories/
│   ├── GenericRepository.cs
│   ├── IGenericRepository.cs
│   ├── IKeukenRepository.cs
│   ├── IReceptRepository.cs
│   ├── KeukenRepository.cs
│   └── ReceptRepository.cs
├── appsettings.json
├── docker-compose.yml
├── Program.cs
└── ReceptenApi.csproj
```

Twee mappen bevatten iets wat je misschien niet verwacht. De cursus zet `IUnitOfWork` en `UnitOfWork` in `Data`, naast de context, en niet in `Repositories`. En `Migrations` vul je nooit zelf: die map maakt EF Core aan bij je eerste migratie.

Program.cs, les na les

Les	Wat er verandert	Waar
03	`AddScoped<IReceptRepository, InMemoryReceptRepository>()` erbij	vóór `builder.Build()`
04	de verbinding ophalen, en `AddDbContext<ReceptContext>(...)` erbij	vóór `builder.Build()`
05	in de regel van les 03 wordt `InMemoryReceptRepository` vervangen door `ReceptRepository`	dezelfde regel
06	`AddScoped` voor elke nieuwe repository	vóór `builder.Build()`
07	alle `AddScoped` voor repositories weg, `AddScoped<IUnitOfWork, UnitOfWork>()` erbij	vóór `builder.Build()`
08	`AddControllers()` krijgt `.AddJsonOptions(...)` met `IgnoreCycles`	de regel met `AddControllers`
09	`IgnoreCycles` weer weg, `TypeAdapterConfig.GlobalSettings.Scan(...)` erbij	net na `CreateBuilder`

Zo ziet `Program.cs` eruit na les 09:

```
using System.Reflection;
using Mapster;
using Microsoft.EntityFrameworkCore;
using ReceptenApi.Data;
using Scalar.AspNetCore;

var builder = WebApplication.CreateBuilder(args);

// Mapster: zoek alle klassen die IRegister beloven (les 09)
TypeAdapterConfig.GlobalSettings.Scan(Assembly.GetExecutingAssembly());

builder.Services.AddControllers();
builder.Services.AddOpenApi();

// De database (les 04)
var connectionString = builder.Configuration.GetConnectionString("Standaard");
builder.Services.AddDbContext<ReceptContext>(options =>
    options.UseNpgsql(connectionString));

// De Unit of Work (les 07)
builder.Services.AddScoped<IUnitOfWork, UnitOfWork>();

var app = builder.Build();

if (app.Environment.IsDevelopment())
{
    app.MapOpenApi();
    app.MapScalarApiReference();
}

app.UseHttpsRedirection();
app.UseAuthorization();
app.MapControllers();

app.Run();
```

Drie regels die in elke les gelden

1. Alles wat met `builder.Services` begint, staat vóór `builder.Build()`. Zie hoofdstuk 27.
2. Vraagt een klasse iets in haar constructor, dan moet dat geregistreerd zijn. Je eigen klassen registreer je zelf. Wat .NET zelf levert, zoals `ILogger<T>` en vanaf les 04 de context via `AddDbContext`, hoef je niet apart te registreren.
3. Vanaf les 04: verander je iets aan een model, dan volgt er een nieuwe migratie en `Update-Database`. De database pas je nooit met de hand aan.

Voorspel de uitkomst

Vraag 1

De regel `builder.Services.AddScoped<IReceptRepository, InMemoryReceptRepository>();` staat onder `var app = builder.Build();`. Wat gebeurt er als je de API start?

Toon het antwoord ›

Vraag 2

Na les 07 staan de oude `AddScoped`-regels voor elke repository nog in `Program.cs`, naast de nieuwe voor de Unit of Work. De controllers vragen enkel nog `IUnitOfWork`. Werkt de API?

Toon het antwoord ›

Vraag 3

Je voegt in les 06 een eigenschap `Bereidingstijd` toe aan `Recept`, en start de API zonder nieuwe migratie. Wat geeft `GET api/recept`?

Toon het antwoord ›

Antwoorden

VRAAG 1

De API start niet. Je krijgt een `InvalidOperationException`: *The service collection cannot be modified because it is read-only.*

Na `builder.Build()` ligt de lijst met diensten vast. Wat je daarna nog registreert, kan er niet meer bij.

‹ Terug naar de vraag

VRAAG 2

Ja. De oude registraties zijn overbodig, want niemand vraagt die repositories nog. Ze doen geen kwaad, maar ze zeggen iets wat niet meer klopt. Haal ze weg, zodat `Program.cs` toont wat je API echt gebruikt.

‹ Terug naar de vraag

VRAAG 3

500 Internal Server Error. In het terminalvenster staat een fout van PostgreSQL: *42703: column r.Bereidingstijd does not exist*.

EF Core vraagt de nieuwe kolom op, maar de tabel heeft ze nog niet. Een migratie maken en `Update-Database` uitvoeren lost het op.

‹ Terug naar de vraag

36. Les 03: repository en dependency injection

IN JE API

Wat je bouwt

Je haalt de lijst, en alles wat ermee gebeurt, uit je controller. Dat komt in een repository, achter een interface. De controller vraagt die repository in zijn constructor, en `Program.cs` zegt welke klasse hij krijgt. Daarna komt er logging bij.

Voor wie je API gebruikt, verandert er niets. Elk endpoint geeft hetzelfde antwoord als voor de les. Zo'n verbouwing zonder ander gedrag heet **refactoren**.

Hoe je dit hoofdstuk leest

De les van school toont het begin en het einde. Eerst de controller van les 02, en daarna meteen de volledige repository en de volledige controller van les 03. Hoe je van het ene naar het andere komt, zegt ze in twee zinnen: *verplaats nu alle data-logica* naar de repository, en *pas nu de rest van de controller aan*. Dit hoofdstuk toont wat er in die twee zinnen zit, stap voor stap.

- Eerst lees je de controller van les 02, en duid je aan wat erin verhuist.
- Dan bouw je de nieuwe bestanden, één stap tegelijk. Bij elke stap zie je de code vóór en ná, wat er verhuist, naar welk bestand, en waarom.
- Na elke stap bouwt je project, en werkt je API nog. Loopt er iets mis, dan weet je meteen in welke stap.

In de code van dit hoofdstuk betekenen twee kleuren telkens hetzelfde:

- **oranje en vet**: dit werkt met de gegevens, en verhuist naar de repository;
- **groen en vet**: dit is nieuw of gewijzigd in deze stap.

Het voorbeeld is de recepten-API van dit boek. De les werkt met laptops in een `ProductController`. De stappen zijn dezelfde.

Waarom je de controller opsplitst

Na les 02 doet je controller alles zelf. Hij bewaart de lijst, zoekt erin, deelt nummers uit, kijkt na wat de client stuurt en kiest het antwoord. Dat werkt zolang de recepten in een lijst staan. Maar in les 05 komen ze in een database. Dan moet je in elke actie het zoekwerk herschrijven, en loop je het risico dat je onderweg ook iets aan je antwoorden verandert.

De les gebruikt het beeld van een ober. De ober neemt je bestelling op, kijkt of ze kan, en brengt je bord. Hij staat niet zelf in de keuken. Je controller is de ober, de repository is de keuken.

Die verdeling heeft een naam: **Separation of Concerns**, elk stuk code één taak. Zolang de controller zelf de lijst bewaart, zijn de controller en de opslag **strak gekoppeld** (*tight coupling*): verander je de opslag, dan verander je de controller. Na deze les zijn ze **los gekoppeld** (*loose coupling*): de controller kent enkel nog de afspraak, niet de opslag.

Na deze les:

- staat al het werk met de gegevens op één plaats, in de repository;
- wissel je later van opslag met één regel in `Program.cs`, zonder iets aan je controller te veranderen;
- lees je in je controller meteen wat elk endpoint antwoordt, zonder zoekwerk ertussen.

Wat er bijkomt in je project

```
ReceptenApi/
├── Controllers/
│   └── ReceptController.cs             ← gewijzigd
├── Models/
│   └── Recept.cs
├── Repositories/                       ← nieuw
│   ├── IReceptRepository.cs            ← nieuw
│   └── InMemoryReceptRepository.cs     ← nieuw
└── Program.cs                          ← gewijzigd
```

De les ruimt ook op: `WeatherForecast.cs` en `WeatherForecastController.cs` gaan weg, als ze er nog stonden.

Wie doet wat

Bestand	Zijn taak	Wat erin staat	Wat er nooit in staat
`Recept.cs`, in `Models`	beschrijven hoe één recept eruitziet	de eigenschappen	een lijst met recepten, code die zoekt
`IReceptRepository.cs`	het contract: welke vragen de controller over recepten mag stellen	enkel methodekoppen: de naam, de parameters en het returntype	code tussen accolades, een lijst, iets over HTTP
`InMemoryReceptRepository.cs`	de gegevens beheren	de lijst met de startgegevens, en alles wat erin zoekt, toevoegt, wijzigt of verwijdert, ook het nieuwe Id	`Ok`, `NotFound`, `BadRequest`, routes, berichten voor de client
`ReceptController.cs`	het gesprek met de client	de routes, de controles op invoer, de vragen aan de repository, de keuze van de statuscode en het bericht, de logging	de lijst, LINQ op de gegevens, `new` voor de repository
`Program.cs`	vastleggen welke klasse er komt als iemand de interface vraagt	`AddScoped<IReceptRepository, InMemoryReceptRepository>()`	zoekwerk of antwoorden voor de client

Kort gezegd: de repository weet alles over de gegevens en niets over HTTP. De controller weet alles over HTTP en niets over waar de gegevens staan. De interface is de afspraak tussen die twee.

Het vertrekpunt: je controller na les 02

Na les 02 heb je een model en een controller. Het model staat in `Models/Recept.cs`, en verandert in deze les niet:

```
namespace ReceptenApi.Models;

public class Recept
{
    public int Id { get; set; }
    public string Titel { get; set; } = default!;
    public decimal Kostprijs { get; set; }
}
```

De controller doet alles zelf. Lees hem rustig, en kijk naar wat oranje is. Dat is alles wat met de gegevens werkt: de lijst zelf, en alles wat erin zoekt, een nummer uitdeelt, toevoegt, wijzigt of verwijdert. Dat verhuist naar de repository. De rest gaat over HTTP, en blijft in de controller.

```
using Microsoft.AspNetCore.Mvc;
using ReceptenApi.Models;

namespace ReceptenApi.Controllers;

[ApiController]
[Route("api/[controller]")]
public class ReceptController : ControllerBase
{
    private static List<Recept> _recepten = new List<Recept>
    {
        new Recept { Id = 1, Titel = "Lasagne", Kostprijs = 12.50m },
        new Recept { Id = 2, Titel = "Pad thai", Kostprijs = 9.80m },
        new Recept { Id = 3, Titel = "Linzensoep", Kostprijs = 4.20m }
    };

    [HttpGet]
    public ActionResult<List<Recept>> GetAll()
    {
        return Ok(_recepten);
    }

    [HttpGet("{id}")]
    public ActionResult<Recept> GetById(int id)
    {
        Recept? recept = _recepten.FirstOrDefault(r => r.Id == id);

        if (recept == null)
        {
            return NotFound($"We vonden geen recept met Id {id}.");
        }

        return Ok(recept);
    }

    [HttpPost]
    public ActionResult<Recept> Create(Recept nieuw)
    {
        if (nieuw.Kostprijs < 0)
        {
            return BadRequest("De kostprijs kan niet negatief zijn.");
        }

        nieuw.Id = _recepten.Max(r => r.Id) + 1;
        _recepten.Add(nieuw);

        return CreatedAtAction(nameof(GetById), new { id = nieuw.Id }, nieuw);
    }

    [HttpPut("{id}")]
    public ActionResult Update(int id, Recept gewijzigd)
    {
```

```
        Recept? bestaand = _recepten.FirstOrDefault(r => r.Id == id);

        if (bestaand == null)
        {
            return NotFound($"We vonden geen recept met Id {id}.");
        }

        if (gewijzigd.Kostprijs < 0)
        {
            return BadRequest("De kostprijs kan niet negatief zijn.");
        }

        bestaand.Titel = gewijzigd.Titel;
        bestaand.Kostprijs = gewijzigd.Kostprijs;

        return NoContent();
    }

    [HttpDelete("{id}")]
    public ActionResult Delete(int id)
    {
        Recept? recept = _recepten.FirstOrDefault(r => r.Id == id);

        if (recept == null)
        {
            return NotFound($"We vonden geen recept met Id {id}.");
        }

        _recepten.Remove(recept);

        return NoContent();
    }
}
```

Kijk naar een paar regels waar oranje en gewoon door elkaar staan:

- In `return Ok(_recepten);` is enkel de lijst oranje. De lijst teruggeven is werk met de gegevens. `Ok` is een HTTP-antwoord: status 200.
- In `Recept? recept = _recepten.FirstOrDefault(r => r.Id == id);` is enkel het zoeken oranje. De variabele `recept` blijft in de controller. Ze krijgt straks haar waarde van de repository.
- `if (recept == null)` is niet oranje, al staat het vlak onder het zoeken. Zoeken is werk met de gegevens. Beslissen wat je de client antwoordt als er niets is, is HTTP.

> **BELANGRIJK**
>
> **Eén actie valt in twee stukken.** Bijna elke actie stelt eerst een vraag aan de gegevens, en kiest dan een antwoord voor de client. Het eerste stuk verhuist naar de repository. Het tweede blijft in de controller. Daarom verdwijnt geen enkele actie uit de controller: elke actie wordt korter, en vraagt de repository wat ze vroeger zelf deed.

Regel per regel: waar hoort dit?

Elk soort regel uit je controller van les 02 staat in deze tabel, met waar het naartoe gaat en waarom.

Wat er in je controller van les 02 staat	Waar het naartoe gaat	Waarom
de lijst, `private static List<Recept> _recepten`, met de startgegevens	de repository	dat zijn de gegevens zelf
zoeken op Id, met `FirstOrDefault(r => r.Id == id)`	de repository, in `GetById`	zoeken in de gegevens
filteren, met `Where(...)`	de repository, in een eigen methode	ook dat is zoeken in de gegevens
het nieuwe Id bepalen, met `Max(r => r.Id) + 1`	de repository, in `Create`	wie de gegevens bewaart, deelt de nummers uit. Vanaf les 05 doet de database dat
toevoegen, met `Add`	de repository, in `Create`	de gegevens veranderen
de eigenschappen overnemen bij bijwerken, zoals `bestaand.Titel = gewijzigd.Titel;`	de repository, in `Update`	dat is de wijziging zelf
verwijderen, met `Remove`	de repository, in `Delete`	de gegevens veranderen
`if (recept == null) return NotFound(...)`	blijft in de controller	een statuscode kiezen is HTTP
je controles op invoer, met `BadRequest`	blijft in de controller	beslissen of een verzoek mag
`Ok`, `CreatedAtAction`, `NoContent`	blijft in de controller	het antwoord aan de client
`[HttpGet]`, `[Route(...)]` en de andere attributen	blijft in de controller	routes zijn HTTP
logging	blijft in de controller	de les logt enkel in de controller
`using Microsoft.AspNetCore.Mvc;`	blijft enkel in de controller	de repository heeft niets uit die namespace nodig
`new InMemoryReceptRepository()`	nergens: `Program.cs` neemt het over	de controller maakt niet zelf wat hij nodig heeft

Staat er een regel in je controller die niet in de tabel staat, stel jezelf dan deze drie vragen:

1. Zou de regel ook bestaan als de recepten in een bestand of een database stonden, alleen anders geschreven? Dan gaat hij over de gegevens, en hoort hij in de repository.
2. Gaat de regel over HTTP: een route, een statuscode, of een bericht dat de client te zien krijgt? Dan hoort hij in de controller.
3. Beslist de regel of iets mag? Dan hoort hij in de controller. De repository voert uit wat de controller vraagt, en beslist zelf niets.

Stap 1: opruimen

Wat je doet. Verwijder `WeatherForecast.cs` en `WeatherForecastController.cs`, als ze er nog staan. Verwijder ook controllers uit les 01 die je niet meer gebruikt.

Waarom. Straks zoek je in je project waar iets staat. Bestanden die je niet gebruikt, staan dan in de weg.

Zo weet je dat het lukte. Je project bouwt (**Ctrl+Shift+B**). Bouwt het niet, dan gebruikte iets nog een bestand dat je verwijderde.

Stap 2: de interface, afgeleid uit je controller

Wat je doet. Maak een map `Repositories`. Klik er met de rechtermuisknop op, kies **Add › New Item**, en dan **Interface**. Noem het bestand `IReceptRepository.cs`.

Welke methodes erin komen. Die verzin je niet. Je leest ze af uit de oranje stukken in je controller. Elk endpoint stelt de gegevens een vraag, en elke vraag wordt één methodekop.

Endpoint	Oranje in de controller	De vraag aan de gegevens	De methodekop
GET alles	`_recepten`	geef alle recepten	`List<Recept> GetAll();`
GET op Id	`_recepten.FirstOrDefault(r => r.Id == id)`	geef het recept met dit Id, als het bestaat	`Recept? GetById(int id);`
POST	`nieuw.Id = _recepten.Max(r => r.Id) + 1;` en `_recepten.Add(nieuw);`	bewaar dit nieuwe recept, en geef het terug met zijn nummer	`Recept Create(Recept recept);`
PUT	het zoeken, en het overnemen van de eigenschappen	neem de nieuwe waarden over in het recept met dit Id	`void Update(int id, Recept recept);`
DELETE	het zoeken, en `_recepten.Remove(recept);`	verwijder het recept met dit Id	`void Delete(int id);`

Elke kop heeft drie delen. Zo kies je ze:

- **De naam** zegt wat er gebeurt, niet hoe. `GetById`, niet `ZoekInLijst`: de database van les 05 zoekt niet in een lijst, en de naam moet dan nog kloppen.
- **De parameters** zijn wat de repository nodig heeft om haar werk te doen: een Id, een recept, of allebei. Nooit iets van HTTP.
- **Het returntype** is wat de controller terug nodig heeft om zijn antwoord te kiezen.

Dat laatste is het moeilijkste. Daarom per methode:

Methode	Returntype	Waarom
`GetAll`	`List<Recept>`	de controller stuurt de lijst door met `Ok`. Zijn er geen recepten, dan is het een lege lijst
`GetById`	`Recept?`	het recept kan ontbreken. Het vraagteken belooft de controller dat hij `null` kan krijgen, zodat hij weet dat hij dat moet nakijken
`Create`	`Recept`	de controller heeft het nieuwe Id nodig voor `CreatedAtAction`. Dat Id bestaat pas als de repository het uitdeelde. Daarom geeft `Create` het recept terug
`Update`	`void`	de controller heeft niets terug nodig voor `NoContent`. Of het recept bestaat, weet hij al op voorhand (zie stap 6)
`Delete`	`void`	net als bij `Update`

Zo ziet de interface eruit:

```
using ReceptenApi.Models;

namespace ReceptenApi.Repositories;

public interface IReceptRepository
{
    List<Recept> GetAll();
    Recept? GetById(int id);
    Recept Create(Recept recept);
    void Update(int id, Recept recept);
    void Delete(int id);
}
```

Let op vier dingen:

- Na elke kop staat een puntkomma, geen accolades. Een interface zegt *wat* er moet bestaan, niet *hoe* het werkt.
- Er staat geen `public` voor de methodes. In een interface is alles vanzelf publiek.
- `using ReceptenApi.Models;` is nodig, want `Recept` staat in een andere namespace.
- `using Microsoft.AspNetCore.Mvc;` staat er niet. Het contract kent geen HTTP.

Waarom een interface, en niet meteen een klasse? Straks kent de controller enkel deze interface. In les 05 komt er een tweede klasse, `ReceptRepository`, die met de database werkt en dezelfde vijf methodes heeft. Omdat de controller enkel het contract kent, merkt hij die wissel niet. De les vergelijkt het met een bakker: jij geeft geld, je krijgt brood. Hoe de bakker het brood maakte, hoef je niet te weten. Daarom kan je naar elke bakker. Zie ook hoofdstuk 19.

Zo weet je dat het lukte. Je project bouwt, en je API werkt nog zoals ervoor. Niemand gebruikt de interface al.

Stap 3: de klasse die het contract belooft

Wat je doet. Maak in de map `Repositories` een klasse `InMemoryReceptRepository`. Schrijf `: IReceptRepository` achter haar naam. Er komt een rode lijn onder `IReceptRepository`, met fout **CS0535**: de klasse belooft het contract, maar heeft de methodes nog niet. Zet je cursor op de rode lijn, druk **Ctrl+.**, en kies **Implement interface**. Visual Studio maakt dan dit:

```
using ReceptenApi.Models;

namespace ReceptenApi.Repositories;

public class InMemoryReceptRepository : IReceptRepository
{
    public List<Recept> GetAll()
    {
        throw new NotImplementedException();
    }

    public Recept? GetById(int id)
    {
        throw new NotImplementedException();
    }

    public Recept Create(Recept recept)
    {
        throw new NotImplementedException();
    }

    public void Update(int id, Recept recept)
    {
        throw new NotImplementedException();
    }

    public void Delete(int id)
    {
        throw new NotImplementedException();
    }
}
```

Staan de methodes bij jou in een andere volgorde, dan maakt dat niet uit.

Waarom deze naam. `InMemory` zegt waar de gegevens staan: in het geheugen. In les 05 komt er een tweede klasse naast, met een andere naam en hetzelfde contract. De les noemt haar klasse kortweg `InMemoryRepository`. Met de naam van het model erin weet je later, met meer modellen, meteen welke repository het is.

Waarom eerst lege methodes. Vanaf nu bewaakt de compiler het contract. Vergeet je een methode, of klopt een returntype niet, dan bouwt je project niet. `throw new NotImplementedException();` is een plaatshouder: bouwen lukt, maar wie de methode oproept, krijgt een fout. Stap 6 vult ze één voor één in.

Zo weet je dat het lukte. Je project bouwt, en je API werkt nog zoals ervoor. Niemand gebruikt de klasse al.

Stap 4: de lijst verhuist

Wat je doet. Kopieer de lijst met de startgegevens uit de controller naar de repository, bovenaan in de klasse. Laat de lijst in de controller voorlopig staan.

```
public class InMemoryReceptRepository : IReceptRepository
{
    private static List<Recept> _recepten = new List<Recept>
    {
        new Recept { Id = 1, Titel = "Lasagne", Kostprijs = 12.50m },
        new Recept { Id = 2, Titel = "Pad thai", Kostprijs = 9.80m },
        new Recept { Id = 3, Titel = "Linzensoep", Kostprijs = 4.20m }
    };

    // ... de vijf methodes van stap 3
}
```

Waarom hier. De lijst is de opslag. Wie de gegevens bewaart, is vanaf nu de repository.

Waarom kopiëren, en niet knippen. Knip je de lijst uit de controller, dan bouwt je project pas weer als alle acties verhuisd zijn. Al die tijd kan je niets testen. Kopieer je ze, dan blijft alles werken, en haal je de lijst uit de controller weg in stap 7.

Waarom `static`. Na deze les krijgt elk verzoek een nieuwe controller, en een nieuwe repository. Volg twee verzoeken na elkaar, eerst zonder `static`:

1. Een POST komt binnen. Er komt een repository A, met een nieuwe lijst van drie recepten. Recept 4 komt erbij. Het antwoord vertrekt, en repository A verdwijnt, met haar lijst.
2. Een GET komt binnen. Er komt een repository B, met een nieuwe lijst van drie recepten. Recept 4 staat er niet in.

Met `static` hoort de lijst bij de klasse, niet bij repository A of B. Er is dan maar één lijst, en elke repository gebruikt die. In les 02 was de lijst in de controller om dezelfde reden `static`: ook toen kwam er bij elk verzoek een nieuwe controller. Zie hoofdstuk 24.

> **BELANGRIJK**
>
> De lijst in de repository moet `static` zijn. De les schrijft ze zonder. Je merkt dat pas als je iets toevoegt of wijzigt, en het bij het volgende verzoek weer weg is.

Zo weet je dat het lukte. Je project bouwt, en je API werkt nog zoals ervoor. Er zijn nu twee lijsten met dezelfde startgegevens, maar enkel die in de controller wordt gebruikt.

Stap 5: de controller krijgt een veld voor de repository

Wat je doet. Geef de controller een veld voor zijn repository, en een constructor die er een maakt. De les doet het eerst zo, met `new`. Bovenaan komt er een `using` bij voor de map `Repositories`.

```
using Microsoft.AspNetCore.Mvc;
using ReceptenApi.Models;
using ReceptenApi.Repositories;

namespace ReceptenApi.Controllers;

[ApiController]
[Route("api/[controller]")]
public class ReceptController : ControllerBase
{
    private static List<Recept> _recepten = new List<Recept>
    {
        // ... de drie recepten, die blijven nog even staan
    };

    private readonly IReceptRepository _repository;

    public ReceptController()
    {
        _repository = new InMemoryReceptRepository();
    }

    // ... de vijf acties, nog ongewijzigd
}
```

Waarom het zo geschreven is:

- **Het veld heeft het type van de interface**, niet van de klasse. Zo kent de rest van de controller enkel het contract. Er is maar één regel die de klasse bij naam noemt: de regel met `new`. In stap 8 verdwijnt ook die.
- `private`**:** niemand buiten de controller heeft iets met zijn repository te maken.
- `readonly`**:** het veld krijgt één keer een waarde, in de constructor, en daarna nooit meer. Probeer je het later toch, dan bouwt je project niet. Zie hoofdstuk 24.
- **De underscore** voor `_repository` is een afspraak voor private velden. De les noemt zo'n veld een *dependency*: iets waarvan de controller afhangt om zijn werk te doen.
- **Eerst met** `new`: zo werkt alles meteen, zonder iets in `Program.cs`. Je kan nu actie per actie omzetten. Maar de controller beslist nog zelf welke klasse hij gebruikt. Dat is nog strakke koppeling. Stap 8 lost het op.

Zo weet je dat het lukte. Je project bouwt, en je API werkt nog zoals ervoor. De acties gebruiken nog hun eigen lijst.

Stap 6: actie per actie verhuizen

Nu verhuis je het werk, één actie tegelijk. Bij elke actie doe je drie dingen:

1. In de repository vervang je `throw new NotImplementedException();` door de oranje regels uit die actie.
2. In de controller vervang je die oranje regels door één vraag aan `_repository`.
3. Je start je API en test die ene actie in Scalar.

Werk in deze volgorde: eerst de twee GET's, dan POST, PUT en DELETE. Zo kan je na elke actie testen met acties die al verhuisd zijn.

> LET OP
>
> **Tijdens deze stap zijn er twee lijsten.** De ene staat in de controller, de andere in de repository. Ze beginnen gelijk, maar lopen uit elkaar zodra je iets toevoegt, wijzigt of verwijdert. Een recept dat je via de repository toevoegt, zie je niet in een actie die nog de lijst van de controller gebruikt. Test daarom telkens met acties die al verhuisd zijn. De volledige test doe je na stap 7, als er nog maar één lijst is.

6a. GET alles

Vóór, in de controller:

```
[HttpGet]
public ActionResult<List<Recept>> GetAll()
{
    return Ok(_recepten);
}
```

In de repository vervang je de plaatshouder:

```
public List<Recept> GetAll()
{
    return _recepten;
}
```

Ná, in de controller:

```
[HttpGet]
public ActionResult<List<Recept>> GetAll()
{
    return Ok(_repository.GetAll());
}
```

Wat verhuisde: de lijst teruggeven. **Wat bleef:** de route en `Ok`.

Waarom. `Ok` maakt er een HTTP-antwoord van, met status 200. De repository weet niet dat er een client is. Ze geeft de lijst, en daar stopt het.

Test: `GET api/recept` geeft 200 met de drie recepten.

6b. GET op Id

Vóór, in de controller:

```
[HttpGet("{id}")]
public ActionResult<Recept> GetById(int id)
{
    Recept? recept = _recepten.FirstOrDefault(r => r.Id == id);

    if (recept == null)
    {
        return NotFound($"We vonden geen recept met Id {id}.");
    }

    return Ok(recept);
}
```

In de repository:

```
public Recept? GetById(int id)
{
    return _recepten.FirstOrDefault(r => r.Id == id);
}
```

Ná, in de controller:

```
[HttpGet("{id}")]
public ActionResult<Recept> GetById(int id)
{
    Recept? recept = _repository.GetById(id);

    if (recept == null)
    {
        return NotFound($"We vonden geen recept met Id {id}.");
    }

    return Ok(recept);
}
```

Wat verhuisde: het zoeken. **Wat bleef:** de controle op `null`, `NotFound` met het bericht, en `Ok`.

Waarom de actie hier in twee valt. Ze stelt twee verschillende vragen:

- *Is er een recept met dit Id?* Dat is een vraag aan de gegevens. De repository antwoordt met het recept, of met `null`.
- *Wat zeg ik de client als het er niet is?* Dat is HTTP. Enkel de controller kent 404 en het bericht.

Voor de repository is `null` geen fout, maar een antwoord: *niet gevonden*. De controller vertaalt dat antwoord naar een 404. `NotFound` kan ook niet in de repository: die methode bestaat enkel in een controller. In de repository geeft ze fout **CS0103**.

Test: `GET api/recept/1` geeft 200 met Lasagne. `GET api/recept/99` geeft 404 met *We vonden geen recept met Id 99.*

6c. POST

Vóór, in de controller:

```
[HttpPost]
public ActionResult<Recept> Create(Recept nieuw)
{
    if (nieuw.Kostprijs < 0)
    {
        return BadRequest("De kostprijs kan niet negatief zijn.");
    }

    nieuw.Id = _recepten.Max(r => r.Id) + 1;
    _recepten.Add(nieuw);

    return CreatedAtAction(nameof(GetById), new { id = nieuw.Id }, nieuw);
}
```

In de repository:

```
public Recept Create(Recept recept)
{
    recept.Id = _recepten.Max(r => r.Id) + 1;
    _recepten.Add(recept);
    return recept;
}
```

Ná, in de controller:

```
[HttpPost]
public ActionResult<Recept> Create(Recept nieuw)
{
    if (nieuw.Kostprijs < 0)
    {
        return BadRequest("De kostprijs kan niet negatief zijn.");
    }

    Recept gemaakt = _repository.Create(nieuw);

    return CreatedAtAction(nameof(GetById), new { id = gemaakt.Id }, gemaakt);
}
```

Wat verhuisde: het nieuwe Id bepalen, en het recept toevoegen. **Wat bleef:** de controle op de kostprijs, `BadRequest`, en `CreatedAtAction`.

Waarom het Id naar de repository gaat. Wie de gegevens bewaart, deelt de nummers uit. In les 05 doet de database dat zelf. Dan verandert enkel de repository, en de controller merkt niets.

Waarom de controle op de kostprijs blijft. Beslissen of een verzoek mag, is de taak van de controller: de ober kijkt of je bestelling kan, voor hij naar de keuken gaat. De repository voert uit wat de controller vraagt. De controle staat ook vóór `Create`. Stond ze erna, dan zat het foute recept al in de lijst voor je *nee* zei.

Waarom `Create` het recept teruggeeft. `CreatedAtAction` heeft het nieuwe Id nodig. `gemaakt` is hetzelfde object als `nieuw`, want een klasse is een referentietype (zie hoofdstuk 16). `nieuw.Id` zou dus ook werken. Maar met `gemaakt` lees je meteen waar het nummer vandaan komt. En in les 05 klopt het nog altijd.

Waarom de parameter in de repository anders heet. In de controller heet hij `nieuw`, in de repository `recept`. Dat mag: een methode kiest zelf de naam van haar parameters. De repository weet niet dat het recept nieuw is voor de client. Ze krijgt een recept, en bewaart het.

Test: een POST naar `api/recept` met `{ "titel": "Stoofvlees", "kostprijs": 15.00 }` geeft 201, met Id 4. Een GET op `api/recept` toont het erbij, want die actie gebruikt al de repository. Een POST met `"kostprijs": -1` geeft 400.

6d. PUT

Vóór, in de controller:

```
[HttpPut("{id}")]
public ActionResult Update(int id, Recept gewijzigd)
{
    Recept? bestaand = _recepten.FirstOrDefault(r => r.Id == id);

    if (bestaand == null)
    {
        return NotFound($"We vonden geen recept met Id {id}.");
    }

    if (gewijzigd.Kostprijs < 0)
    {
        return BadRequest("De kostprijs kan niet negatief zijn.");
    }

    bestaand.Titel = gewijzigd.Titel;
    bestaand.Kostprijs = gewijzigd.Kostprijs;

    return NoContent();
}
```

In de repository:

```
public void Update(int id, Recept recept)
{
    Recept? bestaand = _recepten.FirstOrDefault(r => r.Id == id);

    if (bestaand != null)
    {
        bestaand.Titel = recept.Titel;
        bestaand.Kostprijs = recept.Kostprijs;
    }
}
```

Ná, in de controller:

```
[HttpPut("{id}")]
public ActionResult Update(int id, Recept gewijzigd)
{
    if (_repository.GetById(id) == null)
    {
        return NotFound($"We vonden geen recept met Id {id}.");
    }

    if (gewijzigd.Kostprijs < 0)
    {
        return BadRequest("De kostprijs kan niet negatief zijn.");
    }

    _repository.Update(id, gewijzigd);

    return NoContent();
}
```

Wat verhuisde: het bestaande recept zoeken, en de nieuwe waarden overnemen. **Wat bleef:** nagaan of het recept bestaat, de controle op de kostprijs, `NotFound`, `BadRequest` en `NoContent`.

Hier is de splitsing het moeilijkst. Vóór de verbouwing had de controller het gevonden recept voor twee dingen nodig: om te weten of hij een 404 moest geven, en om de waarden in over te nemen. Dat tweede verhuist. Voor het eerste vraagt de controller nu `_repository.GetById(id)`, en kijkt hij enkel of het antwoord `null` is.

> **OPMERKING**
>
> **Waarom zoekt de repository in `Update` nog eens, als de controller al keek?** Het zijn twee verschillende vragen. De controller wil weten of het recept bestaat, om het juiste antwoord te kiezen. De repository zoekt het op om er iets mee te doen. Ze weet niet wie haar oproept, en mag bij een onbekend Id niet crashen. Daarom kijkt ze zelf na of ze iets vond, en doet ze anders gewoon niets.

Nog twee dingen in de repository:

- **De controle** `if (bestaand != null)`**.** Zonder die controle crasht `bestaand.Titel` met een `NullReferenceException` als iemand `Update` oproept met een onbekend Id. Zie hoofdstuk 17.
- **De eigenschappen één voor één.** Je past het object aan dat al in de lijst staat. `bestaand = recept;` zou dat niet doen: het laat enkel je variabele `bestaand` naar een ander object wijzen. De lijst bevat dan nog altijd het oude recept. Zie hoofdstuk 16.

Test: een PUT naar `api/recept/2` met een nieuwe kostprijs geeft 204. Een GET op `api/recept/2` toont de nieuwe kostprijs. Een PUT naar `api/recept/99` geeft 404.

6e. DELETE

Vóór, in de controller:

```
[HttpDelete("{id}")]
public ActionResult Delete(int id)
{
    Recept? recept = _recepten.FirstOrDefault(r => r.Id == id);

    if (recept == null)
    {
        return NotFound($"We vonden geen recept met Id {id}.");
    }

    _recepten.Remove(recept);

    return NoContent();
}
```

In de repository:

```
public void Delete(int id)
{
    Recept? recept = _recepten.FirstOrDefault(r => r.Id == id);

    if (recept != null)
    {
        _recepten.Remove(recept);
    }
}
```

Ná, in de controller:

```
[HttpDelete("{id}")]
public ActionResult Delete(int id)
{
    if (_repository.GetById(id) == null)
    {
        return NotFound($"We vonden geen recept met Id {id}.");
    }

    _repository.Delete(id);

    return NoContent();
}
```

Wat verhuisde: het recept zoeken en verwijderen. **Wat bleef:** nagaan of het recept bestaat, `NotFound` en `NoContent`. Het werkt net als bij PUT.

Waarom `Delete` een Id krijgt, en geen recept. Het Id is genoeg om te weten welk recept weg moet. De controller hoeft het object dus niet door te geven. De database in les 05 verwijdert ook op Id.

Test: een DELETE op `api/recept/3` geeft 204. Een GET op `api/recept/3` geeft daarna 404. Een DELETE op `api/recept/99` geeft 404.

Na stap 6, endpoint per endpoint

Endpoint	Blijft in de controller	Gaat naar de repository
GET alles	de route, `Ok`	de lijst teruggeven
GET op Id	de route, de controle op `null`, `NotFound` of `Ok`	zoeken op Id
POST	de route, de controles op invoer, `CreatedAtAction` met het nieuwe Id	het nieuwe Id bepalen en het recept toevoegen
PUT	de route, nagaan of het recept bestaat, de controles op invoer, `NotFound` of `NoContent`	het bestaande recept zoeken en de eigenschappen overnemen
DELETE	de route, nagaan of het recept bestaat, `NotFound` of `NoContent`	het recept zoeken en verwijderen

En wat de controller doet met wat de repository teruggeeft:

Methode	Geeft terug	Wat dat betekent	Wat de controller ermee doet
`GetAll`	`List<Recept>`	alle recepten, of een lege lijst	`Ok` met de lijst. Ook een lege lijst is een geldig antwoord
`GetById`	`Recept?`	het recept, of `null` als het er niet is	bij `null` een `NotFound` met een bericht, anders `Ok`
`Create`	`Recept`	het recept, nu met zijn nieuwe Id	`CreatedAtAction` met dat Id
`Update`	niets (`void`)		de repository zegt niet of het lukte. Daarom kijkt de controller vooraf met `GetById`

Methode	Geeft terug	Wat dat betekent	Wat de controller ermee doet
`Delete`	niets (`void`)		net als bij `Update`

Een repository geeft nooit een statuscode of een bericht terug. Ze kent geen HTTP. Dezelfde repository zou je ook in een consoletoepassing kunnen gebruiken.

Stap 7: de lijst uit de controller

Wat je doet. Verwijder de lijst uit de controller. Bouw je project.

Waarom nu pas. Na stap 6 gebruikt geen enkele actie die lijst nog. Er blijft één lijst over, in de repository. Pas nu heeft een volledige test zin.

Bouwt het niet? Dan krijg je fout **CS0103**: *The name '_recepten' does not exist in the current context*. Elke rode lijn wijst een actie aan die nog met de lijst werkt. Die actie vergat je in stap 6.

Zo ziet de controller er nu uit. Dit is ook de tussenversie van de les: alles werkt via de repository, maar de controller maakt ze nog zelf.

```
using Microsoft.AspNetCore.Mvc;
using ReceptenApi.Models;
using ReceptenApi.Repositories;

namespace ReceptenApi.Controllers;

[ApiController]
[Route("api/[controller]")]
public class ReceptController : ControllerBase
{
    private readonly IReceptRepository _repository;

    public ReceptController()
    {
        _repository = new InMemoryReceptRepository();
    }

    [HttpGet]
    public ActionResult<List<Recept>> GetAll()
    {
        return Ok(_repository.GetAll());
    }

    [HttpGet("{id}")]
    public ActionResult<Recept> GetById(int id)
    {
        Recept? recept = _repository.GetById(id);

        if (recept == null)
        {
            return NotFound($"We vonden geen recept met Id {id}.");
        }

        return Ok(recept);
```

```
    }

    [HttpPost]
    public ActionResult<Recept> Create(Recept nieuw)
    {
        if (nieuw.Kostprijs < 0)
        {
            return BadRequest("De kostprijs kan niet negatief zijn.");
        }

        Recept gemaakt = _repository.Create(nieuw);

        return CreatedAtAction(nameof(GetById), new { id = gemaakt.Id }, gemaakt);
    }

    [HttpPut("{id}")]
    public ActionResult Update(int id, Recept gewijzigd)
    {
        if (_repository.GetById(id) == null)
        {
            return NotFound($"We vonden geen recept met Id {id}.");
        }

        if (gewijzigd.Kostprijs < 0)
        {
            return BadRequest("De kostprijs kan niet negatief zijn.");
        }

        _repository.Update(id, gewijzigd);

        return NoContent();
    }

    [HttpDelete("{id}")]
    public ActionResult Delete(int id)
    {
        if (_repository.GetById(id) == null)
        {
            return NotFound($"We vonden geen recept met Id {id}.");
        }

        _repository.Delete(id);

        return NoContent();
    }
}
```

Kijk wat er níet meer in staat: geen lijst, geen `FirstOrDefault`, geen `Max`, geen `Add` of `Remove`. Elke actie stelt de repository een vraag, en kiest dan haar antwoord.

Zo weet je dat het lukte. Doe nu alle verzoeken uit stap 6 opnieuw, na elkaar. Je krijgt dezelfde antwoorden als voor de les. Een recept dat je toevoegt, zie je nu in elk endpoint.

Stap 8: van new naar dependency injection

Het probleem. De controller noemt nog één klasse bij naam: `new InMemoryReceptRepository()`. In les 05 komt `ReceptRepository`. Dan moet je in elke controller die regel aanpassen. Met tien controllers zijn dat tien regels. Vergeet je er één, dan werkt die controller nog met de lijst, zonder foutmelding.

De oplossing. De controller vraagt wat hij nodig heeft, en iemand anders maakt het. De les vergelijkt het met een auto. Een auto heeft een motor nodig, maar bouwt die niet zelf. De fabriek bouwt de motor en zet hem erin. Dat erin zetten heet **injecteren**, en wat je erin zet is een **dependency**. Vandaar de naam: **dependency injection**.

Dat gebeurt in twee delen: de controller vraagt, en `Program.cs` zegt wat hij krijgt.

8a. De constructor vraagt de repository

Vóór:

```
public ReceptController()
{
    _repository = new InMemoryReceptRepository();
}
```

Ná:

```
public ReceptController(IReceptRepository repository)
{
    _repository = repository;
}
```

De constructor zegt nu: *ik werk enkel als je me een* `IReceptRepository` *geeft*. Wie de controller maakt, moet er een meegeven. Na deze stap staat de naam `InMemoryReceptRepository` nergens meer in de controller.

Waarom een parameter én een veld. Het zijn twee verschillende dingen. De parameter `repository` bestaat enkel zolang de constructor loopt. Het veld `_repository` blijft bestaan zolang de controller bestaat, en de acties gebruiken het. Daarom zet de constructor de parameter in het veld.

Wat er nu gebeurt. Je project bouwt. Maar elk verzoek naar de controller geeft 500, met deze fout:

```
InvalidOperationException: Unable to resolve service for type
'ReceptenApi.Repositories.IReceptRepository' while attempting to
activate 'ReceptenApi.Controllers.ReceptController'.
```

ASP.NET Core maakt de controller, en ziet dat de constructor een `IReceptRepository` vraagt. Maar het weet niet welke klasse dat moet zijn. Een interface kan het niet maken: er staat geen code in. Dat vertel je in 8b.

8b. Program.cs zegt welke klasse

Vóór, het begin van `Program.cs`:

```
using Scalar.AspNetCore;

var builder = WebApplication.CreateBuilder(args);

builder.Services.AddControllers();
builder.Services.AddOpenApi();

var app = builder.Build();
```

Ná:

```
using ReceptenApi.Repositories;
using Scalar.AspNetCore;

var builder = WebApplication.CreateBuilder(args);

builder.Services.AddControllers();
builder.Services.AddOpenApi();
builder.Services.AddScoped<IReceptRepository, InMemoryReceptRepository>();

var app = builder.Build();
```

Lees de nieuwe regel zo: *vraagt iemand een* `IReceptRepository`*, maak dan een* `InMemoryReceptRepository`. Links staat wat er gevraagd wordt, de interface. Rechts staat wat er geleverd wordt, de klasse.

- **Waarom vóór** `builder.Build()`**.** `Build` legt de lijst met diensten vast. Daarna kan er niets meer bij: de API start dan niet. Zie hoofdstuk 27.
- **Waarom** `AddScoped`**.** Elk verzoek krijgt zijn eigen repository. Binnen één verzoek krijgt iedereen die erom vraagt dezelfde. Bij het volgende verzoek komt er een nieuwe. Daarom moet de lijst `static` zijn (stap 4). Er bestaan nog andere manieren, die staan in hoofdstuk 27. De cursus gebruikt `AddScoped`.
- **Waarom de** `using`**.** `IReceptRepository` en `InMemoryReceptRepository` staan in de namespace `ReceptenApi.Repositories`. De les vermeldt die regel niet. **Ctrl+.** op de rode lijn zet hem erbij.

Wat dit oplevert. Kijk wie welke klasse kent, voor en na deze stap:

	Na stap 7	Na stap 8
de controller kent	`IReceptRepository` en `InMemoryReceptRepository`	enkel `IReceptRepository`
`Program.cs` kent	geen van beide	allebei, in één regel
om in les 05 naar de database te wisselen, pas je aan	elke controller	één regel in `Program.cs`

In les 05 wordt de regel `AddScoped<IReceptRepository, ReceptRepository>()`. Geen enkele controller verandert. Dat is losse koppeling.

Zo weet je dat het lukte. Start je API en doe alle verzoeken opnieuw. Je krijgt dezelfde antwoorden als na stap 7.

Stap 9: logging

Wat je doet. De controller krijgt een tweede dependency, een logger. Dat gaat net als bij de repository: een veld, en een parameter in de constructor.

Vóór:

```
private readonly IReceptRepository _repository;

public ReceptController(IReceptRepository repository)
{
    _repository = repository;
}
```

Ná:

```
private readonly IReceptRepository _repository;
private readonly ILogger<ReceptController> _logger;

public ReceptController(IReceptRepository repository,
                        ILogger<ReceptController> logger)
{
    _repository = repository;
    _logger = logger;
}
```

Zet daarna in elke actie minstens één logmelding. Zo ziet `GetById` er dan uit:

```
[HttpGet("{id}")]
public ActionResult<Recept> GetById(int id)
{
    _logger.LogInformation("Recept {Id} opgevraagd", id);

    Recept? recept = _repository.GetById(id);

    if (recept == null)
    {
        _logger.LogWarning("Recept {Id} niet gevonden", id);
        return NotFound($"We vonden geen recept met Id {id}.");
    }

    return Ok(recept);
}
```

- **Waarom je de logger niet registreert.** ASP.NET Core heeft hem zelf al klaarstaan. Je eigen klassen, zoals je repository, registreer je zelf. Wat .NET levert, niet.
- **Waarom `<ReceptController>`.** De naam tussen de punthaken komt mee in elke melding, zodat je ziet waar ze vandaan komt. Zet er altijd de klasse in waarin je logt.
- **Waarom in de controller.** De controller weet wat er gevraagd werd en wat hij antwoordde. De les logt enkel daar.
- **Waarom `{Id}` en geen `$`.** De les schrijft de meldingen met een `$` ervoor. Dat werkt ook. Dit boek gebruikt een sjabloon, zoals Microsoft aanraadt. In het terminalvenster zie je hetzelfde. Zie hoofdstuk 28.

Welk niveau je kiest:

Wat er gebeurt	Niveau	Voorbeeld
er komt een verzoek binnen	`LogInformation`	`"Recept {Id} opgevraagd"`
een actie is gelukt	`LogInformation`	`"Recept {Id} toegevoegd"`
een Id bestaat niet	`LogWarning`	`"Recept {Id} niet gevonden"`
de invoer wordt geweigerd	`LogWarning`	`"Negatieve kostprijs geweigerd"`
er loopt iets mis in je eigen code	`LogError`	zie hoofdstuk 28

Een onbekend Id is `LogWarning`, geen `LogError`. Je API werkt correct: de client vroeg iets wat er niet is.

Zo weet je dat het lukte. In het terminalvenster van je API verschijnen je meldingen:

```
info: ReceptenApi.Controllers.ReceptController[0]
      Recept 1 opgevraagd
warn: ReceptenApi.Controllers.ReceptController[0]
      Recept 99 niet gevonden
```

Stap 10: een nieuw endpoint in de nieuwe opbouw

De les voegt na de verbouwing een endpoint toe dat filtert. Hier doe je hetzelfde met de kostprijs: alle recepten die hoogstens een bepaald bedrag kosten, via `GET api/recept/maximum/{bedrag}`.

In de nieuwe opbouw werk je van binnen naar buiten: eerst het contract, dan de repository, dan de controller. Zo wijst de compiler je telkens de volgende stap aan.

10a. Het contract

```
public interface IReceptRepository
{
    List<Recept> GetAll();
    Recept? GetById(int id);
    List<Recept> GetByMaximumKostprijs(decimal bedrag);
    Recept Create(Recept recept);
    void Update(int id, Recept recept);
    void Delete(int id);
}
```

Het returntype is `List<Recept>`, zonder vraagteken. Een filter vindt nul, één of meer recepten. Nul recepten is een lege lijst, geen `null`.

Bouw je nu, dan krijg je **CS0535** in `InMemoryReceptRepository`: de klasse belooft het contract, maar mist de nieuwe methode. Zo herinnert de compiler je aan de volgende stap.

10b. De repository

```
public List<Recept> GetByMaximumKostprijs(decimal bedrag)
{
    return _recepten
        .Where(r => r.Kostprijs <= bedrag)
        .ToList();
}
```

`Where` zoekt in de gegevens, dus het hoort hier. `ToList` is nodig omdat `Where` geen lijst geeft, maar een `IEnumerable`, en het contract een `List` belooft. Zie hoofdstuk 11 en hoofdstuk 12.

10c. De controller

```
[HttpGet("maximum/{bedrag}")]
public ActionResult<List<Recept>> GetByMaximumKostprijs(decimal bedrag)
{
    _logger.LogInformation("Recepten tot {Bedrag} euro opgevraagd", bedrag);
    return Ok(_repository.GetByMaximumKostprijs(bedrag));
}
```

- **Waarom de route `maximum/{bedrag}`.** `GET api/recept/{id}` bestaat al. Elke combinatie van verb en route moet uniek zijn. Anders geeft je API 500, met *The request matched multiple endpoints*.
- **Waarom `Ok`, ook bij een lege lijst.** Een filter dat niets vindt, is hier een geldig antwoord: er zijn geen recepten onder dat bedrag. Wil je bij een lege lijst een 404, dan komt die controle hier, in de controller, net als bij `GetById`. Welke van de twee je kiest, zegt je opgave.

Test: `GET api/recept/maximum/10` geeft 200 met Pad thai en Linzensoep. `GET api/recept/maximum/1` geeft 200 met `[]`.

Het eindresultaat

Na de tien stappen bestaat je verbouwing uit vier bestanden. Hier staan ze volledig.

Het contract, in `Repositories/IReceptRepository.cs`:

```
using ReceptenApi.Models;

namespace ReceptenApi.Repositories;

public interface IReceptRepository
{
    List<Recept> GetAll();
    Recept? GetById(int id);
    List<Recept> GetByMaximumKostprijs(decimal bedrag);
    Recept Create(Recept recept);
    void Update(int id, Recept recept);
    void Delete(int id);
}
```

De repository, in `Repositories/InMemoryReceptRepository.cs`. Hier staat alles wat vroeger in de controller met `_recepten` te maken had.

```
using ReceptenApi.Models;

namespace ReceptenApi.Repositories;

public class InMemoryReceptRepository : IReceptRepository
{
    private static List<Recept> _recepten = new List<Recept>
    {
        new Recept { Id = 1, Titel = "Lasagne", Kostprijs = 12.50m },
        new Recept { Id = 2, Titel = "Pad thai", Kostprijs = 9.80m },
        new Recept { Id = 3, Titel = "Linzensoep", Kostprijs = 4.20m }
    };

    public List<Recept> GetAll()
    {
        return _recepten;
    }
```

```
    public Recept? GetById(int id)
    {
        return _recepten.FirstOrDefault(r => r.Id == id);
    }

    public List<Recept> GetByMaximumKostprijs(decimal bedrag)
    {
        return _recepten
            .Where(r => r.Kostprijs <= bedrag)
            .ToList();
    }

    public Recept Create(Recept recept)
    {
        recept.Id = _recepten.Max(r => r.Id) + 1;
        _recepten.Add(recept);
        return recept;
    }

    public void Update(int id, Recept recept)
    {
        Recept? bestaand = _recepten.FirstOrDefault(r => r.Id == id);

        if (bestaand != null)
        {
            bestaand.Titel = recept.Titel;
            bestaand.Kostprijs = recept.Kostprijs;
        }
    }

    public void Delete(int id)
    {
        Recept? recept = _recepten.FirstOrDefault(r => r.Id == id);

        if (recept != null)
        {
            _recepten.Remove(recept);
        }
    }
}
```

Kijk wat er níet in staat: geen `Ok`, geen `NotFound`, geen bericht, geen route. De repository geeft gegevens terug, of `null`, en daar stopt het.

De controller, in `Controllers/ReceptController.cs`:

```
using Microsoft.AspNetCore.Mvc;
using ReceptenApi.Models;
using ReceptenApi.Repositories;

namespace ReceptenApi.Controllers;

[ApiController]
[Route("api/[controller]")]
public class ReceptController : ControllerBase
{
```

```
    private readonly IReceptRepository _repository;
    private readonly ILogger<ReceptController> _logger;

    public ReceptController(IReceptRepository repository,
                            ILogger<ReceptController> logger)
    {
        _repository = repository;
        _logger = logger;
    }

    [HttpGet]
    public ActionResult<List<Recept>> GetAll()
    {
        _logger.LogInformation("Alle recepten opgevraagd");
        return Ok(_repository.GetAll());
    }

    [HttpGet("{id}")]
    public ActionResult<Recept> GetById(int id)
    {
        _logger.LogInformation("Recept {Id} opgevraagd", id);

        Recept? recept = _repository.GetById(id);

        if (recept == null)
        {
            _logger.LogWarning("Recept {Id} niet gevonden", id);
            return NotFound($"We vonden geen recept met Id {id}.");
        }

        return Ok(recept);
    }

    [HttpGet("maximum/{bedrag}")]
    public ActionResult<List<Recept>> GetByMaximumKostprijs(decimal bedrag)
    {
        _logger.LogInformation("Recepten tot {Bedrag} euro opgevraagd", bedrag);
        return Ok(_repository.GetByMaximumKostprijs(bedrag));
    }

    [HttpPost]
    public ActionResult<Recept> Create(Recept nieuw)
    {
        _logger.LogInformation("Nieuw recept {Titel}", nieuw.Titel);

        if (nieuw.Kostprijs < 0)
        {
            _logger.LogWarning("Negatieve kostprijs geweigerd");
            return BadRequest("De kostprijs kan niet negatief zijn.");
        }

        Recept gemaakt = _repository.Create(nieuw);
        _logger.LogInformation("Recept {Id} toegevoegd", gemaakt.Id);

        return CreatedAtAction(nameof(GetById), new { id = gemaakt.Id }, gemaakt);
    }

    [HttpPut("{id}")]
```

```
    public ActionResult Update(int id, Recept gewijzigd)
    {
        _logger.LogInformation("Recept {Id} bijwerken", id);

        if (_repository.GetById(id) == null)
        {
            _logger.LogWarning("Recept {Id} niet gevonden", id);
            return NotFound($"We vonden geen recept met Id {id}.");
        }

        if (gewijzigd.Kostprijs < 0)
        {
            _logger.LogWarning("Negatieve kostprijs geweigerd");
            return BadRequest("De kostprijs kan niet negatief zijn.");
        }

        _repository.Update(id, gewijzigd);

        return NoContent();
    }

    [HttpDelete("{id}")]
    public ActionResult Delete(int id)
    {
        _logger.LogInformation("Recept {Id} verwijderen", id);

        if (_repository.GetById(id) == null)
        {
            _logger.LogWarning("Recept {Id} niet gevonden", id);
            return NotFound($"We vonden geen recept met Id {id}.");
        }

        _repository.Delete(id);

        return NoContent();
    }
}
```

De routes, de statuscodes en de berichten zijn letterlijk dezelfde als in je controller van les 02. Daarom merkt de client niets van de verbouwing.

Program.cs na deze les

```
using ReceptenApi.Repositories;
using Scalar.AspNetCore;

var builder = WebApplication.CreateBuilder(args);

builder.Services.AddControllers();
builder.Services.AddOpenApi();
builder.Services.AddScoped<IReceptRepository, InMemoryReceptRepository>();

var app = builder.Build();

if (app.Environment.IsDevelopment())
{
    app.MapOpenApi();
    app.MapScalarApiReference();
}

app.UseHttpsRedirection();
app.UseAuthorization();
app.MapControllers();

app.Run();
```

Hoe je ziet dat het werkt

Doe elk verzoek van voor de les opnieuw. Elk antwoord moet hetzelfde zijn. Enkel de laatste rij is nieuw.

Verzoek	Antwoord, voor én na de les
`GET api/recept`	200 met de drie recepten
`GET api/recept/1`	200 met Lasagne
`GET api/recept/99`	404 met *We vonden geen recept met Id 99.*
`POST api/recept` met een kostprijs van 15	201 met het nieuwe recept, Id 4
`POST api/recept` met een kostprijs van -1	400 met *De kostprijs kan niet negatief zijn.*
`PUT api/recept/2` met een nieuwe kostprijs	204, en een GET toont de nieuwe kostprijs
`PUT api/recept/99`	404 met *We vonden geen recept met Id 99.*
`DELETE api/recept/3`	204, en een GET op recept 3 geeft daarna 404
`DELETE api/recept/99`	404 met *We vonden geen recept met Id 99.*
`GET api/recept/maximum/10`	nieuw: 200 met de recepten van hoogstens 10 euro

Daarnaast:

- Een POST, en daarna een GET: het nieuwe recept staat erbij.
- In het terminalvenster van je API verschijnen je logmeldingen.
- In je controller staat geen lijst meer, geen LINQ op je gegevens, en geen `new` voor de repository.

Wat er gebeurt bij één verzoek

Een client stuurt `GET api/recept/1`. Dit gebeurt er, in deze volgorde:

1. ASP.NET Core zoekt welke controller en welke actie bij die route horen: `ReceptController`, en daarin `GetById`.
2. Om een `ReceptController` te maken, kijkt het naar zijn constructor. Die vraagt een `IReceptRepository` en een `ILogger<ReceptController>`.
3. Voor `IReceptRepository` kijkt het in `Program.cs`. Daar staat `AddScoped<IReceptRepository, InMemoryReceptRepository>()`, dus het maakt een nieuwe `InMemoryReceptRepository`. De logger levert het zelf.
4. Het maakt de controller, en geeft beide mee aan de constructor. De constructor bewaart ze in zijn velden.
5. `GetById` loopt: een logmelding, een vraag aan de repository, en de keuze tussen 200 en 404.
6. Het antwoord gaat naar de client. De controller en de repository verdwijnen. De `static` lijst blijft, want die hoort bij de klasse, niet bij één object.

Bij het volgende verzoek begint alles opnieuw, met een nieuwe controller en een nieuwe repository. Zo werkt `AddScoped`: één repository per verzoek.

Alle stappen op een rij

Gebruik dit lijstje terwijl je bouwt. Na elke stap bouwt je project (**Ctrl+Shift+B**), en werkt je API nog.

1. Ruim op: haal de bestanden van het weervoorbeeld weg, en controllers die je niet meer gebruikt.
2. Maak een map `Repositories`, en daarin de interface `IReceptRepository`. Lees de methodes af uit de oranje stukken: één kop per vraag aan de gegevens.
3. Maak de klasse `InMemoryReceptRepository`, die de interface belooft. Laat **Ctrl+.** de lege methodes maken.
4. Kopieer de lijst met de startgegevens naar de repository, `static`. Laat de lijst in de controller nog staan.
5. Geef de controller een veld `private readonly IReceptRepository _repository`, en vul het in de constructor met `new InMemoryReceptRepository()`.

6. Verhuis actie per actie: de oranje regels naar de repository, één vraag aan `_repository` in de controller. Test elke actie in Scalar.
7. Haal de lijst uit de controller weg. Doe de volledige test.
8. Laat de constructor een `IReceptRepository` vragen in plaats van ze te maken. Registreer ze in `Program.cs` met `AddScoped`, vóór `builder.Build()`.
9. Vraag ook een `ILogger<ReceptController>` in de constructor, en zet in elke actie minstens één logmelding. De logger registreer je niet.
10. Voeg nieuwe endpoints toe van binnen naar buiten: contract, repository, controller.

Zo kijk je je eigen code na

Zoek met **Ctrl+F** in elk bestand naar wat er niet meer in hoort.

Waar je zoekt	Waarnaar	Vind je het, dan
je controller	de naam van je lijst, of `List<` als veld	staat de lijst er nog
je controller	`new` gevolgd door de naam van je repository	maakt de controller zijn repository nog zelf
je controller	`FirstOrDefault(`, `Where(`, `Max(`, `.Add(`, `.Remove(` op je gegevens	doet de controller nog werk met de gegevens
je repository	`Ok(`, `NotFound(`, `BadRequest(`, `NoContent(`, `ActionResult`	kiest de repository een antwoord voor de client
je repository	`NotImplementedException`	is een methode nog leeg
je repository	`static` bij je lijst	ontbreekt het, dan verdwijnt wat je toevoegt
je interface	een `{` na een methodekop	staat er code in je contract
`Program.cs`	je `AddScoped`	kijk of ze boven `builder.Build()` staat

Als een stap ontbreekt

Wat je ziet	Waarom	Wat je doet
elk verzoek naar de controller geeft 500, met *Unable to resolve service for type 'ReceptenApi.Repositories.IReceptRepository' while attempting to activate 'ReceptenApi.Controllers.ReceptController'.*	de registratie in `Program.cs` ontbreekt	`AddScoped` toevoegen, vóór `builder.Build()`
de API start niet: *The service collection cannot be modified because it is read-only.*	de registratie staat na `builder.Build()`	de regel erboven zetten
een endpoint geeft 500, met *The method or operation is not implemented.*	de methode in de repository is nog de lege versie van **Ctrl+.**	de methode invullen
CS0535: *'InMemoryReceptRepository' does not implement interface member 'IReceptRepository.Delete(int)'*	de klasse en de interface lopen uit elkaar	de methode toevoegen aan de klasse, of weghalen uit de interface

Wat je ziet	Waarom	Wat je doet
CS0246: *The type or namespace name 'IReceptRepository' could not be found*	de `using` voor de map `Repositories` ontbreekt	**Ctrl+.** op de naam, en de `using` laten toevoegen
CS0103: *The name '_recepten' does not exist in the current context*, in de controller	na stap 7 werkt een actie nog met de lijst	die actie verhuizen, zoals in stap 6
CS0103: *The name 'NotFound' does not exist in the current context*, in de repository	de repository probeert een antwoord voor de client te geven. `NotFound` bestaat enkel in een controller	`null` teruggeven, en de controller de 404 laten kiezen
waarschuwing **CS8600**: *Converting null literal or possible null value to non-nullable type.*	`Recept recept = _repository.GetById(id);` zonder vraagteken, terwijl `GetById` `null` kan geven	`Recept?` schrijven, zoals de interface belooft
tijdens stap 6 ontbreekt een recept dat je net toevoegde	het endpoint dat je test, gebruikt nog de lijst van de controller	ook dat endpoint verhuizen
een toegevoegd recept is bij het volgende verzoek weg	de lijst in de repository is niet `static`	`static` toevoegen, zie stap 4
500 met *The request matched multiple endpoints*	twee acties met hetzelfde verb en dezelfde route	elke combinatie van verb en route moet uniek zijn
geen logmeldingen in de terminal	`LogDebug` en `LogTrace` zie je standaard niet	zie hoofdstuk 28

Let op bij de lescode

LET OP

- De `InMemoryRepository` in de les heeft een lijst zonder `static`, terwijl les 02 er wel een had. Samen met `AddScoped` verdwijnt dan alles wat je toevoegt.
- De les vermeldt niet dat `Program.cs` een `using` nodig heeft voor de map `Repositories`. Visual Studio stelt hem voor met **Ctrl+.**.
- In de tussenversies van de les staat het veld zonder `readonly`, in de laatste versie met. Schrijf `readonly` meteen: dan kan het veld enkel in de constructor een waarde krijgen.
- Het commentaar bij de route zegt `api/laptops`. `[controller]` wordt de naam van de klasse zonder *Controller*, dus `api/Product`.
- In de eerste versie van de controller staat `Laptop laptop = _laptopRepository.GetById(id);` zonder vraagteken. Dat geeft waarschuwing CS8600. Schrijf `Laptop?`, zoals de interface belooft.
- `GetLaptopsByMerk` belooft een `List<Laptop>?`, en de controller kijkt na of het resultaat `null` is. `ToList` geeft nooit `null`, hooguit een lege lijst. De controle op `Count == 0` is degene die telt.
- De DELETE in de les meldt een onbekend Id met `LogError`, de andere endpoints doen dat met `LogWarning`. Een onbekend Id is geen fout in je API, maar een vraag naar iets wat er niet is: `LogWarning` past beter. Zie hoofdstuk 28.
- `Max` op een lege lijst geeft een `InvalidOperationException`. Verwijder je alle items en voeg je er daarna een toe, dan crasht `Create`. Zie hoofdstuk 11.
- De inleiding spreekt over een repository die naar een .txt- of .json-bestand schrijft. De les werkt dat niet uit. Wil je het toch proberen: hoofdstuk 30.

Waar het uitgelegd staat

Onderwerp	Hoofdstuk
`FirstOrDefault`, `Where` en `Max`, ook op een lege lijst	11
`IEnumerable` en `ToList`	12
de constructor en velden	13
een object aanpassen tegenover een variabele laten wijzen	16
`null` en het vraagteken achter `Recept?`	17
de interface als contract	19
`private readonly` en `static`	24
`Ok`, `NotFound`, `CreatedAtAction` en `NoContent`	26
registreren, dependency injection en `AddScoped`	27
logging en de niveaus	28

Voorspel de uitkomst

Vraag 1

`Program.cs` registreert `AddScoped<IReceptRepository, InMemoryReceptRepository>()`. De controller vraagt in zijn constructor een `InMemoryReceptRepository`, niet een `IReceptRepository`. Wat geeft een GET?

Toon het antwoord ›

Vraag 2

De constructor van de controller vraagt de repository, maar de regel `_repository = repository;` ontbreekt. Wat gebeurt er bij het bouwen, en wat geeft een GET?

Toon het antwoord ›

Vraag 3

De lijst in `InMemoryReceptRepository` is niet `static`, en de repository is geregistreerd met `AddScoped`. Je verwijdert recept 1 met een DELETE, en vraagt daarna recept 1 op. Wat krijg je?

Toon het antwoord ›

Vraag 4

De PUT in de controller roept meteen `_repository.Update(id, gewijzigd);` op en geeft `NoContent()` terug. De controle met `GetById` is weg. Een client stuurt een PUT naar `api/recept/99`, en recept 99 bestaat niet. Wat krijgt hij, en wat is er veranderd?

Toon het antwoord ›

Vraag 5

```
[HttpGet("{id}")]
public ActionResult<Recept> GetById(int id)
{
    return Ok(_repository.GetById(id));
}
```

Recept 99 bestaat niet. Wat krijgt de client bij `GET api/recept/99`?

Toon het antwoord ›

Vraag 6

Je zit midden in stap 6, en je verhuist POST vóór GET op Id. POST gebruikt dus al de repository, GET op Id nog de lijst van de controller. Je voegt met een POST een recept toe, en krijgt 201 met Id 4. Daarna vraag je `GET api/recept/4`. Wat krijg je?

Toon het antwoord ›

Vraag 7

In de repository staat deze `Update`:

```
public void Update(int id, Recept recept)
{
    Recept? bestaand = _recepten.FirstOrDefault(r => r.Id == id);

    if (bestaand != null)
    {
        bestaand = recept;
    }
}
```

Je stuurt een PUT naar `api/recept/2` met een nieuwe kostprijs, en daarna een GET op `api/recept/2`. Wat geven de twee verzoeken?

Toon het antwoord ›

Vraag 8

Na stap 8 vraagt de constructor een `IReceptRepository repository`, en `Program.cs` registreert de repository. Maar in de constructor staat nog `_repository = new InMemoryReceptRepository();`, en niet `_repository = repository;`. Werkt de API? En wat gebeurt er in les 05, als je in `Program.cs` wisselt naar `ReceptRepository`?

Toon het antwoord ›

Antwoorden

VRAAG 1

500 Internal Server Error, met *Unable to resolve service for type 'ReceptenApi.Repositories.InMemoryReceptRepository'*.

De registratie zegt: wie een `IReceptRepository` vraagt, krijgt een `InMemoryReceptRepository`. Ze zegt niets over wie de klasse zelf vraagt. Daarom vraagt een controller altijd de interface.

‹ Terug naar de vraag

VRAAG 2

Het bouwt, met waarschuwing **CS8618**: het veld moet een waarde hebben voor de constructor afloopt. Een GET geeft daarna **500**, met een `NullReferenceException`: het veld is `null`, want je hebt er nooit iets in gezet.

‹ Terug naar de vraag

VRAAG 3

200 OK met recept 1. De DELETE heeft het recept uit de lijst van zijn eigen repository gehaald. Het volgende verzoek krijgt een nieuwe repository, met een nieuwe lijst waarin recept 1 gewoon weer staat.

‹ Terug naar de vraag

VRAAG 4

204 No Content, en er is niets veranderd.

`Update` in de repository vindt recept 99 niet en doet dus niets. Ze zegt dat niet, want ze geeft niets terug. De controller weet niet dat er niets gebeurde, en meldt dat alles gelukt is. Daarom kijkt de controller eerst zelf met `GetById`.

‹ Terug naar de vraag

VRAAG 5

204 No Content, zonder inhoud.

`GetById` geeft `null`, en van `Ok(null)` maakt ASP.NET Core vanzelf een 204. Geen foutmelding en geen 404: de client denkt dat het gelukt is, maar krijgt niets. Met `null` zegt de repository dat er niets is. Dat er dan een 404 volgt, beslist de controller.

‹ Terug naar de vraag

VRAAG 6

404 Not Found, met *We vonden geen recept met Id 4.*

De POST zette recept 4 in de lijst van de repository. GET op Id zoekt nog in de lijst van de controller, en daar staan enkel de drie startrecepten. Daarom verhuis je eerst de acties die lezen, en test je telkens met acties die al verhuisd zijn.

‹ Terug naar de vraag

VRAAG 7

De PUT geeft **204 No Content**. De GET geeft **200 OK met het oude recept 2**: de kostprijs is niet veranderd.

`bestaand = recept;` laat enkel de variabele `bestaand` naar een ander object wijzen, het recept uit de body. De lijst bevat nog altijd het oude object, en dat is niet aangepast. Neem de eigenschappen één voor één over, zoals in stap 6d.

‹ Terug naar de vraag

VRAAG 8

Ja, de API werkt, en elk antwoord klopt. Maar de repository die ASP.NET Core meegeeft, wordt niet gebruikt: de controller maakt zijn eigen.

In les 05 wissel je de regel in `Program.cs`, en deze controller merkt er niets van. Hij blijft met de lijst in het geheugen werken, zonder foutmelding. Je denkt dat je API de database gebruikt, maar na een herstart is alles weg. Gebruik dus altijd de parameter, en schrijf nergens `new` voor een repository.

‹ Terug naar de vraag

37. Les 04: Docker, EF Core en de eerste migratie

DATA

Wat je bouwt

Een PostgreSQL-database in Docker, met pgAdmin om erin te kijken. Daarnaast een context die je klassen omzet naar tabellen, en een eerste migratie die de tabel maakt.

Je API gebruikt de database na deze les nog niet. De repository uit les 03 werkt verder met haar lijst. De overstap komt in les 05.

Hoe je dit hoofdstuk leest

In les 04 verandert er niets aan je controller, je interface of je repository. Er wordt dus niets opgesplitst zoals in les 03. Er komt wel een **ketting** van nieuwe stukken bij, die elkaar nodig hebben. Ontbreekt één schakel, of zegt ze iets anders dan de volgende, dan werkt het geheel niet.

De les toont die stukken na elkaar, maar zegt weinig over hoe ze aan elkaar hangen. Dit hoofdstuk toont bij elk nieuw stuk:

- wat erin staat;
- waarom het er moet zijn;
- welk ander stuk het gebruikt, en wat daar letterlijk hetzelfde moet zijn.

In de code betekent **groen en vet** weer: nieuw of gewijzigd in deze stap.

De ketting, in één tabel

Stuk	Waar	Wat het weet	Wie het gebruikt
`docker-compose.yml`	de map van je project	hoe de container met de database eruitziet: de gebruiker, het wachtwoord, de naam van de database, de poort	Docker, bij `docker compose up -d`
`appsettings.json`	de map van je project	waar de database staat, en hoe je binnenkomt: de connection string	`Program.cs`, via `GetConnectionString`
`Program.cs`	de map van je project	dat er een context is, en met welke database ze praat	ASP.NET Core, en de commando's voor migraties
`ReceptContext.cs`	de map `Data`	welke klassen een tabel worden, en hoe elke kolom eruitziet	EF Core, bij elke migratie. Vanaf les 05 ook je repository
de migraties	de map `Migrations`	wat er stap voor stap aan de database veranderd is	`Update-Database`

Lees de tabel van boven naar onder, en je hebt de volgorde waarin je werkt. Elk stuk steunt op het stuk erboven.

Wat er bijkomt in je project

```
ReceptenApi/
├── Data/                                   ← nieuw
│   └── ReceptContext.cs                    ← nieuw
├── Migrations/                             ← nieuw, gemaakt door EF Core
│   ├── 20261104093000_EersteVersie.cs
│   ├── 20261104093000_EersteVersie.Designer.cs
│   └── ReceptContextModelSnapshot.cs
├── appsettings.json                        ← gewijzigd
├── docker-compose.yml                      ← nieuw
├── Program.cs                              ← gewijzigd
└── ReceptenApi.csproj                      ← gewijzigd, door de packages
```

Wat níet verandert: `ReceptController.cs`, `IReceptRepository.cs` en `InMemoryReceptRepository.cs`. Na deze les antwoordt je API nog precies zoals na les 03.

Stap 1: Docker Desktop

Wat je doet. Installeer **Docker Desktop** en start het. Vraagt het een update van WSL 2, volg die dan. Linksonder moet *Engine running* groen staan.

Waarom. Docker start programma's in een **container**: een afgesloten doos met het programma en alles wat het nodig heeft. Zo hoef je PostgreSQL niet zelf te installeren. Stop je de container, dan is je computer weer zoals ervoor. En de database draait bij iedereen op dezelfde manier.

Zo weet je dat het lukte. *Engine running* staat groen.

Stap 2: docker-compose.yml

Wat je doet. Maak in de map van je project, naast het `.csproj`-bestand, een bestand met exact de naam `docker-compose.yml`. Zet er dit in:

```yaml
services:
  postgres-db:
    image: postgres:16-alpine
    container_name: recepten_postgres
    restart: always
    environment:
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: KiesEenWachtwoord
      POSTGRES_DB: ReceptenDb
    ports:
      - "5432:5432"
    volumes:
      - postgres_data:/var/lib/postgresql/data

  pgadmin-tool:
    image: dpage/pgadmin4
    container_name: recepten_pgadmin
    restart: always
    environment:
```

```
      PGADMIN_DEFAULT_EMAIL: student@opleiding.be
      PGADMIN_DEFAULT_PASSWORD: Welkom01!
    ports:
      - "8080:80"
    depends_on:
      - postgres-db

volumes:
  postgres_data:
```

Waarom. Dit bestand is het recept voor twee containers: de database en pgAdmin, het programma waarmee je in de database kijkt. Docker leest het bij elke start.

Regel	Wat ze doet
`image`	welk programma in de container draait, en welke versie
`environment`	de gebruiker, het wachtwoord en de naam van de database die bij de eerste start gemaakt worden
`ports`	links de poort op je computer, rechts die in de container: `"8080:80"` maakt pgAdmin bereikbaar op `http://localhost:8080`
`volumes`	waar de gegevens bewaard worden, buiten de container: ze blijven bestaan na `docker compose down`
`depends_on`	pgAdmin start pas na de database

Onthoud drie waarden: de gebruiker `postgres`, het wachtwoord, en de naam `ReceptenDb`. Je hebt ze nodig in stap 5.

Stap 3: de containers starten

Wat je doet. Open een terminal in de map met `docker-compose.yml`, en voer uit:

```
docker compose up -d
```

Waarom `-d`. Zonder `-d` blijft de terminal vastzitten aan de containers. Met `-d` draaien ze op de achtergrond, en kan je verder werken.

Zo weet je dat het lukte. In Docker Desktop verschijnt een groep met de naam van je map, met twee groene containers. De eerste keer duurt het even: Docker downloadt dan de programma's.

Stap 4: de packages

Wat je doet. Installeer vier packages, via **Tools › NuGet Package Manager › Manage NuGet Packages for Solution**, of in de terminal:

```
dotnet add package Microsoft.EntityFrameworkCore
dotnet add package Microsoft.EntityFrameworkCore.Design
dotnet add package Microsoft.EntityFrameworkCore.Tools
dotnet add package Npgsql.EntityFrameworkCore.PostgreSQL
```

Waarom vier. Elk package doet één ding:

Package	Waarvoor
`Microsoft.EntityFrameworkCore`	EF Core zelf: de context, `DbSet`, `SaveChanges`
`Npgsql.EntityFrameworkCore.PostgreSQL`	de vertaling naar PostgreSQL. Voor een andere database neem je een ander package, de rest blijft
`Microsoft.EntityFrameworkCore.Design`	nodig om migraties te maken
`Microsoft.EntityFrameworkCore.Tools`	de commando's in de Package Manager Console, zoals `Add-Migration`

Wat er in je project verandert. In `ReceptenApi.csproj` komen binnen een `<ItemGroup>` vier regels bij:

```
<PackageReference Include="Microsoft.EntityFrameworkCore" Version="10.0.0" />
<PackageReference Include="Microsoft.EntityFrameworkCore.Design" Version="10.0.0" />
<PackageReference Include="Microsoft.EntityFrameworkCore.Tools" Version="10.0.0" />
<PackageReference Include="Npgsql.EntityFrameworkCore.PostgreSQL" Version="10.0.0" />
```

Je versienummers kunnen hoger zijn. Wat telt: alle vier hebben dezelfde hoofdversie als je .NET, 10 bij .NET 10. Bij `Design` en `Tools` zet Visual Studio er nog een paar regels onder. Die laat je staan.

Stap 5: de connection string in appsettings.json

Wat je doet. Zet in `appsettings.json` waar je database staat.

Vóór:

```
{
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  },
  "AllowedHosts": "*"
}
```

Ná:

```
{
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  },
  "AllowedHosts": "*",
  "ConnectionStrings": {
    "Standaard":
"Host=localhost;Port=5432;Database=ReceptenDb;Username=postgres;Password=KiesEenWachtwoord"
  }
}
```

Vergeet de komma na `"*"` niet. Zonder komma is het geen geldige JSON meer, en start je API niet.

Waarom in appsettings.json, en niet in je code. Het adres van een database is een instelling, geen logica. Verhuist de database, dan pas je één regel tekst aan, en verandert er niets aan je code.

Wat letterlijk hetzelfde moet zijn. De connection string bestaat uit vijf stukjes. Elk stukje moet kloppen met een regel in `docker-compose.yml`:

In de connection string	In docker-compose.yml	Waarom
`Host=localhost`	geen regel	je API draait op je eigen computer, en de container is daar bereikbaar
`Port=5432`	het linkse getal in `"5432:5432"`	de poort op je computer, waar Docker de database doorgeeft
`Database=ReceptenDb`	`POSTGRES_DB: ReceptenDb`	de database die bij de eerste start gemaakt werd
`Username=postgres`	`POSTGRES_USER: postgres`	de gebruiker die bij de eerste start gemaakt werd
`Password=KiesEenWachtwoord`	`POSTGRES_PASSWORD: KiesEenWachtwoord`	het wachtwoord van die gebruiker

De naam `Standaard`**.** Die kies je zelf. Je gebruikt hem in stap 7 letterlijk opnieuw. De les noemt hem `PostgresConnection`.

Stap 6: de context

Wat je doet. Maak een map `Data`, en daarin een klasse `ReceptContext`. Je bouwt ze in drie delen op.

6a. De klasse en haar constructor

```
using Microsoft.EntityFrameworkCore;
using ReceptenApi.Models;

namespace ReceptenApi.Data;

public class ReceptContext : DbContext
{
    public ReceptContext(DbContextOptions<ReceptContext> options)
        : base(options) { }
}
```

- `: DbContext`**.** Je context erft van de klasse `DbContext` van EF Core. Daardoor kan ze alles wat EF Core kan: tabellen lezen, wijzigingen onthouden, en die bewaren met `SaveChanges`. Zie hoofdstuk 20.
- **De constructor vraagt** `options`**.** Daarin zit met welke database de context praat, en waar die staat. De context bepaalt dat niet zelf. `Program.cs` geeft het mee, net zoals `Program.cs` in les 03 besliste welke repository de controller kreeg.
- `: base(options)` geeft die opties door aan `DbContext`. De constructor zelf doet verder niets, daarom staan er lege accolades.

6b. Een DbSet per tabel

```
public class ReceptContext : DbContext
{
    public ReceptContext(DbContextOptions<ReceptContext> options)
        : base(options) { }

    public DbSet<Recept> Recepten { get; set; }
}
```

- **Waarom.** Een `DbSet` zegt tegen EF Core: van deze klasse komt er een tabel. Een model zonder `DbSet` krijgt geen tabel.
- **De naam** `Recepten` is de naam die je repository vanaf les 05 gebruikt: `_context.Recepten`. Zonder andere afspraak wordt het ook de naam van de tabel.

6c. Hoe de tabel eruitziet

```
public class ReceptContext : DbContext
{
    public ReceptContext(DbContextOptions<ReceptContext> options)
        : base(options) { }

    public DbSet<Recept> Recepten { get; set; }

    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        base.OnModelCreating(modelBuilder);

        modelBuilder.Entity<Recept>(entity =>
        {
            entity.ToTable("Recepten");
            entity.Property(r => r.Titel).IsRequired().HasMaxLength(100);
            entity.Property(r => r.Kostprijs).HasPrecision(8, 2);
        });
    }
}
```

Waarom. EF Core heeft vaste afspraken: een `string` wordt een tekstkolom zonder maximum, een `decimal` een getal zonder vaste lengte. In `OnModelCreating` leg je je eigen regels vast. Dat heet de **Fluent API**: je schrijft de regels als een ketting van methodes achter elkaar.

- `override`: `DbContext` heeft deze methode al. Jij schrijft je eigen versie. Typ `override` en een spatie, dan toont Visual Studio de methodes die je kan overschrijven.
- `base.OnModelCreating(modelBuilder);` laat eerst `DbContext` zijn eigen werk doen. Zet het altijd bovenaan.
- `modelBuilder.Entity<Recept>(...)`: alles tussen de haakjes gaat over het model `Recept`.

Zo wordt elke eigenschap van `Recept` een kolom:

In `Recept`	Kolom	Type in PostgreSQL	Waarom
`int Id`	`Id`	`integer`, primaire sleutel, telt zelf op	EF Core herkent een eigenschap met de naam `Id` als sleutel. De database deelt de nummers uit
`string Titel`	`Titel`	`character varying(100)`, verplicht	`HasMaxLength(100)` en `IsRequired()`. Zonder `HasMaxLength` wordt het `text`, zonder maximum
`decimal Kostprijs`	`Kostprijs`	`numeric(8,2)`, verplicht	`HasPrecision(8, 2)`: acht cijfers, waarvan twee na de komma. Een `decimal` kan nooit `null` zijn, dus de kolom is verplicht

Stap 7: de context registreren in Program.cs

Wat je doet. Vertel in `Program.cs` dat er een context is, en met welke database ze praat.

Vóór, zoals na les 03:

```
using ReceptenApi.Repositories;
using Scalar.AspNetCore;

var builder = WebApplication.CreateBuilder(args);

builder.Services.AddControllers();
builder.Services.AddOpenApi();
builder.Services.AddScoped<IReceptRepository, InMemoryReceptRepository>();

var app = builder.Build();
```

Ná:

```
using Microsoft.EntityFrameworkCore;
using ReceptenApi.Data;
using ReceptenApi.Repositories;
using Scalar.AspNetCore;

var builder = WebApplication.CreateBuilder(args);

builder.Services.AddControllers();
builder.Services.AddOpenApi();
builder.Services.AddScoped<IReceptRepository, InMemoryReceptRepository>();

var connectionString = builder.Configuration.GetConnectionString("Standaard");
builder.Services.AddDbContext<ReceptContext>(options =>
    options.UseNpgsql(connectionString));

var app = builder.Build();
```

Regel per regel:

Regel	Wat ze doet	Waarom
`GetConnectionString("Standaard")`	leest de connection string uit `appsettings.json`, onder `ConnectionStrings`	de naam moet letterlijk die uit stap 5 zijn. Anders krijg je `null`
`AddDbContext<ReceptContext>(...)`	registreert je context, zoals `AddScoped` in les 03 je repository registreerde	wie een `ReceptContext` vraagt, krijgt er een. Elk verzoek krijgt zijn eigen context
`options.UseNpgsql(connectionString)`	vult de `options` die je constructor in stap 6a vraagt	zo weet de context dat ze met PostgreSQL praat, en waar die staat
de twee `using`-regels	`Microsoft.EntityFrameworkCore` voor `UseNpgsql`, `ReceptenApi.Data` voor je context	zonder krijg je CS0246 of CS1061

Waarom de repository-regel niet verandert. Je API blijft de lijst gebruiken. Zo kan je de database bouwen en testen, terwijl je API gewoon werkt. In les 05 verander je die ene regel.

Waarom vóór `builder.Build()`**.** Net als in les 03: na `Build` ligt de lijst met diensten vast.

Stap 8: de eerste migratie maken

Wat je doet. Open de **Package Manager Console** (**Tools › NuGet Package Manager › Package Manager Console**), en typ:

```
Add-Migration EersteVersie
```

Wat er dan gebeurt. `Add-Migration` praat nog niet met de database. Het doet dit:

1. Het bouwt je project.
2. Het leest je context: welke `DbSet`'s er zijn, en wat er in `OnModelCreating` staat. Samen heet dat het **model**.
3. Het vergelijkt dat model met de vorige foto van je model, de **snapshot**. De eerste keer is er nog geen.
4. Het schrijft het verschil op als een migratie, en maakt een nieuwe snapshot.

Er komen drie bestanden in de map `Migrations`:

Bestand	Wat erin staat
`20261104093000_EersteVersie.cs`	de migratie zelf: `Up` en `Down`. Het getal is de datum en het uur, zodat de migraties op volgorde staan
`20261104093000_EersteVersie.Designer.cs`	hoe je model eruitzag op het moment van deze migratie. Dit lees je niet
`ReceptContextModelSnapshot.cs`	de foto van je volledige model nu. De volgende `Add-Migration` vergelijkt hiermee

Lees je migratie altijd na. Zo ziet ze eruit. De lange regels zijn hier afgebroken, zodat ze op de pagina passen:

```
public partial class EersteVersie : Migration
{
    protected override void Up(MigrationBuilder migrationBuilder)
    {
        migrationBuilder.CreateTable(
            name: "Recepten",
            columns: table => new
            {
                Id = table.Column<int>(type: "integer", nullable: false)
                    .Annotation("Npgsql:ValueGenerationStrategy",
                        NpgsqlValueGenerationStrategy.IdentityByDefaultColumn),
                Titel = table.Column<string>(type: "character varying(100)",
                    maxLength: 100, nullable: false),
                Kostprijs = table.Column<decimal>(type: "numeric(8,2)",
                    precision: 8, scale: 2, nullable: false)
            },
            constraints: table =>
            {
                table.PrimaryKey("PK_Recepten", x => x.Id);
```

```
            });
    }

    protected override void Down(MigrationBuilder migrationBuilder)
    {
        migrationBuilder.DropTable(
            name: "Recepten");
    }
}
```

In de migratie	Waar het vandaan komt
`CreateTable`, met `name: "Recepten"`	`ToTable("Recepten")` in je context
`Id`, `integer`, met `IdentityByDefaultColumn`	de eigenschap `Id`: de database telt zelf op
`Titel`, `character varying(100)`, `nullable: false`	`HasMaxLength(100)` en `IsRequired()`
`Kostprijs`, `numeric(8,2)`	`HasPrecision(8, 2)`
`PrimaryKey("PK_Recepten", x => x.Id)`	`Id` is de sleutel
`Down` met `DropTable`	het omgekeerde van `Up`: zo kan je de migratie ongedaan maken

Klopt er iets niet, en heb je de migratie nog niet uitgevoerd? Dan haal je ze weg met `Remove-Migration`, pas je je context aan, en maak je ze opnieuw.

Stap 9: de database bijwerken

Wat je doet. Docker moet draaien. Typ in de Package Manager Console:

```
Update-Database
```

Wat er dan gebeurt, in deze volgorde:

1. EF Core bouwt je project, en leest via `Program.cs` je context en je connection string.
2. Het maakt verbinding met PostgreSQL, op `localhost:5432`. Draait Docker niet, dan stopt het hier.
3. Bestaat de database nog niet, dan maakt het ze.
4. Het kijkt in de tabel `__EFMigrationsHistory` welke migraties al uitgevoerd zijn. De eerste keer maakt het die tabel.
5. Het voert `Up` uit van elke migratie die nog niet in die tabel staat, op volgorde.
6. Het schrijft elke uitgevoerde migratie in `__EFMigrationsHistory`.

Waarom die tabel. Zo weet EF Core de volgende keer waar het gebleven was. `Update-Database` voert een migratie nooit twee keer uit.

Liever een gewone terminal? Installeer dan één keer het hulpprogramma `dotnet-ef`, en gebruik deze commando's in de map van je project:

```
dotnet tool install --global dotnet-ef
dotnet ef migrations add EersteVersie
dotnet ef database update
```

Stap 10: kijken in pgAdmin

1. Surf naar `http://localhost:8080` en meld je aan met het e-mailadres en het wachtwoord van pgAdmin uit `docker-compose.yml`.
2. Rechtsklik op **Servers › Register › Server…**. Geef een naam op het tabblad **General**.
3. Op het tabblad **Connection**: Host `postgres-db`, Port `5432`, Username `postgres`, en het wachtwoord van de database. Bewaar.
4. Open **Databases › ReceptenDb › Schemas › public › Tables**.

> **BELANGRIJK**
>
> In pgAdmin is de Host `postgres-db`, in `appsettings.json` is het `localhost`. pgAdmin draait zelf in Docker, en bereikt de database via de naam van de dienst in `docker-compose.yml`. Je API draait op je computer, en bereikt de database via poort 5432 op je eigen computer.

Zo weet je dat het lukte. Je ziet twee tabellen: `Recepten`, met de drie kolommen uit de migratie, en `__EFMigrationsHistory`, met één rij: `EersteVersie`.

En je startgegevens?

De tabel `Recepten` is leeg. Je drie startrecepten staan nog altijd in de `static` lijst van `InMemoryReceptRepository`. De lijst en de tabel bestaan na deze les naast elkaar, en weten niets van elkaar.

- De les vult de tabel met een SQL-script, dat je bijvoorbeeld door een AI laat maken, en in pgAdmin uitvoert.
- Vanaf les 06 komen startgegevens mee met je migraties, via `HasData`. Zie hoofdstuk 39.

Program.cs na deze les

```
using Microsoft.EntityFrameworkCore;
using ReceptenApi.Data;
using ReceptenApi.Repositories;
using Scalar.AspNetCore;

var builder = WebApplication.CreateBuilder(args);

builder.Services.AddControllers();
builder.Services.AddOpenApi();
builder.Services.AddScoped<IReceptRepository, InMemoryReceptRepository>();
```

```
var connectionString = builder.Configuration.GetConnectionString("Standaard");
builder.Services.AddDbContext<ReceptContext>(options =>
    options.UseNpgsql(connectionString));

var app = builder.Build();

if (app.Environment.IsDevelopment())
{
    app.MapOpenApi();
    app.MapScalarApiReference();
}

app.UseHttpsRedirection();
app.UseAuthorization();
app.MapControllers();

app.Run();
```

Hoe je ziet dat het werkt

- In Docker Desktop draaien twee containers, allebei groen.
- Na `Add-Migration` staan er drie bestanden in `Migrations`. In `Up` staat `CreateTable` met de naam van je tabel, en een kolom per eigenschap.
- Na `Update-Database` staat in pgAdmin je tabel, en een tabel `__EFMigrationsHistory` met één rij: je migratie.
- Je API werkt nog zoals na les 03, met de lijst.

Alle stappen op een rij

1. Installeer en start Docker Desktop.
2. Maak `docker-compose.yml` naast je `.csproj`.
3. Start de containers met `docker compose up -d`.
4. Installeer de vier packages.
5. Zet de connection string in `appsettings.json`. Elk stukje klopt met `docker-compose.yml`.
6. Maak de context in de map `Data`: de klasse, een `DbSet` per tabel, en je regels in `OnModelCreating`.
7. Registreer de context in `Program.cs` met `AddDbContext`, vóór `builder.Build()`.
8. Maak de eerste migratie met `Add-Migration`, en lees ze na.
9. Voer ze uit met `Update-Database`. Docker moet draaien.
10. Kijk in pgAdmin of de tabel er staat.

Als een stap ontbreekt

Wat je ziet	Waarom	Wat je doet
`Update-Database`: *Failed to connect to 127.0.0.1:5432*	Docker draait niet, of de container is gestopt	Docker Desktop starten, `docker compose up -d`

Wat je ziet	Waarom	Wat je doet
password authentication failed for user "postgres"	het wachtwoord in `appsettings.json` verschilt van dat in `docker-compose.yml`	gelijk zetten. Het wachtwoord uit `docker-compose.yml` geldt enkel bij de allereerste start. Veranderde je het later, dan wist `docker compose down -v` de volume, met alle gegevens erin
Bind for 0.0.0.0:5432 failed: port is already allocated	een andere PostgreSQL gebruikt poort 5432 al, bijvoorbeeld de container van de les	die andere container stoppen, of links een andere poort nemen, zoals `"5433:5432"`, en `Port=5433` in de connection string
The term 'Add-Migration' is not recognized	het package `Tools` ontbreekt, of je typt in een gewone terminal	het package installeren, en de Package Manager Console gebruiken
Could not execute because the specified command or file was not found. bij `dotnet ef`	het hulpprogramma `dotnet-ef` is niet geïnstalleerd	`dotnet tool install --global dotnet-ef`
Your startup project 'ReceptenApi' doesn't reference Microsoft.EntityFrameworkCore.Design.	het package `Design` ontbreekt	het package installeren
Unable to create a 'DbContext' of type 'ReceptContext'	de registratie met `AddDbContext` ontbreekt, of de context heeft geen constructor met `options`	registreren in `Program.cs`, de constructor toevoegen
een fout over een lege of ontbrekende connection string	de naam in `GetConnectionString` verschilt van die in `appsettings.json`	letterlijk dezelfde naam gebruiken
de API start niet, met een fout over `appsettings.json`	een komma vergeten in de JSON	de komma na `"AllowedHosts": "*"` toevoegen
`Update-Database`: *The model for context 'ReceptContext' has pending changes. Add a new migration before updating the database.*	je veranderde je model of je context, maar maakte geen nieuwe migratie	eerst `Add-Migration`, dan `Update-Database`
geen tabel na `Update-Database`	de context heeft geen `DbSet` voor het model	de `DbSet` toevoegen, een nieuwe migratie maken

Let op bij de lescode

LET OP

- De commando's voor de terminal staan in de les als `dotnet Entity Framework migrations add` en `dotnet Entity Framework database update`. Dat moet `dotnet ef migrations add` en `dotnet ef database update` zijn, en daarvoor heb je het hulpprogramma `dotnet-ef` nodig. De les vermeldt het niet.
- De les schrijft `string connectionString = builder.Configuration.GetConnectionString(...)`. `GetConnectionString` kan `null` geven, dus dat geeft waarschuwing CS8600. Met `var`, zoals in dit boek, krijgt de variabele het type `string?`, en verdwijnt die waarschuwing.
- De namespace wisselt in de les tussen `WebApiDemo.Data` en `WebAPIDemo.Data`. C# let op hoofdletters. Kies één schrijfwijze en gebruik ze overal, anders krijg je CS0246.
- `Prijs` is in de les een `double`. `HasPrecision(18, 2)` hoort bij een `decimal`. Kijk in je migratie welk kolomtype er komt. Zie hoofdstuk 2 voor het verschil.
- De samenvatting zegt dat pgAdmin op `localhost:5432` draait. Dat is de database. pgAdmin draait op `localhost:8080`.
- De les zegt in stap 5 dat Docker later in het hoofdstuk aan bod komt, maar Docker staat ervoor.

OPMERKING

De les geeft vier vaste regels: Docker draait voor je iets met de database doet, een migratie krijgt een naam die zegt wat er verandert, je leest elke migratie na, en je past de database nooit met de hand aan.

Voor het examen zegt de les: de packages, de connection string en `Program.cs` zijn gegeven. Jij moet een database op Docker kunnen maken, migraties maken en uitvoeren, en gegevens lezen en wegschrijven.

Waar het uitgelegd staat

Onderwerp	Hoofdstuk
de context, `DbSet`, de Fluent API en migraties	31
`GetConnectionString` en registreren	27
`decimal` tegenover `double`	2
`override` en `base`	20

Voorspel de uitkomst

Vraag 1

Je voegt na les 04 een eigenschap `Bereidingstijd` toe aan `Recept`, zonder nieuwe migratie, en test je API in Scalar. Wat merk je?

Toon het antwoord ›

Vraag 2

Je voert `docker compose down` uit en daarna weer `docker compose up -d`. Staat je tabel er nog? En na `docker compose down -v`?

Toon het antwoord ›

Vraag 3

In pgAdmin vul je bij Host `localhost` in in plaats van `postgres-db`. Wat gebeurt er?

Toon het antwoord ›

Vraag 4

Docker draait niet. Je typt `Add-Migration EersteVersie`, en daarna `Update-Database`. Wat lukt er, en wat niet?

Toon het antwoord ›

Vraag 5

Na je eerste migratie verander je in je context `HasMaxLength(100)` in `HasMaxLength(150)`. Je typt meteen `Update-Database`, zonder `Add-Migration`. Wat gebeurt er?

Toon het antwoord ›

Antwoorden

VRAAG 1

Niets. Je API gebruikt in les 04 de database nog niet: de repository uit les 03 werkt met haar lijst.

Pas in les 05, als de API de database gebruikt, geeft een GET een 500 over een kolom die niet bestaat.

‹ Terug naar de vraag

VRAAG 2

Na `down`: ja. De gegevens staan in de volume, buiten de container.

Na `down -v`: nee. `-v` wist ook de volume. De database is dan leeg, en `Update-Database` bouwt ze opnieuw op.

‹ Terug naar de vraag

VRAAG 3

pgAdmin kan geen verbinding maken. Binnen de container van pgAdmin is `localhost` die container zelf, en daar draait geen database. pgAdmin vindt de database via de naam van de dienst: `postgres-db`.

‹ Terug naar de vraag

VRAAG 4

Add-Migration lukt, Update-Database niet. `Add-Migration` vergelijkt je model met de snapshot. Daarvoor heeft het de database niet nodig. `Update-Database` moet de migratie uitvoeren in de database, en stopt met *Failed to connect to 127.0.0.1:5432*.

‹ Terug naar de vraag

VRAAG 5

Update-Database weigert, met *The model for context 'ReceptContext' has pending changes. Add a new migration before updating the database.*

Je model zegt 150, maar geen enkele migratie zegt dat al. Sinds EF Core 9 stopt `Update-Database` dan, zodat je model en je database niet ongemerkt uit elkaar lopen. Eerst `Add-Migration`, dan `Update-Database`.

‹ Terug naar de vraag

38. Les 05: de repository op de database, en async

DATA

Wat je bouwt

Een tweede repository die hetzelfde contract volgt, maar met de context werkt. Met één regel in `Program.cs` gebruikt je API dan de database, en blijven je gegevens bewaard na een herstart.

Daarna maak je de hele keten asynchroon: de repository, de interface en de controller.

Hoe je dit hoofdstuk leest

Les 05 doet twee verbouwingen na elkaar. Elk heeft in dit hoofdstuk een eigen deel:

- **Deel A: van de lijst naar de database.** Je repository werkt niet meer met een lijst, maar met de context uit les 04. De controller en de interface veranderen niet.
- **Deel B: van synchroon naar asynchroon.** Elke methode die op de database wacht, krijgt `async` en `await`. Dat werkt als een domino: van de repository, via de interface, tot in de controller.

De les doet het in die volgorde, en dit hoofdstuk ook. Zo zie je elke verandering apart.

De kleuren betekenen hetzelfde als in hoofdstuk 36:

- **oranje en vet**: dit wordt vervangen;
- **groen en vet**: dit is nieuw of gewijzigd in deze stap.

Wat er bijkomt in je project

```
ReceptenApi/
├── Controllers/
│   └── ReceptController.cs            ← gewijzigd: async
├── Repositories/
│   ├── IReceptRepository.cs           ← gewijzigd: async
│   ├── InMemoryReceptRepository.cs    ← weg, aan het einde van de les
│   └── ReceptRepository.cs            ← nieuw
└── Program.cs                         ← gewijzigd: één regel
```

Het vertrekpunt: je repository met de lijst

Zo ziet `InMemoryReceptRepository` eruit na les 03. Alles wat oranje is, werkt met de lijst. Dat moet straks met de database werken.

```
public class InMemoryReceptRepository : IReceptRepository
{
    private static List<Recept> _recepten = new List<Recept>
    {
```

```
        new Recept { Id = 1, Titel = "Lasagne", Kostprijs = 12.50m },
        new Recept { Id = 2, Titel = "Pad thai", Kostprijs = 9.80m },
        new Recept { Id = 3, Titel = "Linzensoep", Kostprijs = 4.20m }
    };

    public List<Recept> GetAll()
    {
        return _recepten;
    }

    public Recept? GetById(int id)
    {
        return _recepten.FirstOrDefault(r => r.Id == id);
    }

    public List<Recept> GetByMaximumKostprijs(decimal bedrag)
    {
        return _recepten
            .Where(r => r.Kostprijs <= bedrag)
            .ToList();
    }

    public Recept Create(Recept recept)
    {
        recept.Id = _recepten.Max(r => r.Id) + 1;
        _recepten.Add(recept);
        return recept;
    }

    public void Update(int id, Recept recept)
    {
        Recept? bestaand = _recepten.FirstOrDefault(r => r.Id == id);

        if (bestaand != null)
        {
            bestaand.Titel = recept.Titel;
            bestaand.Kostprijs = recept.Kostprijs;
        }
    }

    public void Delete(int id)
    {
        Recept? recept = _recepten.FirstOrDefault(r => r.Id == id);

        if (recept != null)
        {
            _recepten.Remove(recept);
        }
    }
}
```

Kijk hoeveel er níet oranje is: de koppen van de methodes, de controles op `null`, het overnemen van de eigenschappen. Die blijven. Enkel de plaatsen waar de lijst gebruikt wordt, veranderen.

Deel A, stap 1: een nieuwe klasse

Wat je doet. Maak in de map `Repositories` een klasse `ReceptRepository`, die `IReceptRepository` belooft. Laat Visual Studio de lege methodes maken met **Ctrl+.** en **Implement interface**, zoals in les 03.

Waarom een nieuwe klasse, en niet de oude aanpassen. Dat is de reden waarom je in les 03 een interface maakte. De oude repository blijft werken tot de nieuwe af is. Daarna wissel je met één regel in `Program.cs`. Loopt er iets mis, dan wissel je even terug.

Waarom de naam `ReceptRepository`. Deze repository wordt de gewone. `InMemory` zei waar de gegevens stonden. Vanaf nu staan ze in de database, en dat is wat je verwacht.

Deel A, stap 2: de context als dependency

Wat je doet. Geef de repository een veld voor de context, en vraag de context in haar constructor.

```
using ReceptenApi.Data;
using ReceptenApi.Models;

namespace ReceptenApi.Repositories;

public class ReceptRepository : IReceptRepository
{
    private readonly ReceptContext _context;

    public ReceptRepository(ReceptContext context)
    {
        _context = context;
    }

    // ... de zes lege methodes van stap 1
}
```

Waarom. De repository heeft de context nodig om met de database te praten. Ze maakt die niet zelf met `new`, ze vraagt ze. Dat is hetzelfde patroon als de controller in les 03, één schakel dieper.

Zo vraagt elk stuk wat het nodig heeft, en levert `Program.cs` het:

Wie vraagt	Wat	Wie levert	Geregistreerd met
`ReceptController`	`IReceptRepository`	`ReceptRepository`, vanaf stap 4	`AddScoped`, in les 03
`ReceptRepository`	`ReceptContext`	de context, met de opties uit `Program.cs`	`AddDbContext`, in les 04
`ReceptController`	`ILogger<ReceptController>`	.NET zelf	niets

Je schrijft nergens `new ReceptContext(...)`. ASP.NET Core maakt bij elk verzoek de hele ketting: eerst de context, dan de repository, dan de controller.

Deel A, stap 3: methode per methode

Wat je doet. Vul de zes methodes in. Kopieer telkens de methode uit `InMemoryReceptRepository`, en vervang wat oranje is.

3a. GetAll

Met de lijst:

```
public List<Recept> GetAll()
{
    return _recepten;
}
```

Met de database:

```
public List<Recept> GetAll()
{
    return _context.Recepten
        .OrderBy(r => r.Titel)
        .ToList();
}
```

- `_context.Recepten` **is geen lijst.** Het is de `DbSet` uit les 04: de tabel. Er wordt nog niets gelezen.
- `ToList()` **stuurt de vraag naar de database**, als SQL, en maakt van het antwoord een lijst. Zonder `ToList` bouwt je project niet: het contract belooft een `List<Recept>`, en een `DbSet` is er geen.
- `OrderBy` **zet de recepten op volgorde.** De les doet dat ook. Een database belooft geen volgorde, tenzij je erom vraagt. De lijst gaf de recepten terug zoals je ze toevoegde.

> **LET OP**
>
> Met `OrderBy` kies jij de volgorde. Kies je een andere volgorde dan je lijst had, dan krijgt de client de recepten in een andere volgorde dan voor de les. Hier is dat alfabetisch op titel. Beslis dus bewust waarop je sorteert.

3b. GetById

Met de lijst:

```
public Recept? GetById(int id)
{
    return _recepten.FirstOrDefault(r => r.Id == id);
}
```

Met de database:

```
public Recept? GetById(int id)
{
    return _context.Recepten.Find(id);
}
```

- `Find` **zoekt op de primaire sleutel**, het `Id`. Vindt het niets, dan geeft het `null`, net als `FirstOrDefault`. Voor de controller verandert er dus niets.
- `Find` **kijkt eerst in het geheugen van de context.** Heeft de context dit recept tijdens dit verzoek al opgehaald, dan stuurt `Find` geen nieuwe vraag naar de database. `FirstOrDefault(r => r.Id == id)` werkt ook, maar vraagt het altijd opnieuw.

3c. GetByMaximumKostprijs

Met de lijst:

```
public List<Recept> GetByMaximumKostprijs(decimal bedrag)
{
    return _recepten
        .Where(r => r.Kostprijs <= bedrag)
        .ToList();
}
```

Met de database:

```
public List<Recept> GetByMaximumKostprijs(decimal bedrag)
{
    return _context.Recepten
        .Where(r => r.Kostprijs <= bedrag)
        .ToList();
}
```

Enkel de bron verandert. De LINQ is letterlijk dezelfde. Wat er gebeurt, is wel helemaal anders:

- Met de lijst haalde je alle recepten op, en filterde C# ze in het geheugen van je API.
- Met de database vertaalt EF Core je `Where` naar SQL. De database filtert, en enkel de recepten die passen, komen naar je API.

In het terminalvenster van je API zie je de SQL die EF Core stuurt, ongeveer zo:

```
SELECT r."Id", r."Kostprijs", r."Titel"
FROM "Recepten" AS r
WHERE r."Kostprijs" <= @bedrag
```

Waarom pas bij `ToList`**.** `Where` en `OrderBy` bouwen de vraag, maar sturen ze nog niet. Pas `ToList`, `Find`, `Count` of `Any` sturen ze echt. Dat heet **uitgestelde uitvoering**. Zie hoofdstuk 32.

3d. Create

Met de lijst:

```
public Recept Create(Recept recept)
{
    recept.Id = _recepten.Max(r => r.Id) + 1;
    _recepten.Add(recept);
    return recept;
}
```

Met de database:

```
public Recept Create(Recept recept)
{
    _context.Recepten.Add(recept);
    _context.SaveChanges();
    return recept;
}
```

Waarom `Max + 1` verdwijnt. In les 03 deelde de repository de nummers uit, omdat de lijst dat niet kon. Nu doet de database het: je migratie in les 04 gaf de kolom `Id` een teller (`IdentityByDefaultColumn`). Die is ook veiliger. Komen er twee POST's tegelijk binnen, dan kunnen ze met `Max + 1` allebei hetzelfde nummer berekenen. De teller van de database geeft nooit twee keer hetzelfde nummer.

Waarom `Add` én `SaveChanges`. `Add` schrijft nog niets. Het noteert in de context: *dit recept moet erbij*. Pas `SaveChanges` stuurt alles wat genoteerd is naar de database. Voor één recept is dat:

```
INSERT INTO "Recepten" ("Kostprijs", "Titel")
VALUES (@p0, @p1)
RETURNING "Id";
```

Hoe het recept dan zijn Id krijgt. `RETURNING "Id"` vraagt de database om het nieuwe nummer terug te sturen. EF Core zet het in `recept.Id`. Daarom heeft het recept dat `Create` teruggeeft, het juiste Id, en kan `CreatedAtAction` in de controller het gebruiken. De controller hoeft daarvoor niets te veranderen.

> **LET OP**
>
> In les 03 overschreef `Max + 1` altijd het Id dat de client meestuurde. Nu doet niemand dat nog. Stuurt de client een Id mee dat niet 0 is, dan probeert EF Core precies dat nummer te bewaren. Kijk dus wat je met een meegestuurd Id wil doen.

3e. Update

Met de lijst:

```
public void Update(int id, Recept recept)
{
    Recept? bestaand = _recepten.FirstOrDefault(r => r.Id == id);

    if (bestaand != null)
    {
        bestaand.Titel = recept.Titel;
        bestaand.Kostprijs = recept.Kostprijs;
    }
}
```

Met de database:

```
public void Update(int id, Recept recept)
{
    Recept? bestaand = _context.Recepten.Find(id);

    if (bestaand != null)
    {
        bestaand.Titel = recept.Titel;
        bestaand.Kostprijs = recept.Kostprijs;
        _context.SaveChanges();
    }
}
```

Waarom het overnemen hetzelfde blijft. Het recept dat `Find` teruggeeft, wordt gevolgd door de context. De context onthoudt hoe het eruitzag toen het uit de database kwam. Bij `SaveChanges` vergelijkt ze, en stuurt ze een `UPDATE` voor de kolommen die veranderd zijn. Dat heet de **change tracker**. Daarom pas je ook hier de eigenschappen één voor één aan. `bestaand = recept;` verandert enkel je variabele, en de context ziet geen wijziging.

Twee keer zoeken, één keer vragen. In hoofdstuk 36 zocht de controller eerst met `GetById`, en zocht de repository in `Update` nog eens. Met de database kost dat niets extra. De controller en de repository gebruiken tijdens één verzoek dezelfde context. De tweede `Find` vindt het recept in het geheugen van die context, en stuurt geen tweede vraag.

3f. Delete

Met de lijst:

```
public void Delete(int id)
{
    Recept? recept = _recepten.FirstOrDefault(r => r.Id == id);

    if (recept != null)
    {
        _recepten.Remove(recept);
    }
}
```

Met de database:

```
public void Delete(int id)
{
    Recept? recept = _context.Recepten.Find(id);

    if (recept != null)
    {
        _context.Recepten.Remove(recept);
        _context.SaveChanges();
    }
}
```

Net als bij `Add`: `Remove` noteert enkel. `SaveChanges` stuurt de `DELETE`.

Na stap 3, alles naast elkaar

Met de lijst	Met de database	Waarom
`_recepten`	`_context.Recepten`	de tabel in plaats van de lijst
`return _recepten;`	`.ToList()` erachter	een `DbSet` is geen lijst. `ToList` stuurt de vraag
`FirstOrDefault(r => r.Id == id)`	`Find(id)`	zoekt op de sleutel, en eerst in het geheugen van de context
`Max(r => r.Id) + 1`	weg	de database deelt de nummers uit
`Add`, `Remove`	`Add`, `Remove`, en daarna `SaveChanges`	zonder `SaveChanges` komt er niets in de database
eigenschappen overnemen	hetzelfde, en daarna `SaveChanges`	de context ziet de wijziging en stuurt een `UPDATE`
de lijst is `static`	geen lijst meer	de gegevens staan in de database, niet in je API

Deel A, stap 4: één regel in Program.cs

Wat je doet. Laat `Program.cs` de nieuwe repository leveren.

Vóór:

```
builder.Services.AddScoped<IReceptRepository, InMemoryReceptRepository>();
```

Ná:

```
builder.Services.AddScoped<IReceptRepository, ReceptRepository>();
```

Waarom er verder niets verandert. De controller vraagt een `IReceptRepository`. Welke klasse hij krijgt, weet hij niet. Dat is wat je in les 03 bouwde, en hier betaalt het zich uit: geen letter in je controller verandert.

Waarom `AddScoped`, en geen `AddSingleton`. `AddDbContext` geeft elk verzoek een eigen context. Een repository met `AddSingleton` zou de hele tijd blijven bestaan, en dus de context van één verzoek proberen te bewaren. ASP.NET Core weigert dat: de API start dan niet.

Deel A, stap 5: testen

Wat je doet. Start je API en test elk endpoint in Scalar.

Wat je merkt:

- **De tabel is leeg.** Je drie startrecepten stonden in de lijst van `InMemoryReceptRepository`, en die gebruik je niet meer. `GET api/recept` geeft `[]`, tenzij je de tabel in les 04 al vulde. Voeg een paar recepten toe met POST. In les 06 komen startgegevens mee met je migraties.
- **Je gegevens blijven bewaard.** Voeg een recept toe, stop je API, start ze opnieuw, en vraag de lijst op. Het recept staat er nog.
- **In het terminalvenster** verschijnt bij elk verzoek *Executed DbCommand*, met de SQL die EF Core stuurde.

Waarom `static` niet meer nodig is. In les 03 moest de lijst `static` zijn, omdat elk verzoek een nieuwe repository kreeg. Nu krijgt elk verzoek nog steeds een nieuwe repository, en een nieuwe context. Maar de gegevens staan niet in die objecten. Ze staan in de database, en die blijft.

Deel B: waarom asynchroon

Een vraag aan de database duurt even. Synchroon wacht de thread van je API al die tijd, en kan hij niets anders doen. Komen er veel verzoeken tegelijk, dan raken de threads op, en moet elk nieuw verzoek wachten.

Met `async` en `await` geeft de thread zich vrij terwijl de database werkt. Hij helpt intussen een ander verzoek. Zodra de database antwoordt, gaat je code verder. Zie hoofdstuk 29.

Zo zet je een methode om:

Synchroon	Asynchroon	Waarom
`List<Recept> GetAll()`	`async Task<List<Recept>> GetAllAsync()`	een `Task<T>` is een belofte: het resultaat komt later

Synchroon	Asynchroon	Waarom
`Recept? GetById(int id)`	`async Task<Recept?> GetByIdAsync(int id)`	ook het vraagteken gaat mee in de `Task`
`void Update(...)`	`async Task UpdateAsync(...)`	`void` wordt `Task`. Schrijf nooit `async void`
`.ToList()`	`await ... .ToListAsync()`	EF Core heeft een asynchrone versie van elke methode die de database echt aanspreekt
`.Find(id)`	`await ... .FindAsync(id)`	idem
`.SaveChanges()`	`await ... .SaveChangesAsync()`	idem
`.Add(...)`, `.Remove(...)`	blijven	ze spreken de database niet aan, ze noteren enkel
`.Where(...)`, `.OrderBy(...)`	blijven	ze bouwen de vraag, ze sturen ze nog niet

De naam krijgt `Async` erachter. Dat is een afspraak: wie de methode oproept, ziet meteen dat hij `await` nodig heeft.

Het domino-effect. `await` mag enkel in een methode met `async`. Een `async`-methode geeft een `Task` terug. Wie die oproept, moet dus zelf `await` gebruiken, en wordt dus zelf `async`. Zo valt de ene steen op de volgende: eerst de repository, dan de interface, dan de controller.

Deel B, stap 6: de repository asynchroon

Wat je doet. Zet de methodes van `ReceptRepository` één voor één om. Voeg bovenaan `using Microsoft.EntityFrameworkCore;` toe: daar staan `ToListAsync`, `FindAsync` en de andere.

GetAll, vóór:

```
public List<Recept> GetAll()
{
    return _context.Recepten
        .OrderBy(r => r.Titel)
        .ToList();
}
```

Ná:

```
public async Task<List<Recept>> GetAllAsync()
{
    return await _context.Recepten
        .OrderBy(r => r.Titel)
        .ToListAsync();
}
```

GetById, vóór:

```
public Recept? GetById(int id)
{
    return _context.Recepten.Find(id);
}
```

Ná:

```
public async Task<Recept?> GetByIdAsync(int id)
{
    return await _context.Recepten.FindAsync(id);
}
```

Create, vóór:

```
public Recept Create(Recept recept)
{
    _context.Recepten.Add(recept);
    _context.SaveChanges();
    return recept;
}
```

Ná:

```
public async Task<Recept> CreateAsync(Recept recept)
{
    _context.Recepten.Add(recept);
    await _context.SaveChangesAsync();
    return recept;
}
```

`return recept;` blijft zo. In een `async`-methode geef je het resultaat zelf terug. C# pakt het in een `Task` in.

Update, vóór:

```
public void Update(int id, Recept recept)
{
    Recept? bestaand = _context.Recepten.Find(id);

    if (bestaand != null)
    {
        bestaand.Titel = recept.Titel;
        bestaand.Kostprijs = recept.Kostprijs;
        _context.SaveChanges();
    }
}
```

Ná:

```
public async Task UpdateAsync(int id, Recept recept)
{
    Recept? bestaand = await _context.Recepten.FindAsync(id);

    if (bestaand != null)
    {
        bestaand.Titel = recept.Titel;
        bestaand.Kostprijs = recept.Kostprijs;
        await _context.SaveChangesAsync();
    }
}
```

`GetByMaximumKostprijs` gaat zoals `GetAll`, en `Delete` zoals `Update`. Je ziet ze in het eindresultaat.

Visual Studio helpt. Typ eerst enkel `await` voor de oproep. Er komt een rode lijn, want de methode is nog niet `async`. Druk **Ctrl+.**, en Visual Studio past `async` en het returntype voor je aan.

Bouwt het? Nee. `ReceptRepository` belooft `IReceptRepository`, maar heeft de oude methodes niet meer: fout **CS0535**. Dat is de eerste dominosteen. De interface volgt in stap 7.

Deel B, stap 7: de interface

Vóór:

```
public interface IReceptRepository
{
    List<Recept> GetAll();
    Recept? GetById(int id);
    List<Recept> GetByMaximumKostprijs(decimal bedrag);
    Recept Create(Recept recept);
    void Update(int id, Recept recept);
    void Delete(int id);
}
```

Ná:

```
public interface IReceptRepository
{
    Task<List<Recept>> GetAllAsync();
    Task<Recept?> GetByIdAsync(int id);
    Task<List<Recept>> GetByMaximumKostprijsAsync(decimal bedrag);
    Task<Recept> CreateAsync(Recept recept);
    Task UpdateAsync(int id, Recept recept);
    Task DeleteAsync(int id);
}
```

In een interface staat geen `async`. `async` zegt iets over hoe een methode werkt, en een interface zegt enkel wat er moet bestaan. De `Task` hoort wel bij het contract.

Bouwt het? `ReceptRepository` klopt nu met het contract. Maar twee andere bestanden niet meer:

- `InMemoryReceptRepository` heeft de oude methodes nog: **CS0535**.
- De controller roept `GetAll`, `GetById` en de andere op, en die bestaan niet meer: **CS1061**.

Deel B, stap 8: de controller

Wat je doet. Elke actie die de repository oproept, krijgt `async`, een `Task` rond haar returntype, en `await` voor elke oproep.

GetAll, vóór:

```
[HttpGet]
public ActionResult<List<Recept>> GetAll()
{
    _logger.LogInformation("Alle recepten opgevraagd");
    return Ok(_repository.GetAll());
}
```

Ná:

```
[HttpGet]
public async Task<ActionResult<List<Recept>>> GetAll()
{
    _logger.LogInformation("Alle recepten opgevraagd");
    return Ok(await _repository.GetAllAsync());
}
```

GetById, vóór:

```
[HttpGet("{id}")]
public ActionResult<Recept> GetById(int id)
{
    _logger.LogInformation("Recept {Id} opgevraagd", id);

    Recept? recept = _repository.GetById(id);

    if (recept == null)
    {
        _logger.LogWarning("Recept {Id} niet gevonden", id);
        return NotFound($"We vonden geen recept met Id {id}.");
    }

    return Ok(recept);
}
```

Ná:

```
[HttpGet("{id}")]
public async Task<ActionResult<Recept>> GetById(int id)
{
    _logger.LogInformation("Recept {Id} opgevraagd", id);

    Recept? recept = await _repository.GetByIdAsync(id);

    if (recept == null)
    {
        _logger.LogWarning("Recept {Id} niet gevonden", id);
        return NotFound($"We vonden geen recept met Id {id}.");
    }

    return Ok(recept);
}
```

Create, vóór:

```
[HttpPost]
public ActionResult<Recept> Create(Recept nieuw)
{
    _logger.LogInformation("Nieuw recept {Titel}", nieuw.Titel);

    if (nieuw.Kostprijs < 0)
    {
        _logger.LogWarning("Negatieve kostprijs geweigerd");
        return BadRequest("De kostprijs kan niet negatief zijn.");
    }

    Recept gemaakt = _repository.Create(nieuw);
    _logger.LogInformation("Recept {Id} toegevoegd", gemaakt.Id);

    return CreatedAtAction(nameof(GetById), new { id = gemaakt.Id }, gemaakt);
}
```

Ná:

```
[HttpPost]
public async Task<ActionResult<Recept>> Create(Recept nieuw)
{
    _logger.LogInformation("Nieuw recept {Titel}", nieuw.Titel);

    if (nieuw.Kostprijs < 0)
    {
        _logger.LogWarning("Negatieve kostprijs geweigerd");
        return BadRequest("De kostprijs kan niet negatief zijn.");
    }

    Recept gemaakt = await _repository.CreateAsync(nieuw);
    _logger.LogInformation("Recept {Id} toegevoegd", gemaakt.Id);

    return CreatedAtAction(nameof(GetById), new { id = gemaakt.Id }, gemaakt);
}
```

Update, vóór:

```
[HttpPut("{id}")]
public ActionResult Update(int id, Recept gewijzigd)
{
    _logger.LogInformation("Recept {Id} bijwerken", id);

    if (_repository.GetById(id) == null)
    {
        _logger.LogWarning("Recept {Id} niet gevonden", id);
        return NotFound($"We vonden geen recept met Id {id}.");
    }

    if (gewijzigd.Kostprijs < 0)
    {
        _logger.LogWarning("Negatieve kostprijs geweigerd");
        return BadRequest("De kostprijs kan niet negatief zijn.");
    }

    _repository.Update(id, gewijzigd);

    return NoContent();
}
```

Ná:

```
[HttpPut("{id}")]
public async Task<ActionResult> Update(int id, Recept gewijzigd)
{
    _logger.LogInformation("Recept {Id} bijwerken", id);

    if (await _repository.GetByIdAsync(id) == null)
    {
        _logger.LogWarning("Recept {Id} niet gevonden", id);
        return NotFound($"We vonden geen recept met Id {id}.");
    }

    if (gewijzigd.Kostprijs < 0)
    {
        _logger.LogWarning("Negatieve kostprijs geweigerd");
        return BadRequest("De kostprijs kan niet negatief zijn.");
    }

    await _repository.UpdateAsync(id, gewijzigd);

    return NoContent();
}
```

`GetByMaximumKostprijs` gaat zoals `GetAll`, en `Delete` zoals `Update`.

Drie dingen blijven bewust hetzelfde:

- **De namen van de acties.** `GetById` heet niet `GetByIdAsync`. Zie het kader hieronder.
- **Wat er binnen `ActionResult<...>` staat.** `ActionResult<Recept>` wordt `Task<ActionResult<Recept>>`: de `Task` komt er rond, de rest blijft.
- **De logging, de controles en de antwoorden.** Ze wachten op niets, dus ze krijgen geen `await`.

> **BELANGRIJK**
>
> `CreatedAtAction` **en het achtervoegsel** `Async`**.** ASP.NET Core knipt `Async` van het einde van de naam van een controlleractie. Heet je GET-actie `GetByIdAsync`, dan kent de router ze als `GetById`.
>
> `CreatedAtAction(nameof(GetByIdAsync), ...)` zoekt dan een actie die niet bestaat. De client krijgt een 500 met *No route matches the supplied values.*, terwijl het recept wél bewaard is.
>
> Daarom krijgen in dit boek de acties in een controller geen `Async`, de methodes in een repository wel. Zie hoofdstuk 26.

Deel B, stap 9: InMemoryReceptRepository weg

Wat je doet. Verwijder `InMemoryReceptRepository.cs`.

Waarom. Ze volgt het contract niet meer, en `Program.cs` gebruikt ze niet meer. Je zou ze ook asynchroon kunnen maken, maar ze heeft haar werk gedaan.

Bouwt het nu? Ja. Pas nu valt de laatste dominosteen. Zo verliep de bouw onderweg:

Na stap	Bouwt je project?	Wat Visual Studio zegt
6: de repository asynchroon	nee	CS0535 in `ReceptRepository`: ze belooft nog de oude methodes
7: de interface asynchroon	nee	CS0535 in `InMemoryReceptRepository`, en CS1061 in de controller
8: de controller asynchroon	nee	enkel nog CS0535 in `InMemoryReceptRepository`
9: `InMemoryReceptRepository` weg	ja	

Doe stap 6 tot 9 dus in één keer. Tussendoor kan je niet testen.

Wat er gebeurt bij één verzoek

Een client stuurt `GET api/recept/1`. Dit gebeurt er, in deze volgorde:

1. ASP.NET Core zoekt de actie bij de route: `GetById` in `ReceptController`.
2. Het maakt de ketting: een nieuwe `ReceptContext`, een `ReceptRepository` die die context krijgt, en een `ReceptController` die die repository krijgt.
3. `GetById` loopt tot `await _repository.GetByIdAsync(id)`. Daarin stuurt `FindAsync` een `SELECT` naar de database.
4. Terwijl de database zoekt, geeft de thread zich vrij, en helpt hij een ander verzoek.
5. De database antwoordt. Je code gaat verder na de `await`, met het recept of met `null`.
6. De controller kiest 200 of 404, en het antwoord vertrekt.
7. De controller, de repository en de context verdwijnen. De gegevens blijven in de database.

Het eindresultaat

Het contract, in `Repositories/IReceptRepository.cs`:

```
using ReceptenApi.Models;

namespace ReceptenApi.Repositories;

public interface IReceptRepository
{
    Task<List<Recept>> GetAllAsync();
    Task<Recept?> GetByIdAsync(int id);
    Task<List<Recept>> GetByMaximumKostprijsAsync(decimal bedrag);
    Task<Recept> CreateAsync(Recept recept);
    Task UpdateAsync(int id, Recept recept);
    Task DeleteAsync(int id);
}
```

De repository, in `Repositories/ReceptRepository.cs`:

```
using Microsoft.EntityFrameworkCore;
using ReceptenApi.Data;
using ReceptenApi.Models;

namespace ReceptenApi.Repositories;

public class ReceptRepository : IReceptRepository
{
    private readonly ReceptContext _context;

    public ReceptRepository(ReceptContext context)
    {
        _context = context;
    }

    public async Task<List<Recept>> GetAllAsync()
    {
        return await _context.Recepten
            .OrderBy(r => r.Titel)
            .ToListAsync();
    }

    public async Task<Recept?> GetByIdAsync(int id)
    {
        return await _context.Recepten.FindAsync(id);
    }

    public async Task<List<Recept>> GetByMaximumKostprijsAsync(decimal bedrag)
    {
        return await _context.Recepten
            .Where(r => r.Kostprijs <= bedrag)
            .ToListAsync();
    }

    public async Task<Recept> CreateAsync(Recept recept)
    {
        _context.Recepten.Add(recept);
        await _context.SaveChangesAsync();
        return recept;
    }

    public async Task UpdateAsync(int id, Recept recept)
    {
        Recept? bestaand = await _context.Recepten.FindAsync(id);

        if (bestaand != null)
        {
            bestaand.Titel = recept.Titel;
            bestaand.Kostprijs = recept.Kostprijs;
            await _context.SaveChangesAsync();
        }
    }

    public async Task DeleteAsync(int id)
    {
```

```
        Recept? recept = await _context.Recepten.FindAsync(id);

        if (recept != null)
        {
            _context.Recepten.Remove(recept);
            await _context.SaveChangesAsync();
        }
    }
}
```

De controller, in `Controllers/ReceptController.cs`:

```
using Microsoft.AspNetCore.Mvc;
using ReceptenApi.Models;
using ReceptenApi.Repositories;

namespace ReceptenApi.Controllers;

[ApiController]
[Route("api/[controller]")]
public class ReceptController : ControllerBase
{
    private readonly IReceptRepository _repository;
    private readonly ILogger<ReceptController> _logger;

    public ReceptController(IReceptRepository repository,
                            ILogger<ReceptController> logger)
    {
        _repository = repository;
        _logger = logger;
    }

    [HttpGet]
    public async Task<ActionResult<List<Recept>>> GetAll()
    {
        _logger.LogInformation("Alle recepten opgevraagd");
        return Ok(await _repository.GetAllAsync());
    }

    [HttpGet("{id}")]
    public async Task<ActionResult<Recept>> GetById(int id)
    {
        _logger.LogInformation("Recept {Id} opgevraagd", id);

        Recept? recept = await _repository.GetByIdAsync(id);

        if (recept == null)
        {
            _logger.LogWarning("Recept {Id} niet gevonden", id);
            return NotFound($"We vonden geen recept met Id {id}.");
        }

        return Ok(recept);
    }

    [HttpGet("maximum/{bedrag}")]
```

```
    public async Task<ActionResult<List<Recept>>> GetByMaximumKostprijs(
        decimal bedrag)
    {
        _logger.LogInformation("Recepten tot {Bedrag} euro opgevraagd", bedrag);
        return Ok(await _repository.GetByMaximumKostprijsAsync(bedrag));
    }

    [HttpPost]
    public async Task<ActionResult<Recept>> Create(Recept nieuw)
    {
        _logger.LogInformation("Nieuw recept {Titel}", nieuw.Titel);

        if (nieuw.Kostprijs < 0)
        {
            _logger.LogWarning("Negatieve kostprijs geweigerd");
            return BadRequest("De kostprijs kan niet negatief zijn.");
        }

        Recept gemaakt = await _repository.CreateAsync(nieuw);
        _logger.LogInformation("Recept {Id} toegevoegd", gemaakt.Id);

        return CreatedAtAction(nameof(GetById), new { id = gemaakt.Id }, gemaakt);
    }

    [HttpPut("{id}")]
    public async Task<ActionResult> Update(int id, Recept gewijzigd)
    {
        _logger.LogInformation("Recept {Id} bijwerken", id);

        if (await _repository.GetByIdAsync(id) == null)
        {
            _logger.LogWarning("Recept {Id} niet gevonden", id);
            return NotFound($"We vonden geen recept met Id {id}.");
        }

        if (gewijzigd.Kostprijs < 0)
        {
            _logger.LogWarning("Negatieve kostprijs geweigerd");
            return BadRequest("De kostprijs kan niet negatief zijn.");
        }

        await _repository.UpdateAsync(id, gewijzigd);

        return NoContent();
    }

    [HttpDelete("{id}")]
    public async Task<ActionResult> Delete(int id)
    {
        _logger.LogInformation("Recept {Id} verwijderen", id);

        if (await _repository.GetByIdAsync(id) == null)
        {
            _logger.LogWarning("Recept {Id} niet gevonden", id);
            return NotFound($"We vonden geen recept met Id {id}.");
        }

        await _repository.DeleteAsync(id);
```

```
        return NoContent();
    }
}
```

Program.cs na deze les

Ten opzichte van les 04 verandert één regel:

```
using Microsoft.EntityFrameworkCore;
using ReceptenApi.Data;
using ReceptenApi.Repositories;
using Scalar.AspNetCore;

var builder = WebApplication.CreateBuilder(args);

builder.Services.AddControllers();
builder.Services.AddOpenApi();
builder.Services.AddScoped<IReceptRepository, ReceptRepository>();

var connectionString = builder.Configuration.GetConnectionString("Standaard");
builder.Services.AddDbContext<ReceptContext>(options =>
    options.UseNpgsql(connectionString));

var app = builder.Build();

if (app.Environment.IsDevelopment())
{
    app.MapOpenApi();
    app.MapScalarApiReference();
}

app.UseHttpsRedirection();
app.UseAuthorization();
app.MapControllers();

app.Run();
```

Hoe je ziet dat het werkt

Verzoek	Antwoord na de les	Ten opzichte van les 03
`GET api/recept`	200 met de recepten uit de database, alfabetisch op titel	andere volgorde, door `OrderBy`
`GET api/recept/1`	200 met het recept, of 404 met het bericht	hetzelfde
`GET api/recept/maximum/10`	200 met de recepten tot 10 euro	hetzelfde
`POST api/recept`	201 met het nieuwe recept en zijn Id	het Id komt van de database
`PUT` en `DELETE`	204, of 404 met het bericht	hetzelfde

Daarnaast:

- Een recept dat je toevoegt, staat er nog na een herstart van je API.
- In pgAdmin zie je het staan: rechtsklik op de tabel, **View/Edit Data › All Rows**.

- In het terminalvenster verschijnt bij elk verzoek *Executed DbCommand*, met de SQL die EF Core stuurde.
- In je code staat nergens `.Result`, `.Wait()` of `async void`.

Alle stappen op een rij

1. Maak `ReceptRepository`, die `IReceptRepository` belooft. Laat **Ctrl+.** de lege methodes maken.
2. Geef ze een veld `private readonly ReceptContext _context`, en vraag de context in de constructor.
3. Vul de methodes in, eerst synchroon: `_context.Recepten` in plaats van de lijst, `Find` in plaats van `FirstOrDefault`, geen `Max + 1`, en `SaveChanges` na elke wijziging.
4. Verander in `Program.cs` één regel: `InMemoryReceptRepository` wordt `ReceptRepository`.
5. Test: voeg een recept toe, stop je API, start ze opnieuw, en vraag de lijst op.
6. Maak de repository asynchroon, met `using Microsoft.EntityFrameworkCore;`.
7. Pas de interface mee aan.
8. Maak de acties van de controller asynchroon, met `await` voor elke oproep naar de repository. De namen van de acties blijven.
9. Haal `InMemoryReceptRepository` weg. Nu bouwt je project weer.

Als een stap ontbreekt

Wat je ziet	Waarom	Wat je doet
een POST geeft 201, maar het recept staat niet in de database	`SaveChanges` of `SaveChangesAsync` ontbreekt	na `Add`, `Remove` en het bijwerken bewaren
alles werkt, maar na een herstart is alles weg	`Program.cs` registreert nog `InMemoryReceptRepository`	de regel aanpassen
`GET api/recept` geeft `[]`	de tabel is leeg: de startrecepten stonden in de lijst	recepten toevoegen. In les 06 komen startgegevens mee met je migraties
500 met *No route matches the supplied values.*, en het recept is wel bewaard	`CreatedAtAction` verwijst naar een actie met `Async` in haar naam	zie het kader bij stap 8
CS0029: *Cannot implicitly convert type 'DbSet<Recept>' to 'List<Recept>'*	`ToList` of `ToListAsync` vergeten	het erachter zetten
waarschuwing **CS4014**: *Because this call is not awaited, execution of the current method continues before the call is completed.*	`await` vergeten	`await` ervoor zetten
CS0029: *Cannot implicitly convert type 'Task<Recept?>' to 'Recept?'*	`await` vergeten bij een methode die een `Task` geeft	`await` ervoor zetten
CS4032: *The 'await' operator can only be used within an async method.*	`async` vergeten in de methode zelf	`async` toevoegen, en het returntype een `Task` maken
CS1061: *'IReceptRepository' does not contain a definition for 'GetAll'*	de interface is asynchroon, de controller nog niet	de oproep aanpassen, met `await`

Wat je ziet	Waarom	Wat je doet
CS0738: *does not have the matching return type of 'Task<List<Recept>>'*	de interface en de repository zeggen iets anders	ze gelijk maken
de API start niet: *Cannot consume scoped service 'ReceptenApi.Data.ReceptContext' from singleton 'ReceptenApi.Repositories.IReceptRepository'.*	de repository is geregistreerd met `AddSingleton`	`AddScoped` gebruiken
500 met *42P01: relation "Recepten" does not exist*	de migratie is niet uitgevoerd	`Update-Database`
500 met *42703: column r.Bereidingstijd does not exist*	het model is gewijzigd zonder nieuwe migratie	een migratie maken en uitvoeren

Let op bij de lescode

LET OP

- De lescode zet `CreatedAtAction(nameof(GetLaptopByIdAsync), ...)`. Dat geeft de 500 uit het kader bij stap 8.
- In de les staat `private readonly \WebAPIDemoContext _context;`, met een schuine streep te veel. Zo compileert het niet.
- In de tabel met LINQ-opdrachten staat `_context.Laptops.Count(id)`. Bedoeld is `Find(id)`.
- De namespaces wisselen tussen `WebApiDemo` en `WebAPIDemo`. Kies één schrijfwijze.
- De DELETE in de les meldt een onbekend Id nog altijd met `LogError`. Zie hoofdstuk 36.

Waar het uitgelegd staat

Onderwerp	Hoofdstuk
`async`, `await` en `Task`	29
`ToListAsync`, `FindAsync`, `SaveChangesAsync` en wat de context noteert	32
`CreatedAtAction` en de andere helpers	26
registreren, `AddScoped` en de levensduur van een dienst	27
LINQ: `Where`, `OrderBy` en `ToList`	11

Voorspel de uitkomst

Vraag 1

Je POST geeft `CreatedAtAction(nameof(GetByIdAsync), new { id = recept.Id }, recept)`, en je GET-actie heet `GetByIdAsync`. Wat krijgt de client? Staat het recept in de database?

Toon het antwoord ›

Vraag 2

`ReceptRepository` is af, maar in `Program.cs` staat nog `AddScoped<IReceptRepository, InMemoryReceptRepository>()`. Je voegt een recept toe. Waar komt het terecht?

Toon het antwoord ›

Vraag 3

```
List<Recept> recepten = _context.Recepten.ToListAsync();
```

Deze regel staat in een `async`-methode van `ReceptRepository`. Wat zegt Visual Studio?

Toon het antwoord ›

Vraag 4

`CreateAsync` in de repository doet `_context.Recepten.Add(recept);` en `return recept;`, maar `SaveChangesAsync` ontbreekt. Een client stuurt een POST met een nieuw recept. Wat krijgt hij terug, en wat staat er daarna in de database?

Toon het antwoord ›

Vraag 5

```
public async Task<List<Recept>> GetAllAsync()
{
    return _context.Recepten.OrderBy(r => r.Titel).ToList();
}
```

Bouwt het? Werkt `GET api/recept`? Wat is er mis?

Toon het antwoord ›

Antwoorden

VRAAG 1

500 Internal Server Error, met *No route matches the supplied values.* **Ja**, het recept staat in de database.

De POST bewaart eerst, en maakt pas daarna het adres voor de Location-header. Dat adres kan niet gemaakt worden, want voor de router heet de actie `GetById`.

‹ Terug naar de vraag

VRAAG 2

In de lijst van `InMemoryReceptRepository`**.** De database blijft leeg. Je controller krijgt de klasse die `Program.cs` registreert, en de nieuwe repository wordt nooit gebruikt.

‹ Terug naar de vraag

VRAAG 3

Compileert niet: CS0029, *Cannot implicitly convert type 'Task<List<Recept>>' to 'List<Recept>'*.

`ToListAsync` geeft een `Task`: de belofte van een lijst, nog niet de lijst zelf. Met `await` ervoor wacht je op het resultaat, en dan klopt het type.

‹ Terug naar de vraag

VRAAG 4

201 Created, met `"id": 0`**.** De database blijft leeg.

`Add` noteerde het recept enkel in de context. Zonder `SaveChangesAsync` gaat er geen `INSERT` naar de database, en komt er dus ook geen nummer terug. Het antwoord ziet er gelukt uit, maar na het verzoek verdwijnt de context, met alles wat ze noteerde.

‹ Terug naar de vraag

VRAAG 5

Het bouwt, met waarschuwing CS1998: *This async method lacks 'await' operators and will run synchronously.* En `GET api/recept` werkt.

Maar de methode is enkel asynchroon in naam. `ToList` wacht synchroon op de database, en de thread kan intussen niets anders doen. Gebruik `await` en `ToListAsync`.

‹ Terug naar de vraag

39. Les 06: relaties en startgegevens

DATA

Wat je bouwt

Meer modellen, met relaties ertussen, vastgelegd met de Fluent API. Startgegevens die met je migraties meekomen. En een repository en een controller voor een nieuw model.

In het voorbeeld heeft een keuken veel recepten, en hebben recepten en ingrediënten een veel-op-veel-relatie via een tussentabel met een hoeveelheid. De les doet hetzelfde met klanten, bestellingen, orderlijnen en producten.

Hoe je dit hoofdstuk leest

Les 06 bouwt in drie migraties. Elke migratie is hier een stap, en bij elke stap zie je wat er in de database verandert, en waarom.

Daarna splitst de les opnieuw iets op, net als les 03:

- **De startgegevens verhuizen een tweede keer.** In les 03 stonden ze in de lijst van de repository. In les 05 vielen ze weg. Nu komen ze in de context.
- `OnModelCreating` **valt uiteen in twee methodes**: één die de tabellen opbouwt, en één met de startgegevens.

En de les werkt één keer in omgekeerde volgorde: eerst de klasse van de repository, en dan pas de interface.

De kleuren:

- **oranje en vet**: dit verhuist of wordt vervangen;
- **groen en vet**: dit is nieuw of gewijzigd in deze stap;
- **blauw en vet**: dit verhuist ook, maar naar een tweede plaats. Je ziet blauw enkel in stap 5, waar code naar twee methodes gaat.

Wat er bijkomt in je project

```
ReceptenApi/
├── Controllers/
│   └── KeukenController.cs            ← nieuw
├── Data/
│   └── ReceptContext.cs               ← gewijzigd
├── Migrations/                        ← drie migraties erbij
├── Models/
│   ├── Ingredient.cs                  ← nieuw
│   ├── Keuken.cs                      ← nieuw
│   ├── Recept.cs                      ← gewijzigd: sleutel en navigaties
│   └── ReceptIngredient.cs            ← nieuw: de tussentabel
├── Repositories/
│   ├── IKeukenRepository.cs           ← nieuw
│   └── KeukenRepository.cs            ← nieuw
└── Program.cs                         ← gewijzigd
```

Stap 1: de modellen

Wat je doet. Maak drie nieuwe modellen, en geef `Recept` er drie eigenschappen bij.

`Recept` vóór, zoals na les 05:

```
public class Recept
{
    public int Id { get; set; }
    public string Titel { get; set; } = default!;
    public decimal Kostprijs { get; set; }
}
```

`Recept` ná:

```
public class Recept
{
    public int Id { get; set; }
    public string Titel { get; set; } = default!;
    public decimal Kostprijs { get; set; }

    public int KeukenId { get; set; }
    public Keuken? Keuken { get; set; }

    public List<ReceptIngredient>? ReceptIngredienten { get; set; }
}
```

De drie nieuwe modellen:

```
public class Keuken
{
    public int Id { get; set; }
    public string Naam { get; set; } = default!;
```

```
    public List<Recept>? Recepten { get; set; }
}

public class Ingredient
{
    public int Id { get; set; }
    public string Naam { get; set; } = default!;

    public List<ReceptIngredient>? ReceptIngredienten { get; set; }
}

public class ReceptIngredient
{
    public int Id { get; set; }
    public string Hoeveelheid { get; set; } = default!;

    public int ReceptId { get; set; }
    public Recept? Recept { get; set; }

    public int IngredientId { get; set; }
    public Ingredient? Ingredient { get; set; }
}
```

Elk model in zijn eigen bestand, in de map `Models`.

Welke eigenschap een kolom wordt. Niet elke eigenschap komt in de database. Kijk naar `Recept`:

Eigenschap	Een kolom?	Waarom
`Id`, `Titel`, `Kostprijs`	ja	zoals in les 04
`int KeukenId`	ja: de **vreemde sleutel**	het nummer van de keuken waar dit recept bij hoort. EF Core herkent de naam: de navigatie `Keuken`, met `Id` erachter
`Keuken? Keuken`	nee: een **navigatie**	in C# kan je van een recept naar zijn keuken. In de database is dat gewoon `KeukenId`
`List<ReceptIngredient>? ReceptIngredienten`	nee: een navigatie	de veel-kant. De vreemde sleutel staat aan de andere kant, in de tussentabel

De regel van de les. Een één-kant heeft een vreemde sleutel én een navigatie. Een veel-kant heeft enkel een lijst. Logisch: een vreemde sleutel bevat één nummer, dus ze kan maar naar één ding wijzen.

De twee relaties:

Relatie	Soort	Waar de vreemde sleutel staat
een keuken heeft veel recepten, een recept hoort bij één keuken	één-op-veel	`KeukenId`, in `Recepten`
een recept heeft veel ingrediënten, een ingrediënt zit in veel recepten	veel-op-veel	in de tussentabel `ReceptIngredienten`: `ReceptId` en `IngredientId`

Een veel-op-veel-relatie kan niet rechtstreeks: geen van beide tabellen kan een lijst van nummers bewaren. Daarom komt er een tussentabel, die de relatie opknipt in twee één-op-

veel-relaties. Die tussentabel bewaart ook wat bij de koppeling zelf hoort: de `Hoeveelheid` van een ingrediënt in een recept.

Waarom een vraagteken bij elke navigatie. Een client stuurt bij een POST een recept zonder keuken-object, enkel met een `keukenId`. Zonder vraagteken maakt `[ApiController]` de navigatie verplicht, en krijgt de client een 400. En je repository laadt de navigaties niet mee: ze blijven `null` tot les 08. Zie hoofdstuk 17.

Stap 2: de tabellen erbij

Wat je doet. Geef de context een `DbSet` per nieuw model, en een blok in `OnModelCreating` met de tabelnaam en de regels voor de kolommen.

```
public class ReceptContext : DbContext
{
    public ReceptContext(DbContextOptions<ReceptContext> options)
        : base(options) { }

    public DbSet<Recept> Recepten { get; set; }
    public DbSet<Keuken> Keukens { get; set; }
    public DbSet<Ingredient> Ingredienten { get; set; }
    public DbSet<ReceptIngredient> ReceptIngredienten { get; set; }

    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        base.OnModelCreating(modelBuilder);

        modelBuilder.Entity<Recept>(entity =>
        {
            entity.ToTable("Recepten");
            entity.Property(r => r.Titel).IsRequired().HasMaxLength(100);
            entity.Property(r => r.Kostprijs).HasPrecision(8, 2);
        });

        modelBuilder.Entity<Keuken>(entity =>
        {
            entity.ToTable("Keukens");
            entity.Property(k => k.Naam).IsRequired().HasMaxLength(50);
        });

        modelBuilder.Entity<Ingredient>(entity =>
        {
            entity.ToTable("Ingredienten");
            entity.Property(i => i.Naam).IsRequired().HasMaxLength(50);
        });

        modelBuilder.Entity<ReceptIngredient>(entity =>
        {
            entity.ToTable("ReceptIngredienten");
            entity.Property(ri => ri.Hoeveelheid).IsRequired().HasMaxLength(30);
        });
    }
}
```

> **BELANGRIJK**
>
> **Begin met een lege database.** Je nieuwe migratie geeft de tabel `Recepten` een kolom `KeukenId`. Elk recept dat er al in staat, krijgt daar 0 in. Een keuken met nummer 0 bestaat niet, dus de database weigert de vreemde sleutel, en `Update-Database` mislukt.
>
> Wis daarom eerst je database met `Drop-Database`, en bevestig. Je migraties bouwen ze straks volledig opnieuw op, en je startgegevens komen in stap 4.

Maak dan de migratie, en voer ze uit:

```
Add-Migration TabellenErbij
Update-Database
```

Lees de migratie na. Je vindt er meer in dan je misschien verwacht. Dit is het deel over `Recepten`:

```
migrationBuilder.AddColumn<int>(
    name: "KeukenId",
    table: "Recepten",
    type: "integer",
    nullable: false,
    defaultValue: 0);

// ... CreateTable voor Keukens, Ingredienten en ReceptIngredienten

migrationBuilder.AddForeignKey(
    name: "FK_Recepten_Keukens_KeukenId",
    table: "Recepten",
    column: "KeukenId",
    principalTable: "Keukens",
    principalColumn: "Id",
    onDelete: ReferentialAction.Cascade);
```

- `AddColumn` **met** `defaultValue: 0`**:** daarom moest je database leeg zijn.
- `AddForeignKey`**:** de relatie staat er al, terwijl je in je context nog niets over relaties schreef. EF Core vond ze zelf, via zijn afspraken: een eigenschap `KeukenId` naast een navigatie `Keuken`.
- `onDelete: ReferentialAction.Cascade`**:** omdat `KeukenId` een `int` is, en dus verplicht, kiest EF Core zelf voor `Cascade`. Verwijder je een keuken, dan verwijdert de database al haar recepten mee. In stap 3 verander je dat.

Zo weet je dat het lukte. In pgAdmin staan vier tabellen. Bij `Recepten`, onder **Constraints**, staat een vreemde sleutel naar `Keukens`.

Stap 3: de relaties met de Fluent API

Wat je doet. Schrijf de relaties zelf uit in `OnModelCreating`. Twee blokken krijgen er regels bij:

```
modelBuilder.Entity<Recept>(entity =>
{
    entity.ToTable("Recepten");
    entity.Property(r => r.Titel).IsRequired().HasMaxLength(100);
    entity.Property(r => r.Kostprijs).HasPrecision(8, 2);

    entity.HasOne(r => r.Keuken)
          .WithMany(k => k.Recepten)
          .HasForeignKey(r => r.KeukenId)
          .OnDelete(DeleteBehavior.Restrict);
});

modelBuilder.Entity<ReceptIngredient>(entity =>
{
    entity.ToTable("ReceptIngredienten");
    entity.Property(ri => ri.Hoeveelheid).IsRequired().HasMaxLength(30);

    entity.HasOne(ri => ri.Recept)
          .WithMany(r => r.ReceptIngredienten)
          .HasForeignKey(ri => ri.ReceptId)
          .OnDelete(DeleteBehavior.Restrict);

    entity.HasOne(ri => ri.Ingredient)
          .WithMany(i => i.ReceptIngredienten)
          .HasForeignKey(ri => ri.IngredientId)
          .OnDelete(DeleteBehavior.Restrict);
});
```

Hoe je het leest. Altijd vanuit de entiteit van het blok. In het blok van `Recept`:

Regel	Lees als
`entity.HasOne(r => r.Keuken)`	een recept heeft één keuken
`.WithMany(k => k.Recepten)`	die keuken heeft veel recepten
`.HasForeignKey(r => r.KeukenId)`	de kolom die ze verbindt, is `KeukenId`
`.OnDelete(DeleteBehavior.Restrict)`	een keuken met recepten mag niet weg

De tussentabel krijgt twee zulke regels: één naar elke kant. Samen maken ze de veel-op-veel-relatie.

Waarom zelf uitschrijven, als EF Core de relatie al vond? Om twee redenen. Je kiest zelf wat er bij verwijderen gebeurt. En wie je context leest, ziet meteen welke relaties er zijn, zonder de afspraken van EF Core te kennen.

Cascade tegenover Restrict:

	Je verwijdert een keuken met recepten	Wanneer kiezen
`Cascade`	de database verwijdert ook alle recepten van die keuken, zonder te waarschuwen	bijna nooit voor gegevens die ertoe doen
`Restrict`	de database weigert, en de keuken blijft	de les kiest dit overal

De les vertelt het verhaal van klant Gerrit: met `Cascade` verdwijnen met zijn account ook al zijn bestellingen van vijf jaar. Met `Restrict` houdt de database je tegen.

Maak de migratie, en voer ze uit:

```
Add-Migration RelatiesErbij
Update-Database
```

Wat erin staat. Enkel wat veranderde: het gedrag bij verwijderen. Per vreemde sleutel zie je dit:

```
migrationBuilder.DropForeignKey(
    name: "FK_Recepten_Keukens_KeukenId",
    table: "Recepten");

migrationBuilder.AddForeignKey(
    name: "FK_Recepten_Keukens_KeukenId",
    table: "Recepten",
    column: "KeukenId",
    principalTable: "Keukens",
    principalColumn: "Id",
    onDelete: ReferentialAction.Restrict);
```

De database kan het gedrag van een vreemde sleutel niet zomaar aanpassen. Daarom haalt de migratie ze weg, en zet ze ze opnieuw, met `Restrict`.

Stap 4: de startgegevens verhuizen

Waar ze vandaan komen. In les 03 stonden je startrecepten in de lijst van de repository:

```
private static List<Recept> _recepten = new List<Recept>
{
    new Recept { Id = 1, Titel = "Lasagne", Kostprijs = 12.50m },
    new Recept { Id = 2, Titel = "Pad thai", Kostprijs = 9.80m },
    new Recept { Id = 3, Titel = "Linzensoep", Kostprijs = 4.20m }
};
```

In les 05 verdween die lijst, en de tabel bleef leeg. Nu komen ze terug, in `OnModelCreating`, met een keuken erbij:

```
modelBuilder.Entity<Keuken>().HasData(
    new Keuken { Id = 1, Naam = "Italiaans" },
    new Keuken { Id = 2, Naam = "Thais" },
```

```
    new Keuken { Id = 3, Naam = "Belgisch" });

modelBuilder.Entity<Recept>().HasData(
    new Recept { Id = 1, Titel = "Lasagne", Kostprijs = 12.50m, KeukenId = 1 },
    new Recept { Id = 2, Titel = "Pad thai", Kostprijs = 9.80m, KeukenId = 2 },
    new Recept { Id = 3, Titel = "Linzensoep", Kostprijs = 4.20m, KeukenId = 3 });

modelBuilder.Entity<Ingredient>().HasData(
    new Ingredient { Id = 1, Naam = "Pasta" },
    new Ingredient { Id = 2, Naam = "Tomaat" },
    new Ingredient { Id = 3, Naam = "Rijstnoedels" },
    new Ingredient { Id = 4, Naam = "Linzen" });

modelBuilder.Entity<ReceptIngredient>().HasData(
    new ReceptIngredient { Id = 1, ReceptId = 1, IngredientId = 1,
                           Hoeveelheid = "250 g" },
    new ReceptIngredient { Id = 2, ReceptId = 1, IngredientId = 2,
                           Hoeveelheid = "400 g" },
    new ReceptIngredient { Id = 3, ReceptId = 2, IngredientId = 3,
                           Hoeveelheid = "200 g" },
    new ReceptIngredient { Id = 4, ReceptId = 3, IngredientId = 4,
                           Hoeveelheid = "300 g" },
    new ReceptIngredient { Id = 5, ReceptId = 3, IngredientId = 2,
                           Hoeveelheid = "1 stuk" });
```

Waarom in de context, en niet meer in de repository. Startgegevens horen bij de database, niet bij één klasse die ze toevallig gebruikt. Met `HasData` komen ze mee met je migraties: wie je project opbouwt met `Update-Database`, heeft meteen dezelfde gegevens. Zonder SQL-script, en zonder iets met de hand toe te voegen.

De regels van `HasData`:

Regel	Waarom
elk startgegeven krijgt zelf een `Id`	EF Core moet de rij later kunnen terugvinden, om ze bij te werken of te verwijderen in een volgende migratie
een relatie geef je mee met de vreemde sleutel: `KeukenId = 1`	een object, zoals `Keuken = new Keuken {...}`, mag niet in `HasData`
een `decimal` krijgt een `m` achter het getal	zonder `m` is `12.50` een `double`, en bouwt je project niet. Zie hoofdstuk 2
het nummer in een vreemde sleutel moet bestaan	anders weigert de database bij `Update-Database`

Maak de migratie, en voer ze uit:

```
Add-Migration Startgegevens
Update-Database
```

In de migratie zie je per tabel een `InsertData`, bijvoorbeeld:

```
migrationBuilder.InsertData(
    table: "Keukens",
    columns: new[] { "Id", "Naam" },
    values: new object[,]
    {
        { 1, "Italiaans" },
        { 2, "Thais" },
        { 3, "Belgisch" }
    });
```

> **BELANGRIJK**
>
> Startgegevens met een vaste Id vertellen de teller van PostgreSQL niet welke nummers al gebruikt zijn. De eerste keuken die je daarna via de API toevoegt, kan daardoor een nummer krijgen dat al bestaat. Dan krijg je een 500 over een dubbele sleutel. Hoofdstuk 31 legt uit hoe je dat oplost.

Stap 5: OnModelCreating opsplitsen

Na stap 4 staat alles in één methode: vier blokken die tabellen opbouwen, en vier blokken met startgegevens. Ze past niet meer op één scherm. De les geeft een vuistregel: een methode is idealiter niet langer dan één schermlengte. Is ze langer, dan is het tijd om op te splitsen.

Wat je doet. Je splitst `OnModelCreating` in twee methodes, elk met één taak:

- `GenerateTables`: alles wat de tabellen en hun relaties opbouwt;
- `SeedData`: alle startgegevens.

Zo ziet `OnModelCreating` eruit voor je splitst. **Blauw** gaat naar `GenerateTables`, **oranje** naar `SeedData`. Wat gewoon blijft, blijft in `OnModelCreating`:

```
protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    base.OnModelCreating(modelBuilder);

    modelBuilder.Entity<Recept>(entity =>
    {
        entity.ToTable("Recepten");
        entity.Property(r => r.Titel).IsRequired().HasMaxLength(100);
        entity.Property(r => r.Kostprijs).HasPrecision(8, 2);

        entity.HasOne(r => r.Keuken)
              .WithMany(k => k.Recepten)
              .HasForeignKey(r => r.KeukenId)
              .OnDelete(DeleteBehavior.Restrict);
    });

    modelBuilder.Entity<Keuken>(entity =>
    {
        entity.ToTable("Keukens");
```

```
        entity.Property(k => k.Naam).IsRequired().HasMaxLength(50);
    });

    modelBuilder.Entity<Ingredient>(entity =>
    {
        entity.ToTable("Ingredienten");
        entity.Property(i => i.Naam).IsRequired().HasMaxLength(50);
    });

    modelBuilder.Entity<ReceptIngredient>(entity =>
    {
        entity.ToTable("ReceptIngredienten");
        entity.Property(ri => ri.Hoeveelheid).IsRequired().HasMaxLength(30);

        entity.HasOne(ri => ri.Recept)
              .WithMany(r => r.ReceptIngredienten)
              .HasForeignKey(ri => ri.ReceptId)
              .OnDelete(DeleteBehavior.Restrict);

        entity.HasOne(ri => ri.Ingredient)
              .WithMany(i => i.ReceptIngredienten)
              .HasForeignKey(ri => ri.IngredientId)
              .OnDelete(DeleteBehavior.Restrict);
    });

    modelBuilder.Entity<Keuken>().HasData(
        new Keuken { Id = 1, Naam = "Italiaans" },
        new Keuken { Id = 2, Naam = "Thais" },
        new Keuken { Id = 3, Naam = "Belgisch" });

    modelBuilder.Entity<Recept>().HasData(
        // ... de drie recepten uit stap 4
    );

    modelBuilder.Entity<Ingredient>().HasData(
        // ... de vier ingrediënten uit stap 4
    );

    modelBuilder.Entity<ReceptIngredient>().HasData(
        // ... de vijf koppelingen uit stap 4
    );
}
```

Ná, drie methodes. `OnModelCreating` roept de twee andere op:

```
protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    base.OnModelCreating(modelBuilder);
    GenerateTables(modelBuilder);
    SeedData(modelBuilder);
}

private void GenerateTables(ModelBuilder modelBuilder)
{
    modelBuilder.Entity<Recept>(entity =>
    {
        // ... alles van het blok voor Recept
    });

    // ... de blokken voor Keuken, Ingredient en ReceptIngredient
}

private void SeedData(ModelBuilder modelBuilder)
{
    modelBuilder.Entity<Keuken>().HasData(
        // ... de drie keukens
    );

    // ... de recepten, de ingrediënten en de koppelingen
}
```

De volledige context staat bij het eindresultaat.

Waarom opsplitsen. Elke methode heeft nu één taak. Wil je een kolom aanpassen, dan zoek je in `GenerateTables`. Wil je een startgegeven bijzetten, dan zoek je in `SeedData`. `OnModelCreating` leest als een inhoudstafel. Dat heet het **Single Responsibility Principle**: één verantwoordelijkheid per stuk code. In les 03 splitste je een klasse op. Hier splits je een methode op, om dezelfde reden.

Waarom het zo geschreven is:

- `base.OnModelCreating(modelBuilder);` **blijft bovenaan in** `OnModelCreating`**.** Het hoort bij het overschrijven van de methode, niet bij je tabellen of je gegevens.
- **Beide methodes krijgen** `modelBuilder` **als parameter.** Ze hebben hem nodig, en hij bestaat enkel in `OnModelCreating`. Daarom geeft `OnModelCreating` hem door.
- **Ze zijn** `private`**.** Enkel de context zelf roept ze op. EF Core kent alleen `OnModelCreating`.
- **Eerst** `GenerateTables`**, dan** `SeedData`**.** Voor EF Core maakt de volgorde niet uit: het leest alles, en bouwt dan het model. Voor wie leest wel: eerst hoe de tabellen eruitzien, dan wat erin komt.

Visual Studio doet het werk. Selecteer de blauwe blokken, en druk **Ctrl+R, Ctrl+M** (**Extract Method**). Visual Studio maakt er een methode van, met `modelBuilder` als parameter, en zet de oproep op de plaats waar de code stond. Geef de methode de naam `GenerateTables`. Doe hetzelfde met de oranje blokken, en noem die methode `SeedData`.

Is er een migratie nodig? Nee. Je verplaatste enkel code, en het model is hetzelfde gebleven. Een migratie zou leeg zijn. Zie vraag 2.

Stap 6: een repository, de klasse eerst

Wat je doet. Maak in de map `Repositories` een klasse `KeukenRepository`, asynchroon, zoals `ReceptRepository` uit les 05. Schrijf de klasse eerst, nog zonder interface:

```
using Microsoft.EntityFrameworkCore;
using ReceptenApi.Data;
using ReceptenApi.Models;

namespace ReceptenApi.Repositories;

public class KeukenRepository
{
    private readonly ReceptContext _context;

    public KeukenRepository(ReceptContext context)
    {
        _context = context;
    }

    public async Task<List<Keuken>> GetAllAsync()
    {
        return await _context.Keukens.ToListAsync();
    }

    public async Task<Keuken?> GetByIdAsync(int id)
    {
        return await _context.Keukens.FindAsync(id);
    }

    public async Task<Keuken> CreateAsync(Keuken keuken)
    {
        _context.Keukens.Add(keuken);
        await _context.SaveChangesAsync();
        return keuken;
    }

    public async Task UpdateAsync(int id, Keuken keuken)
    {
        Keuken? bestaand = await _context.Keukens.FindAsync(id);

        if (bestaand != null)
        {
            bestaand.Naam = keuken.Naam;
            await _context.SaveChangesAsync();
        }
    }

    public async Task DeleteAsync(int id)
    {
        Keuken? keuken = await _context.Keukens.FindAsync(id);
```

```
        if (keuken != null)
        {
            _context.Keukens.Remove(keuken);
            await _context.SaveChangesAsync();
        }
    }
}
```

Zet dan je cursor op de naam van de klasse, en druk **Ctrl+R, Ctrl+I** (**Extract Interface**). Vink alle methodes aan. Visual Studio maakt dit bestand:

```
using ReceptenApi.Models;

namespace ReceptenApi.Repositories;

public interface IKeukenRepository
{
    Task<List<Keuken>> GetAllAsync();
    Task<Keuken?> GetByIdAsync(int id);
    Task<Keuken> CreateAsync(Keuken keuken);
    Task UpdateAsync(int id, Keuken keuken);
    Task DeleteAsync(int id);
}
```

En het past de klasse aan:

```
public class KeukenRepository : IKeukenRepository
```

Waarom het zo kan. Een interface bevat de koppen van de publieke methodes. Die staan al in je klasse. Visual Studio schrijft ze over, zonder body. Het resultaat is hetzelfde als wanneer je eerst de interface typte. Staan de methodes bij jou in een andere volgorde, dan maakt dat niet uit.

De twee richtingen naast elkaar:

	In les 03	In les 06
eerst	de interface: welke vragen stelt de controller?	de klasse, met alle methodes
dan	de klasse, met **Ctrl+.**: **Implement interface**	de interface, met **Ctrl+R, Ctrl+I**: **Extract Interface**
handig als	je nog moet uitzoeken wat er in het contract hoort	je al precies weet wat de klasse moet doen, zoals hier: ze lijkt op `ReceptRepository`

Waarom ze zo op `ReceptRepository` lijkt. Elke repository doet hetzelfde: alles ophalen, één ophalen, toevoegen, bijwerken, verwijderen. Enkel de tabel en het model verschillen. In les 07 haal je die herhaling weg, met één generieke repository.

Stap 7: registreren in Program.cs

Vóór:

```
builder.Services.AddScoped<IReceptRepository, ReceptRepository>();
```

Ná:

```
builder.Services.AddScoped<IReceptRepository, ReceptRepository>();
builder.Services.AddScoped<IKeukenRepository, KeukenRepository>();
```

Waarom. Zoals in les 03: wie een `IKeukenRepository` vraagt, moet weten welke klasse hij krijgt. Elke repository krijgt haar eigen regel, vóór `builder.Build()`. In les 07 verdwijnen ze allemaal weer.

Stap 8: de controller

Wat je doet. Maak in de map `Controllers` een `KeukenController`. Hij werkt net als `ReceptController`.

```
using Microsoft.AspNetCore.Mvc;
using ReceptenApi.Models;
using ReceptenApi.Repositories;

namespace ReceptenApi.Controllers;

[ApiController]
[Route("api/[controller]")]
public class KeukenController : ControllerBase
{
    private readonly IKeukenRepository _repository;

    public KeukenController(IKeukenRepository repository)
    {
        _repository = repository;
    }

    [HttpGet]
    public async Task<ActionResult<List<Keuken>>> GetAll()
    {
        return Ok(await _repository.GetAllAsync());
    }

    [HttpGet("{id}")]
    public async Task<ActionResult<Keuken>> GetById(int id)
    {
        Keuken? keuken = await _repository.GetByIdAsync(id);

        if (keuken == null)
        {
            return NotFound($"We vonden geen keuken met Id {id}.");
        }
```

```
        return Ok(keuken);
    }

    [HttpPost]
    public async Task<ActionResult<Keuken>> Create(Keuken nieuw)
    {
        Keuken gemaakt = await _repository.CreateAsync(nieuw);

        return CreatedAtAction(nameof(GetById), new { id = gemaakt.Id }, gemaakt);
    }

    [HttpPut("{id}")]
    public async Task<ActionResult> Update(int id, Keuken gewijzigd)
    {
        if (await _repository.GetByIdAsync(id) == null)
        {
            return NotFound($"We vonden geen keuken met Id {id}.");
        }

        await _repository.UpdateAsync(id, gewijzigd);

        return NoContent();
    }

    [HttpDelete("{id}")]
    public async Task<ActionResult> Delete(int id)
    {
        if (await _repository.GetByIdAsync(id) == null)
        {
            return NotFound($"We vonden geen keuken met Id {id}.");
        }

        await _repository.DeleteAsync(id);

        return NoContent();
    }
}
```

De les geeft deze controller geen logger. Wil je er toch een, zie stap 9 van hoofdstuk 36.

> **LET OP**
>
> Een DELETE op een keuken die nog recepten heeft, geeft hier een 500. `Restrict` doet zijn werk: de database weigert, en die weigering wordt een fout. De keuken blijft wel bewaard. Wil je de client een duidelijk antwoord geven, dan moet de controller vóór `DeleteAsync` nagaan of er nog recepten aan de keuken hangen, en zelf een statuscode met een reden kiezen. Beslissen of iets mag, is de taak van de controller.

Zo weet je dat het lukte. In Scalar staat het endpoint `api/Keuken`. Een GET geeft de drie keukens uit je startgegevens.

Wat dit betekent voor je recepten

`Recept` kreeg drie eigenschappen bij. Dat merk je ook op plaatsen die je in deze les niet aanraakte:

- **De JSON van een recept** heeft nu ook `keukenId`, `keuken` en `receptIngredienten`. De twee navigaties zijn `null`: je repository laadt ze niet mee. Gerelateerde gegevens laden komt in les 08.
- **Een POST of PUT van een recept** moet een `keukenId` meesturen dat bestaat. Anders weigert de database, met *23503: violates foreign key constraint*, en krijgt de client een 500. Ook hier: wil je een duidelijk antwoord, dan kijkt de controller het vooraf na.
- **Scalar zet de navigaties in zijn voorbeeld.** Laat je `keuken` of `receptIngredienten` staan in je verzoek, dan probeert EF Core ook die objecten toe te voegen. Haal ze weg voor je een verzoek verstuurt.
- **Je repository neemt eigenschappen één voor één over** bij bijwerken. Krijgt een model een eigenschap bij, kijk dan welke methodes in je repository dat doen.

Het eindresultaat

De context, in `Data/ReceptContext.cs`:

```
using Microsoft.EntityFrameworkCore;
using ReceptenApi.Models;

namespace ReceptenApi.Data;

public class ReceptContext : DbContext
{
    public ReceptContext(DbContextOptions<ReceptContext> options)
        : base(options) { }

    public DbSet<Recept> Recepten { get; set; }
    public DbSet<Keuken> Keukens { get; set; }
    public DbSet<Ingredient> Ingredienten { get; set; }
    public DbSet<ReceptIngredient> ReceptIngredienten { get; set; }

    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        base.OnModelCreating(modelBuilder);
        GenerateTables(modelBuilder);
        SeedData(modelBuilder);
    }

    private void GenerateTables(ModelBuilder modelBuilder)
    {
        modelBuilder.Entity<Recept>(entity =>
        {
            entity.ToTable("Recepten");
            entity.Property(r => r.Titel).IsRequired().HasMaxLength(100);
            entity.Property(r => r.Kostprijs).HasPrecision(8, 2);

            entity.HasOne(r => r.Keuken)
```

```
                .WithMany(k => k.Recepten)
                .HasForeignKey(r => r.KeukenId)
                .OnDelete(DeleteBehavior.Restrict);
        });

        modelBuilder.Entity<Keuken>(entity =>
        {
            entity.ToTable("Keukens");
            entity.Property(k => k.Naam).IsRequired().HasMaxLength(50);
        });

        modelBuilder.Entity<Ingredient>(entity =>
        {
            entity.ToTable("Ingredienten");
            entity.Property(i => i.Naam).IsRequired().HasMaxLength(50);
        });

        modelBuilder.Entity<ReceptIngredient>(entity =>
        {
            entity.ToTable("ReceptIngredienten");
            entity.Property(ri => ri.Hoeveelheid)
                  .IsRequired().HasMaxLength(30);

            entity.HasOne(ri => ri.Recept)
                  .WithMany(r => r.ReceptIngredienten)
                  .HasForeignKey(ri => ri.ReceptId)
                  .OnDelete(DeleteBehavior.Restrict);

            entity.HasOne(ri => ri.Ingredient)
                  .WithMany(i => i.ReceptIngredienten)
                  .HasForeignKey(ri => ri.IngredientId)
                  .OnDelete(DeleteBehavior.Restrict);
        });
    }

    private void SeedData(ModelBuilder modelBuilder)
    {
        modelBuilder.Entity<Keuken>().HasData(
            new Keuken { Id = 1, Naam = "Italiaans" },
            new Keuken { Id = 2, Naam = "Thais" },
            new Keuken { Id = 3, Naam = "Belgisch" });

        modelBuilder.Entity<Recept>().HasData(
            new Recept { Id = 1, Titel = "Lasagne", Kostprijs = 12.50m,
                         KeukenId = 1 },
            new Recept { Id = 2, Titel = "Pad thai", Kostprijs = 9.80m,
                         KeukenId = 2 },
            new Recept { Id = 3, Titel = "Linzensoep", Kostprijs = 4.20m,
                         KeukenId = 3 });

        modelBuilder.Entity<Ingredient>().HasData(
            new Ingredient { Id = 1, Naam = "Pasta" },
            new Ingredient { Id = 2, Naam = "Tomaat" },
            new Ingredient { Id = 3, Naam = "Rijstnoedels" },
            new Ingredient { Id = 4, Naam = "Linzen" });

        modelBuilder.Entity<ReceptIngredient>().HasData(
            new ReceptIngredient { Id = 1, ReceptId = 1, IngredientId = 1,
```

```
                                    Hoeveelheid = "250 g" },
            new ReceptIngredient { Id = 2, ReceptId = 1, IngredientId = 2,
                                    Hoeveelheid = "400 g" },
            new ReceptIngredient { Id = 3, ReceptId = 2, IngredientId = 3,
                                    Hoeveelheid = "200 g" },
            new ReceptIngredient { Id = 4, ReceptId = 3, IngredientId = 4,
                                    Hoeveelheid = "300 g" },
            new ReceptIngredient { Id = 5, ReceptId = 3, IngredientId = 2,
                                    Hoeveelheid = "1 stuk" });
    }
}
```

De modellen staan bij stap 1, de repository en haar interface bij stap 6, de controller bij stap 8.

Program.cs na deze les

Ten opzichte van les 05 komt er één regel bij:

```
builder.Services.AddScoped<IReceptRepository, ReceptRepository>();
builder.Services.AddScoped<IKeukenRepository, KeukenRepository>();
```

De rest van `Program.cs` blijft zoals na les 05.

Hoe je ziet dat het werkt

- Na elke `Update-Database` zie je in pgAdmin wat er veranderde: nieuwe tabellen, en bij **Constraints** de vreemde sleutels.
- In de migratie van de relaties staat bij elke vreemde sleutel `onDelete: ReferentialAction.Restrict`.
- Je startgegevens staan in de tabellen.
- In Scalar staat het nieuwe endpoint. Een GET geeft de keukens, maar hun recepten zijn `null`. Gerelateerde gegevens laden komt in les 08.
- Je recepten staan er weer, nu met een `keukenId`.

Alle stappen op een rij

1. Maak de nieuwe modellen, met hun vreemde sleutels en navigaties. Een navigatie krijgt een vraagteken.
2. Geef de context een `DbSet` per model, en een blok met `ToTable` per entiteit. Wis eerst je database met `Drop-Database`. Maak de migratie `TabellenErbij`, lees ze na, en voer ze uit.
3. Leg de relaties vast in `OnModelCreating`: `HasOne`, `WithMany`, `HasForeignKey` en `OnDelete(DeleteBehavior.Restrict)`. Een tussentabel krijgt twee relaties. Migratie `RelatiesErbij`, en `Update-Database`.
4. Geef startgegevens mee met `HasData`: elk een eigen Id, relaties via de vreemde sleutel, een `m` achter een `decimal`. Migratie `Startgegevens`, en `Update-Database`.
5. Splits `OnModelCreating` in `GenerateTables` en `SeedData`, met **Ctrl+R, Ctrl+M**. Geen migratie nodig.

6. Schrijf de klasse `KeukenRepository`, asynchroon, en laat Visual Studio er de interface van maken met **Ctrl+R, Ctrl+I**.
7. Registreer de nieuwe repository in `Program.cs`.
8. Maak een `KeukenController`.
9. Open Scalar: het nieuwe endpoint staat erbij.

Als een stap ontbreekt

Wat je ziet	Waarom	Wat je doet
`Update-Database` na `TabellenErbij`: *23503: insert or update on table "Recepten" violates foreign key constraint "FK_Recepten_Keukens_KeukenId"*	er stonden al recepten in de tabel, en ze kregen `KeukenId` 0	de database wissen met `Drop-Database`, en opnieuw `Update-Database`
`Add-Migration`: *The seed entity for entity type 'Keuken' cannot be added because a default value was provided for the required property 'Id'.*	een startgegeven zonder Id	elk startgegeven een eigen Id geven
`Add-Migration`: een fout dat de seed entity *cannot be added because it has the navigation 'Keuken' set*	een object meegegeven in plaats van de vreemde sleutel	enkel `KeukenId` meegeven
`Update-Database`: *23503: insert or update on table "Recepten" violates foreign key constraint*	een startgegeven verwijst naar een Id dat niet bestaat	de vreemde sleutel verbeteren
`Update-Database`: *23505: duplicate key value violates unique constraint*	dezelfde startgegevens staan er al, bijvoorbeeld via een SQL-script én `HasData`	één van de twee kiezen
de eerste POST na de startgegevens: 500 met *23505: duplicate key value violates unique constraint "PK_Keukens"*	de teller van PostgreSQL	zie het kader bij stap 4
een DELETE geeft 500 met *23503: update or delete on table "Keukens" violates foreign key constraint*	`Restrict` doet zijn werk: er hangen nog recepten aan	zie het kader bij stap 8
een POST geeft 400 met *The Recepten field is required.*	een navigatie zonder vraagteken. `[ApiController]` maakt ze dan verplicht	een vraagteken zetten, zoals in stap 1
na een POST staat er een keuken *string* bij	het voorbeeld van Scalar stuurde de navigatie mee	de navigaties weglaten uit je verzoek
elk verzoek naar de nieuwe controller geeft 500 met *Unable to resolve service*	de nieuwe repository is niet geregistreerd	`AddScoped` toevoegen
CS0535 in `KeukenRepository` na **Extract Interface**	een methode niet aangevinkt, of later een methode toegevoegd aan de interface	de interface en de klasse gelijk maken

Let op bij de lescode

LET OP

- De configuratie van `OrderLijn` zet de relatie met `Bestelling` twee keer, en die met `Product` niet. EF Core maakt die laatste toch, via zijn afspraken, maar dan met `Cascade` in plaats van `Restrict`. Kijk in je migratie welke `onDelete` er bij elke vreemde sleutel staat.
- `UpdateAsync` in de les doet `existingBestelling = updatedBestelling;`. Dat verandert enkel de variabele. EF Core ziet geen wijziging, en de PUT geeft 204 terwijl er niets bewaard is. Neem de eigenschappen één voor één over, zoals in les 05.
- De PUT in de les kijkt niet of de bestelling bestaat.
- `Klant? Klant { get; set; } = default!;` zet een vraagteken én `default!` op dezelfde eigenschap. Ze spreken elkaar tegen: kies één van de twee. Voor een navigatie is dat het vraagteken.
- Het bestand heet `BestellingenController.cs`, maar de klasse `BestellingController`. De route komt van de klasse: `api/Bestelling`, niet `api/bestellingen` zoals het commentaar zegt.
- Heeft een model een `DateTime`, dan kan PostgreSQL een startgegeven weigeren met *Cannot write DateTime with Kind=Unspecified to PostgreSQL type 'timestamp with time zone', only UTC is supported.* Geef de datum dan mee met `DateTimeKind.Utc`. Zie hoofdstuk 3.

Waar het uitgelegd staat

Onderwerp	Hoofdstuk
relaties, de Fluent API, `Cascade` en `Restrict`, `HasData`	31
navigaties met een vraagteken of `= default!`	17
`decimal` en de `m`	2
datums	3
een interface laten volgen door een klasse	19

Voorspel de uitkomst

Vraag 1

In `SeedData` staat een recept met `KeukenId = 4`, maar er zijn enkel keukens met Id 1, 2 en 3. `Add-Migration` lukt. Wat gebeurt er bij `Update-Database`?

Toon het antwoord ›

Vraag 2

Je splitst `OnModelCreating` in `GenerateTables` en `SeedData`, zoals in stap 5. Je maakt daarna toch een nieuwe migratie. Wat staat erin?

Toon het antwoord ›

Vraag 3

Een recept met Id 1 hoort bij keuken 1. De relatie staat op `Restrict`. Een DELETE op keuken 1 kijkt niet vooraf of er nog recepten zijn. Wat krijgt de client?

Toon het antwoord ›

Vraag 4

Je slaat het kader bij stap 2 over. In je tabel `Recepten` staan nog twee recepten uit les 05. Je maakt de migratie `TabellenErbij`, en typt `Update-Database`. Wat gebeurt er?

Toon het antwoord ›

Vraag 5

Je maakt na stap 2 nog geen relaties met de Fluent API. Staat er toch al een vreemde sleutel tussen `Recepten` en `Keukens` in de database? En wat gebeurt er met de recepten als je dan een keuken verwijdert?

Toon het antwoord ›

Antwoorden

VRAAG 1

De database weigert, met *23503: insert or update on table "Recepten" violates foreign key constraint*. De migratie wordt niet uitgevoerd.

EF Core controleert bij het maken van de migratie niet of keuken 4 bestaat. Dat doet pas de database.

‹ Terug naar de vraag

VRAAG 2

Niets: een `Up` en een `Down` zonder inhoud. Je verplaatste enkel code, en het model is hetzelfde gebleven. Een migratie bevat enkel wat er in het model veranderd is. Zo'n lege migratie haal je weg met `Remove-Migration`.

‹ Terug naar de vraag

VRAAG 3

500 Internal Server Error. De database weigert de keuken te verwijderen, met *23503: update or delete on table "Keukens" violates foreign key constraint*.

`Restrict` beschermt je gegevens, maar het is de database die weigert, en dat wordt een 500. De controller moet vooraf nagaan of het mag, en zelf een duidelijk antwoord kiezen.

‹ Terug naar de vraag

VRAAG 4

Update-Database mislukt, met *23503: insert or update on table "Recepten" violates foreign key constraint "FK_Recepten_Keukens_KeukenId"*.

De migratie geeft de bestaande recepten `KeukenId` 0. Daarna probeert ze de vreemde sleutel te maken, maar keuken 0 bestaat niet. Wis de database met `Drop-Database`, en voer `Update-Database` opnieuw uit.

‹ Terug naar de vraag

VRAAG 5

Ja, de vreemde sleutel staat er al, met `Cascade`. Verwijder je een keuken, dan verwijdert de database al haar recepten mee.

EF Core vond de relatie zelf, via zijn afspraken: `KeukenId` naast de navigatie `Keuken`. Omdat `KeukenId` een `int` is, is de relatie verplicht, en dan kiest EF Core voor `Cascade`. Daarom schrijf je de relatie in stap 3 zelf uit, met `Restrict`.

‹ Terug naar de vraag

40. Les 07: generieke repository en Unit of Work

DATA

Wat je bouwt

De vijf standaardbewerkingen schrijf je één keer, in een generieke repository. Een specifieke repository erft daarvan, en voegt enkel toe wat ontbreekt. Een Unit of Work biedt alle repositories aan en bewaart alles in één keer. Je controllers vragen enkel nog de Unit of Work.

Wat er bijkomt in je project

```
ReceptenApi/
├── Controllers/
│   ├── KeukenController.cs            ← gewijzigd: vraagt IUnitOfWork
│   └── ReceptController.cs            ← gewijzigd: vraagt IUnitOfWork
├── Data/
│   ├── IUnitOfWork.cs                 ← nieuw
│   └── UnitOfWork.cs                  ← nieuw
├── Repositories/
│   ├── GenericRepository.cs           ← nieuw
│   ├── IGenericRepository.cs          ← nieuw
│   ├── IKeukenRepository.cs           ← weg of leeg, zie stap 4
│   ├── IReceptRepository.cs           ← gewijzigd: erft, met de extra's
│   ├── KeukenRepository.cs            ← weg of leeg, zie stap 4
│   └── ReceptRepository.cs            ← gewijzigd: erft van de generieke
└── Program.cs                         ← gewijzigd
```

Stap voor stap

1. Maak `IGenericRepository<TEntity>` met `where TEntity : class`. Ze belooft vijf bewerkingen: alles ophalen, één ophalen op Id, toevoegen, bijwerken en verwijderen.
2. Maak `GenericRepository<TEntity>`, die ze belooft. De context is een `protected` veld, zodat ook de klassen die ervan erven erbij kunnen.
3. Laat de specifieke repository erven. De interface erft van `IGenericRepository<Recept>` en houdt enkel de extra methodes. De klasse erft van `GenericRepository<Recept>` en belooft de interface. Haar constructor geeft de context door met `: base(context)`.
4. Een model zonder extra methodes heeft geen eigen repository meer nodig. De Unit of Work geeft er de generieke voor. De les maakt voor een ander model toch een lege specifieke repository, voor later.
5. Maak in de map `Data` de interface `IUnitOfWork`: één eigenschap per repository, en `SaveChangesAsync`.
6. Maak in `Data` de klasse `UnitOfWork`. Ze vraagt de context, en maakt elke repository pas als iemand ze opvraagt.
7. `Program.cs`: haal de registraties van de repositories weg, en registreer de Unit of Work. `AddDbContext` blijft.
8. De controllers vragen `IUnitOfWork` in plaats van een repository. Na toevoegen, bijwerken of verwijderen roepen ze `await _uow.SaveChangesAsync()` op.

De generieke interface:

```
public interface IGenericRepository<TEntity> where TEntity : class
{
    Task<IEnumerable<TEntity>> GetAllAsync();
    Task<TEntity?> GetByIdAsync(int id);
    void Add(TEntity entity);
    void Update(TEntity entity);
    void Delete(TEntity entity);
}
```

`Add`, `Update` en `Delete` zijn niet asynchroon en bewaren niets. Ze vertellen de context enkel wat er moet gebeuren. Het bewaren doet de Unit of Work. Dat is bewust.

Wat er in `GenericRepository` staat:

Methode	Wat erin staat
`GetAllAsync`	`_context.Set<TEntity>()`, en dan `ToListAsync()`
`GetByIdAsync`	`_context.Set<TEntity>()`, en dan `FindAsync(id)`
`Add`	`Add(entity)` op de `Set<TEntity>()`
`Update`	`Attach(entity)`, en de toestand van het object op `EntityState.Modified`
`Delete`	`Remove(entity)` op de `Set<TEntity>()`

`Set<TEntity>()` geeft de `DbSet` van het type dat ingevuld werd. Zo weet de generieke repository niet welke tabel ze krijgt, en werkt ze toch.

De Unit of Work:

```
public interface IUnitOfWork
{
    IReceptRepository ReceptRepository { get; }
    IGenericRepository<Keuken> KeukenRepository { get; }
    Task<int> SaveChangesAsync();
}

public class UnitOfWork : IUnitOfWork
{
    private readonly ReceptContext _context;
    private IReceptRepository? _receptRepository;
    private IGenericRepository<Keuken>? _keukenRepository;

    public UnitOfWork(ReceptContext context)
    {
        _context = context;
    }

    public IReceptRepository ReceptRepository =>
        _receptRepository ??= new ReceptRepository(_context);

    public IGenericRepository<Keuken> KeukenRepository =>
        _keukenRepository ??= new GenericRepository<Keuken>(_context);

    public async Task<int> SaveChangesAsync() =>
        await _context.SaveChangesAsync();
}
```

Elke repository krijgt dezelfde context. Daarom noteert die ene context alle wijzigingen, en bewaart één `SaveChangesAsync` ze allemaal samen: alles lukt, of niets.

> **BELANGRIJK**
>
> **Een PUT die eerst ophaalt en dan `Update` oproept.** Haal je eerst het bestaande recept op met `GetByIdAsync`, dan volgt de context dat object. Roep je daarna `Update` op met een ánder object met hetzelfde Id, dan probeert `Attach` dat tweede object ook te volgen. Je krijgt een 500 met *The instance of entity type 'Recept' cannot be tracked because another instance with the same key value for {'Id'} is already being tracked.*
>
> Werk dus met één object per Id. Zie hoofdstuk 32, bij wat de context noteert.

Program.cs na deze les

```
builder.Services.AddControllers();
builder.Services.AddOpenApi();

var connectionString = builder.Configuration.GetConnectionString("Standaard");
builder.Services.AddDbContext<ReceptContext>(options =>
    options.UseNpgsql(connectionString));

builder.Services.AddScoped<IUnitOfWork, UnitOfWork>();
```

Hoe je ziet dat het werkt

- `Program.cs` registreert enkel nog de context en de Unit of Work.
- Elke controller vraagt nog maar één ding in zijn constructor, naast de logger.
- Geen enkele repository roept nog zelf `SaveChangesAsync` op.
- Alle endpoints werken zoals voorheen. Een PUT wijzigt echt: een GET daarna toont de nieuwe gegevens.

Als een stap ontbreekt

Wat je ziet	Waarom	Wat je doet
een POST geeft 201, maar er staat niets in de database	`SaveChangesAsync` van de Unit of Work vergeten	na elke wijziging `await _uow.SaveChangesAsync()`
een PUT geeft 500 met *cannot be tracked because another instance with the same key value for {'Id'} is already being tracked*	twee objecten met hetzelfde Id	zie het kader hierboven
CS1061: *'IGenericRepository' does not contain a definition for 'GetVanKeukenAsync'*	de Unit of Work geeft de generieke repository, niet de specifieke	de eigenschap het type `IReceptRepository` geven, in de interface én de klasse
CS7036: *There is no argument given that corresponds to the required parameter 'context' of 'GenericRepository.GenericRepository(ReceptContext)'*	`: base(context)` vergeten	de context doorgeven aan de basisklasse
CS0122: *'_context' is inaccessible due to its protection level*	het veld in `GenericRepository` is `private`	`protected` maken
CS0452: *The type 'X' must be a reference type in order to use it as parameter 'TEntity'*	een type dat geen klasse is	enkel modellen gebruiken
een controller geeft 500 met *Unable to resolve service for type 'IReceptRepository'*	de controller vraagt nog een repository, maar die registratie is weg	`IUnitOfWork` vragen
navigaties zijn `null` in de JSON	er wordt niets meegeladen	dat komt in les 08

Let op bij de lescode

LET OP

- De PUT in de les haalt eerst de laptop op en roept dan `Update` op met het object uit de body. Dat geeft de 500 uit het kader hierboven.
- `CreatedAtAction(nameof(CreateLaptopAsync), ...)` verwijst naar de POST zelf in plaats van naar de GET, en heeft het probleem met `Async` uit les 05. Verwijs naar je GET met een Id.
- De les maakt een lege `BestellingRepository`, maar de Unit of Work gebruikt `new GenericRepository<Bestelling>`. De lege klasse wordt dus nog niet gebruikt. Les 08 verandert dat.
- `IUnitOfWork` en `UnitOfWork` staan in de map `Data`, maar met de namespace `...Repositories`. In les 08 wordt het `...Data`. Kies één en hou ze gelijk met de map.
- De les noemt het pas maken van een repository met `??=` *lazy loading*. Dat is iets anders dan lazy loading in EF Core. Hier gaat het enkel over een object pas maken als je het nodig hebt.

Waar het uitgelegd staat

Onderwerp	Hoofdstuk
generics en `where`	21
erven, `base(...)` en `protected`	20
`??=`	17
`Set<TEntity>()`, `Attach`, wat de context noteert, de Unit of Work	32
`IEnumerable<T>`	12

Voorspel de uitkomst

Vraag 1

In `UnitOfWork` staat `public IReceptRepository ReceptRepository => new ReceptRepository(_context);`, zonder `??=`. Een controller voegt een recept toe en roept daarna `SaveChangesAsync` op. Wordt het recept bewaard?

Toon het antwoord ›

Vraag 2

Een POST doet `_uow.ReceptRepository.Add(recept);` en geeft meteen `CreatedAtAction(nameof(GetById), new { id = recept.Id }, recept)` terug, zonder `SaveChangesAsync`. Wat staat er in de Location-header, en wat in de database?

Toon het antwoord ›

Antwoorden

VRAAG 1

Ja. Elke keer dat je de eigenschap opvraagt, krijg je een nieuwe repository. Maar ze krijgen allemaal dezelfde context, en die noteert de wijziging. `SaveChangesAsync` op die context bewaart ze. Met `??=` maak je enkel minder objecten.

‹ Terug naar de vraag

VRAAG 2

De Location-header eindigt op `/api/Recept/0`, en de database is leeg.

`Add` noteert het recept enkel. Pas bij `SaveChangesAsync` gaat het naar de database, en pas dan krijgt het een Id. Zonder bewaren blijft het Id 0.

‹ Terug naar de vraag

41. Les 08: Include en kringlopen

DATA

Wat je bouwt

Methodes in je specifieke repositories die gerelateerde gegevens in één keer meeladen, met `Include` en `ThenInclude`, en endpoints die ze gebruiken.

Daarbij komt een tijdelijke instelling die kringlopen in je JSON doorknipt. In les 09 gaat ze weer weg.

Wat er bijkomt in je project

```
ReceptenApi/
├── Controllers/
│   ├── KeukenController.cs            ← gewijzigd: nieuw endpoint
│   └── ReceptController.cs            ← gewijzigd: nieuw endpoint
├── Data/
│   ├── IUnitOfWork.cs                 ← gewijzigd: specifieke types
│   └── UnitOfWork.cs                  ← gewijzigd
├── Repositories/
│   ├── IKeukenRepository.cs           ← nieuw of gewijzigd
│   ├── IReceptRepository.cs           ← gewijzigd: een methode met Include
│   ├── KeukenRepository.cs            ← nieuw of gewijzigd
│   └── ReceptRepository.cs            ← gewijzigd
└── Program.cs                         ← gewijzigd: AddJsonOptions
```

Stap voor stap

1. Beslis per endpoint welke gegevens je samen nodig hebt.
2. Zet in de interface van de specifieke repository een methode die ze samen ophaalt, en schrijf ze in de klasse met `Include`, `ThenInclude` en `FirstOrDefaultAsync`. Zo'n methode hoort nooit in de generieke repository: die weet niets van relaties.
3. Krijgt een model nu een eigen methode, dan krijgt het een eigen interface en klasse die erven, zoals in les 07.
4. Pas de Unit of Work aan: die eigenschap krijgt het specifieke type, in de interface én in de klasse.
5. Maak het endpoint in de controller, bijvoorbeeld `[HttpGet("{id}/details")]`.
6. Test het endpoint. Krijg je een fout over een kringloop, zet dan in `Program.cs` tijdelijk `IgnoreCycles` aan.

Een methode met `Include`:

```
public async Task<Recept?> GetMetDetailsAsync(int id)
{
    return await _context.Recepten
        .Include(r => r.Keuken)
        .Include(r => r.ReceptIngredienten)
            .ThenInclude(ri => ri.Ingredient)
        .FirstOrDefaultAsync(r => r.Id == id);
}
```

In de Unit of Work wordt de eigenschap van de keukens dan:

```
IKeukenRepository KeukenRepository { get; }
```

Program.cs na deze les

```
builder.Services.AddControllers()
    .AddJsonOptions(options =>
    {
        // tijdelijk: knip kringlopen door (in les 09 gaat dit weer weg)
        options.JsonSerializerOptions.ReferenceHandler =
            System.Text.Json.Serialization.ReferenceHandler.IgnoreCycles;
    });
```

> **OPMERKING**
>
> Voor het examen zegt de les: `IgnoreCycles` volstaat daar niet, je gebruikt DTO's. Dat is les 09.
>
> De les toont ook Dapper. Op het examen moet je het begrip kunnen uitleggen, niet toepassen. Zie hoofdstuk 34.

Hoe je ziet dat het werkt

- Het nieuwe endpoint geeft het recept met zijn keuken en zijn ingrediënten ingevuld.
- In het terminalvenster staat per verzoek één *Executed DbCommand*: één vraag met JOIN's, geen reeks losse vragen.
- Met `IgnoreCycles` staat er in de JSON soms `null` waar je een object verwacht. Dat is de kringloop die doorgeknipt wordt, geen fout.
- Je oude endpoints geven hun navigaties nog altijd als `null`. Ze laden niets mee, en dat is juist.

Als een stap ontbreekt

Wat je ziet	Waarom	Wat je doet
de keuken is `null`, de lijst met ingrediënten is leeg	`Include` vergeten	`Include` voor elke navigatie die je nodig hebt
de koppelobjecten staan erin, maar elk `ingredient` is `null`	`ThenInclude` vergeten, of op het verkeerde niveau	`ThenInclude` onder de `Include` van de koppellijst
500 met *A possible object cycle was detected. This can either be due to a cycle or if the object depth is larger than the maximum allowed depth of 64.*	een kringloop in je JSON	tijdelijk `IgnoreCycles`, in les 09 DTO's
CS1061: *'IGenericRepository' does not contain a definition for 'GetMetReceptenAsync'*	de Unit of Work geeft nog de generieke repository	het specifieke type geven, in de interface én de klasse
CS1061 op `TEntity` in de generieke repository	`Include` in de generieke repository gezet	de methode naar de specifieke repository verhuizen

Let op bij de lescode

LET OP

- `CreatedAtAction(nameof(PostBestelling), ...)` verwijst naar de POST zelf. Die controller heeft geen GET met een Id. Verwijs naar een GET die bestaat.
- De PUT kijkt niet of de bestelling bestaat.
- `private IKlantRepository _klantRepository;` heeft geen vraagteken, en geeft daardoor waarschuwing CS8618. De andere velden in de Unit of Work hebben er wel een.
- `IKlantRepository` belooft `Task<Klant>`, de klasse geeft `Task<Klant?>`. Zet het vraagteken ook in de interface.
- De les zegt dat EF Core standaard aan lazy loading doet. In EF Core wordt een navigatie zonder `Include` gewoon niet geladen: ze blijft `null`.
- De Dapper-code in de les volgt de interface niet, en gebruikt SQL van SQL Server (`SCOPE_IDENTITY`). Neem ze niet over.
- De namespace van `IUnitOfWork` en `UnitOfWork` verandert hier van `...Repositories` naar `...Data`.

Waar het uitgelegd staat

Onderwerp	Hoofdstuk
`Include`, `ThenInclude` en kringlopen in JSON	32
Dapper	34
`FirstOrDefault` en lambda's	11

Voorspel de uitkomst

Vraag 1

`GetMetDetailsAsync` heeft enkel `.Include(r => r.ReceptIngredienten)`, zonder `ThenInclude`. Wat staat er bij elk koppelobject in de JSON?

Toon het antwoord ›

Vraag 2

Je zet een methode met `.Include(r => r.Keuken)` in `GenericRepository<TEntity>`, zodat alle repositories ze hebben. Wat gebeurt er?

Toon het antwoord ›

Vraag 3

Zonder `IgnoreCycles` geeft een endpoint een recept terug, opgehaald met `.Include(r => r.Keuken)`. Wat krijgt de client?

Toon het antwoord ›

Antwoorden

VRAAG 1

De `receptId`, de `ingredientId` en de hoeveelheid, maar `"ingredient": null`. `Include` laadt de koppelobjecten, en niet wat er daarachter zit. Daarvoor dient `ThenInclude`.

‹ Terug naar de vraag

VRAAG 2

Compileert niet: CS1061, *'TEntity' does not contain a definition for 'Keuken'*. De generieke repository weet niet welk type ze krijgt, dus ook niet welke navigaties het heeft. Daarom hoort `Include` in een specifieke repository.

‹ Terug naar de vraag

VRAAG 3

500, met *A possible object cycle was detected.*

Het recept wijst naar zijn keuken. EF Core vult in die keuken ook de lijst met recepten aan, met het recept dat al gevolgd wordt. Dat recept wijst weer naar de keuken, en zo verder: een kringloop. Dat aanvullen heet *relationship fix-up*. Je hebt er niet om gevraagd, maar het gebeurt vanzelf.

‹ Terug naar de vraag

42. Les 09: DTO’s en Mapster

DATA

Wat je bouwt

Aparte klassen voor wat je API toont en voor wat ze ontvangt: DTO’s. Mapster zet je modellen om naar DTO’s en terug. Wat niet vanzelf past, leg je vast in een `MapperProfile`. `IgnoreCycles` verdwijnt.

Wat er bijkomt in je project

```
ReceptenApi/
├── Configuration/                    ← nieuw
│   └── MapperProfile.cs              ← nieuw
├── Controllers/                      ← gewijzigd: DTO's in en uit
├── DTOs/                             ← nieuw
│   ├── Keukens/
│   │   ├── KeukenDto.cs
│   │   └── KeukenWriteDto.cs
│   └── Recepten/
│       ├── ReceptDetailDto.cs
│       ├── ReceptDto.cs
│       └── ReceptWriteDto.cs
├── Program.cs                        ← gewijzigd
└── ReceptenApi.csproj                ← gewijzigd: Mapster
```

Stap voor stap

1. Haal in `Program.cs` de regel met `IgnoreCycles` weg. De lege `AddJsonOptions` eromheen mag mee weg: dan staat er weer gewoon `AddControllers()`.
2. Maak een map `DTOs`, met een submap per model. Maak per endpoint de DTO’s die het nodig heeft.
3. Een read-DTO bevat enkel wat de client moet zien, zo plat mogelijk, en mag een Id hebben. Een write-DTO bevat enkel wat de client mag invullen: geen Id, en geen berekende velden.
4. Neem de namen en de types van de eigenschappen letterlijk over uit je model, waar het kan. Wat gelijk heet, zet Mapster vanzelf om.
5. Installeer Mapster: `dotnet add package Mapster`, of via NuGet.
6. Laat je controllers DTO’s teruggeven en ontvangen. Bovenaan komt `using Mapster;`, en het returntype wordt de DTO.
7. Test. Wat niet op naam past, blijft leeg, zonder foutmelding.
8. Maak een map `Configuration` met `MapperProfile`, die `IRegister` belooft. Leg daarin vast wat niet vanzelf past.
9. Laat Mapster in `Program.cs` naar je `MapperProfile` zoeken, net na `CreateBuilder`.
10. Test opnieuw: alles is ingevuld, en enkel met wat je wil tonen.

De cursus maakt de DTO’s als `class` met `{ get; set; }`. Een `record` mag ook. Zie hoofdstuk 14.

Een POST die een write-DTO ontvangt en een read-DTO teruggeeft:

```
[HttpPost]
public async Task<ActionResult<ReceptDto>> Create(ReceptWriteDto writeDto)
{
    Recept recept = writeDto.Adapt<Recept>();
    _uow.ReceptRepository.Add(recept);
    await _uow.SaveChangesAsync();

    ReceptDto dto = recept.Adapt<ReceptDto>();
    return CreatedAtAction(nameof(GetById), new { id = dto.Id }, dto);
}
```

De regel in `Program.cs`, met `using Mapster;` en `using System.Reflection;` bovenaan:

```
TypeAdapterConfig.GlobalSettings.Scan(Assembly.GetExecutingAssembly());
```

> **OPMERKING**
>
> De samenvatting van de les zegt dat je de invoer valideert met DataAnnotations op je write-DTO's, maar werkt het niet uit. Het gaat om attributen boven een eigenschap, zoals `[Required]` of `[Range(1, 20)]`, uit `System.ComponentModel.DataAnnotations`. Met `[ApiController]` geeft een write-DTO die er niet aan voldoet vanzelf een 400, nog voor je methode start.

Program.cs na deze les

Zie hoofdstuk 35: daar staat `Program.cs` volledig zoals na les 09.

Hoe je ziet dat het werkt

- Geen enkel endpoint geeft nog een model terug of ontvangt er een.
- `IgnoreCycles` staat nergens meer, en toch zijn er geen kringlopen.
- Scalar toont bij elk endpoint de DTO, met enkel de velden die je wil tonen.
- Een POST met een Id in de body: het Id wordt genegeerd, want je write-DTO heeft er geen.

Als een stap ontbreekt

Wat je ziet	Waarom	Wat je doet
een veld is leeg, 0 of `null`, zonder foutmelding	de naam of het type verschilt van het model, en er is geen eigen regel	de naam gelijk maken, of een regel in je `MapperProfile`
je eigen regels doen niets	de regel met `Scan` ontbreekt in `Program.cs`	toevoegen, net na `CreateBuilder`
een eigen regel doet niets, en er staat een andere voor hetzelfde paar	twee keer `NewConfig` voor dezelfde bron en hetzelfde doel: enkel de laatste geldt	één regel per paar
CS0246: *The type or namespace name 'IRegister' could not be found*	`using Mapster;` ontbreekt, of het package	toevoegen
500 met *A possible object cycle was detected.*	een DTO bevat een model, of twee DTO's wijzen naar elkaar	een DTO bevat enkel waarden en andere DTO's, nooit een model
een platgeslagen veld, zoals `KeukenNaam`, blijft leeg	de repository laadt de keuken niet mee	`Include` in de repository
400 met een lijst fouten, nog voor je methode start	de write-DTO voldoet niet aan zijn DataAnnotations, of een verplicht veld ontbreekt	zo hoort het: dat is de automatische validatie

Let op bij de lescode

LET OP

- `if (products == null || !products.Any()))` heeft een haakje te veel. Zo compileert het niet.
- `GetAllProducts(int id)` heeft een parameter die nergens gebruikt wordt.
- `config.NewConfig<OrderLijn, BesteldProductDto>()` staat er twee keer. De tweede vervangt de eerste.
- `GetBestellingenVanKlant` belooft nog `ActionResult<Klant>`, maar geeft een `KlantDto` terug. Pas het returntype aan.
- `Aantal` is een `int` in de DTO en een `double` in het model. Kijk na of de waarde in je JSON klopt.
- `KlantDto` staat in `DTOs/Klanten`, maar gebruikt `BesteldProductDto` uit `DTOs/Bestellingen`. Daarvoor is een `using` nodig.
- De les berekent een totaalprijs in de mapping, terwijl ze dat zelf afraadt.

Waar het uitgelegd staat

Onderwerp	Hoofdstuk
DTO's, Mapster, platslaan en de `MapperProfile`	33
`record`	14
`Include`	32
`SelectMany` om lijsten plat te slaan	11

Voorspel de uitkomst

Vraag 1

`ReceptWriteDto` heeft geen eigenschap `Id`. Een client stuurt toch `"id": 50` mee in de body van een POST. Wat gebeurt er met die 50?

Toon het antwoord ›

Vraag 2

In `ReceptDto` heet de eigenschap `Naam`, in `Recept` heet ze `Titel`. Er is geen eigen regel. Wat staat er in de JSON?

Toon het antwoord ›

Vraag 3

De regel met `Scan` ontbreekt in `Program.cs`, maar elke DTO heeft enkel eigenschappen die op naam en type passen. Werkt alles?

Toon het antwoord ›

Antwoorden

VRAAG 1

Niets. De write-DTO heeft geen plaats voor een Id, dus de 50 wordt bij het inlezen gewoon overgeslagen. Het nieuwe recept krijgt een Id van de database. Zo houdt een write-DTO buiten wat de client niet mag invullen.

‹ Terug naar de vraag

VRAAG 2

`"naam": null`, zonder foutmelding. Mapster vindt in `Recept` geen eigenschap `Naam`, en laat ze op haar standaardwaarde staan. Hetzelfde woord gebruiken, of een regel met `.Map(dest => dest.Naam, src => src.Titel)`, lost het op.

‹ Terug naar de vraag

VRAAG 3

Ja. `Scan` is enkel nodig om je eigen regels uit de `MapperProfile` te vinden. Wat op naam en type past, zet Mapster altijd vanzelf om.

‹ Terug naar de vraag

Veelgemaakte verwarringen

Tweeënveertig paren die op elkaar lijken en iets anders doen. Dit is het deel waar je het vaakst naar terug zal bladeren.

`Length`, `Count` en `Count()`

Een array en een tekst zeggen `Length`. Een `List` zegt `Count`, zonder haakjes, en weet het meteen. `Count()` met haakjes is LINQ: dat werkt op elke verzameling, maar loopt ze daarvoor helemaal af. Op een lijst neem je de eigenschap. Hoofdstukken 5, 6 en 11.

Array tegenover List

Een array heeft een vaste lengte, een `List` groeit mee. In de praktijk kies je bijna altijd de `List`, tenzij de hoeveelheid echt vastligt. Hoofdstukken 5 en 6.

List tegenover Dictionary

Zoek je in een `List` telkens met `Find` of `FirstOrDefault` op dezelfde eigenschap — een naam, een code — dan wil je eigenlijk een `Dictionary` met die eigenschap als sleutel. De `List` kijkt elk item na tot hij het vindt, de `Dictionary` weet meteen waar hij moet zijn. Zoek je afwisselend op verschillende eigenschappen, of doet de volgorde ertoe, dan blijft de `List` de betere keuze. Hoofdstukken 6 en 7.

Queue tegenover Stack

Allebei laten ze je enkel aan één kant bij. Stel jezelf de vraag: *wie komt er als volgende aan de beurt?* Wie het langst wacht — `Queue`. Wie het laatst kwam — `Stack`. Hoofdstukken 9 en 10.

`Dequeue` tegenover `First()`

`Dequeue` haalt de eerste uit de rij: daarna is hij weg. `First()` van LINQ kijkt enkel, net als `Peek`. Op een lege rij crashen ze alle drie. Bij een `Stack` geldt hetzelfde voor `Pop` tegenover `First()`. Hoofdstukken 9 en 10.

`UnionWith` tegenover `Union`

`a.UnionWith(b)` verandert de set `a` zelf en geeft niets terug. `a.Union(b)` komt van LINQ: `a` blijft zoals hij was, en je krijgt een nieuwe reeks zonder dubbels. Hetzelfde geldt voor `IntersectWith` tegenover `Intersect`, en `ExceptWith` tegenover `Except`. Hoofdstukken 8 en 11.

`SetEquals` tegenover `SequenceEqual`

`SetEquals` vraagt of dezelfde dingen erin zitten; volgorde en dubbels tellen niet. `SequenceEqual` vraagt of alles in dezelfde volgorde staat, met even veel items. Hoofdstukken 8 en 11.

`Remove`, `RemoveAt` en `RemoveAll`

`Remove` wil het item, `RemoveAt` wil een positie, `RemoveAll` wil een voorwaarde. Ze geven ook iets anders terug: `Remove` een ja of nee, `RemoveAll` het aantal dat weg is, `RemoveAt` niets. En `Remove` haalt enkel het eerste voorkomen weg. Hoofdstuk 6.

`Reverse` op een lijst, een array en in LINQ

`lijst.Reverse()` draait de lijst zelf om en geeft niets terug. Op een array geeft `getallen.Reverse()` in .NET 10 een nieuwe, omgekeerde reeks, en blijft de array zelf zoals hij was; `Array.Reverse(getallen)` draait de array zelf om. Wil je op een lijst de versie van LINQ, schrijf dan `lijst.AsEnumerable().Reverse()`. Hoofdstukken 5, 6 en 11.

`Exists` tegenover `Any`

Hetzelfde antwoord: is er minstens één die voldoet. `Exists` is een methode van `List` en van `Array`, `Any` komt van LINQ en werkt op elke verzameling, ook op een `Dictionary`, een `HashSet` en een vraag aan de database. Hoofdstukken 6 en 11.

`FindAll` tegenover `Where`

`FindAll` is een methode van `List` en geeft meteen een nieuwe lijst. `Where` komt van LINQ en geeft een vraag die pas rekent als je ze overloopt (hoofdstuk 12). Wil je een lijst, schrijf dan `Where(...).ToList()`. Hoofdstukken 6 en 11.

`Parse` tegenover `TryParse`

`Parse` gelooft je en crasht als je ongelijk hebt. `TryParse` vraagt het na en geeft `false`. Alles wat van buiten komt, gaat door `TryParse`. Hoofdstuk 2.

`TryParse` tegenover `try`/`catch`

Allebei voorkomen ze dat je programma stopt op een tekst die geen getal is. `TryParse` kijkt vooraf en geeft `false`; `try`/`catch` laat de fout gebeuren en vangt ze daarna op. Kan je vooraf kijken, doe dat dan: het is sneller en zegt duidelijker wat je bedoelt. Houd `try`/`catch` voor wat je vooraf niet kan nakijken, zoals een bestand. Hoofdstukken 2 en 23.

`=` tegenover `==`

`=` geeft een variabele een waarde, `==` vraagt of twee waarden gelijk zijn. In een `if` hoort altijd `==`. C# weigert `if (x = 5)`, want een toewijzing is geen ja of nee. Hoofdstuk 4.

`decimal` tegenover `double`

Allebei zijn het kommagetallen. Een `double` is snel, maar wijkt soms een heel klein beetje af. Een `decimal` rekent tot op de cent juist. Voor geld en prijzen neem je `decimal`, en dan schrijf je een `m` achter elk getal: `19.99m`. Hoofdstuk 2.

`(int)` tegenover `Convert.ToInt32`

`(int)2.9` kapt af: `2`. `Convert.ToInt32(2.9)` rondt af: `3`. En `Convert.ToInt32(2.5)` geeft `2`, want bij een halve rondt C# af naar het even getal. Hoofdstuk 2.

`Hours` tegenover `TotalHours`

`Hours` is enkel het uren-deel van een duur, van 0 tot 23. `TotalHours` is de hele duur in uren, met komma. Bij 14 dagen en 6 uur is `Hours` 6 en `TotalHours` 342. Hoofdstuk 3.

`==` tegenover `Equals` bij tekst

Allebei vergelijken ze letterlijk, hoofdletters inbegrepen. Het verschil is dat `Equals` een tweede argument aanvaardt waarmee je die regel kan versoepelen. Zodra hoofdletters er niet toe mogen doen, heb je `Equals` nodig. Hoofdstuk 1.

`string.Join` op een lijst tegenover op één tekst

`string.Join("-", fruit)` zet een streepje tussen de items van de lijst. `string.Join("-", "kiwi")` geeft gewoon `"kiwi"`: één tekst is één item. Wil je tussen de tekens, geef dan de tekens mee: `string.Join("-", "kiwi".ToCharArray())`. Hoofdstuk 1.

`Find` tegenover `FirstOrDefault`

Op een lijst doen ze hetzelfde. `Find` bestaat enkel op een `List` en een array; `FirstOrDefault` werkt op elke verzameling, ook op een databasequery. Omdat je later met databases werkt en `Find` daar niet bestaat, went `FirstOrDefault` beter aan. Hoofdstukken 6 en 11.

`First` tegenover `Single`

`First` zegt: er is er minstens één, geef mij de eerste. `Single` zegt: er is er precies één, en als dat niet klopt is er iets grondig mis. Gebruik `Single` op een Id, en `First` op een sortering. Hoofdstuk 11.

`Where` tegenover `Select`

`Where` houdt items over, `Select` verbouwt ze. Krijg je mínder items terug dan je erin stak, dan heb je gefilterd — dat is `Where`. Krijg je er evenveel terug maar van een ander soort, bijvoorbeeld enkel de namen, dan heb je verbouwd — dat is `Select`. Hoofdstuk 11.

`Max` tegenover `MaxBy`

`Max` geeft de waarde, `MaxBy` geeft het item dat die waarde draagt. Dezelfde verhouding geldt voor `Min` en `MinBy`. Hoofdstuk 11.

`Any()` tegenover `Count() > 0`

Hetzelfde antwoord, maar `Any` stopt bij het eerste item dat voldoet. Op een lijst van vijf maakt dat niets uit; op een databasetabel van vijftigduizend wel. Hoofdstuk 11.

Wat een methode teruggeeft, en wat ze verandert

`lijst.Sort()` verandert je lijst en geeft niets terug. `lijst.OrderBy(...)` laat je lijst met rust en geeft een nieuw resultaat. Hetzelfde geldt voor `Trim` en `ToUpper` bij tekst: die veranderen niets, ze geven een nieuwe tekst terug. Compileert je regel niet, of lijkt hij niets te doen, dan heb je meestal de verkeerde van de twee te pakken. Hoofdstukken 1, 6 en 11.

`IEnumerable` tegenover `List`

Een `IEnumerable` is een vraag die nog gesteld moet worden. Een `List` is een antwoord dat al klaarligt. `Where` voert niets uit tot je het resultaat doorloopt; verandert je bron intussen, dan verandert je antwoord mee. `ToList()` stelt de vraag meteen en legt het antwoord vast. Hoofdstuk 12.

`Contains` bij een class tegenover een record

Bij een `class` zoekt `Contains` **hetzelfde object**: een tweede object met dezelfde inhoud vindt hij niet. Bij een `record` zoekt hij **dezelfde inhoud**. Hetzelfde verschil zit in `Distinct`, `HashSet`, een sleutel van een `Dictionary` en `==`. Hoofdstukken 13 en 14.

`OfType` tegenover `Cast`

`OfType<Kat>()` houdt enkel de katten over en slaat de rest over. `Cast<Kat>()` probeert alles om te zetten, en crasht met een `InvalidCastException` zodra er iets anders tussen zit. Hoofdstuk 22.

Abstracte klasse tegenover `interface`

Een abstracte klasse zegt wat iets **is**, en geeft gegevens en uitgewerkte methodes mee. Een interface zegt wat iets **kan**, en geeft niets mee. Een klasse erft van hoogstens één abstracte klasse, maar kan zoveel interfaces beloven als ze wil. Delen je klassen gegevens, dan is het een abstracte klasse; delen ze enkel een vaardigheid, dan een interface. Hoofdstukken 19 en 20.

`private` tegenover `protected`

Allebei verbergen ze iets voor de buitenwereld. Het verschil zit bij wie erft: `private` is enkel zichtbaar in de klasse zelf, `protected` ook in elke klasse die ervan erft. Kies `private`, tenzij een afgeleide klasse het echt nodig heeft. Hoofdstuk 20.

`static` tegenover een gewoon veld

Een gewoon veld heeft elk object apart. Een `static` veld is er één keer, voor de hele klasse. Daarom onthoudt een `static` lijst in een controller wat er toegevoegd werd, terwijl een gewone lijst bij elk verzoek opnieuw leeg begint. Hoofdstuk 24.

`const` tegenover `readonly`

Allebei veranderen ze niet meer. Een `const` ligt al vast bij het compileren en kan enkel een getal, een tekst of een `bool` zijn. Een `readonly` veld vul je één keer in, bij de declaratie of in de constructor. Het kan dus ook iets zijn dat pas bekend is als het programma draait, zoals een repository. Hoofdstuk 24.

`readonly` tegenover `AsReadOnly`

`readonly` voor een veld beschermt het veld: je kan er geen andere lijst in zetten. Aan de lijst zelf kan je nog altijd toevoegen. `AsReadOnly()` geeft een kijk op de lijst zonder `Add` en `Remove`. Hoofdstukken 6 en 24.

`{id}` tegenover `{id:int}`

Allebei halen ze een waarde uit de route. Zonder beperking past de route bij elke tekst, en geeft een tekst die geen getal is een 400. Met `:int` past de route enkel bij een geheel getal, en geeft al de rest een 404. Hoofdstuk 25.

`ActionResult<T>` tegenover `IActionResult`

`ActionResult<T>` zegt wat er terugkomt, en laat je dat ook rechtstreeks teruggeven. `IActionResult` kent enkel antwoorden, dus daar pak je je gegevens zelf in met `Ok(...)`. Geeft een methode gegevens terug, neem dan `ActionResult<T>`. Geeft ze enkel een statuscode terug, dan volstaat `IActionResult`. Hoofdstuk 26.

`AddScoped` tegenover `AddSingleton`

`AddScoped` maakt één object per verzoek, `AddSingleton` één voor de hele app. Bewaart een dienst zelf gegevens in het geheugen, dan is een scoped dienst ze na elk verzoek kwijt, terwijl een singleton ze onthoudt. `AddTransient` maakt elke keer een nieuw object. Hoofdstuk 27.

`await` tegenover `.Result`

Allebei geven ze je wat er in een `Task` zit. `await` laat de thread intussen vrij voor ander werk. `.Result` houdt hem vast tot het antwoord er is, en kan een programma zelfs laten vastlopen. In een API gebruik je altijd `await`. Hoofdstuk 29.

`IEnumerable` tegenover `IQueryable`

Allebei zijn het vragen die nog niet gesteld zijn. Een `IEnumerable` voert je programma zelf uit, op wat al in het geheugen zit. Een `IQueryable` wordt naar SQL vertaald, en de database voert hem uit. Filter je pas na `ToListAsync`, dan komt eerst alles uit de database, en filtert je programma daarna. Hoofdstukken 12 en 32.

`Add` tegenover `SaveChangesAsync`

`Add` legt iets in het mandje, `SaveChangesAsync` rekent af. Na `Add` staat er nog niets in de database, en heeft het object nog geen `Id`. Hetzelfde geldt voor `Update` en `Remove`. Hoofdstuk 32.

`Include` tegenover `ThenInclude`

`Include` vertrekt altijd van het object dat je opvraagt. `ThenInclude` gaat één stap verder vanaf de vorige `Include`. De keuken van een recept is een `Include`. De ingrediënten achter de koppelobjecten van een recept zijn een `Include`, gevolgd door een `ThenInclude`. Hoofdstuk 32.

`Cascade` tegenover `Restrict`

Allebei zeggen ze wat er gebeurt als je iets verwijdert waar nog andere rijen aan hangen. `Cascade` neemt die rijen mee, zonder iets te vragen. `Restrict` weigert, en laat jou eerst opruimen. Bij een verplichte relatie is `Cascade` de standaard. Hoofdstuk 31.

`-1`, `null` en `0`

`IndexOf` geeft `-1` wanneer er niets gevonden is. `FirstOrDefault` geeft `null` bij objecten, maar `0` bij getallen. `TryGetValue` zet zijn uitkomst op de standaardwaarde. Drie manieren om "niets" te zeggen, en elk vraagt een andere controle. Hoofdstukken 1, 7, 11 en 17.

Index

Alfabetisch. Het **vette** nummer is het hoofdstuk met de volledige uitleg. De andere nummers zijn hoofdstukken waar je het ook tegenkomt, bijvoorbeeld als LINQ-methode op die verzameling of in een tabel "via iets anders".

Naam	Hoofdstuk
`abstract`	**20**
`ActionResult<T>`	**26**
`Adapt`	**33**, 42
`Add`	3, **6**, **7**, 8, **32**
`Add-Migration`	**31**, 37, 39
`AddAsync`	**32**
`AddControllers`	**27**
`AddDays`	**3**
`AddDbContext`	**31**, 37
`AddHours`	**3**
`AddMinutes`	**3**
`AddMonths`	**3**
`AddOpenApi`	**27**
`AddRange`	**6**, 32
`AddRangeAsync`	**32**
`AddScoped`	**27**, 36, 38, 39, 40
`AddSeconds`	**3**
`AddSingleton`	**27**
`AddTransient`	**27**
`AddYears`	**3**
`Aggregate`	5, 6, 7, 8, 9, 10, **11**
`AggregateBy`	5, 6, 7, 8, 9, 10, **11**
`All`	1, 5, 6, 7, 8, 9, 10, **11**
`AllAsync`	**32**
`and`	**4**
anoniem object	**13**
`Any`	1, 5, 6, 7, 8, 9, 10, **11**, 34
`AnyAsync`	**32**
`[ApiController]`	**25**
`Append`	1, 5, 6, 7, 8, 9, 10, **11**

Naam	Hoofdstuk
AppendJoin	1
AppendLine	1
appsettings.json	27, 28, 31, 35, **37**
array	5
Array.AsReadOnly	5
Array.BinarySearch	5
Array.Clear	5
Array.ConvertAll	5
Array.Copy	5
Array.Empty	5
Array.Exists	5
Array.Fill	5
Array.Find	5
Array.FindAll	5
Array.FindIndex	5
Array.FindLast	5
Array.FindLastIndex	5
Array.ForEach	5
Array.IndexOf	5
Array.LastIndexOf	5
Array.Resize	5
Array.Reverse	5
Array.Sort	5
Array.TrueForAll	5
as	22
AsAsyncEnumerable	32
AsList	34
AsNoTracking	32
AsReadOnly	6
AsSplitQuery	32
Async	29
async	29
Attach	32, 40
AttachRange	32

Naam	Hoofdstuk
attribuut	**25**
Average	2, 5, 6, 7, 8, 9, 10, **11**
AverageAsync	**32**
await	**29**
BadRequest	**26**
base	**20**
basisklasse	**20**
BinarySearch	**6**
break	**4**
Build	**27**
Capacity	**6**, 9, 10
Cascade	**31**
Cast	**11**, 22
catch	**23**
change tracker	**32**
char.GetNumericValue	**1**
char.IsAsciiDigit	**1**
char.IsAsciiLetter	**1**
char.IsDigit	**1**
char.IsLetter	**1**
char.IsLetterOrDigit	**1**
char.IsLower	**1**
char.IsPunctuation	**1**
char.IsUpper	**1**
char.IsWhiteSpace	**1**
char.ToLower	**1**
char.ToUpper	**1**
checked	**2**
Chunk	1, 5, 6, 7, 8, 9, 10, **11**
class	**13**
Clear	1, **6**, 7, 8, 9, 10
Clone	**5**
Comparer	**7**, 8
CompareTo	**1**, 2, 3, 18

Naam	Hoofdstuk
Concat	1, 5, 6, 7, 8, 9, 10, **11**
const	**24**
constructor	**13**
Contains	**1**, 3, 5, **6**, 7, 8, 9, 10, 11, 13, 14
ContainsAsync	**32**
ContainsKey	**7**
ContainsValue	**7**
continue	**4**
Convert.ToInt32	**2**
ConvertAll	**6**
CopyTo	**5**, 6, 9, 10
Count	1, 2, 3, 5, **6**, 7, 8, 9, 10, **11**, 34
CountAsync	**32**
CountBy	1, 5, 6, 7, 8, 9, 10, **11**
CreatedAtAction	**26**, 38
CultureInfo.InvariantCulture	**2**
Dapper	**34**
Date	**3**
DateOnly	**3**
DateOnly.FromDateTime	**3**
DateTime	**3**
DateTime.DaysInMonth	**3**
DateTime.IsLeapYear	**3**
DateTime.MaxValue	**3**
DateTime.MinValue	**3**
DateTime.Now	**3**
DateTime.ParseExact	**3**
DateTime.Today	**3**
DateTime.TryParse	**3**
DateTime.TryParseExact	**3**
DateTime.UtcNow	**3**
Day	**3**
DayNumber	**3**
DayOfWeek	**3**

Naam	Hoofdstuk
DayOfYear	**3**
Days	**3**
DbContext	**31**, 37
DbSet	**31**, 37
decimal	**2**
decimal.Parse	**2**
decimal.TryParse	**2**
Deconstruct	**14**
default	**21**
DefaultIfEmpty	**11**
dependency injection	**27**, 36
Dequeue	**9**
Dictionary	**7**
Distinct	1, 2, 5, 6, 7, 8, 9, 10, **11**, 13, 14, 32
DistinctBy	5, 6, 8, 9, 10, **11**
do	**4**
docker compose	**37**
Docker Desktop	**37**
docker-compose.yml	35, **37**
dotnet add package	33, **37**, 42
dotnet ef	**31**, 37
double	**2**
double.Parse	**2**
DTO	**33**, 42
Duration	**3**
ElementAt	1, 5, 6, 7, 8, 9, 10, **11**
ElementAtAsync	**32**
ElementAtOrDefault	5, 6, 7, 8, 9, 10, **11**
ElementAtOrDefaultAsync	**32**
else	**4**
EndsWith	**1**
Enqueue	**9**
EnsureCapacity	**6**, 9, 10
Entity<T>	**31**

Naam	Hoofdstuk
EntityState	**32**, 40
Entry	**32**
enum	**18**
Enum.GetNames	**18**
Enum.GetValues	**18**
Enum.IsDefined	**18**
Enum.Parse	**18**
Enum.TryParse	**18**
Enumerable.Empty	**11**
Enumerable.InfiniteSequence	**11**
Enumerable.Range	2, **11**
Enumerable.Repeat	**11**
Enumerable.Sequence	**11**
Equals	**1**, 2, 5, 6, 7, 8, 9, 10, 12, 13, 14, 15, 18
Except	1, 5, 6, 7, 8, 9, 10, **11**
ExceptBy	5, 6, 7, 8, **11**
Exception	**23**
ExceptWith	**8**
ExecuteAsync	**34**
ExecuteDeleteAsync	**32**
ExecuteReaderAsync	**34**
ExecuteScalarAsync	**34**
ExecuteUpdateAsync	**32**
Exists	**6**
File.AppendAllText	**30**
File.Exists	**30**
File.ReadAllLines	**30**
File.ReadAllText	**30**
File.ReadLines	**30**
File.WriteAllLines	**30**
File.WriteAllText	**30**
finally	**23**
Find	**6**, 32
FindAll	**6**

Naam	Hoofdstuk
FindAsync	**32**
FindIndex	**6**
FindLast	**6**
FindLastIndex	**6**
First	1, 5, 6, 7, 8, 9, 10, **11**, 34
FirstAsync	**32**
FirstOrDefault	1, 5, 6, 7, 8, 9, 10, **11**, 34
FirstOrDefaultAsync	**32**
Fluent API	**31**, 39
for	**4**
ForEach	**6**
foreach	**4**
ForEachAsync	**32**
ForType	**33**
[FromBody]	**25**
FromDays	**3**
FromHours	**3**
[FromQuery]	**25**
FromSql	**32**
GenericRepository	**40**, 41
generics	**21**
GetConnectionString	**27**, 37
GetHashCode	**1**, 5, 6, 7, 8, 9, 10, 13, 14
GetLength	**5**
GetLowerBound	**5**
GetRange	**6**
GetType	1, 2, 3, 5, 6, 7, 8, 9, 10, 12, 13, 15, 18, 20, **22**, 23
GetUpperBound	**5**
GetValueOrDefault	7, **17**
GroupBy	1, 3, 5, 6, 7, 8, 9, 10, **11**, 32
GroupJoin	5, 6, 7, 8, 9, **11**
HasData	**31**, 39
HasForeignKey	**31**
HashSet	**8**

Naam	Hoofdstuk
HasKey	**31**
HasMaxLength	**31**
HasOne	**31**
HasPrecision	**31**
HasValue	**17**
Hour	**3**
Hours	**3**
[HttpDelete]	**25**
[HttpGet]	**25**
[HttpPost]	**25**
[HttpPut]	**25**
IActionResult	**26**
IEnumerable	**12**
if	**4**
IGenericRepository<TEntity>	**40**, 41
Ignore	**33**
IgnoreCycles	**32**, 41, 42
ILogger	**28**, 36
Include	**32**, 41
Index	5, 6, 8, 9, 10, **11**
IndexOf	**1**, **6**, 13, 14
IndexOfAny	**1**
Insert	1, **6**
InsertRange	**6**
int.MaxValue	**2**
int.MinValue	**2**
int.Parse	1, **2**
int.TryParse	1, **2**
interface	**19**
Intersect	1, 5, 6, 7, 8, 9, 10, **11**
IntersectBy	5, 6, 7, 8, **11**
IntersectWith	**8**
IQueryable	**32**
IRegister	**33**, 42

Naam	Hoofdstuk
is	20, **22**
IsBetween	**3**
IsDevelopment	**27**
IsNullOrEmpty	**1**
IsNullOrWhiteSpace	**1**
IsProperSubsetOf	**8**
IsProperSupersetOf	**8**
IsRequired	**31**
IsSubsetOf	**8**
IsSupersetOf	**8**
IUnitOfWork	**40**, 41
Join	5, 6, 7, 8, 9, 10, **11**
JsonSerializer.Deserialize	**30**
JsonSerializer.Serialize	**30**
Keys	**7**
Keys.Contains	**7**
Keys.ToList	**7**
KeyValuePair	**7**
Kind	**3**
Last	1, 5, 6, 7, 8, 9, 10, **11**
LastAsync	**32**
LastIndexOf	**1**, 6
LastIndexOfAny	**1**
LastOrDefault	1, 5, 6, 7, 8, 9, 10, **11**
LastOrDefaultAsync	**32**
lazy loading	**32**
LeftJoin	5, 6, 7, 8, 9, **11**
Length	**1**, 5
List	**6**
LoadAsync	**32**
Local	**32**
LogCritical	**28**
LogDebug	**28**
LogError	**28**

Naam	Hoofdstuk
LogInformation	**28**
LogTrace	**28**
LogWarning	**28**
long	**2**
LongCount	**11**
LongCountAsync	**32**
m	**2**
Map	**33**
MapControllers	**27**
MapOpenApi	**27**
MapperProfile	33, **42**
MapScalarApiReference	**27**
Mapster	**33**
Math.Abs	**2**
Math.Ceiling	**2**
Math.Clamp	**2**
Math.Floor	**2**
Math.Max	**2**
Math.Min	**2**
Math.Pow	**2**
Math.Round	**2**
Math.Sign	**2**
Math.Sqrt	**2**
Math.Truncate	**2**
Max	1, 2, 3, 5, 6, 7, 8, 9, 10, **11**, 13
MaxAsync	**32**
MaxBy	3, 5, 6, 7, 8, 9, 10, **11**
Message	**23**
micro-ORM	**34**
MidpointRounding	**2**
migratie	**31**, 37, 39
Millisecond	**3**
Min	1, 2, 3, 5, 6, 7, 8, 9, 10, **11**, 13
MinAsync	**32**

Naam	Hoofdstuk
MinBy	5, 6, 7, 8, 9, 10, **11**
Minute	**3**
Minutes	**3**
Month	**3**
N+1-probleem	**32**
nameof	**22**
namespace	**13**
navigatie-eigenschap	**31**
new()	**21**
NewConfig	**33**
NoContent	**26**
NotFound	**26**
OfType	5, 6, 8, 9, 10, **11**, 20, 22
Ok	**26**
OnDelete	**31**
OnModelCreating	**31**, 39
or	**4**
Order	1, 2, 3, 5, 6, 7, 8, 9, 10, **11**, 13, 14, 18, 32
OrderBy	3, 5, 6, 7, 8, 9, 10, **11**, 32, 34
OrderByDescending	5, 6, 7, 8, 9, **11**, 32
OrderDescending	1, 2, 3, 5, 6, 7, 8, 9, **11**, 32
overerving	**20**
Overlaps	**8**
override	**20**
Package Manager Console	31, **37**
PadLeft	**1**
PadRight	**1**
Parse	**2**
ParseExact	**3**
Path.Combine	**30**
Peek	**9**, **10**
pgAdmin	**37**, 38, 39
platslaan	**33**
Pop	**10**

Naam	Hoofdstuk
Prepend	1, 5, 6, 7, 8, 9, 10, **11**
primaire sleutel	**31**
Property	**31**
protected	**20**
Push	**10**
QueryAsync	**34**
QueryFirstAsync<T>	**34**
QueryFirstOrDefaultAsync	**34**
QueryMultipleAsync	**34**
QuerySingleAsync	**34**
QuerySingleOrDefaultAsync<T>	**34**
Queue	**9**
Random.Shared.Next	**2**
Random.Shared.NextDouble	**2**
Rank	**5**
readonly	**24**
record	**14**
refactoren	**36**
ReferenceEquals	**13**
referentietype	**16**
Register	**33**
Remove	1, **6**, **7**, 8, 13, 14, **32**
Remove-Migration	**31**, 39
RemoveAll	**6**
RemoveAt	**6**
RemoveRange	**6**, 32
RemoveWhere	**8**
Replace	**1**
repository	35, **36**, 38, 40, 41
Restrict	**31**
Reverse	1, 5, **6**, 7, 8, 9, 10, 11
RightJoin	5, 6, 7, 8, 9, **11**
[Route]	**25**
routeparameter	**25**

Naam	Hoofdstuk
Run	**27**
SaveChangesAsync	**32**, 38, 40
Scan	**33**, 42
sealed	**20**
Second	**3**
Select	1, 5, 6, 7, 8, 9, 10, **11**, 32, 34
SelectMany	5, 6, 7, 8, 9, 10, **11**
SequenceEqual	1, 5, 6, 8, 9, 10, **11**
Set<T>	**32**
Set<TEntity>()	**32**, 40
SetEquals	**8**
Shuffle	5, 6, 7, 8, 9, 10, **11**
Single	1, 5, 6, 7, 8, 9, 10, **11**
SingleAsync	**32**
SingleOrDefault	1, 5, 6, 7, 8, 9, 10, **11**
SingleOrDefaultAsync	**32**
Skip	1, 5, 6, 7, 8, 9, 10, **11**, 32
SkipLast	1, 5, 6, 8, 9, 10, **11**
SkipWhile	1, 5, 6, 7, 8, 9, 10, **11**
Slice	**6**
Sort	**6**, 13, 14
Split	**1**
splitOn	**34**
SQL-injectie	**34**
SqlQuery	**32**
Stack	**10**
StartsWith	**1**
static	**24**
StatusCode	**26**
string.Compare	**1**
string.Concat	**1**
string.Empty	**1**
string.Equals	**1**
string.Format	**1**

Naam	Hoofdstuk
string.IsNullOrEmpty	1
string.IsNullOrWhiteSpace	1
string.Join	1
struct	15
Substring	1
Subtract	3
Sum	1, 2, 5, 6, 7, 8, 9, 10, 11
SumAsync	32
switch	4
SymmetricExceptWith	8
Take	1, 5, 6, 7, 8, 9, 10, 11, 32
TakeLast	1, 5, 6, 8, 9, 10, 11
TakeWhile	1, 5, 6, 7, 8, 9, 10, 11
Task	29
Task.Delay	29
ThenBy	5, 6, 7, 8, 9, 11, 32
ThenByDescending	5, 6, 7, 8, 9, 11, 32
ThenInclude	32, 41
throw	23
TimeOfDay	3
TimeOnly	3
TimeOnly.AddMinutes	3
TimeSpan	3
TimeSpan.FromMinutes	3
TimeSpan.Zero	3
ToArray	5, 6, 7, 8, 9, 10, 11
ToArrayAsync	32
ToDateTime	3
ToDictionary	5, 6, 7, 8, 9, 10, 11, 34
ToDictionaryAsync	32
ToHashSet	5, 6, 8, 9, 10, 11
ToHashSetAsync	32
ToList	5, 6, 7, 8, 9, 10, 11, 34
ToListAsync	32

Naam	Hoofdstuk
ToLookup	5, 6, 7, 8, 9, 10, **11**
ToLower	**1**
ToLowerInvariant	**1**
ToQueryString	**32**
ToShortDateString	**3**
ToString	1, 2, 3, 5, 6, 7, 8, 9, 10, **12**, 13, 14, 15, 18
ToTable	**31**
TotalDays	**3**
TotalHours	**3**
TotalMinutes	**3**
ToUpper	**1**
ToUpperInvariant	**1**
Trim	**1**
TrimEnd	**1**
TrimExcess	**6**, 9, 10
TrimStart	**1**
TrueForAll	**6**
try	**23**
TryAdd	**7**
TryDequeue	**9**
TryGetValue	**7**, 8
TryPeek	**9**, 10
TryPop	**10**
typeof	**22**
uitzondering	**23**
Union	1, 5, 6, 7, 8, 9, 10, **11**
UnionBy	5, 6, 7, 8, **11**
UnionWith	**8**
Unit of Work	**32**, 40, 41
Update	**32**
Update-Database	**31**, 37, 39
UpdateRange	**32**
UseAuthorization	**27**
UseHttpsRedirection	**27**

Naam	Hoofdstuk
UseNpgsql	31
using	13
Values	7
Values.Contains	7
Values.ToHashSet	7
Values.ToList	7
virtual	20
vreemde sleutel	31
waardetype	16
WebApplication.CreateBuilder	27
Where	1, 3, 5, 6, 7, 8, 9, 10, **11**, 32, 34
where	21
while	4
with	14
WithMany	31
WithOne	31
Year	3
Zip	1, 5, 6, 7, 8, 9, 10, **11**
! en = default!	17
"""	34
%	2
&& en ||	4
.Result	29
=>	13
? :	4
?.	17
?? en ??=	17